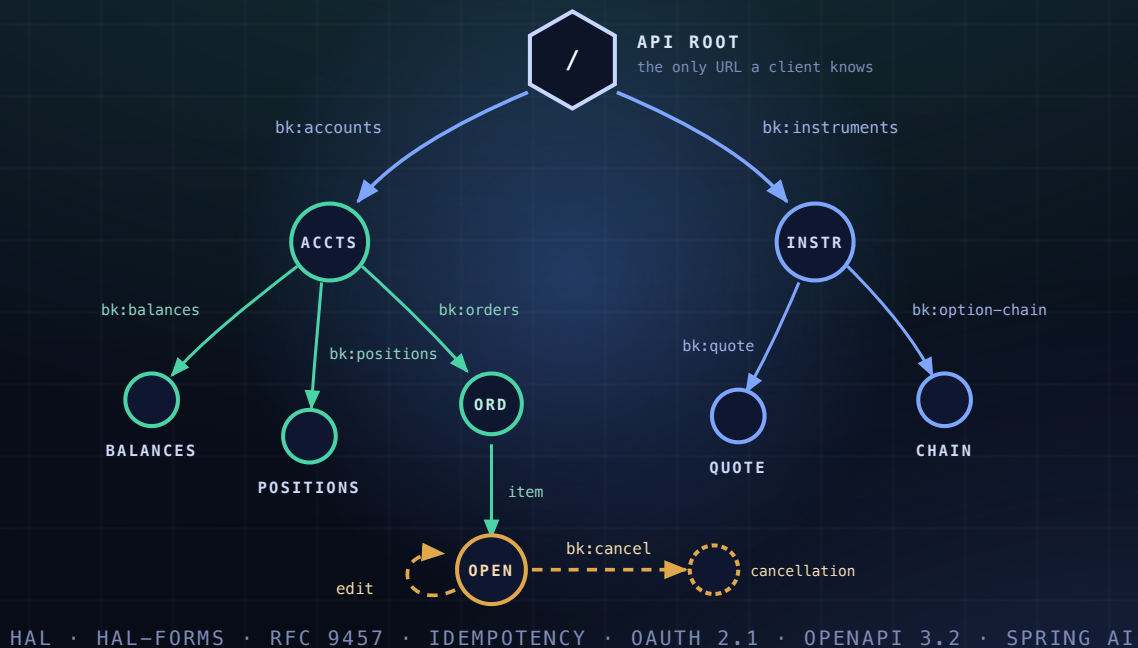


REST API Standards with HATEOAS

Design, build and consume hypermedia APIs the standard way. A complete brokerage API in Spring Boot 4 and Java 27 takes you from first principles to idempotent trading, OAuth 2.1, OpenAPI 3.2 contracts and an AI agent built with Spring AI.



26 chapters · a complete Spring Boot 4 brokerage API · every exchange captured from the running code

AUTHOR

Smiroh Dev

Smiroh Publishers
FIRST EDITION

REST API Standards with HATEOAS

Smiroh Dev

PUBLISHER	Smiroh Publishers
EDITION	First edition
PUBLISHED	September 2026
FORMAT	Electronic book (PDF), 256 pages
ISBN (EBOOK)	To be assigned
SUBJECT	Computers / Programming / Web Services & APIs
CODE	Java 27, Spring Boot 4.1.1, Spring AI 2.0.1; complete listings in Appendix E

Copyright © 2026 Smiroh Dev. All rights reserved.

Published by Smiroh Publishers. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

TRADEMARKS

Java is a registered trademark of Oracle and/or its affiliates. Spring, Spring Boot and related names are trademarks of Broadcom Inc. and/or its subsidiaries. Webull is a trademark of its owner. Claude and Anthropic are trademarks of Anthropic, PBC. All other trademarks are the property of their respective owners and are used editorially, with no intention of infringement. The publisher is not affiliated with any company or product named in this book.

ABOUT THE SAMPLE APPLICATION

The brokerage in this book is fictional. Its customers, accounts and prices are invented, and its market is a simulation. The code is a teaching implementation: state lives in memory, and the authorization server is configured for local development only. It is not a trading system and must never be connected to real accounts, real markets or real credentials.

STANDARDS AND THIRD-PARTY APIS

The RFCs, Internet-Drafts and specifications cited here are the normative sources; where this book and a specification disagree, the specification governs. Chapter 24 describes the publicly documented Webull OpenAPI as of September 2026, for comparison only. Product features, model names and draft standards change, so check current documentation before relying on them.

LIMIT OF LIABILITY AND DISCLAIMER OF WARRANTY

The information in this book is provided on an “as is” basis, without warranty of any kind, express or implied. While every precaution has been taken in its preparation, neither the author nor the publisher shall have any liability to any person or entity with respect to any loss or damage caused, or alleged to have been caused, directly or indirectly, by the information or code it contains. Nothing in this book is investment, legal or compliance advice.

Set in Helvetica Neue and Charter, with Menlo for code. Every HTTP exchange in this book was recorded from the running sample application, and every code excerpt is drawn from the tested source at build time. All diagrams are original vector artwork created for this edition.

First edition · September 2026 · Smiroh Publishers

What is in this book

–	Preface	Who this book is for, what you will build, and how to read it	5
PART I Foundations			
01	Why API standards matter	A duplicate order, a broken client, and the case for standards	9
02	REST in one chapter	Constraints, resources and the Richardson maturity model	12
03	HTTP, the parts that matter	Methods, status codes, headers and media types	15
04	Your workbench	Build, run and call the brokerage API	18
PART II Designing Resources			
05	Modeling a brokerage	From screens and verbs to resources and links	25
06	Representations	JSON style, money as strings, time in UTC	28
07	Accounts and history	Balances, positions and transactions: the read side	32
08	Market data	Instruments, quotes and option chains	39
PART III Hypermedia			
09	HATEOAS	Links as the engine of application state	47
10	HAL and link relations	_links, _embedded, CURIEs and registered relations	52
11	Hypermedia with Spring HATEOAS	Models, link builders and assemblers	56
12	Affordances with HAL-FORMS	Telling clients what they can do next	60
PART IV Trading Safely			
13	Placing orders	Validation, business rules and 201 Created	65
14	Idempotency	Retries that can never buy twice	74
15	Changing and canceling orders	ETags, merge patch and 202 Accepted	80
16	Problem details	Errors as RFC 9457 documents	87

CONTENTS · CONTINUED

PART V Operating at Scale

17	Collections	Cursor pagination, filtering and caching	94
18	Live order events	Server-sent events with replay	100
19	Evolution and quotas	Versions, deprecation and rate limits	104
20	Security	OAuth 2.1, scopes, PKCE and protected resource metadata	110

PART VI Contracts, Clients and Agents

21	The OpenAPI 3.2 contract	Describing a hypermedia API precisely	119
22	Testing the API	From MockMvcTester to contract tests	125
23	A hypermedia client	Following links from Java	131
24	RPC or resources?	Lessons from the Webull OpenAPI	135
25	Agentic AI with Spring AI	A trading assistant with a human in the loop	138
26	The standard on one page	Every rule in this book, as a checklist	146

APPENDICES

A	Quick reference	Commands, clients, scopes and every endpoint	148
B	Status codes and problem types	What each error means and what a client should do	150
C	Glossary	Every term in one place	152
D	Further reading	The RFCs, drafts and specifications behind each chapter	154
E	Complete source code	67 files and 6,424 lines, every one tested	156

How the chapters build on each other

Part I needs no prior knowledge beyond Java and a little HTTP. Each later part assumes the ones before it. If you already design REST APIs for a living, skim Part I, read Chapter 4 to get the code running, and start in earnest at Part III. If you only want the agent, read Chapters 4, 9, 14 and 20 first: Chapter 25 leans on all four.

About this book

Most APIs that call themselves RESTful stop at JSON over HTTP. This book goes the whole way, and shows with one complete, tested application why the last steps are the ones that pay.

Who this book is for

You write or review HTTP APIs, probably in Java, and you want them to be *boringly predictable*: every error shaped the same way, every retry safe, every collection paged the same way, and clients that do not break when the server changes. You might be a developer building your first public API, an architect writing your organization's API guidelines, or a tech lead who has to decide whether hypermedia is worth it. You need Java and a working idea of what an HTTP request is. Everything else is explained as it comes up.

What you will build

The running example is a retail brokerage, the kind of API that sits behind a trading app: accounts, balances, positions, transaction history, market data, and orders for stocks and options. A brokerage makes a demanding teacher. Money must never be rounded by accident, a retried request must never buy twice, two devices editing one order must not overwrite each other, and every action must be authorized to the cent. The standards in this book exist because domains like this one need them.

MODULE	WHAT IT IS	WHERE IT IS COVERED
auth-server	An OAuth 2.1 authorization server (Spring Security) with three registered clients and one demo customer.	Chapters 4 and 20
brokerage-api	The REST API itself: HAL and HAL-FORMS hypermedia, RFC 9457 errors, idempotent orders, ETags, cursors, server-sent events, versioning and rate limits.	Chapters 5–20
brokerage-client	A small Java client that navigates the API by following links and never builds a URL.	Chapter 23
brokerage-agent	An AI assistant built with Spring AI and Claude that uses the client as its tools, with a human confirming every trade.	Chapter 25
openapi	The OpenAPI 3.2 contract, served by the API and checked by its tests.	Chapter 21

How this book is built

Two rules kept the book honest. First, **no code in it is typed by hand**. Every code excerpt is pulled from the tested source when the book is built, and Appendix E prints every source file in full. Second, **no HTTP exchange is invented**. Each request and response was recorded from the running application, against

freshly started servers. When you run the same request, you will see the same shape, though prices and timestamps will differ, because the market in the sample application never stops moving.

api/orders/Order.java

source excerpt

```
@JsonIgnore
public String etag() {
    return "\"" + id + "-v" + version + "\"";
}
```

HTTP

recorded exchange

```
GET /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1
If-None-Match: "ORD-100002-v1"
-----
HTTP/1.1 304 Not Modified
```

Light blocks are source code, captioned with the file they come from. Dark blocks are traffic on the wire, captioned HTTP. Long HAL documents are sometimes shortened: a comment such as `/* 3 more */` marks where array items were left out, and the HAL `curies` entry, which every response repeats, is usually omitted. Chapter 10 explains what it is for.

THE STANDARD

Chapters that set a rule end with a box like this one. The boxes add up to a complete API style guide, gathered in Chapter 26 as a single checklist you can adopt, adapt or argue with.

What you need

- **JDK 27** (the sample code uses Java 27 as its release level) and **Maven 3.9** or later, or the Maven Wrapper included with the code.
- **curl**, or any HTTP client, to follow along with the exchanges.
- For Chapter 25 only, an **Anthropic API key**. Everything else runs without one.

COMPONENT	VERSION USED IN THIS BOOK
Java	27 (general availability September 15, 2026)
Spring Boot	4.1.1, with Spring Framework 7.0.9
Spring HATEOAS	3.1.2
Spring Security (including the authorization server)	7.1.1
Jackson	3.1.5
Spring AI	2.0.1, with the Anthropic Java SDK 2.52.0
JUnit	6.0.3
OpenAPI Specification	3.2.1

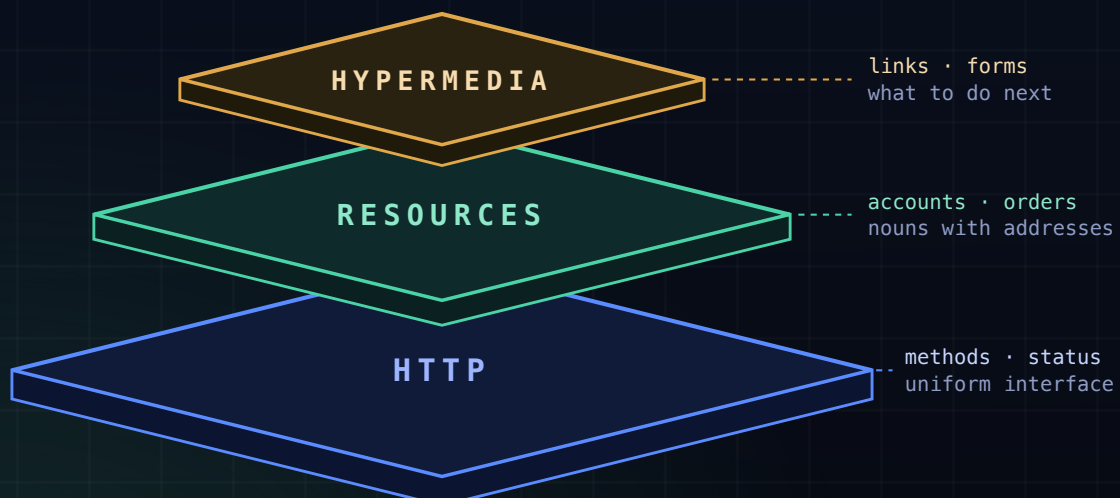
A note on the standards

Where an Internet standard exists, this book uses it and names it: RFC 9110 for HTTP semantics, RFC 9457 for errors, RFC 7396 for merge patches, RFC 8288 for links, and so on. A few conventions are still Internet-Drafts, notably the `Idempotency-Key` and `RateLimit` header fields. The book says so each time, uses the

shape the draft describes, and explains what to watch for. Where no standard exists, as with pagination cursors, the book states a convention and the reasoning behind it, so you can adopt it or replace it knowingly.

Foundations

Before a single annotation or link: why standards matter, what REST actually asks of an API, the parts of HTTP that an API designer must know cold, and a workbench on which the rest of the book runs.



Why API standards matter

An API is a promise made to strangers. Standards are how you keep that promise when you are not in the room: when the network drops, when the server changes, and when the client was written by someone you will never meet.

TUESDAY, 9:31 A.M. • A COMMUTER TRAIN

Maya taps *Buy* on 100 shares. The train enters a tunnel. Her trading app waits thirty seconds, gives up, and, because its developers were careful people, tries again. The second request goes through. So, it turns out, did the first. The brokerage now holds two orders for 100 shares, and by the time the train leaves the tunnel both have filled.

That afternoon the app's developers read the API documentation looking for a way to retry safely. There isn't one. They find something else instead: errors arrive as `{"code": 417, "msg": "failed"}` for a dozen different reasons, so the app cannot tell “insufficient funds” from “market closed.” And a release note mentions that `/v1/orders/place` will move to `/v2/trade/submit` next quarter, which means another app release, another app-store review, and another month in which old versions of the app will break.

None of these is a bug in the usual sense. Each is a decision the API's designers did not know they were making.

Every one of those problems has a published, well-understood solution. A client that sends an `Idempotency-Key` header can retry an order as often as it likes, and the server will create it once. An error written as an RFC 9457 problem document carries a stable type identifier that a program can branch on. A client that follows links instead of building URLs does not care where the server moves a resource. This book is about adopting those solutions deliberately and consistently, so that the next Maya only buys once.

What a standard buys you

“Standard” here means two things. Some are **Internet standards**: RFCs published by the IETF, such as HTTP semantics (RFC 9110) or problem details (RFC 9457), that thousands of engineers have already argued over. Others are **house conventions**: choices such as “money is always a decimal string” that no RFC makes for you but that you make once, write down, and apply everywhere. Both kinds pay off the same way.

WITHOUT A STANDARD	WITH ONE	WHO BENEFITS
Every endpoint invents its own error shape	One problem-details format, one catalog of type URIs	Client developers, support engineers
Retries might duplicate orders	Idempotency keys make retries safe by construction	Customers, and the firm's risk team

WITHOUT A STANDARD	WITH ONE	WHO BENEFITS
Clients hard-code URL patterns	Clients follow links from one root URL	API teams, who can reorganize freely
Two devices silently overwrite an order change	ETags and <code>If-Match</code> turn a lost update into a clear 412	Customers with more than one screen
Each list pages differently	One cursor convention and a <code>next</code> link	Anyone who has written a sync job
Documentation drifts from behavior	An OpenAPI contract tested against real responses	Everyone who reads the documentation

There is a quieter benefit too. Standards shrink documentation. When your API returns `428 Precondition Required`, you do not need a page explaining what went wrong: RFC 6585 explains it, every HTTP library knows it, and an experienced developer recognizes it on sight. Every invented convention is something your documentation must teach and your users must remember.

The standards in this book

STANDARD	WHAT IT GOVERNS	CHAPTER
RFC 9110, 9111	HTTP semantics and caching: methods, status codes, conditional requests	3, 15, 17
RFC 8288, RFC 6570	Web linking and link relations; URI templates such as <code>{?status}</code>	9, 10
HAL, HAL-FORMS	JSON hypermedia: <code>_links</code> , <code>_embedded</code> and <code>_templates</code>	10–12
RFC 9457	Problem details: one machine-readable shape for every error	16
RFC 7396	JSON merge patch, for partial updates	15
Idempotency-Key DRAFT	Safe retries of non-idempotent requests	14
WHATWG HTML (SSE)	Server-sent events and <code>Last-Event-ID</code> replay	18
RFC 9745, RFC 8594	The <code>Deprecation</code> and <code>Sunset</code> header fields	19
RateLimit fields DRAFT	Advertising quotas before clients hit them	19
OAuth 2.1 DRAFT , RFC 7636	Authorization with scopes; PKCE for the authorization code grant	20
RFC 9728, RFC 8631	Protected resource metadata; <code>service-desc</code> and <code>service-doc</code> links	10, 20
OpenAPI 3.2	A machine-readable description of the whole API	21

The brokerage you will build

Standards are easiest to judge in a real system, so this book builds one. The sample brokerage has four parts, shown in Figure 1.1, and every one of them runs on your laptop.

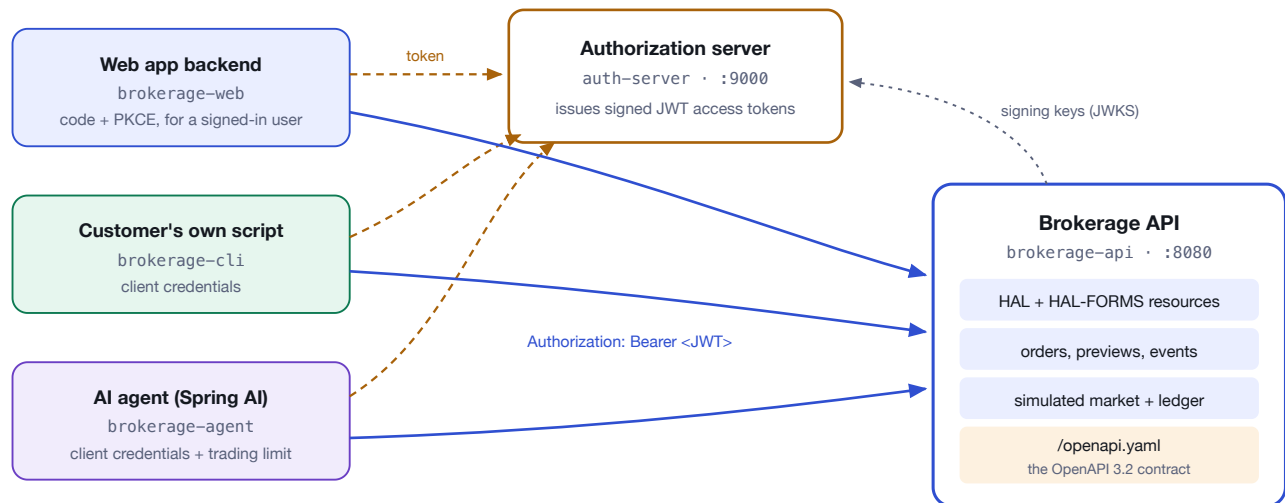


Figure 1.1 — The sample brokerage. Three kinds of client obtain tokens from the authorization server and call the API with them. The API checks each token's signature with the authorization server's published keys, and never sees a password.

The **API** serves accounts, balances, positions, transaction history, instruments, quotes, option chains and orders. Its market is a simulation: prices take a small random step every second and orders fill against them, so the order lifecycle is real enough to test cancellations and fills. The **authorization server** issues OAuth 2.1 tokens to three registered clients, each representing a different way customers reach a brokerage. The **hypermedia client** is a small Java library that navigates the API by following links. The **agent** is a conversational assistant, built with Spring AI and Claude, that uses the client as its tools and needs a person to confirm every trade.

Why a brokerage? Because it is unforgiving. A social network can show a stale like count. A brokerage that rounds a price, repeats an order or lets two edits collide costs somebody money. If a standard earns its keep here, it will earn its keep anywhere.

How to read this book

The chapters build on each other. Part I sets out REST and HTTP and gets the code running. Part II designs the resources. Part III adds hypermedia, the part of REST most APIs skip. Part IV makes trading safe: idempotency, concurrency control and errors. Part V covers what happens at scale: paging, streaming, versioning, quotas and security. Part VI ties it together with the OpenAPI contract, tests, a client, a comparison with a real-world RPC-style brokerage API, and an AI agent.

THE STANDARD

1. Where an Internet standard exists, use it, and name it in your documentation.
2. Where none exists, write the convention down once and apply it to every endpoint.
3. Treat consistency as a feature: the second endpoint should hold no surprises for anyone who has used the first.

REST in one chapter

REST is not a protocol, a format or a URL style. It is a short list of constraints, and the payoff for accepting them is an API that can change on the server without breaking the clients.

Roy Fielding described Representational State Transfer in his 2000 doctoral dissertation as the architectural style of the Web itself: the set of choices that let millions of independently written browsers and servers keep working together while both sides evolved. An API is RESTful to the degree that it adopts the same constraints. Each one costs you something, and each one buys you something specific.

The constraints

Client–server

The client owns the user interface and the server owns the data and rules. In the brokerage, buying power is computed only by the server. A client can display it, but never decides it.

Stateless

Every request carries everything the server needs to understand it: the access token, the account, the order. The server keeps no conversation state between requests. That is why the brokerage API disables HTTP sessions entirely, and why any instance behind a load balancer can answer any request.

Cacheable

Responses say whether, and for how long, they may be reused. A quote is good for one second, and only for the user who asked (`Cache-Control: max-age=1, private`). An account balance must never be stored by a shared cache at all.

Uniform interface

Every resource is manipulated the same way: identified by a URI, read and changed through representations, described by self-descriptive messages, and connected by links. This is the constraint that separates REST from “JSON over HTTP,” and it has four parts, covered below.

Layered system

A client cannot tell whether it is talking to the server itself, a cache, a gateway or a rate limiter. The brokerage adds its rate limiting and request IDs as layers that no endpoint has to know about.

Code on demand (optional)

A server may send code for the client to run, as web pages send JavaScript. APIs rarely use it, and this one does not.

The uniform interface, in four parts

- 1 **Resources are identified by URIs.** The order `ORD-100002` in account `ACC-1001` is `/accounts/ACC-1001/orders/ORD-100002`, and only that.
- 2 **Resources are manipulated through representations.** The client never touches the order itself. It receives a JSON representation, and it changes the order by sending one: a merge patch that says `{"limitPrice": "455.00"}`.

- 3 **Messages are self-descriptive.** A response says what it is. For example, `Content-Type: application/problem+json` announces an error document before the client reads a byte of it, and `ETag` says which version of the order this is.
- 4 **Hypermedia is the engine of application state.** Responses contain links to what the client can do next. An open order links to its cancellation; a filled order does not. This is HATEOAS, the subject of Part III.

The Richardson maturity model

Leonard Richardson proposed a simple way to see how far an API has gone toward REST, in four levels. Martin Fowler's write-up made it famous. Here is the brokerage's order placement at each level.

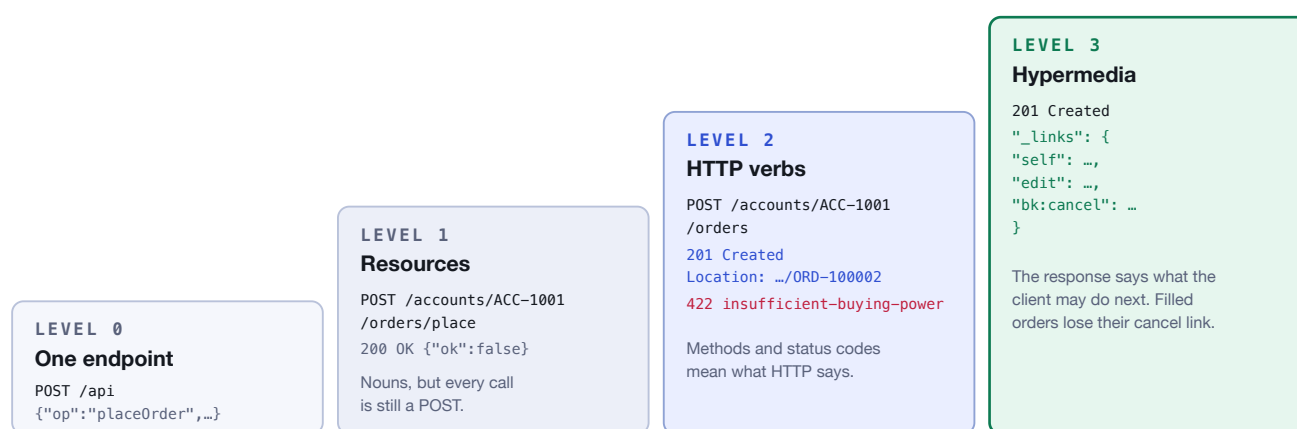


Figure 2.1 — The Richardson maturity model, applied to placing an order. Most APIs described as RESTful stop at level 2. This book's API is at level 3.

Level 0 uses HTTP only as a tunnel: one URL, one method, and the real request hidden in the body. **Level 1** gives things addresses but still treats HTTP as a pipe. **Level 2** uses methods and status codes as HTTP defines them, so caches, proxies and client libraries understand what is going on; `201 Created` with a `Location` header is a sentence every HTTP developer can read. **Level 3** adds hypermedia controls. The response to a new order links to the order, to a way to change it and to a way to cancel it, and the client learns what it may do from the response rather than from documentation.

Levels are not grades. A level 2 API can be excellent, and plenty of successful public APIs stop there. The argument of this book is narrower: the step to level 3 is cheaper than it looks with modern frameworks, and in a domain where actions depend on state and permissions, as trading does, it removes a whole class of client bugs. Chapter 9 makes that case in detail.

Resources and representations

A **resource** is anything important enough to have its own identifier: an account, an order, a quote, even an order *preview*. A **representation** is a document describing the resource's current state in some format. One resource can have several representations. The brokerage serves every order as plain HAL (`application/hal+json`) and, to clients that ask for it, as HAL-FORMS (`application/prs.hal-forms+json`), which adds descriptions of the forms a client may submit. The URI stays the same; the `Accept` header chooses the representation.

This distinction pays off in design. You do not have to expose your database tables. A *balances* resource can be computed on every request from the ledger and the market, because to the client a resource is whatever the representation says, however it came to be.

What REST is not

- **Not CRUD over tables.** The brokerage has no `DELETE` for orders. A canceled order still exists, so canceling is a state change modeled as a resource of its own: `POST .../orders/ORD-100002/cancellation`.
- **Not a URL style.** Pretty URLs are pleasant, but a REST client must not depend on them. Clients follow links; the server may reorganize its URLs at will.
- **Not JSON.** REST is format-neutral. What matters is that the format is described by a media type and can carry links.
- **Not the opposite of RPC.** Some operations are awkward as resources. Chapter 24 compares the brokerage with a real RPC-style trading API, and says when RPC is the better fit.

THE STANDARD

1. Every request is self-contained: no server-side sessions, and the token travels with every call.
2. Every important thing is a resource with a stable URI, including processes such as previews and cancellations.
3. Methods and status codes mean exactly what RFC 9110 says they mean.
4. Responses carry links to related resources and to the actions available right now.

HTTP, the parts that matter

HTTP already defines most of what an API needs: verbs with guarantees, a vocabulary of outcomes, and headers for caching, concurrency and negotiation. Using them as defined is most of the work of a good API.

Methods and their guarantees

RFC 9110 gives each method two properties that clients, proxies and retry libraries rely on. A **safe** method does not change anything the client is responsible for, so it can be prefetched, cached and retried freely. An **idempotent** method has the same effect whether it is sent once or ten times, so a client can retry it after a timeout.

METHOD	SAFE	IDEMPOTENT	HOW THE BROKERAGE USES IT
GET	yes	yes	Every read: accounts, positions, quotes, orders, the event stream.
HEAD	yes	yes	Like GET without the body; Spring answers it for every GET mapping.
POST	no	no	Placing an order, previewing one, requesting a cancellation. Made retry-safe with an <code>Idempotency-Key</code> (Chapter 14).
PATCH	no	no	Changing an open order, as a JSON merge patch guarded by <code>If-Match</code> (Chapter 15).
PUT	no	yes	Not used: no resource here is replaced wholesale by the client.
DELETE	no	yes	Not used: orders are never deleted, only canceled, and history is never erased.

Notice what drives the choice of method: the effect, not the URL. Canceling an order is not a `DELETE`, because the order survives with the status `CANCELLED` and stays readable forever. It is not a `PATCH` of `status`, because a client does not get to set a status; it can only ask for a cancellation, which the market may overtake with a fill. So it is a `POST` that creates a cancellation request, answered with `202 Accepted`.

Status codes: the vocabulary of outcomes

A client should be able to handle a response correctly from its status code alone, and then refine its behavior from the body. These are all the codes the brokerage returns.

STATUS	MEANING IN THIS API	CLIENT'S NEXT STEP
200 OK	The read or change succeeded; the body is the current state.	Use it.
201 Created	An order was placed. <code>Location</code> names it; <code>ETag</code> is its version.	Store the Location, not a URL you built.
202 Accepted	A cancellation was requested and is in progress.	Watch the order until it is final.

STATUS	MEANING IN THIS API	CLIENT'S NEXT STEP
304 Not Modified	Your cached copy (by <code>If-None-Match</code>) is still current.	Reuse the cached copy.
400 Bad Request	The request is malformed or invalid: a missing field, a bad cursor.	Fix the request; do not retry as is.
401 Unauthorized	No valid access token.	Obtain a token, then retry.
403 Forbidden	The token is valid but lacks a scope, or exceeds a delegated trading limit.	Ask for more authority, or a smaller order.
404 Not Found	No such resource <i>for this caller</i> , including other customers' accounts.	Stop; re-navigate from the root.
409 Conflict	The order's state forbids the action: it has already filled or been canceled.	Re-read the order.
412 Precondition Failed	The order changed since you read it (<code>If-Match</code> mismatch).	Re-read, reapply, resend.
422 Unprocessable Content	Well-formed but refused by a business rule: buying power, position, a reused key.	Show the reason to the user.
428 Precondition Required	This change needs an <code>If-Match</code> header.	Read the ETag first.
429 Too Many Requests	A rate limit was reached.	Wait for <code>Retry-After</code> , then retry.

Never return 200 for an error. A response like `200 OK {"success": false}` hides failures from every tool that relies on status codes: monitoring, caches, retry policies, API gateways and the developer's own debugger. If something went wrong, the status line must say so.

The headers this API speaks

REQUEST HEADER	PURPOSE	RESPONSE HEADER	PURPOSE
Authorization	The OAuth bearer token.	Location	Where the new order lives.
Accept	HAL or HAL-FORMS.	ETag	The version of this state.
Content-Type	JSON or merge patch.	Cache-Control	Whether and how long to cache.
Idempotency-Key	Makes an order retry-safe.	Idempotent-Replayed	This is a stored result.
If-Match	Change only this version.	Deprecation, Sunset	This version is going away.
If-None-Match	Send it only if changed.	RateLimit, -Policy	Your quota and what is left.
API-Version	Pin the API version.	Retry-After	How long to wait.
Last-Event-ID	Resume an event stream.	WWW-Authenticate	Why a token was refused.
X-Request-Id	Correlate your logs with ours.	X-Request-Id	Quote it to support.

Media types and content negotiation

The `Content-Type` of every message tells the receiver how to read it. The brokerage uses these types, each defined by a specification you can cite.

MEDIA TYPE	USED FOR
<code>application/hal+json</code>	Every resource representation (the default).
<code>application/prs.hal-forms+json</code>	The same representations plus <code>_templates</code> , on request.
<code>application/problem+json</code>	Every error (RFC 9457).
<code>application/json</code>	Request bodies for new orders and previews.
<code>application/merge-patch+json</code>	Changes to an order (RFC 7396).
<code>text/event-stream</code>	The live order event stream.
<code>application/openapi+yaml</code>	The API's own OpenAPI description.

A client picks among the representations the server offers with the `Accept` header. Ask for an account as plain HAL and you receive its links. Ask for `application/prs.hal-forms+json` and the same account also describes the order form you may submit, as Chapter 12 shows. If a client asks for a type the server cannot produce, the answer is `406 Not Acceptable`; if it sends a body in a type the endpoint does not read, it is `415 Unsupported Media Type`. Both come back as problem documents like every other error.

THE STANDARD

1. Choose methods by their guarantees: GET for reads, POST for non-idempotent actions, PATCH for partial changes.
2. Use the most specific status code that applies, and never report an error with a 2xx status.
3. Label every body with a precise media type; use registered types and headers before inventing new ones.

Your workbench

Twenty minutes from now you will have the whole brokerage running on your machine, a token in hand, and your first hypermedia response on the screen. Every later chapter assumes you can reproduce what it shows.

What you need

The code targets **Java 27**, the release that became generally available on September 15, 2026, so you need a JDK 27. Install one with your platform's package manager or a version manager such as SDKMAN, and check it:

terminal

```
$ java -version
openjdk version "27" 2026-09-15
```

You do not need Maven installed: the project includes the **Maven Wrapper**, a small script (`mvnw`, or `mvnw.cmd` on Windows) that downloads the exact Maven version the project was built with, 3.9.16, the first time you run it. You will also want `curl`.

If your IDE complains. IDE support for a new Java release often lags by a few weeks. If your editor reports errors that Maven does not, trust Maven, and update the IDE's Java tooling. The book's code compiles and its tests pass with the command-line build.

A tour of the project

The code lives in the `brokerage-apis` folder of the book's repository. It is a multi-module Maven build whose parent is Spring Boot 4.1.1.

brokerage-apis/

project layout

<code>pom.xml</code>	<i>parent: Spring Boot 4.1.1, Java 27, Spring AI 2.0.1</i>
<code>mvnw, mvnw.cmd, .mvn/</code>	<i>the Maven Wrapper</i>
<code>openapi/brokerage-api.yaml</code>	<i>the OpenAPI 3.2 contract (Chapter 21)</i>
<code>auth-server/</code>	<i>OAuth 2.1 authorization server, port 9000 (Chapter 20)</i>
<code>brokerage-api/</code>	<i>the REST API, port 8080</i>
<code>src/main/java/com/example/brokerage/api/</code>	
<code>RootController.java</code>	<i>the API root and the contract</i>
<code>platform/</code>	<i>errors, security, versions, paging, rate limits</i>
<code>market/</code>	<i>instruments, quotes, option chains, the simulator</i>
<code>accounts/</code>	<i>accounts, the ledger, balances, positions</i>
<code>orders/</code>	<i>orders, previews, idempotency, events</i>
<code>src/test/java/...</code>	<i>31 tests, including contract tests</i>
<code>brokerage-client/</code>	<i>the hypermedia client and its demo (Chapter 23)</i>
<code>brokerage-agent/</code>	<i>the Spring AI agent (Chapter 25)</i>

The API's packages follow its domain, not its layers. Everything about orders, from the controller to the idempotency store, lives in `orders`; cross-cutting machinery such as error handling lives in `platform`. When you want to know how cancellation works, there is one package to read.

Build and test

From the `brokerage-apis` folder, build everything and run every test:

terminal

```
$ ./mvnw verify
...
[INFO] Brokerage Authorization Server ..... SUCCESS
[INFO] Brokerage API ..... SUCCESS
[INFO] Brokerage Hypermedia Client ..... SUCCESS
[INFO] Brokerage AI Agent ..... SUCCESS
[INFO] BUILD SUCCESS
```

The first run downloads Maven and the dependencies, so it takes a few minutes; after that, a full build with tests takes well under a minute. Each module also produces an executable jar under its `target` folder.

Start the servers

Open two terminals in `brokerage-apis`. Start the authorization server first: the API fetches its configuration and signing keys from it to validate tokens.

terminal 1

```
$ java -jar auth-server/target/auth-server-1.0.0.jar
```

terminal 2

```
$ java -jar brokerage-api/target/brokerage-api-1.0.0.jar
```

The API seeds two demo customers with accounts, cash, positions and a year of transaction history, and starts a market simulation that ticks once a second. Everything is in memory: restart the API and you start fresh.

Get a token

Every call except the API root needs an OAuth access token. The quickest way to get one is the `brokerage-cli` client, which stands for a customer's own script. It uses the client credentials grant: it authenticates as itself and receives a token bound to its customer.

terminal

```
$ curl -s -u brokerage-cli:cli-secret http://localhost:9000/oauth2/token \
  -d grant_type=client_credentials \
  -d scope="accounts:read market-data:read orders:read orders:write"
```

HTTP

the token request, as the server saw it

```
POST /oauth2/token HTTP/1.1
Host: localhost:9000
Authorization: Basic YnJva2VyYWdlLWNSaTpjbGktd2VjcmV0
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=accounts:read+market-data:read+orders:read+orders:write
-----
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8
Cache-Control: no-cache, no-store, max-age=0, must-revalidate

{
  "access_token": "eyJraWQiOiI3MTE1NmVLYi02...",
  "scope": "orders:read market-data:read orders:write accounts:read",
  "token_type": "Bearer",
  "expires_in": 899
}
```

The access token is a signed JWT. Its payload carries the claims the API relies on. You can decode the middle section of any JWT with a base64 decoder; for this token it reads:

access token claims

decoded payload

```
{
  "sub": "brokerage-cli",
  "aud": "brokerage-api",
  "nbf": 1790678728,
  "scope": [
    "orders:read",
    "market-data:read",
    "orders:write",
    "accounts:read"
  ],
  "iss": "http://localhost:9000",
  "exp": 1790679628,
  "customer_id": "cust-1001",
  "iat": 1790678728,
  "jti": "3aa3db4f-3a7b-4d89-af7f-65e48a9d2ab2"
}
```

Three claims matter most. `aud` says the token was minted for the brokerage API and nothing else, and the API rejects tokens meant for other audiences. `scope` lists what the client may do. `customer_id` says whose accounts it may see. Chapter 20 explains where each comes from. For the rest of this book, assume a shell variable holds the token:

terminal

```
$ TOKEN=$(curl -s -u brokerage-cli:cli-secret http://localhost:9000/oauth2/token \
  -d grant_type=client_credentials \
  -d scope="accounts:read market-data:read orders:read orders:write" | jq -r .access_token)
```

Your first hypermedia response

The API root is the one URL a client needs to know, and the only one that needs no token:

```
GET / HTTP/1.1
Host: localhost:8080
API-Version: 2
Accept: application/hal+json

-----

HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/"
    },
    "bk:accounts": {
      "href": "http://localhost:8080/accounts"
    },
    "bk:instruments": {
      "href": "http://localhost:8080/instruments{?symbol,type}",
      "templated": true
    },
    "service-desc": {
      "href": "http://localhost:8080/openapi.yaml",
      "type": "application/openapi+yaml"
    },
    "service-doc": {
      "href": "https://docs.brokerage.example",
      "type": "text/html"
    },
    "bk:oauth-protected-resource": {
      "href": "http://localhost:8080/.well-known/oauth-protected-resource"
    },
    "curies": [
      {
        "href": "https://docs.brokerage.example/rels/{rel}",
        "name": "bk",
        "templated": true
      }
    ]
  },
  "description": "Accounts, positions, market data and trading, as hypermedia",
  "name": "Brokerage API"
}
```

Read the response as a menu. There is a link to the accounts, a templated link for searching instruments, a machine-readable description of the API (`service-desc`), human documentation (`service-doc`), and the protected resource metadata that tells a client where to get tokens. The `curies` entry explains the `bk:` prefix, and Chapter 10 covers it. From here, a client can reach every resource in the API without building a single URL. Follow the accounts link with your token:

terminal

```
$ curl -s -H "Authorization: Bearer $TOKEN" -H "API-Version: 2" http://localhost:8080/accounts
```

Take the guided tour

The client module includes a demo that exercises the API end to end through the hypermedia client of Chapter 23: it lists accounts, reads balances and positions, fetches a quote, previews and places a limit order below the market, cancels it, and waits for the cancellation to complete. With both servers running:

terminal

```
$ ./mvnw -q -pl brokerage-client compile exec:java \
-Dexec.mainClass=com.example.brokerage.client.Demo
```

demo output

recorded from a real run

Accounts
ACC-1001 Individual Margin MARGIN
ACC-1002 Roth IRA CASH

ACC-1001: cash 30982.11, buying power 52364.22, equity 67074.39

Positions
AAPL 53 @ 231.95 P&L 2584.73
MSFT 20 @ 514.09 P&L 2239.80
SPY 15 @ 667.59 P&L 1913.85
AMZN 12 @ 230.40 P&L 300.00
NVDA 4 @ 184.62 P&L -3.84

AAPL 231.95 bid 231.93 ask 231.97 (0.80%)

Preview: 5 AAPL at 208.75 costs 1043.75 (acceptable: true)
Placed ORD-100005: OPEN, cancelable: true
Cancel requested: PENDING_CANCEL
Now CANCELLED, cancelable: false

Not one URL in the demo was built by hand. Every request followed a link that the previous response provided, which is the promise of the next ten chapters.

When something goes wrong

SYMPTOM	CAUSE AND FIX
UnsupportedClassVersionError, class file version 71	You ran the jar with an older Java. Use JDK 27.
Port 8080 was already in use	Stop the other process, or add --server.port=8081 to the command.
Every call returns 401	The token expired (15 minutes for brokerage-cli), or the authorization server was restarted and its signing keys changed. Get a new token.
Calls with a fresh token still fail with 401	The API fetches the authorization server's configuration and signing keys on the first authenticated request. Make sure the authorization server is running on port 9000.

THE STANDARD

1. An API ships with a one-command build that runs every test, and a way to run it locally with realistic data.
2. Local development never needs production credentials: tokens come from a local authorization server.

Designing Resources

A brokerage modeled as resources: what gets an address, what the representations look like, and how money, time and identity are written down. Then the read side of the API, from accounts to option chains, one captured exchange at a time.



Modeling a brokerage

Resource design starts from what users need to see and do, not from database tables. The trick is to turn every verb that matters into a noun with an address.

From screens to resources

Picture the trading app your API will serve. A home screen lists the customer's accounts. An account screen shows cash, buying power, holdings and recent activity. A ticket lets the customer pick a stock, see an estimate and place an order. An orders screen shows what is working and what has filled, with a button to cancel. A quote screen shows the price; an options screen shows contracts by expiration. Each of those is information a client needs, and each becomes a resource.

WHAT THE USER SEES OR DOES	RESOURCE	KIND
My accounts	/accounts	Collection
One account	/accounts/{accountId}	Item
Cash and buying power	/accounts/{accountId}/balances	Computed view
Holdings	/accounts/{accountId}/positions	Collection
Activity statement	/accounts/{accountId}/transactions	Paged collection
What would this order cost?	/accounts/{accountId}/order-previews	Process (POST)
My orders; place one	/accounts/{accountId}/orders	Paged collection
One order; change it	/accounts/{accountId}/orders/{orderId}	Item
Cancel it	.../orders/{orderId}/cancellation	Process (POST)
Tell me when things happen	/accounts/{accountId}/order-events	Event stream
Find a stock	/instruments?symbol=AAPL	Filtered collection
Its price	/instruments/{instrumentId}/quote	Item (changes every second)
Its options	/instruments/{instrumentId}/option-chain	Filtered collection

Two design decisions hide in that table. First, **processes become resources**. “Preview an order” and “cancel an order” are verbs, but REST has no custom verbs, so each becomes a noun you `POST` to: an *order preview*, a *cancellation*. This is not a trick to satisfy a style guide. A cancellation really is a thing with its own life. It is requested, it may fail because the order filled first, and it completes later. Naming it makes that life visible.

Second, **account data hangs off the account, market data does not**. Balances, positions, transactions and orders belong to one account, and their URIs say so. Instruments and quotes are the same for everyone, so they live at the top level. The URI structure mirrors ownership, and ownership is what authorization checks.

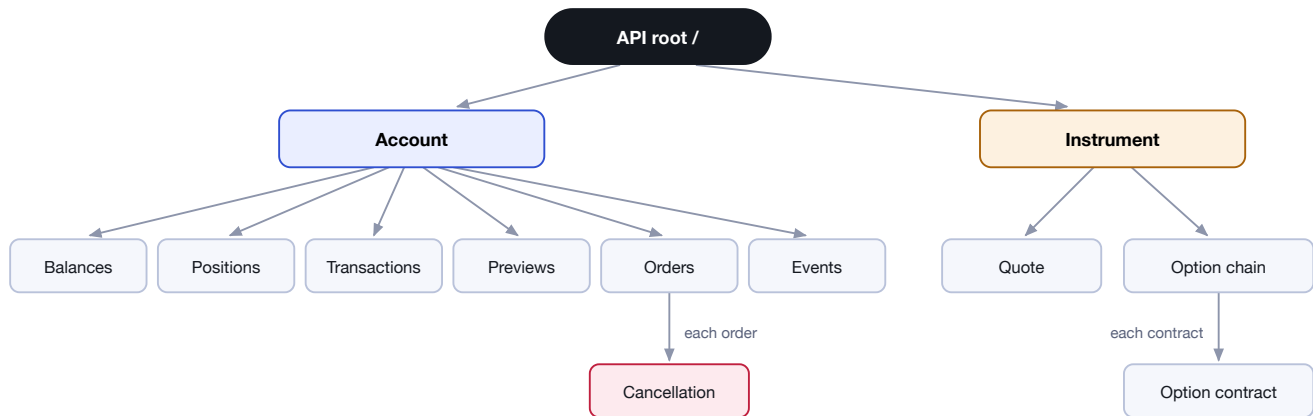


Figure 5.1 — The brokerage's resources as a tree of links from the root. Every arrow is a link a client can follow. Some links also cross the tree: a position links to its instrument and its quote, and an order to its instrument. The graph, not a list of URL patterns, is what a client needs to understand.

Rules for URIs

URIs are for the server to design and for clients to follow, but people still read them in logs and debuggers, so a few conventions keep them humane.

- **Nouns, plural for collections:** `/accounts`, `/orders`. Singular for a resource of which there is one per parent: `/balances` (one balance sheet), `/quote`, `/cancellation`.
- **Lowercase, with hyphens:** `/order-previews`, `/option-chain`. Never camel case, never underscores, never a file extension.
- **Nest only for ownership, and not deeply:** an order belongs to an account, so `/accounts/{accountId}/orders/{orderId}`. Nothing in this API is nested more deeply than that.
- **Filters go in the query string:** `?status=OPEN&symbol=AAPL`, not `/orders/open/aapl`. Query parameters are optional, combinable and self-describing.
- **No verbs**, except as the nouns of processes, like `cancellation`.
- **No version in the path:** `/v2/accounts` would give one account two URIs. Versions travel in a header instead (Chapter 19).

Identifiers are opaque

The brokerage's identifiers look like `ACC-1001`, `ORD-100002` and `OPT-AAPL-261016-C-240`. They are readable, which helps in logs and support tickets, but they are **opaque** to clients: a client stores an ID and sends it back, and never takes it apart. The option contract's ID happens to contain its expiration and strike, but a client that parses them out has written a bug that will fire the day the ID scheme changes. The contract's terms are in its representation, as fields, for exactly that reason.

```

/**
 * Something that can be traded: a stock, an ETF, or an option contract. The ID is
 * opaque: clients store it and send it back, but never take it apart.
 *
 * @param multiplier shares controlled by one unit: 1 for stocks, 100 for a US equity option
 * @param option      present only for option contracts
 */
@Relation(collectionRelation = "instruments", itemRelation = "instrument")
public record Instrument(String id, String symbol, String name, Type type, String exchange,
    String currency, boolean tradable, boolean fractionable, BigDecimal multiplier, OptionTerms option) {

    public enum Type { EQUITY, ETF, OPTION }

    public enum Right { CALL, PUT }

    /** The contract terms of an option. */
    public record OptionTerms(String underlyingId, String underlyingSymbol, Right right,
        BigDecimal strike, LocalDate expiration, String style) {
    }

    @JsonIgnore // otherwise Jackson would also write a boolean "option" property
    public boolean isOption() {
        return type == Type.OPTION;
    }
}

```

One resource, one URI; one URI, one resource

Each resource has exactly one canonical URI, the one in its `self` link. That rule is why the API has no `/v1` in its paths, and why an order has no second address such as `/orders/ORD-100002` outside its account. Two URIs for one thing split caches, confuse logs and double the authorization rules. When a client needs a shortcut, the server gives it a link, not a second address.

Resources are not tables. `balances` has no table behind it: it is computed on each request from the ledger and live prices. `order-previews` stores nothing at all. The API promises what a representation contains, not how it is stored, which leaves the server free to change its internals.

THE STANDARD

1. Model what clients need to see and do; give every such thing, including processes, a noun and a URI.
2. URIs are lowercase, hyphenated, plural for collections, nested only for ownership, with filters in the query string.
3. Identifiers are opaque strings; everything a client needs is a field in the representation.
4. Every resource has exactly one canonical URI: the `self` link.

Representations: money, time and identity

A representation is a contract written in JSON. The decisions that matter most are small ones: how to write a price, a timestamp, an absent value. Get them wrong once and every client inherits the mistake.

Money is a string

JSON has one number type, and most JSON parsers read it as a binary floating-point double. In binary floating point, `0.1 + 0.2` is `0.30000000000000004`, and a price such as `232.42` cannot be represented exactly at all. A server that writes prices as JSON numbers is therefore trusting every client's parser not to round them. In a brokerage, that trust is misplaced.

So every price, amount and quantity in this API is a **decimal string**: `"limitPrice": "450.00"`. A string survives every JSON parser intact, and the client decides how to read it, usually into `BigDecimal` in Java or a decimal library elsewhere. One bean configures this for every representation:

`api/platform/JsonConfig.java`

```
/**
 * Money, prices and quantities are written as decimal strings ("187.50"), never as
 * JSON numbers. Many JSON parsers read numbers into binary floating point, where
 * 0.1 + 0.2 is not 0.3; a string survives every parser intact.
 */
@Bean
JsonMapperBuilderCustomizer decimalsAsStrings() {
    return builder -> builder.withConfigOverride(BigDecimal.class,
        override -> override.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)));
}
```

The OpenAPI contract describes such strings with a `Decimal` schema that carries a pattern, so a generated client knows to expect `"187.50"` and not `187.5`. Counts that are naturally integers, such as a quote's traded volume, stay JSON numbers.

VALUE	EXAMPLE	SCALE
Cash, amounts, market values	"32420.55"	2 decimal places, rounded half-even
Prices, limits, stops	"232.42"	As entered, up to 4 decimal places
Average fill price	"232.0400"	4 decimal places: an average needs more precision than a price
Quantities	"10", "0.5"	Up to 6 decimal places, for fractional shares
Percentages	"29.14"	2 decimal places, as percent (not a fraction)
Counts	1250184	A JSON integer

Every amount also travels with an explicit `currency` (an ISO 4217 code such as `USD`) on the account, the instrument or the balance sheet it belongs to. A client never guesses a currency from a symbol, and never formats one into a value.

Time is UTC

Moments are written in ISO 8601 in UTC with a `Z`, as Java's `Instant` serializes them: `"createdAt": "2026-09-29T10:03:28.574985Z"`. Calendar dates that have no time of day, such as the date an account was opened or the date a trade settles, are plain ISO dates: `"settlesOn": "2026-09-30"`. The API never sends local times or time-zone offsets. A client converts to the user's zone for display, and only for display.

The distinction matters. A trade executed at 3:59 p.m. in New York is `19:59Z`; the date it *settles* is a business date defined by market rules, one business day later under the T+1 cycle US equity markets use:

`api/accounts/Ledger.java`

```
/** US equity and option trades settle one business day after the trade date (T+1). */
private static LocalDate settlementDate(Instant tradeTime) {
    LocalDate date = tradeTime.atZone(ZoneOffset.UTC).toLocalDate().plusDays(1);
    while (date.getDayOfWeek() == DayOfWeek.SATURDAY || date.getDayOfWeek() == DayOfWeek.SUNDAY) {
        date = date.plusDays(1);
    }
    return date;
}
```

Names and enumerations

Property names are **camelCase**, matching the Java records that produce them one for one. Enumerated values are **UPPER_SNAKE_CASE** strings such as `MARKET`, `STOP_LIMIT` and `PENDING_CANCEL`. Both choices are common; what matters is picking one and never mixing them. An order written in this style looks like this:

HTTP

a filled market order

```
HTTP/1.1 200 OK
ETag: "ORD-100003-v2"
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100003"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-AAPL"
    }
  },
  "id": "ORD-100003",
  "accountId": "ACC-1001",
  "instrumentId": "EQ-AAPL",
  "symbol": "AAPL",
  "instrumentType": "EQUITY",
```

```

"side": "BUY",
"type": "MARKET",
"timeInForce": "DAY",
"quantity": "3",
"status": "FILLED",
"filledQuantity": "3",
"averageFillPrice": "232.0400",
"fills": [
  {
    "id": "FIL-500001",
    "quantity": "3",
    "price": "232.04",
    "executedAt": "2026-09-29T10:45:32.234747Z"
  }
],
"createdAt": "2026-09-29T10:45:31.592779Z",
"updatedAt": "2026-09-29T10:45:32.234747Z",
"remainingQuantity": "0"
}

```

Absent is not null

A market order has no limit price, and an unfilled order has no average fill price. The API omits such fields rather than sending `null`, which you can see in the capture above. One line of Spring Boot configuration applies the rule everywhere:

```

brokerage-api/src/main/resources/application.yaml

jackson:
  # Absent optional fields are omitted rather than sent as null.
  default-property-inclusion: non_null

```

Omission keeps representations small and removes a question every client would otherwise ask: does `null` mean “none,” “unknown” or “not loaded”? Here, a missing field means the concept does not apply to this resource in its current state.

What a representation leaves out

A representation is a public contract, so it should show what clients need and nothing else. The `Order` record shows three techniques at once:

```

api/orders/Order.java

@Relation(collectionRelation = "orders", itemRelation = "order")
public record Order(String id, String accountId, String clientOrderId, String instrumentId, String symbol,
    Instrument.Type instrumentType, Side side, Type type, TimeInForce timeInForce, BigDecimal quantity,
    BigDecimal limitPrice, BigDecimal stopPrice, Status status, BigDecimal filledQuantity,
    BigDecimal averageFillPrice, List<Fill> fills, Instant createdAt, Instant updatedAt,
    @JsonIgnore boolean stopTriggered, @JsonIgnore long version) {

```

- **Internal state is ignored.** `stopTriggered` is bookkeeping for the matching engine, and `version` surfaces only as the `ETag` header (Chapter 15).

- **Derived values are computed.** `remainingQuantity` is not stored; a method marked `@JsonProperty` computes it, so it can never disagree with `quantity` and `filledQuantity`.
- **Sensitive values are masked or hidden.** An account's number is always sent as `"****4821"`, and its `customerId` is never sent at all: the caller already knows who they are.

`api/orders/Order.java`

```
@JsonProperty
public BigDecimal remainingQuantity() {
    return quantity.subtract(filledQuantity);
}
```

Signed amounts and self-contained entries

In a transaction, `amount` is signed from the account's point of view: money in is positive, money out is negative. A client can total a statement without a lookup table of transaction types. Each transaction also repeats the symbol it concerns, so a statement renders without one request per line.

The tolerant reader

The other half of a representation contract is on the client side. A client must ignore properties it does not recognize and treat unknown enumeration values gracefully, because adding a field or a status is how an API grows without breaking anyone. The book's Java client is configured that way from the start (Chapter 23). It even reads statuses as plain strings, so a status the server adds next year cannot crash it.

THE STANDARD

1. Prices, amounts and quantities are decimal strings with a documented scale; every amount has a currency.
2. Moments are ISO 8601 UTC with `Z`; dates without a time are ISO dates.
3. Properties are camelCase; enumerated values are UPPER_SNAKE_CASE strings.
4. Absent values are omitted, never `null`; internal fields never appear; sensitive values are masked.
5. Clients are tolerant readers: unknown properties and values never break them.

Accounts, balances, positions and history

The read side of a brokerage API is where most traffic goes. It should be fast, obvious and impossible to misuse, and above all it must never show one customer another customer's money.

The accounts collection

A customer can own several accounts: here, an individual margin account and a Roth IRA. `GET /accounts` returns them embedded in a HAL collection, each with its own links.

HTTP

```
GET /accounts HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_embedded": {
    "bk:accounts": [
      {
        "_links": {
          "self": {
            "href": "http://localhost:8080/accounts/ACC-1001"
          },
          "bk:balances": {
            "href": "http://localhost:8080/accounts/ACC-1001/balances"
          },
          "bk:positions": {
            "href": "http://localhost:8080/accounts/ACC-1001/positions"
          },
          "bk:transactions": {
            "href": "http://localhost:8080/accounts/ACC-1001/transactions{?type,from,to,cursor,limit}",
            "templated": true
          },
          "bk:orders": {
            "href": "http://localhost:8080/accounts/ACC-1001/orders{?status,symbol,cursor,limit}",
            "templated": true
          },
          "bk:order-previews": {
            "href": "http://localhost:8080/accounts/ACC-1001/order-previews"
          },
          "bk:order-events": {
            "href": "http://localhost:8080/accounts/ACC-1001/order-events"
          }
        },
        "id": "ACC-1001",
        "number": "****4821",
        "nickname": "Individual Margin",
        "type": "MARGIN",
        "registration": "INDIVIDUAL",
        "currency": "USD",
        "openedOn": "2021-03-15"
      }
    ]
  }
}
```

```

    },
    /* 1 more */
  ]
},
"_links": {
  "self": {
    "href": "http://localhost:8080/accounts"
  }
}
}
}

```

Each account's links are its table of contents: balances, positions, transactions, orders, order previews and the order event stream. A client that wants an account's positions follows `bk:positions` and never needs to know that the path ends in `/positions`. The `bk:transactions` and `bk:orders` links are **templated**: the parameters in braces are optional filters the client may fill in, following RFC 6570.

Whose account is it?

The access token carries a `customer_id` claim, and every account lookup goes through one method that compares it with the account's owner. The comparison is deliberately strict about what it reveals:

`api/accounts/Ledger.java`

```

/**
 * An account the customer owns. Someone else's account is reported as not found,
 * not as forbidden: a 403 would confirm that the account exists.
 */
public Account account(String customerId, String accountId) {
    Account account = accounts.get(accountId);
    if (account == null || !account.customerId().equals(customerId)) {
        throw ApiException.notFound("Account", accountId);
    }
    return account;
}

```

HTTP

another customer's account

```

GET /accounts/ACC-2001 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
-----
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "detail": "Account ACC-2001 was not found",
  "instance": "/accounts/ACC-2001",
  "status": 404,
  "title": "The resource does not exist",
  "type": "https://docs.brokerage.example/problems/not-found",
  "requestId": "d3ec8878-ab83-45f1-9987-415c81e67b13"
}

```

Answering `403 Forbidden` would be more literal, and it would also confirm that `ACC-2001` exists. A `404` tells an unauthorized caller nothing, which is what RFC 9110 suggests when a server does not want to reveal that a resource exists. Because every controller asks the ledger for the account first, there is no path to another customer's balances, positions or orders that skips this check.

Balances: a computed view

HTTP

```
GET /accounts/ACC-1001/balances HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/balances"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:positions": {
      "href": "http://localhost:8080/accounts/ACC-1001/positions"
    }
  },
  "accountId": "ACC-1001",
  "currency": "USD",
  "cash": "32420.55",
  "reservedForOrders": "4800.00",
  "buyingPower": "55241.10",
  "positionsMarketValue": "34662.28",
  "totalEquity": "67082.83",
  "unrealizedPnl": "7042.98",
  "asOf": "2026-09-29T10:45:28.885254Z"
}
```

No table stores these numbers. The `Portfolio` service derives them on every request from the ledger's cash, the open orders and live prices, so they can never drift from the books of record:

api/accounts/Portfolio.java

```
public Balances balances(Account account) {
    List<Position> positions = positions(account);
    BigDecimal cash = ledger.cash(account.id());
    BigDecimal reserved = reservations.reservedCash(account.id());
    BigDecimal marketValue = positions.stream().map(Position::marketValue)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
    BigDecimal unrealized = positions.stream().map(Position::unrealizedPnl)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
    return new Balances(account.id(), account.currency(), cash, reserved,
        Holdings.buyingPower(account.type(), cash, reserved), marketValue, cash.add(marketValue),
        unrealized, clock.instant());
}
```

`reservedForOrders` is the cash promised to open buy orders: the account's resting order for 10 MSFT at 480.00 holds back 4,800.00. `buyingPower` is what a new purchase may use, and it depends on the account type:

`api/accounts/Holdings.java`

```
/**
 * Buying power: unreserved cash in a cash account, twice that in a margin account
 * (the 50% initial margin of Regulation T). A deliberate simplification.
 */
public static BigDecimal buyingPower(Account.Type type, BigDecimal cash, BigDecimal reserved) {
    BigDecimal free = cash.subtract(reserved).max(BigDecimal.ZERO);
    return type == Account.Type.MARGIN ? free.multiply(BigDecimal.TWO) : free;
}
```

In the capture, the margin account has 32,420.55 in cash, 4,800.00 of it reserved, so its buying power is twice the remaining 27,620.55, or 55,241.10. The same numbers reappear when an order is previewed (Chapter 13), because both go through the same rules.

Positions: holdings at market value

HTTP

```
GET /accounts/ACC-1001/positions HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQ0iI3MTE1NmVL...
API-Version: 2
```

HTTP/1.1 200 OK

Content-Type: application/hal+json

```
{
  "_embedded": {
    "bk:positions": [
      {
        "_links": {
          "self": {
            "href": "http://localhost:8080/accounts/ACC-1001/positions/EQ-AAPL"
          },
          "bk:instrument": {
            "href": "http://localhost:8080/instruments/EQ-AAPL"
          },
          "bk:quote": {
            "href": "http://localhost:8080/instruments/EQ-AAPL/quote"
          }
        },
        "instrumentId": "EQ-AAPL",
        "symbol": "AAPL",
        "instrumentType": "EQUITY",
        "quantity": "50",
        "averageCost": "180.25",
        "costBasis": "9012.50",
        "lastPrice": "232.78",
        "marketValue": "11639.00",
        "unrealizedPnl": "2626.50",
        "unrealizedPnlPercent": "29.14"
      },
      {
        "_links": {
```

```

    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/positions/EQ-MSFT"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "bk:quote": {
      "href": "http://localhost:8080/instruments/EQ-MSFT/quote"
    }
  },
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "quantity": "20",
  "averageCost": "402.10",
  "costBasis": "8042.00",
  "lastPrice": "513.04",
  "marketValue": "10260.80",
  "unrealizedPnl": "2218.80",
  "unrealizedPnlPercent": "27.59"
},
/* 2 more */
]
},
"_links": {
  "self": {
    "href": "http://localhost:8080/accounts/ACC-1001/positions"
  },
  "bk:account": {
    "href": "http://localhost:8080/accounts/ACC-1001"
  }
}
}
}

```

Each position carries its quantity, average cost and cost basis, and its value at the latest price, with unrealized profit and loss. It links to its instrument and to its quote, so a client can refresh a single price without re-reading the whole portfolio. Every position also has its own URI, `/accounts/ACC-1001/positions/EQ-AAPL`, for clients that track one holding.

Transactions: the statement

History is the part of an account that grows forever, so it is **paged** and **filterable**. It is sorted newest first, and it can be filtered by type and by an inclusive date range in UTC. Ask for two at a time and the response ends with a `next` link:

```

HTTP

GET /accounts/ACC-1001/transactions?limit=2 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVL...
API-Version: 2
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_embedded": {
    "bk:transactions": [
      {

```

```

    "id": "TXN-1008",
    "type": "TRADE",
    "description": "Bought 12 AMZN @ 205.40",
    "amount": "-2464.80",
    "instrumentId": "EQ-AMZN",
    "symbol": "AMZN",
    "quantity": "12",
    "price": "205.40",
    "occurredAt": "2026-09-20T09:42:26.876265Z",
    "settlesOn": "2026-09-21"
  },
  {
    "id": "TXN-1007",
    "type": "DIVIDEND",
    "description": "Cash dividend SPY",
    "amount": "26.85",
    "instrumentId": "ETF-SPY",
    "symbol": "SPY",
    "occurredAt": "2026-09-09T08:25:26.876260Z",
    "settlesOn": "2026-09-09"
  }
]
},
"_links": {
  "self": {
    "href": "http://localhost:8080/accounts/ACC-1001/transactions?limit=2"
  },
  "bk:account": {
    "href": "http://localhost:8080/accounts/ACC-1001"
  },
  "next": {
    "href": "http://localhost:8080/accounts/ACC-1001/transactions?
cursor=MTc4ODk0MjMyNjg3NjpwUE4tMTAwNw&limit=2"
  }
}
}
}

```

The client does not compute the next page; it follows `next`, which carries an opaque cursor. When there is no `next` link, there are no more pages. Chapter 17 explains why a cursor beats `page=2`. Filters combine with paging:

HTTP

```

GET /accounts/ACC-1001/transactions?type=DIVIDEND HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQ0iI3MTE1NmVl...
API-Version: 2

```

The controller method shows how little code this takes. Note the two `expand()` calls: the `self` and `next` links are concrete URLs, not templates with the unused filters left open.

api/accounts/AccountController.java

```

/**
 * Transaction history, newest first, filterable by type and by an inclusive date range
 * (UTC), paged with an opaque cursor.
 */
@GetMapping("/accounts/{accountId}/transactions")
public CollectionModel<EntityModel<Transaction>> transactions(@PathVariable String accountId,
    @RequestParam(required = false) Transaction.Type type,

```

```

    @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate from,
    @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate to,
    @RequestParam(required = false) String cursor, @RequestParam(required = false) Integer limit,
    @AuthenticationPrincipal Jwt jwt) {
    Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
    List<Transaction> matching = ledger.transactions(account.id()).stream()
        .filter(t -> type == null || t.type() == type)
        .filter(t -> from == null || !utcDate(t).isBefore(from))
        .filter(t -> to == null || !utcDate(t).isAfter(to))
        .toList();

    CursorPage<Transaction> page = CursorPage.of(matching, Transaction::occurredAt, Transaction::id,
        cursor, limit);

    CollectionModel<EntityModel<Transaction>> model = CollectionModel.of(
        page.items().stream().map(EntityModel::of).toList(),
        linkTo(methodOn(AccountController.class)
            .transactions(accountId, type, from, to, cursor, limit, null))
            .withSelfRel().expand(),
        linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
    if (page.hasNext()) {
        // expand(): "next" is a concrete URL, not a template with the unused filters left open
        model.add(linkTo(methodOn(AccountController.class)
            .transactions(accountId, type, from, to, page.nextCursor(), limit, null))
            .withRel("next").expand());
    }
    return model;
}

```

Where are the transaction links? Transactions are embedded without links of their own. They are immutable entries in a statement, never fetched or changed individually, so the API gives them no separate URI. If a future version needs one, adding a `self` link will be a backward-compatible change.

THE STANDARD

1. Every read checks ownership through one code path; a resource the caller may not see does not exist (404).
2. Derived views such as balances are computed from the books of record, never stored beside them.
3. Collections that grow without bound are paged, with a `next` link when more exists.
4. Collection items embed their own links, so clients never assemble URIs from IDs.

Market data: instruments, quotes and option chains

Market data is shared by every customer and changes every second. That combination shapes its design: global URIs, aggressive but private caching, and links that let a client walk from a stock to its options without knowing a single date.

Finding an instrument

The API root's `bk:instruments` link is a template, `/instruments{?symbol,type}`. A client expands it with the symbol the user typed:

HTTP

```
GET /instruments?symbol=AAPL HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2

-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_embedded": {
    "bk:instruments": [
      {
        "_links": {
          "self": {
            "href": "http://localhost:8080/instruments/EQ-AAPL"
          },
          "bk:quote": {
            "href": "http://localhost:8080/instruments/EQ-AAPL/quote"
          },
          "bk:option-chain": {
            "href": "http://localhost:8080/instruments/EQ-AAPL/option-chain{?expiration}",
            "templated": true
          }
        },
        "id": "EQ-AAPL",
        "symbol": "AAPL",
        "name": "Apple Inc.",
        "type": "EQUITY",
        "exchange": "XNAS",
        "currency": "USD",
        "tradable": true,
        "fractionable": true,
        "multiplier": "1"
      }
    ]
  },
  "_links": {
    "self": {
      "href": "http://localhost:8080/instruments?symbol=AAPL"
    }
  }
}
```

```
}  
}  
}
```

Three properties answer the questions a trading screen asks before it offers a ticket. `tradable` says whether orders are accepted at all, and a halted security is not tradable. `fractionable` says whether fractional quantities are allowed. `multiplier` says how many shares one unit controls: 1 for a stock, 100 for a US equity option. The client reads these instead of hard-coding them per symbol.

Quotes: fresh for one second

HTTP

```
GET /instruments/EQ-AAPL/quote HTTP/1.1  
Host: localhost:8080  
Authorization: Bearer eyJraWQiOiIzMTE1NmVl...  
API-Version: 2  
  
-----  
HTTP/1.1 200 OK  
ETag: "q3"  
Content-Type: application/hal+json  
Cache-Control: max-age=1, private  
  
{  
  "_links": {  
    "self": {  
      "href": "http://localhost:8080/instruments/EQ-AAPL/quote"  
    },  
    "bk:instrument": {  
      "href": "http://localhost:8080/instruments/EQ-AAPL"  
    }  
  },  
  "instrumentId": "EQ-AAPL",  
  "symbol": "AAPL",  
  "last": "232.78",  
  "bid": {  
    "price": "232.76",  
    "size": "500"  
  },  
  "ask": {  
    "price": "232.80",  
    "size": "500"  
  },  
  "change": "2.68",  
  "changePercent": "1.16",  
  "volume": 1250184,  
  "asOf": "2026-09-29T10:45:28.216609Z"  
}
```

Two response headers make polling cheap. `Cache-Control: max-age=1, private` tells the client, and any cache, that the quote is good for one second and must never be stored by a shared cache, because market data is licensed per user. The `ETag` identifies this particular tick of the market. A client that polls sends the tag back, and while the price has not moved it receives a bodiless `304`:

HTTP

```
GET /instruments/EQ-AAPL/quote HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
If-None-Match: "q3"
```

```
-----
HTTP/1.1 304 Not Modified
ETag: "q3"
```

api/market/InstrumentController.java

```
/**
 * Quotes change every second, so they may be cached only briefly and only privately
 * (market data is licensed per user). The ETag lets a client poll cheaply: an unchanged
 * quote costs a 304 with no body.
 */
private <T> ResponseEntity<EntityModel<T>> quoteResponse(Instrument instrument, MarketData.Tick tick,
    T quote, WebRequest request) {
    String etag = "\"q\" + tick.sequence() + \"\"";
    if (request.checkNotModified(etag)) {
        return null;
    }
    EntityModel<T> model = EntityModel.of(quote,
        linkTo(methodOn(InstrumentController.class).quote(instrument.id(), null)).withSelfRel(),
        linkTo(methodOn(InstrumentController.class).instrument(instrument.id()))
            .withRel("instrument"));
    return ResponseEntity.ok()
        .cacheControl(CacheControl.maxAge(1, TimeUnit.SECONDS).cachePrivate())
        .eTag(etag)
        .body(model);
}
```

`checkNotModified` compares the request's `If-None-Match` with the current tag and, when they match, completes the response as a 304. Returning `null` afterward is Spring's convention for “the response is already written.” Chapter 19 returns to quotes, because they are the one representation that changed shape between API versions.

Option chains: links instead of dates

An option chain is the list of option contracts for one underlying stock and one expiration. A naive design would make the client compute expiration dates, which are usually the third Friday of a month and are sometimes moved for holidays. Instead the chain offers every available expiration as a link, all with the same relation and each distinguished by its `name`:

HTTP

```
GET /instruments/EQ-AAPL/option-chain HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
```

```
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
```

```

    "_embedded": {
      "bk:instruments": [
        {
          "_links": {
            "self": {
              "href": "http://localhost:8080/instruments/OPT-AAPL-261016-C-220"
            },
            "bk:quote": {
              "href": "http://localhost:8080/instruments/OPT-AAPL-261016-C-220/quote"
            },
            "bk:underlying": {
              "href": "http://localhost:8080/instruments/EQ-AAPL"
            }
          },
          "id": "OPT-AAPL-261016-C-220",
          "symbol": "AAPL261016C00220000",
          "name": "AAPL 2026-10-16 $220 CALL",
          "type": "OPTION",
          "exchange": "OPRA",
          "currency": "USD",
          "tradable": true,
          "fractionable": false,
          "multiplier": "100",
          "option": {
            "underlyingId": "EQ-AAPL",
            "underlyingSymbol": "AAPL",
            "right": "CALL",
            "strike": "220",
            "expiration": "2026-10-16",
            "style": "AMERICAN"
          }
        },
        /* 5 more */
      ]
    },
    "_links": {
      "self": {
        "href": "http://localhost:8080/instruments/EQ-AAPL/option-chain?expiration=2026-10-16"
      },
      "bk:underlying": {
        "href": "http://localhost:8080/instruments/EQ-AAPL"
      },
      "bk:expiration": [
        {
          "href": "http://localhost:8080/instruments/EQ-AAPL/option-chain?expiration=2026-10-16",
          "name": "2026-10-16"
        },
        {
          "href": "http://localhost:8080/instruments/EQ-AAPL/option-chain?expiration=2026-11-20",
          "name": "2026-11-20"
        }
      ]
    }
  }
}

```

HAL allows a relation to hold an array of links, and `name` is the standard way to tell them apart. A client renders one tab per `bk:expiration` link and follows the one the user picks. When a new expiration is listed, the client shows it with no code change.

api/market/InstrumentController.java

```
/**
 * The option chain for one expiration. Every available expiration is offered as a
 * link with the same relation, told apart by its "name": the client picks one instead
 * of guessing dates.
 */
@GetMapping("/instruments/{instrumentId}/option-chain")
public CollectionModel<EntityModel<Instrument>> optionChain(@PathVariable String instrumentId,
    @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    LocalDate expiration) {
    Instrument underlying = market.instrument(instrumentId);
    List<LocalDate> expirations = market.expirations(underlying.id());
    LocalDate chosen = expiration != null ? expiration
        : expirations.isEmpty() ? null : expirations.getFirst();
    List<EntityModel<Instrument>> contracts = chosen == null ? List.of()
        : market.optionChain(underlying.id(), chosen).stream().map(this::toModel).toList();

    CollectionModel<EntityModel<Instrument>> chain = CollectionModel.of(contracts,
        linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), chosen))
            .withSelfRel(),
        linkTo(methodOn(InstrumentController.class).instrument(underlying.id()))
            .withRel("underlying"));
    for (LocalDate date : expirations) {
        chain.add(linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), date))
            .withRel("expiration").withName(date.toString()));
    }
    return chain;
}
```

The option contract

Each contract is an instrument in its own right, with an `option` object for its terms and a link back to its underlying stock:

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/instruments/OPT-AAPL-261016-C-240"
    },
    "bk:quote": {
      "href": "http://localhost:8080/instruments/OPT-AAPL-261016-C-240/quote"
    },
    "bk:underlying": {
      "href": "http://localhost:8080/instruments/EQ-AAPL"
    }
  },
  "id": "OPT-AAPL-261016-C-240",
  "symbol": "AAPL261016C00240000",
  "name": "AAPL 2026-10-16 $240 CALL",
  "type": "OPTION",
  "exchange": "OPRA",
  "currency": "USD",
  "tradable": true,
  "fractionable": false,
  "multiplier": "100",
```

```

"option": {
  "underlyingId": "EQ-AAPL",
  "underlyingSymbol": "AAPL",
  "right": "CALL",
  "strike": "240",
  "expiration": "2026-10-16",
  "style": "AMERICAN"
}
}

```

Its `symbol` follows the symbology of the Options Clearing Corporation (OCC): the root symbol, the expiration as `yyMMdd`, `C` or `P`, and the strike times 1,000 in eight digits. `AAPL261016C00240000` is the AAPL call expiring October 16, 2026 at a strike of 240. Industry-standard symbols matter because they are what traders, statements and other systems use; the API's own `id` remains the key for every call.

`api/market/MarketData.java`

```

private void option(Instrument underlying, LocalDate expiration, Right right, int strike) {
    String callOrPut = right == Right.CALL ? "C" : "P";
    String id = "OPT-%s-%s-%s-%d".formatted(underlying.symbol(), expiration.format(OPTION_DATE),
        callOrPut, strike);
    // The OCC option symbol: root, expiration (yyMMdd), C or P, and the strike times 1,000.
    String symbol = "%s%s%s%08d".formatted(underlying.symbol(), expiration.format(OPTION_DATE),
        callOrPut, strike * 1000);
    String name = "%s %s $%d %s".formatted(underlying.symbol(), expiration, strike, right);
    OptionTerms terms = new OptionTerms(underlying.id(), underlying.symbol(), right,
        BigDecimal.valueOf(strike), expiration, "AMERICAN");
    instruments.put(id, new Instrument(id, symbol, name, Type.OPTION, "OPRA", "USD",
        true, false, BigDecimal.valueOf(100), terms));
    priceOption(instruments.get(id));
}

```

A market that never stops

Prices in the sample application come from a simulation. Once a second, each stock takes a small random step, option prices follow their underlying, and the matching engine checks open orders against the new prices. Tests switch the simulation off and drive the matching engine by hand, so every price in a test is known in advance.

```

/**
 * Drives the simulated market: every tick, prices move and open orders are matched.
 * Tests switch this off (brokerage.simulator.enabled=false) and call match() directly,
 * so that every price is known in advance.
 */
@Component
@ConditionalOnBooleanProperty(name = "brokerage.simulator.enabled", matchIfMissing = true)
class TradingSession {

    private final MarketData market;
    private final OrderService orders;

    TradingSession(MarketData market, OrderService orders) {
        this.market = market;
        this.orders = orders;
    }

    @Scheduled(fixedDelayString = "${brokerage.simulator.tick-millis:1000}")
    void tick() {
        market.move();
        orders.match();
    }
}

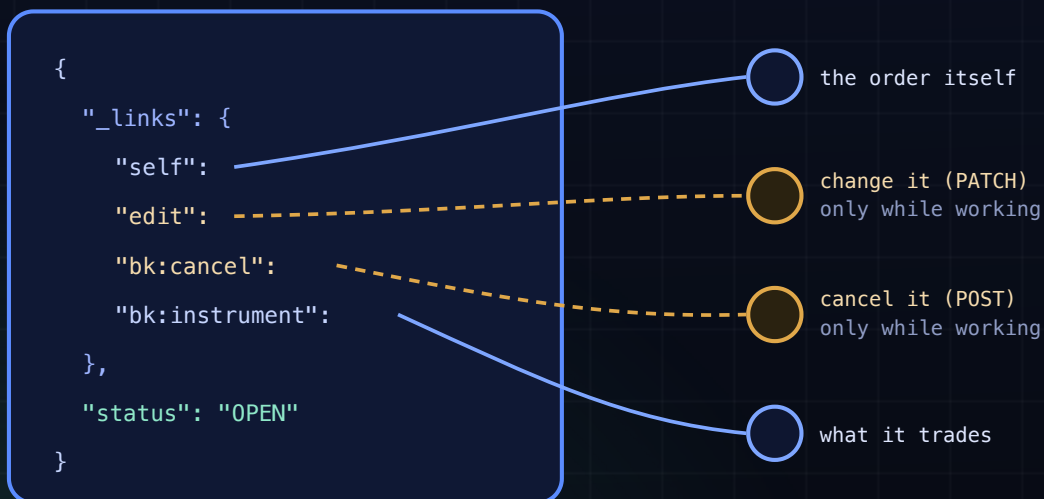
```

THE STANDARD

1. Shared reference data lives at the top level; per-customer data lives under the customer's account.
2. Volatile resources carry an `ETag` and an explicit, short `Cache-Control`; licensed data is always `private`.
3. When the valid choices change over time, as expirations do, offer them as named links instead of making clients compute them.
4. Use industry-standard identifiers, such as OCC option symbols, as properties beside the API's own IDs.

Hypermedia

The constraint most APIs skip, and the one that pays for itself soonest in a trading system: responses that tell the client what it may do next. HAL for links, HAL-FORMS for forms, and Spring HATEOAS to produce both without ceremony.



HATEOAS: links as the engine of application state

A client that knows only one URL, and learns everything else from the responses it receives, can never offer an action the server would refuse, and never breaks when the server reorganizes. That is the whole promise of hypermedia.

What the acronym means

Hypermedia as the engine of application state is the last of REST's uniform-interface constraints.

Application state is where the client is in its work: looking at an account, holding an open order, waiting for a cancellation to complete. The constraint says the client moves from one state to the next by following links and submitting forms that the server put in its responses, just as you move through a website by clicking links rather than typing URLs.

Consider what a trading screen needs to know about an order: whether a *Cancel* button should appear, and whether a *Change* button should. Without hypermedia, the client reimplements the server's rules: “show Cancel if the status is `OPEN` or `PARTIALLY_FILLED`, and if the user's token has `orders:write`, and if...”. Every copy of that rule is a place for it to go stale. With hypermedia, the rule lives in one place, and the client asks one question: *is there a `bk:cancel` link?*

The order's state machine, as links

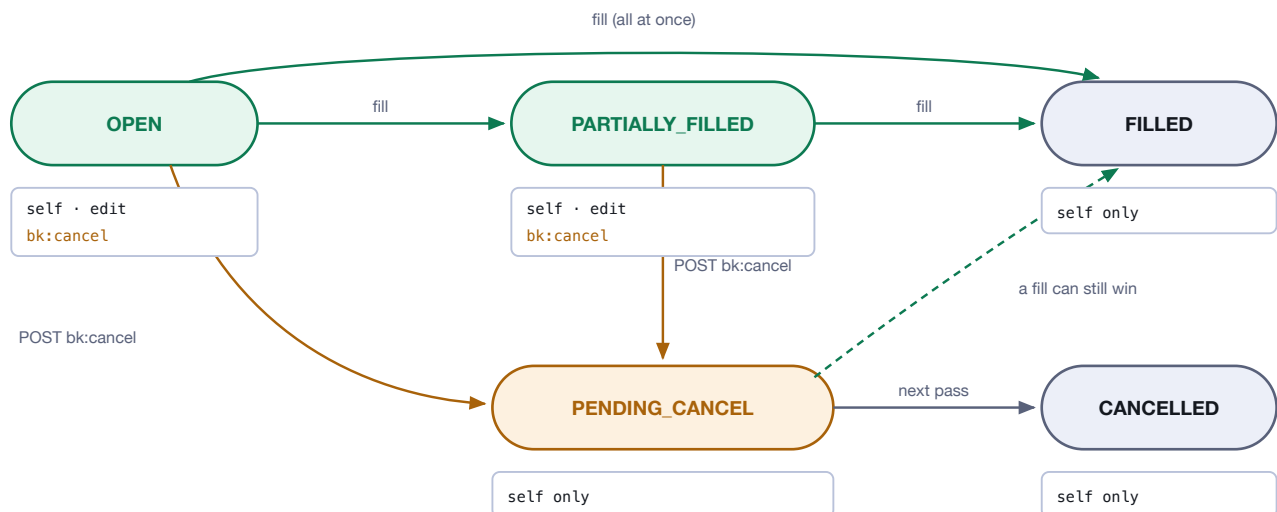


Figure 9.1 — The order lifecycle. The boxes under each state list the links an order carries in that state (every order also links to its account and instrument). Working orders offer `edit` and `bk:cancel`; nothing else does.

Here is the same order before and after it is canceled. While it is working, it offers two transitions:

HTTP

a working order

```
HTTP/1.1 200 OK
ETag: "ORD-100002-v1"
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "450.00",
  "status": "OPEN",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-29T10:45:28.998175Z",
  "updatedAt": "2026-09-29T10:45:28.998175Z",
  "remainingQuantity": "10"
}
```

After the cancellation completes, the transitions are gone. Nothing else about the representation's shape changed:

HTTP

the same order, canceled

```
HTTP/1.1 200 OK
ETag: "ORD-100002-v4"
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    }
  }
}
```

```

    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "455.00",
  "status": "CANCELLED",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-29T10:45:28.998175Z",
  "updatedAt": "2026-09-29T10:45:29.220209Z",
  "remainingQuantity": "10"
}

```

The code that makes this decision is short, and it is the only place in the system where the decision is made:

`api/orders/OrderModelAssembler.java`

```

public EntityModel<Order> toModel(Order order, Caller caller) {
    String accountId = order.accountId();
    String orderId = order.id();
    boolean changeable = order.status().isWorking() && caller.canTrade();

    Link self = linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
        .withSelfRel();
    if (changeable) {
        self = self.andAffordance(afford(methodOn(OrderController.class)
            .replaceOrder(accountId, orderId, null, null, null)));
    }
    EntityModel<Order> model = EntityModel.of(order, self,
        linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"),
        linkTo(methodOn(InstrumentController.class).instrument(order.instrumentId()))
            .withRel("instrument"));

    if (changeable) {
        model.add(linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
            .withRel(IanaLinkRelations.EDIT));
        model.add(linkTo(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))
            .withRel("cancel")
            .andAffordance(
                afford(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))));
    }
    return model;
}

```

Two inputs decide the links: the order's state (`isWorking()`) and the caller's permissions (`canTrade()`), true when the token has `orders:write`. A read-only token sees a working order with no `edit` and no `bk:cancel`, because for that caller those actions do not exist.

Links reflect permissions too

The same principle applies one level up. An account read with a token that has only `accounts:read` and `market-data:read` does not mention orders at all:

HTTP

a read-only token's view of the account

```
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:balances": {
      "href": "http://localhost:8080/accounts/ACC-1001/balances"
    },
    "bk:positions": {
      "href": "http://localhost:8080/accounts/ACC-1001/positions"
    },
    "bk:transactions": {
      "href": "http://localhost:8080/accounts/ACC-1001/transactions{?type,from,to,cursor,limit}",
      "templated": true
    }
  },
  "id": "ACC-1001",
  "number": "****4821",
  "nickname": "Individual Margin",
  "type": "MARGIN",
  "registration": "INDIVIDUAL",
  "currency": "USD",
  "openedOn": "2021-03-15"
}
```

Compare it with the full view in Chapter 7, which adds `bk:orders`, `bk:order-previews` and `bk:order-events`. The client for a read-only dashboard never has to be told that it cannot trade. There is simply nothing to click.

The client's side of the bargain

FOUR RULES FOR HYPERMEDIA CLIENTS

1. **Bookmark only the root.** Store the API root URL in configuration; discover everything else.
2. **Follow relations, not paths.** Find a link by its relation (`bk:orders`), never by what its URL looks like.
3. **Offer an action only if its link is present.** The absence of `bk:cancel` means “you cannot cancel this, now.”
4. **Store resource URIs, not IDs to rebuild them from.** Keep the `self` link of anything you will need again.

Links are guidance, not security

A missing link tells an honest client what it cannot do. It does not stop a dishonest or out-of-date one. The server still enforces every rule on every request: a client that posts to a cancellation URL it saved earlier, after the order filled, gets a clear refusal.

```
POST /accounts/ACC-1001/orders/ORD-100002/cancellation HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2

-----
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "detail": "Order ORD-100002 is CANCELLED and cannot be canceled",
  "instance": "/accounts/ACC-1001/orders/ORD-100002/cancellation",
  "status": 409,
  "title": "The order can no longer be canceled",
  "type": "https://docs.brokerage.example/problems/order-not-cancellable",
  "requestId": "bc30e11d-f95a-4c52-b7c5-201d136c9789"
}
```

This is also why hypermedia and caching get along. A client may act on a representation that was accurate a second ago. The server's answer, `409 Conflict` with a problem type naming the reason, tells it to re-read the order, whose links will now be accurate again.

Objections, answered

“Clients hard-code URLs anyway.”

Some will. They lose the benefits, and nothing breaks for anyone else. The clients in this book, a Java library and an AI agent, follow links, and Chapter 23 shows that doing so costs no more code than building URLs.

“Links make responses bigger.”

A few hundred bytes of links compress extremely well, and they save the round trips a client would otherwise spend asking what it may do.

“We will have to document the relations.”

Yes, instead of documenting URL patterns, parameters and state rules. A relation is a stable name with a meaning; a URL pattern is an implementation detail with a lifetime.

“It is slower to discover everything from the root.”

Discovery results can be cached like any other representation. A client that already holds an account's representation follows its links directly.

THE STANDARD

1. Every representation links to itself and to its related resources.
2. State transitions appear as links only when the resource's state and the caller's permissions allow them.
3. The decision about which links a resource carries is made in exactly one place in the server.
4. The server enforces every rule regardless of links; a stale link earns a precise problem, such as 409.

HAL and link relations

Hypermedia needs a format that says, in a way every client can parse, “here is a link, and here is what it means.” HAL does that with two reserved properties and a naming scheme borrowed from the Web itself.

HAL in two properties

The JSON Hypertext Application Language, HAL, was proposed by Mike Kelly and is described in the Internet-Draft *draft-kelly-json-hal*. Its media type is `application/hal+json`. It adds exactly two reserved properties to an ordinary JSON object:

- `_links`, an object whose keys are **link relations** and whose values are link objects, or arrays of them when a relation has several targets.
- `_embedded`, an object whose keys are link relations and whose values are resources, or arrays of resources, included in the response to save round trips.

Everything else is the resource's own state. A HAL client needs no schema to find the links: they are always in `_links`, keyed by what they mean.

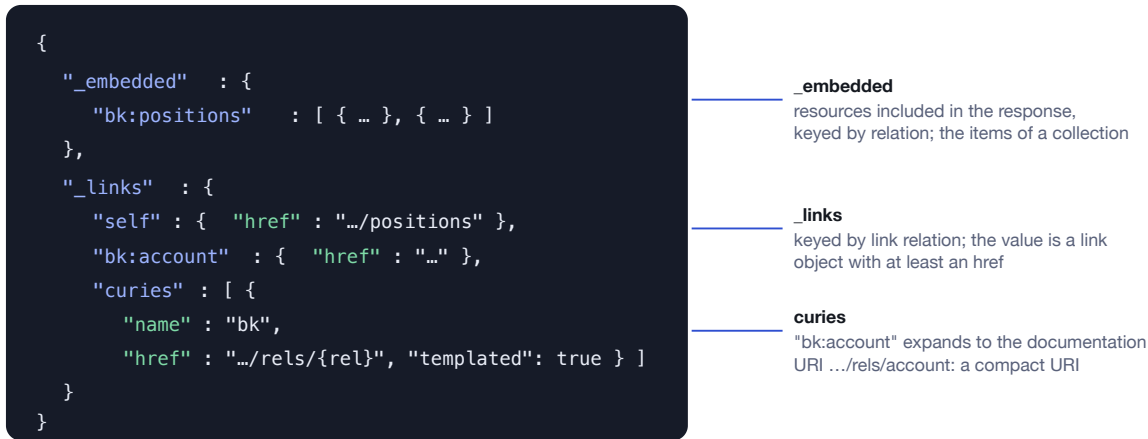


Figure 10.1 — The anatomy of a HAL document: embedded resources, links keyed by relation, and the CURIE that shortens the API's own relation names.

Link objects

A link object must have an `href`. HAL defines a handful of optional properties, and the brokerage uses four of them:

PROPERTY	MEANING	EXAMPLE IN THIS API
href	The target URI, or a URI template	<code>.../orders/ORD-100002</code>

PROPERTY	MEANING	EXAMPLE IN THIS API
templated	true when href is an RFC 6570 template	.../orders{?status,symbol,cursor,limit}
type	The media type the target will return	application/openapi+yaml
name	Distinguishes links that share a relation	"2026-10-16" on bk:expiration
title, profile, deprecation, hreflang	Labels, profiles, retirement notices, languages	not used

Link relations: the vocabulary

A link relation names the *meaning* of a link: not “a URL,” but “the next page,” “the way to edit this,” “this order’s instrument.” RFC 8288, *Web Linking*, defines two kinds. **Registered** relations are short names in the IANA Link Relations registry, such as `self` and `next`, with meanings every client already knows. **Extension** relations are URIs that the API owner defines and documents, which keeps them globally unique.

Extension URIs are long, so HAL provides **CURIEs** (compact URIs): a named template declared once in `curies`. In this API, `bk:orders` expands to `https://docs.brokerage.example/rels/orders`, a page that documents the relation. Here is the API root again, where the declaration appears:

HTTP

the curies entry declares the bk: prefix

```

HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/"
    },
    "bk:accounts": {
      "href": "http://localhost:8080/accounts"
    },
    "bk:instruments": {
      "href": "http://localhost:8080/instruments{?symbol,type}",
      "templated": true
    },
    "service-desc": {
      "href": "http://localhost:8080/openapi.yaml",
      "type": "application/openapi+yaml"
    },
    "service-doc": {
      "href": "https://docs.brokerage.example",
      "type": "text/html"
    },
    "bk:oauth-protected-resource": {
      "href": "http://localhost:8080/.well-known/oauth-protected-resource"
    },
    "curies": [
      {
        "href": "https://docs.brokerage.example/rels/{rel}",
        "name": "bk",
        "templated": true
      }
    ]
  }
}

```

```

    }
  ],
  "description": "Accounts, positions, market data and trading, as hypermedia",
  "name": "Brokerage API"
}

```

RELATION	KIND	MEANING
self	IANA	This resource's canonical URI.
next	IANA	The next page of a paged collection.
edit	IANA (RFC 5023)	Where to send changes to this resource (a merge patch).
service-desc	IANA (RFC 8631)	A machine-readable description of the API: the OpenAPI document.
service-doc	IANA (RFC 8631)	Human-readable documentation.
bk:accounts, bk:account	extension	The customer's accounts; the account a resource belongs to.
bk:balances, bk:positions, bk:transactions	extension	An account's balance sheet, holdings and statement.
bk:orders, bk:order-previews, bk:order-events	extension	An account's orders; where to preview one; its event stream.
bk:cancel	extension	POST here to cancel this order. Present only while it can be canceled.
bk:place-order	extension	On an acceptable preview: where to POST the order.
bk:instruments, bk:instrument	extension	Instrument search; the instrument an order or position concerns.
bk:quote, bk:option-chain	extension	An instrument's latest quote; its option contracts.
bk:expiration, bk:underlying	extension	One expiration's chain (named by date); an option's underlying stock.
bk:oauth-protected-resource	extension	The RFC 9728 metadata: where to get tokens for this API.

Prefer registered relations. Before inventing a relation, check the IANA registry. `edit`, `next`, `prev`, `first`, `last`, `collection`, `item`, `related` and `describedby` cover a great deal. The brokerage invents relations only for domain concepts, such as a balance sheet or a cancellation, that no registry could know about.

Templated links

Some links need input from the client. RFC 6570 URI templates express that without giving away the URL structure. The template `.../orders{?status,symbol,cursor,limit}` uses *form-style query expansion*: each variable the client supplies becomes a query parameter, and each one it omits disappears.

VARIABLES SUPPLIED	EXPANDED URI
none	.../accounts/ACC-1001/orders
symbol=AAPL	.../orders?symbol=AAPL
status=[OPEN, PARTIALLY_FILLED]	.../orders?status=OPEN,PARTIALLY_FILLED

The client expands templates with a library (Spring's `UriTemplate` in Java) and never concatenates strings. The server, for its part, sends templates only for things a client chooses. Links a client should follow as-is, such as `next`, are always expanded.

Why HAL?

HAL is not the only hypermedia format. It is the simplest one that does the job, and it has the best support in the Spring ecosystem.

FORMAT	STRENGTHS	TRADE-OFFS
HAL	Minimal; plain JSON plus <code>_links</code> / <code>_embedded</code> ; wide library support	Links only; actions need HAL-FORMS
HAL-FORMS	Adds <code>_templates</code> : method, fields, validation	A separate media type clients opt into
Siren	Entities, links and <i>actions</i> in one format	A stricter envelope around every resource
JSON:API	Complete conventions for relationships, sparse fieldsets, paging	Opinionated document structure; weaker affordances
JSON-LD + Hydra	Linked-data semantics; rich vocabularies	Steep learning curve for most teams

THE STANDARD

1. Representations are `application/hal+json`; links live in `_links`, included resources in `_embedded`.
2. Use IANA-registered relations where one fits; otherwise define extension relations as URIs, shortened with one CURIE.
3. Every extension relation's URI resolves to documentation of its meaning.
4. Templated links use RFC 6570; links meant to be followed as-is are always expanded.

Hypermedia with Spring HATEOAS

Spring HATEOAS turns hypermedia from hand-written JSON into a handful of types and one link builder. Links are built from controller methods, so they cannot drift from the routes they point to.

Enabling the formats

Spring Boot's HATEOAS starter configures HAL by default. The brokerage turns on HAL-FORMS as well, declares its CURIE, and describes a few HAL-FORMS details, all in one configuration class:

api/platform/HypermediaConfig.java

```
/**
 * Hypermedia formats and link relations.
 *
 * <p>HAL (application/hal+json) is the default. Clients that ask for HAL-FORMS
 * (application/prs.hal-forms+json) also receive "_templates": machine-readable
 * descriptions of the state transitions available right now, such as placing,
 * changing or canceling an order.
 */
@Configuration(proxyBeanMethods = false)
@EnableHypermediaSupport(type = { HypermediaType.HAL, HypermediaType.HAL_FORMS })
public class HypermediaConfig {

    /** Documentation for every custom link relation lives at this URI template. */
    public static final String RELS = "https://docs.brokerage.example/rels/{rel}";
```

api/platform/HypermediaConfig.java

```
/**
 * Relations that are not registered with IANA (like "orders") are custom, and HAL
 * requires custom relations to be URIs. A CURIE keeps them short on the wire:
 * "bk:orders" expands to https://docs.brokerage.example/rels/orders.
 */
@Bean
CurieProvider curieProvider() {
    return new DefaultCurieProvider("bk", UriTemplate.of(RELS)) {
        @Override
        public HalLinkRelation getNamespacedRelFor(LinkRelation relation) {
            // Registered by RFC 8631, but missing from Spring HATEOAS's list of IANA relations.
            return RFC_8631.contains(relation.value()) ? HalLinkRelation.uncuried(relation.value())
                : super.getNamespacedRelFor(relation);
        }
    };
}
```

The override is a small workaround worth knowing about. RFC 8631 registered `service-desc` and `service-doc` with IANA, but Spring HATEOAS's built-in list of IANA relations does not include them, so without the override they would come out as `bk:service-desc`: a custom relation, which is wrong. Registered relations must never be prefixed.

Four model types

TYPE	REPRESENTS	IN THIS API
<code>RepresentationModel</code>	A resource defined by its links, with its own properties	The API root
<code>EntityModel<T></code>	A domain object plus links	An account, an order, a quote
<code>CollectionModel<T></code>	Items (embedded) plus links	Accounts, positions, orders
<code>PagedModel<T></code>	A page of items with page metadata	Not used: pages here are cursors

Domain records stay free of web concerns: `Order` knows nothing about links. The `@Relation` annotation on a record names it inside `_embedded` (`orders`), which the CURIE provider turns into `bk:orders`, and nothing else about the record needs to change.

Links from methods, not strings

`WebMvcLinkBuilder` builds links by *calling* the controller method they point to, on a proxy that records the call instead of executing it:

the pattern

```
linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null)).withSelfRel()
```

The builder reads the method's mapping, fills in the path variables from the arguments, adds any query parameters, and produces an absolute URI based on the current request. Arguments that are not part of the URI, such as the request body or the `Jwt`, are passed as `null`. If someone renames the route, every link to it follows; if someone deletes the method, the links stop compiling. Absolute URIs are derived from the incoming request, so behind a proxy you set `server.forward-headers-strategy` to have Spring honor the `X-Forwarded-*` or `Forwarded` headers.

The root: a resource that is all links

`api/RootController.java`

```
/** The root's own properties, beside its links. */
public static class ApiRoot extends RepresentationModel<ApiRoot> {

    public final String name = "Brokerage API";
    public final String description = "Accounts, positions, market data and trading, as hypermedia";
}
```

api/RootController.java

```
@GetMapping("/")
public ApiRoot root() {
    String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
    return new ApiRoot()
        .add(linkTo(methodOn(RootController.class).root()).withSelfRel())
        .add(linkTo(methodOn(AccountController.class).accounts(null)).withRel("accounts"))
        .add(linkTo(methodOn(InstrumentController.class).search(null, null)).withRel("instruments"))
        // RFC 8631: the machine-readable description of this API ...
        .add(Link.of(base + "/openapi.yaml", "service-desc").withType("application/openapi+yaml"))
        // ... and its human-readable documentation.
        .add(Link.of("https://docs.brokerage.example", "service-doc").withType("text/html"))
        // RFC 9728: where clients learn which authorization server issues tokens for this API.
        .add(Link.of(base + "/.well-known/oauth-protected-resource", "oauth-protected-resource"));
}
```

Three links point outside the controllers: the OpenAPI document, the documentation site and the protected resource metadata. For those, `Link.of(href, rel)` takes the URI directly.

Assemblers: one place for the link rules

When a resource appears in several responses, its links should be built in one place. The brokerage uses an assembler class for orders (you saw it in Chapter 9) and a method on the controller for accounts, whose links depend on the caller's scopes:

api/accounts/AccountController.java

```
/**
 * An account's links depend on what the caller may do: a token without orders:read is
 * not shown the orders at all, and only a token with orders:write is offered the
 * "placeOrder" affordance. Clients discover their capabilities instead of hard-coding them.
 */
EntityModel<Account> toModel(Account account, Caller caller) {
    String id = account.id();
    EntityModel<Account> model = EntityModel.of(account,
        linkTo(methodOn(AccountController.class).account(id, null)).withSelfRel(),
        linkTo(methodOn(AccountController.class).balances(id, null)).withRel("balances"),
        linkTo(methodOn(AccountController.class).positions(id, null)).withRel("positions"),
        linkTo(methodOn(AccountController.class).transactions(id, null, null, null, null, null))
            .withRel("transactions"));
    if (caller.canReadOrders()) {
        Link orders = linkTo(methodOn(OrderController.class).orders(id, null, null, null, null, null))
            .withRel("orders");
        if (caller.canTrade()) {
            orders = orders.andAffordance(
                afford(methodOn(OrderController.class).placeOrder(id, null, null, null)));
        }
        model.add(orders,
            linkTo(methodOn(OrderController.class).previewOrder(id, null, null))
                .withRel("order-previews"),
            linkTo(methodOn(OrderEventController.class).events(id, null, null))
                .withRel("order-events"));
    }
    return model;
}
```

`andAffordance(afford(...))` attaches a description of the `placeOrder` operation to the `bk:orders` link. Plain HAL ignores it; HAL-FORMS renders it as a template, as the next chapter shows.

Collections and embedded items

A collection is a `CollectionModel` of item models, each carrying its own links:

`api/accounts/AccountController.java`

```
@GetMapping("/accounts")
public CollectionModel<EntityModel<Account>> accounts(@AuthenticationPrincipal Jwt jwt) {
    Caller caller = Caller.of(jwt);
    List<EntityModel<Account>> accounts = ledger.accounts(caller.customerId()).stream()
        .map(account -> toModel(account, caller))
        .toList();
    return CollectionModel.of(accounts,
        linkTo(methodOn(AccountController.class).accounts(null)).withSelfRel());
}
```

Testing the links

Links are part of the contract, so they are tested like any other output. This test checks that links follow the token's scopes, and that a HAL-FORMS client sees the order template:

`test: api/HypermediaTest.java`

```
@Test
void linksReflectWhatTheTokenMayDo() {
    // Read-only: accounts are visible, but orders are not even mentioned.
    MvcTestResult readOnly = mvc.get().uri("/accounts/ACC-1001").with(alice(ACCOUNTS)).exchange();
    assertThat(readOnly).hasStatusOk();
    assertThat(readOnly.bodyJson()
        .hasPath("$. _links['bk:positions']")
        .doesNotHavePath("$. _links['bk:orders']"));

    // A full trading token, asking for HAL-FORMS: the placeOrder template appears.
    MvcTestResult trading = mvc.get().uri("/accounts/ACC-1001").with(alice(ALL))
        .accept(MediaTypes.HAL_FORMS_JSON).exchange();
    assertThat(trading).hasStatusOk();
    assertThat(trading.bodyJson().hasPath("$. _links['bk:orders']"));
    assertThat(trading.bodyJson().extractingPath("$. _templates.placeOrder.method").isEqualTo("POST"));
}
```

THE STANDARD

1. Build links from controller methods, never from string templates, so routes and links cannot diverge.
2. Keep domain types free of links; wrap them in models at the edge.
3. Put each resource's link rules in one assembler or method, and test them.
4. Registered relations are never prefixed with a CURIE.

Affordances with HAL-FORMS

A link tells a client where it can go. An affordance tells it what it can do there: which method to use, what to send, and which values are allowed. HAL-FORMS adds exactly that to HAL.

From links to forms

A plain HAL link to `bk:cancel` says an action exists. It does not say whether to use `POST` or `DELETE`, or what body to send. Documentation fills that gap for people, but a generic client, or an AI agent, benefits from a machine-readable answer. **HAL-FORMS**, a specification by Mike Amundsen with the media type `application/prs.hal-forms+json`, supplies it: an extra `_templates` object whose entries describe the requests a client may send.

Clients opt in with the `Accept` header. Ask for the working order that way:

HTTP

the same order, with templates

```
GET /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
Accept: application/prs.hal-forms+json

-----
HTTP/1.1 200 OK
ETag: "ORD-100002-v1"
Content-Type: application/prs.hal-forms+json

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "450.00",
```

```

"status": "OPEN",
"filledQuantity": "0",
"fills": [],
"createdAt": "2026-09-29T10:45:28.998175Z",
"updatedAt": "2026-09-29T10:45:28.998175Z",
"remainingQuantity": "10",
"_templates": {
  "replaceOrder": {
    "method": "PATCH",
    "contentType": "application/merge-patch+json",
    "properties": [
      {
        "name": "limitPrice",
        "regex": "^[0-9]+(\\.[0-9]+)?$",
        "type": "text"
      },
      {
        "name": "quantity",
        "regex": "^[0-9]+(\\.[0-9]+)?$",
        "type": "text"
      },
      {
        "name": "stopPrice",
        "regex": "^[0-9]+(\\.[0-9]+)?$",
        "type": "text"
      },
      {
        "name": "timeInForce"
      }
    ]
  },
  "cancelOrder": {
    "method": "POST",
    "properties": [],
    "target": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
  }
}
}

```

Reading a template

MEMBER	MEANING
method	The HTTP method to use.
contentType	The body's media type, <code>application/json</code> when absent. Here, a merge patch.
target	Where to send it. When absent, the resource's own <code>self</code> URL.
properties	The fields: <code>name</code> , <code>required</code> , <code>regex</code> , <code>type</code> , <code>min / max</code> , <code>options</code> .

`replaceOrder` has no `target`, so it applies to the order itself, and its `contentType` is `application/merge-patch+json`, taken from what the controller method consumes. Its properties are the fields of a change, with the pattern every decimal must match. `cancelOrder` is a `POST` with no fields. A filled or canceled order has no templates at all, for the same reason it has no links.

The account carries two templates for a trading token. `placeOrder` comes from the explicit affordance in Chapter 11. `previewOrder` appears on its own: Spring HATEOAS records an affordance for the handler method behind every link built with `linkTo(methodOn(...))`, and HAL-FORMS renders a template for each one that is not a `GET`. The `bk:order-previews` link points at a `POST` handler, so it brings its template along. Here is one of the `placeOrder` template's properties, from the same capture:

HTTP

from GET /accounts/ACC-1001 (HAL-FORMS)

```
{
  "name": "side",
  "options": { "inline": [ "BUY", "SELL" ] },
  "required": true
}
```

Where the template comes from

Spring HATEOAS derives each property from the request type of the handler method, the class bound with `@RequestBody`. Bean Validation annotations become constraints: `@NotNull` and `@NotBlank` become `required`, `@Size` becomes `min` and `max`, and `@Pattern` becomes `regex`.

api/orders/OrderRequest.java

```
/**
 * The body of POST .../orders and POST .../order-previews. Field-level rules are declared
 * here and checked by Bean Validation; rules that involve several fields or the account
 * (a limit order needs a limit price, a sale needs shares to sell) live in OrderService.
 *
 * <p>This is a JavaBean rather than a record on purpose: Spring HATEOAS derives HAL-FORMS
 * templates from bean properties, and marks a property without a setter as read-only.
 */
public class OrderRequest {

    @NotBlank
    private String instrumentId;
    @NotNull
    private Order.Side side;
    @NotNull
    private Order.Type type;
    private Order.TimeInForce timeInForce;
    @InputType("text") @NotNull @Positive @Digits(integer = 9, fraction = 6)
    private BigDecimal quantity;
    @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
    private BigDecimal limitPrice;
    @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
    private BigDecimal stopPrice;
    /** The caller's own reference for the order, echoed back unchanged. */
    @InputType("text") @Size(max = 64) @Pattern(regexp = "[A-Za-z0-9._-]*")
    private String clientOrderId;
```

The remaining details come from configuration: the list of allowed values for each enumeration, and a pattern that describes a decimal-as-string:

```

/** Offer the allowed values of enum fields inside HAL-FORMS templates. */
@Bean
HalFormsConfiguration halFormsConfiguration() {
    return new HalFormsConfiguration()
        // Decimals travel as strings: describe them as text that must look like a number.
        .withPattern(BigDecimal.class, "[0-9]+(\\.[0-9]+)?$")
        .withOptions(OrderRequest.class, "side",
            metadata -> HalFormsOptions.inline(Order.Side.values()))
        .withOptions(OrderRequest.class, "type",
            metadata -> HalFormsOptions.inline(Order.Type.values()))
        .withOptions(OrderRequest.class, "timeInForce",
            metadata -> HalFormsOptions.inline(Order.TimeInForce.values()));
}

```

A JavaBean, on purpose. `OrderRequest` is a class with getters and setters, not a record. Spring HATEOAS decides whether a property is writable by looking for a setter, and a record has none, so every field of a record-based request would be rendered `readOnly`, a template no client could fill in. The class Javadoc says so, to save the next developer from “simplifying” it. Similarly, `@InputType("text")` stops Spring HATEOAS from describing a `BigDecimal` as a number input, which would contradict the API's decimals-as-strings rule.

What clients do with templates

- **Generic user interfaces** render a form per template: a dropdown for `options`, a validated text box for `regex`, a required marker. When the server adds a field, the form grows without a client release.
- **Client-side validation** uses the same rules the server enforces, stated once.
- **Agents and tools** learn the shape of an action at run time. A tool-using model can be given a template instead of a hand-written parameter list.

Templates describe the *shape* of a request, not every business rule. A limit order needs a limit price, and a sale needs shares to sell; those rules involve several fields or the account's state, so the server checks them and reports violations as problems (Chapter 16). The template narrows the space of wrong requests; it does not replace validation.

THE STANDARD

1. Offer `application/prs.hal-forms+json` beside HAL; describe every state-changing action as a template.
2. Derive templates from the request types and their validation annotations, so forms and validation cannot disagree.
3. A template appears only when its action is available, exactly like its link.
4. Templates describe shape; the server still validates everything.

Trading Safely

Orders move money, so every weakness in an API becomes somebody's loss. This part places orders through layered validation, makes retries harmless with idempotency keys, stops concurrent edits from colliding, and turns every failure into a precise, documented problem.

`problem+json`
every failure

`202 Accepted`
async cancel



validation
400 · 422

idempotency
retry safely

If-Match
412 · 428

Placing orders

Placing an order is the most consequential request a brokerage API accepts. The design goal is simple to state: a well-formed, permitted order is accepted in one round trip, and anything else is refused with a reason a program can act on.

The order request

An order names an instrument, a side, a type, a quantity and, depending on the type, prices. The four order types behave as they do at any US broker:

TYPE	EXECUTES	PRICES REQUIRED
MARKET	Immediately, at the best available price: the ask when buying, the bid when selling.	none
LIMIT	Only at the limit price or better.	<code>limitPrice</code>
STOP	As a market order once the last price reaches the stop.	<code>stopPrice</code>
STOP_LIMIT	As a limit order once the last price reaches the stop.	both

`timeInForce` is `DAY` (the default) or `GTC`, good till canceled. `clientOrderId` is the caller's own reference, up to 64 safe characters, which the API stores and echoes but never interprets. Options use the same request: the instrument decides the multiplier and the commission. Fractional quantities are accepted only for market orders on fractionable stocks.

Preview first

A good trading screen shows the estimated cost before the customer commits. That estimate is a resource of its own, an *order preview*, computed by exactly the rules that placing the order will apply, and stored nowhere:

HTTP

```
POST /accounts/ACC-1001/order-previews HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
Content-Type: application/json
```

```
{
  "instrumentId": "EQ-MSFT",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "10",
  "limitPrice": "450.00",
  "timeInForce": "GTC",
  "clientOrderId": "rebalance-2026-09-28"
}
```

```

HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:place-order": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders"
    }
  },
  "accountId": "ACC-1001",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "10",
  "estimatedPrice": "450.00",
  "estimatedNotional": "4500.00",
  "estimatedCommission": "0.00",
  "estimatedTotal": "4500.00",
  "buyingPower": "55241.10",
  "acceptable": true,
  "issues": []
}

```

An acceptable preview carries a `bk:place-order` link: the next step, offered only when it would succeed and only to a caller allowed to trade. A preview that breaks a business rule is still a *successful* response. The preview itself worked; the answer is “no,” with each issue carrying the same problem type that placing the order would return:

```

HTTP

HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_links": {
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    }
  },
  "accountId": "ACC-1001",
  "instrumentId": "EQ-AMZN",
  "symbol": "AMZN",
  "side": "SELL",
  "type": "MARKET",
  "quantity": "40",
  "estimatedPrice": "228.47",
  "estimatedNotional": "9138.80",
  "estimatedCommission": "0.00",
  "estimatedTotal": "9138.80",
  "buyingPower": "55241.10",
  "acceptable": false,
  "issues": [
    {
      "type": "https://docs.brokerage.example/problems/insufficient-position",
      "detail": "Only 12 AMZN are available to sell; short selling is not supported"
    }
  ]
}

```

```
}  
]  
}
```

For an option, the same request shape yields option economics: two contracts at 3.10 control 200 shares, so the notional is 620.00, plus a commission of 0.65 per contract.

HTTP

POST /accounts/ACC-1001/order-previews HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Content-Type: application/json

```
{  
  "instrumentId": "OPT-AAPL-261016-C-240",  
  "side": "BUY",  
  "type": "LIMIT",  
  "quantity": "2",  
  "limitPrice": "3.10",  
  "timeInForce": "DAY"  
}
```

HTTP/1.1 200 OK

Content-Type: application/hal+json

```
{  
  "_links": {  
    "bk:account": {  
      "href": "http://localhost:8080/accounts/ACC-1001"  
    },  
    "bk:place-order": {  
      "href": "http://localhost:8080/accounts/ACC-1001/orders"  
    }  
  },  
  "accountId": "ACC-1001",  
  "instrumentId": "OPT-AAPL-261016-C-240",  
  "symbol": "AAPL261016C00240000",  
  "side": "BUY",  
  "type": "LIMIT",  
  "quantity": "2",  
  "estimatedPrice": "3.10",  
  "estimatedNotional": "620.00",  
  "estimatedCommission": "1.30",  
  "estimatedTotal": "621.30",  
  "buyingPower": "53848.86",  
  "acceptable": true,  
  "issues": []  
}
```

Placing the order: 201 Created

HTTP

POST /accounts/ACC-1001/orders HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Idempotency-Key: 7d0e4c9a-3b1f-4e6d-8a52-9c0f1e2b3a4d

Content-Type: application/json

```
{
  "instrumentId": "EQ-MSFT",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "10",
  "limitPrice": "450.00",
  "timeInForce": "GTC",
  "clientOrderId": "rebalance-2026-09-28"
}
```

HTTP/1.1 201 Created

Location: http://localhost:8080/accounts/ACC-1001/orders/ORD-100002

ETag: "ORD-100002-v1"

Content-Type: application/hal+json

Idempotent-Replayed: false

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "450.00",
  "status": "OPEN",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-29T10:45:28.998175Z",
  "updatedAt": "2026-09-29T10:45:28.998175Z",
  "remainingQuantity": "10"
}
```

Everything in that response has a job. `201 Created` says a new resource exists. `Location` says where, and it is the URI the client should keep. `ETag` is the order's version, needed to change it later (Chapter 15). `Idempotent-Replayed: false` says this request created the order rather than replaying an earlier result (Chapter 14). The body is the order itself, with its links, so the client needs no second request to display it.

api/orders/OrderController.java

```
/**
 * Places an order: 201 Created with a Location header and the order's ETag. Retrying
 * with the same Idempotency-Key returns the original order, marked Idempotent-Replayed.
 */
@PostMapping("/accounts/{accountId}/orders")
public ResponseEntity<EntityModel<Order>> placeOrder(@PathVariable String accountId,
    @RequestHeader(name = IDEMPOTENCY_KEY, required = false) String idempotencyKey,
    @Valid @RequestBody OrderRequest request, @AuthenticationPrincipal Jwt jwt) {
    Caller caller = Caller.of(jwt);
    OrderService.Placement placement = service.place(caller, accountId, request, idempotencyKey);
    EntityModel<Order> model = assembler.toModel(placement.order(), caller);
    return ResponseEntity.created(model.getRequiredLink(IanaLinkRelations.SELF).toUri())
        .eTag(placement.order().eTag())
        .header("Idempotent-Replayed", Boolean.toString(placement.replayed()))
        .body(model);
}
```

Five layers of refusal

An order can be wrong in five different ways, and each is caught by a different layer with a different status code. Keeping the layers distinct is what lets a client react sensibly.

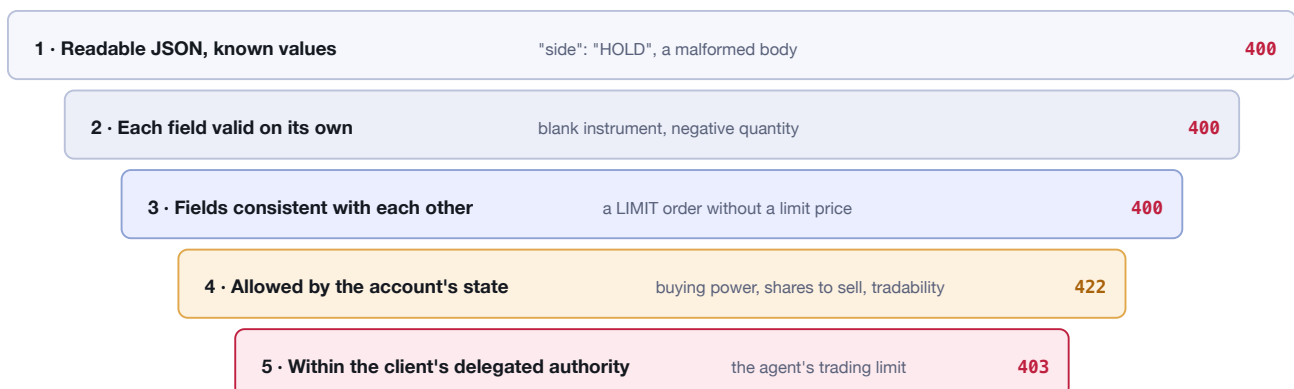


Figure 13.1 — The layers an order passes through. The first three are about the request (400: fix it). The fourth is about the account (422: the request was fine, the answer is no). The fifth is about authority (403: a bigger order needs a different client).

Layers 1 and 2: the request itself

A value of the wrong type never reaches the controller; Jackson rejects it while reading the body. The API reports it in the same `errors` shape as any other invalid field, so a client has one format to handle:

HTTP

HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```
{
  "detail": "One or more fields are invalid",
  "instance": "/accounts/ACC-1001/orders",
  "status": 400,
  "title": "The request is not valid",
  "type": "https://docs.brokerage.example/problems/invalid-request",
  "requestId": "73b19d0e-2869-4944-86d0-c483c63a2fcc",
  "errors": [
    {
      "field": "side",
      "message": "must be one of BUY, SELL"
    }
  ]
}
```

Field rules come from Bean Validation annotations on `OrderRequest`, which you met in Chapter 12. Every failing field is reported at once, sorted by field name, so a user can fix them all in one pass:

HTTP

POST /accounts/ACC-1001/orders HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Idempotency-Key: b1c2d3e4-0000-4000-8000-000000000001

Content-Type: application/json

```
{
  "instrumentId": "",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "-5",
  "limitPrice": "231.123456"
}
```

HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```
{
  "detail": "One or more fields are invalid",
  "instance": "/accounts/ACC-1001/orders",
  "status": 400,
  "title": "The request is not valid",
  "type": "https://docs.brokerage.example/problems/invalid-request",
  "requestId": "2f62a854-3bff-4fea-a15e-2f61afb0ec02",
  "errors": [
    {
      "field": "instrumentId",
      "message": "must not be blank"
    },
    {
      "field": "limitPrice",
      "message": "numeric value out of bounds (<9 digits>.<4 digits> expected)"
    },
    {
      "field": "quantity",
      "message": "must be greater than 0"
    }
  ]
}
```

```
}  
]  
}
```

Layers 3 and 4: the rules

Rules that involve several fields, or the account, live in `OrderService`. Consistency between type and prices is still a malformed request, so it is a 400:

HTTP

HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```
{  
  "detail": "limitPrice is required for LIMIT orders",  
  "instance": "/accounts/ACC-1001/orders",  
  "status": 400,  
  "title": "The request is not valid",  
  "type": "https://docs.brokerage.example/problems/invalid-request",  
  "requestId": "e236d3c7-73e1-4637-a19f-a115d090ca20"  
}
```

Rules about the account's state are different in kind. The request is valid; the account simply cannot do it. That is `422 Unprocessable Content`, with the numbers a client needs in extension members:

HTTP

HTTP/1.1 422 Unprocessable Content

Content-Type: application/problem+json

```
{  
  "detail": "This order needs 330000.00 but only 46241.10 is available",  
  "instance": "/accounts/ACC-1001/orders",  
  "status": 422,  
  "title": "The account does not have enough buying power",  
  "type": "https://docs.brokerage.example/problems/insufficient-buying-power",  
  "requestId": "0c046df7-81a8-4d9a-820a-8adff189860a",  
  "required": "330000.00",  
  "available": "46241.10"  
}
```

All of these checks live in one method, used by both previews and placements, so a preview can never promise what placement will refuse:

api/orders/OrderService.java

```
private Assessment assess(Caller caller, Account account, OrderRequest request) {  
    List<ApiException> issues = new ArrayList<>();  
    Instrument instrument;  
    try {  
        instrument = market.instrument(request.getInstrumentId());  
    } catch (ApiException notFound) {  
        issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,  
            "Instrument " + request.getInstrumentId() + " does not exist"));  
        return Assessment.failed(null, issues);  
    }  
  
    checkPrices(request, issues);  
}
```

```

if (!instrument.tradable()) {
    issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,
        instrument.symbol() + " is not tradable"));
}
boolean wholeUnits = request.getQuantity().stripTrailingZeros().scale() <= 0;
if (!wholeUnits && (!instrument.fractionable() || request.getType() != Type.MARKET)) {
    issues.add(new ApiException(ProblemType.INVALID_REQUEST,
        "Fractional quantities are allowed only in market orders for fractionable stocks"));
}
if (!issues.isEmpty()) {
    return Assessment.failed(instrument, issues);
}

BigDecimal price = referencePrice(request.getType(), request.getSide(),
    request.getLimitPrice(), request.getStopPrice(), instrument);
BigDecimal notional = price.multiply(request.getQuantity())
    .multiply(instrument.multiplier())
    .setScale(2, RoundingMode.HALF_EVEN);
BigDecimal commission = commission(instrument, request.getQuantity());
boolean buy = request.getSide() == Side.BUY;
BigDecimal total = buy ? notional.add(commission) : notional.subtract(commission);
BigDecimal buyingPower = Holdings.buyingPower(account.type(), ledger.cash(account.id()),
    reservedCash(account.id()));

if (buy && total.compareTo(buyingPower) > 0) {
    issues.add(new ApiException(ProblemType.INSUFFICIENT_BUYING_POWER,
        "This order needs " + total.toPlainString() + " but only "
            + buyingPower.toPlainString() + " is available")
        .with("required", total).with("available", buyingPower));
}
if (!buy) {
    BigDecimal available = ledger.quantityHeld(account.id(), instrument.id())
        .subtract(reservedQuantity(account.id(), instrument.id()));
    if (request.getQuantity().compareTo(available) > 0) {
        issues.add(new ApiException(ProblemType.INSUFFICIENT_POSITION,
            "Only " + available.toPlainString() + " " + instrument.symbol()
                + " are available to sell; short selling is not supported")
            .with("available", available));
    }
}
caller.tradingLimit().filter(limit -> notional.compareTo(limit) > 0).ifPresent(limit ->
    issues.add(new ApiException(ProblemType.TRADING_LIMIT_EXCEEDED,
        "This client may place orders up to " + limit.toPlainString()
            + " USD; this one is " + notional.toPlainString()
            .with("tradingLimit", limit)));
return new Assessment(instrument, price, notional, commission, total, buyingPower, issues);
}

```

Execution

A new order starts `OPEN`. Each second the matching engine looks at every working order. A buy executes at the ask and a sell at the bid, never beyond a limit, and never for more than the size displayed at that price, which is how partial fills happen. Every execution is booked to the ledger as a trade with a T+1 settlement date, and published as an event.

`api/orders/OrderService.java`

```

private void tryToFill(Order order) {
    Instrument instrument = market.instrument(order.instrumentId());
    MarketData.Tick tick = market.tick(instrument.id());
}

```

```

boolean buy = order.side() == Side.BUY;
Order current = order;

if ((order.type() == Type.STOP || order.type() == Type.STOP_LIMIT) && !order.stopTriggered()) {
    boolean triggered = buy ? tick.last().compareTo(order.stopPrice()) >= 0
        : tick.last().compareTo(order.stopPrice()) <= 0;
    if (!triggered) {
        return;
    }
    current = current.withStopTriggered();
    save(current);
}

BigDecimal price = buy ? tick.ask() : tick.bid();
if (order.type() == Type.LIMIT || order.type() == Type.STOP_LIMIT) {
    boolean marketable = buy ? price.compareTo(order.limitPrice()) <= 0
        : price.compareTo(order.limitPrice()) >= 0;
    if (!marketable) {
        return;
    }
}

BigDecimal displayed = buy ? tick.askSize() : tick.bidSize();
BigDecimal quantity = current.remainingQuantity().min(displayed);
Instant now = clock.instant();
ledger.recordTrade(order.accountId(), instrument, buy ? quantity : quantity.negate(), price,
    commission(instrument, quantity), order.id(), now);
String fillId = "FIL-" + fillIds.incrementAndGet();
Order filled = current.withFill(new Order.Fill(fillId, quantity, price, now));
save(filled);
boolean complete = filled.status() == Status.FILLED;
events.publish(complete ? "order.filled" : "order.partially-filled", filled);
}

```

THE STANDARD

1. Offer a preview resource that applies exactly the rules placement applies, and link from an acceptable preview to placement.
2. A new order returns `201 Created` with `Location`, `ETag` and the full representation.
3. Malformed or invalid requests are 400 with every field error listed; valid requests the account cannot honor are 422; exceeding delegated authority is 403.
4. Every rule lives in one server-side method shared by every path that needs it.

Idempotency: retries that can never buy twice

Networks fail in the middle of requests. When they do, the client cannot know whether its order was placed. An idempotency key turns that uncertainty from a hazard into a non-event.

The problem with a timeout

Remember Maya's train from Chapter 1. When a `POST` times out, one of three things has happened, and the client cannot tell which:

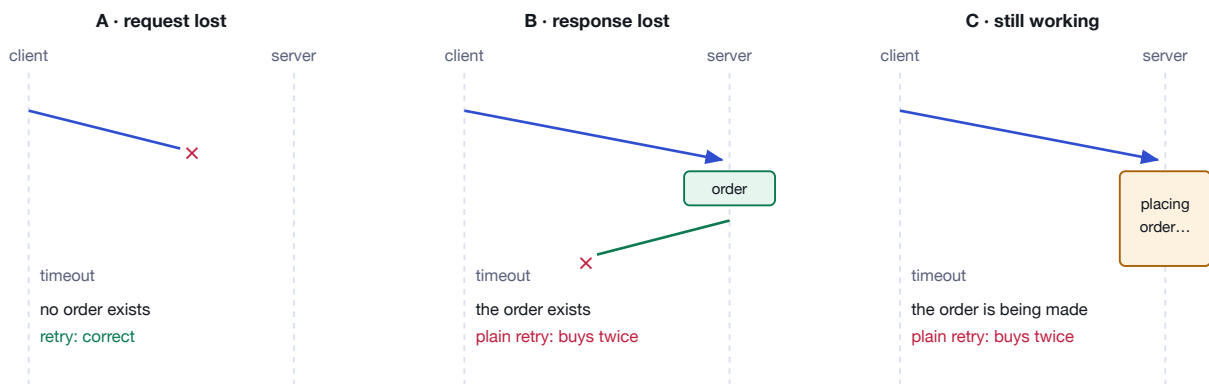


Figure 14.1 — Three failures that look identical to the client. Only in case A is a plain retry correct; only an idempotency key makes a retry correct in all three.

`GET`, `PUT` and `DELETE` are idempotent by definition, so retrying them is always safe. `POST` is not, and placing an order must be a `POST`, because each order is new. The fix is to let the client make any particular `POST` idempotent by naming it.

The Idempotency-Key header

The IETF HTTP APIs working group's Internet-Draft *The Idempotency-Key HTTP Header Field* standardizes a practice that payment APIs have used for years. The client generates a unique key, a UUID works well, and sends it with the request. The server remembers the key along with the result. If the same key arrives again, the server returns the stored result instead of doing the work again.

- **Same key, same request:** the original order, marked `Idempotent-Replayed: true`.
- **Same key, different request:** a client bug; refused with 422 rather than guessed at.
- **No key:** refused with 400. In this API every new order must be retry-safe.

HTTP

the same key and body, sent again

```
HTTP/1.1 201 Created
Location: http://localhost:8080/accounts/ACC-1001/orders/ORD-100002
ETag: "ORD-100002-v1"
```

```

Content-Type: application/hal+json
Idempotent-Replayed: true

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "450.00",
  "status": "OPEN",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-29T10:45:28.998175Z",
  "updatedAt": "2026-09-29T10:45:28.998175Z",
  "remainingQuantity": "10"
}

```

HTTP

the same key, a different quantity

```

POST /accounts/ACC-1001/orders HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
Idempotency-Key: 7d0e4c9a-3b1f-4e6d-8a52-9c0f1e2b3a4d
Content-Type: application/json

```

```

{
  "instrumentId": "EQ-MSFT",
  "side": "BUY",
  "type": "LIMIT",
  "quantity": "12",
  "limitPrice": "450.00",
  "timeInForce": "GTC",
  "clientOrderId": "rebalance-2026-09-28"
}

```

```

HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

```

```
{
```

```

    "detail": "Idempotency-Key 7d0e4c9a-3b1f-4e6d-8a52-9c0f1e2b3a4d was already used for a different order request",
    "instance": "/accounts/ACC-1001/orders",
    "status": 422,
    "title": "The Idempotency-Key was already used for a different request",
    "type": "https://docs.brokerage.example/problems/idempotency-key-reused",
    "requestId": "80c1c494-17af-4a86-9ead-56e6600b8613"
  }

```

HTTP

HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```

{
  "detail": "Send a unique Idempotency-Key header (up to 255 characters) with every new order",
  "instance": "/accounts/ACC-1001/orders",
  "status": 400,
  "title": "An Idempotency-Key header is required",
  "type": "https://docs.brokerage.example/problems/idempotency-key-missing",
  "requestId": "5c2e986a-4189-407e-9c88-98ac0dae6796"
}

```

How the server remembers

Recognizing “the same request” needs a definition. `OrderRequest` computes a **fingerprint** from its normalized fields. Quantities are stripped of trailing zeros, so `"10"` and `"10.00"` count as the same order, and the default time in force is filled in:

api/orders/OrderRequest.java

```

/** A stable fingerprint, so a retried request can be recognized as the same request. */
String fingerprint() {
    return String.join("|", instrumentId, side.name(), type.name(), timeInForceOrDefault().name(),
        plain(quantity), plain(limitPrice), plain(stopPrice), String.valueOf(clientOrderId));
}

```

The key store keeps each key for 24 hours, scoped to the client and the account, so two different applications can never collide on a key:

api/orders/IdempotencyKeys.java

```

/**
 * Remembers which order each Idempotency-Key created, for 24 hours.
 *
 * <p>A network timeout leaves a client unsure whether its order was placed. With a key,
 * retrying is always safe: the same key and the same request return the original order
 * instead of buying twice. The same key with a different request is a client bug, and is
 * refused rather than guessed at.
 */
@Component
public class IdempotencyKeys {

    static final Duration RETENTION = Duration.ofHours(24);

    private record Entry(String fingerprint, String orderId, Instant createdAt) {
    }
}

```

```

private final Map<String, Entry> entries = new ConcurrentHashMap<>();
private final Clock clock;

public IdempotencyKeys(Clock clock) {
    this.clock = clock;
}

/** The order already created with this key, if any. */
Optional<String> existingOrder(String scope, String key, String fingerprint) {
    Instant now = clock.instant();
    entries.values().removeIf(entry -> entry.createdAt().plus(RETENTION).isBefore(now));
    Entry entry = entries.get(scope + "|" + key);
    if (entry == null) {
        return Optional.empty();
    }
    if (!entry.fingerprint().equals(fingerprint)) {
        throw new ApiException(ProblemType.IDEMPOTENCY_KEY_REUSED,
            "Idempotency-Key " + key + " was already used for a different order request");
    }
    return Optional.of(entry.orderId());
}

void remember(String scope, String key, String fingerprint, String orderId) {
    entries.put(scope + "|" + key, new Entry(fingerprint, orderId, clock.instant()));
}
}

```

The placement method checks the key before anything else, and records it only after the order exists:

api/orders/OrderService.java

```

public synchronized Placement place(Caller caller, String accountId, OrderRequest request,
    String idempotencyKey) {
    Account account = ledger.account(caller.customerId(), accountId);
    if (idempotencyKey == null || idempotencyKey.isBlank() || idempotencyKey.length() > 255) {
        throw new ApiException(ProblemType.IDEMPOTENCY_KEY_MISSING,
            "Send a unique Idempotency-Key header (up to 255 characters) with every new order");
    }
    String scope = caller.clientId() + "|" + accountId;
    String fingerprint = request.fingerprint();
    Optional<String> earlier = idempotencyKeys.existingOrder(scope, idempotencyKey, fingerprint);
    if (earlier.isPresent()) {
        return new Placement(orders.get(earlier.get()), true);
    }

    Assessment assessment = assess(caller, account, request);
    assessment.issues().stream().findFirst().ifPresent(issue -> {
        throw issue;
    });

    Instant now = clock.instant();
    Instrument instrument = assessment.instrument();
    Order order = new Order(nextOrderId(), accountId, request.getClientOrderId(), instrument.id(),
        instrument.symbol(), instrument.type(), request.getSide(), request.getType(),
        request.timeInForceOrDefault(), request.getQuantity(), request.getLimitPrice(),
        request.getStopPrice(), Status.OPEN, BigDecimal.ZERO, null, List.of(), now, now,
        false, 1);
    save(order);
    idempotencyKeys.remember(scope, idempotencyKey, fingerprint, order.id());
    events.publish("order.created", order);
    return new Placement(order, false);
}

```

In production. This store is an in-memory map, and the service is synchronized, so two concurrent requests with one key can never both succeed. A real deployment runs several instances, so the store belongs in a shared database, and the key must be claimed atomically *before* the order is created. A unique constraint on (client, account, key) is the usual tool. The draft specifies 409 Conflict for a request that arrives while an earlier one with the same key is still in flight.

The client's half

An idempotency key is only useful if the client reuses it on retries. The book's client generates one key per order and retries timeouts and transient failures with that same key:

client/BrokerageClient.java

```
/**
 * Places an order. On a timeout or a transient error the request is retried with the
 * same Idempotency-Key, so a retry can never place a second order.
 */
public Order place(String accountId, OrderTicket ticket, String idempotencyKey) {
    Link orders = account(accountId).requiredLink("bk:orders").expand();
    for (int attempt = 1; ; attempt++) {
        try {
            return http.post(orders, ticket, headers -> headers.set("Idempotency-Key", idempotencyKey))
                .as(Order.class);
        } catch (ResourceAccessException | ApiProblem failure) {
            boolean retryable = !(failure instanceof ApiProblem problem) || problem.isTransient();
            if (!retryable || attempt == ATTEMPTS) {
                throw failure;
            }
            long seconds = failure instanceof ApiProblem problem
                ? problem.retryAfterSeconds().orElse(1L) : attempt;
            pause(seconds);
        }
    }
}
```

- Retry network failures and transient statuses (429, 502, 503, 504), never 400s or 422s.
- Honor `Retry-After` when the server sends it; otherwise back off progressively.
- Generate the key when the user presses *Buy*, not when the request is sent, so that every retry, even after an app restart, carries the same key.

Is clientOrderId enough?

Many brokerage APIs, including the RPC-style one compared in Chapter 24, deduplicate on a client-supplied order ID instead. It is a workable idea: the ID must be unique per account, so a retry with the same ID cannot create a second order. The `Idempotency-Key` header is the more general version of the same idea. It works for any `POST`, not only orders; it separates “my reference for this order” from “this attempt”; and it defines what a replay returns, which a unique-ID rule usually leaves unsaid.

THE STANDARD

1. Every non-idempotent request that creates something valuable requires an `Idempotency-Key` .
2. The same key and request replay the original response, marked `Idempotent-Replayed: true` ; the same key with a different request is 422.
3. Keys are scoped to the client and retained for at least 24 hours; claim them atomically.
4. Clients generate the key once per user intent and reuse it on every retry.

Changing and canceling orders

A working order is shared state: the customer's phone, their laptop and the exchange can all touch it at once. HTTP's conditional requests keep those writers from silently overwriting each other, and `202 Accepted` tells the truth about cancels that take time.

The lost update

Maya's phone and laptop both show her open order for 10 MSFT at 450.00. On the laptop she raises the limit to 455.00. A moment later, on the phone, which still shows the old order, she cuts the quantity to 8. Without protection, the phone's request is applied to an order it has never seen, and nobody is told.

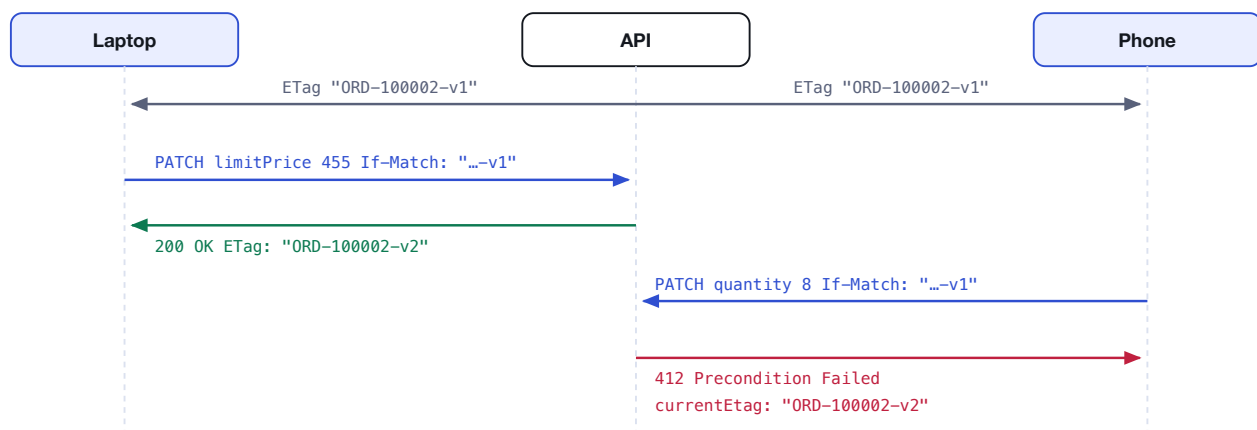


Figure 15.1 — Optimistic concurrency with ETags. The laptop's change succeeds and produces version 2. The phone's change was based on version 1, so it is refused, and the phone must re-read the order before deciding again.

ETags and If-Match

Every order carries a **strong entity tag** that changes whenever the order does:

```

api/orders/Order.java

/** A strong entity tag: changes whenever the order changes. */
@JsonIgnore
public String etag() {
    return "\"" + id + "-v" + version + "\"";
}
  
```

A client that changes an order must send the tag of the version it saw in `If-Match`. The server compares it with the current tag before applying anything. The API goes one step further than HTTP requires and makes the header *mandatory*: a change without `If-Match` is refused with `428 Precondition Required`, defined for exactly this purpose by RFC 6585.

HTTP

PATCH /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Content-Type: application/merge-patch+json

```
{
  "limitPrice": "455.00"
}
```

HTTP/1.1 428 Precondition Required

Content-Type: application/problem+json

```
{
  "detail": "Send If-Match with the order's current ETag to change it",
  "instance": "/accounts/ACC-1001/orders/ORD-100002",
  "status": 428,
  "title": "An If-Match header is required",
  "type": "https://docs.brokerage.example/problems/precondition-required",
  "requestId": "07fb9406-3b3d-4745-9fb9-9aaccf1924b1"
}
```

HTTP

with the current ETag: accepted, and the ETag moves on

PATCH /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Content-Type: application/merge-patch+json

If-Match: "ORD-100002-v1"

```
{
  "limitPrice": "455.00"
}
```

HTTP/1.1 200 OK

ETag: "ORD-100002-v2"

Content-Type: application/hal+json

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    },
    "edit": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:cancel": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002/cancellation"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
}
```

```

"symbol": "MSFT",
"instrumentType": "EQUITY",
"side": "BUY",
"type": "LIMIT",
"timeInForce": "GTC",
"quantity": "10",
"limitPrice": "455.00",
"status": "OPEN",
"filledQuantity": "0",
"fills": [],
"createdAt": "2026-09-29T10:45:28.998175Z",
"updatedAt": "2026-09-29T10:45:29.064534Z",
"remainingQuantity": "10"
}

```

HTTP

the phone, still holding version 1

PATCH /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

Content-Type: application/merge-patch+json

If-Match: "ORD-100002-v1"

```

{
  "quantity": "8"
}

```

HTTP/1.1 412 Precondition Failed

Content-Type: application/problem+json

```

{
  "detail": "The order has changed since you read it; fetch it again and reapply your change",
  "instance": "/accounts/ACC-1001/orders/ORD-100002",
  "status": 412,
  "title": "The resource has changed",
  "type": "https://docs.brokerage.example/problems/precondition-failed",
  "requestId": "02a60603-4e99-4fb1-a10a-3f7ede34eae2",
  "currentEtag": "\"ORD-100002-v2\""
}

```

The `currentEtag` extension member saves the phone a guess: it can re-read the order, show the user what changed, and let them decide.

Merge patch: sending only what changes

The change itself is a **JSON merge patch** (RFC 7396), sent as `application/merge-patch+json`. A merge patch is a partial document: each member present replaces the same member of the resource, and members that are absent stay as they are. That makes the request both minimal and self-evident: `{"limitPrice": "455.00"}` changes the limit price and nothing else.

api/orders/OrderAmendment.java

```
/**
 * The body of PATCH ../orders/{orderId}, sent as application/merge-patch+json (RFC 7396):
 * only the fields present are changed. The instrument, side and type of a working order
 * cannot change; cancel it and place a new one instead.
 */
public class OrderAmendment {

    @InputType("text") @Positive @Digits(integer = 9, fraction = 6)
    private BigDecimal quantity;
    @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
    private BigDecimal limitPrice;
    @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
    private BigDecimal stopPrice;
    private Order.TimeInForce timeInForce;
```

Only a working order's quantity, prices and time in force can change. The instrument, side and type cannot. A different instrument is a different order, so the client cancels and places a new one. The server re-runs the business rules on the new terms, discounting the cash the order had already reserved:

api/orders/OrderService.java

```
/**
 * Changes a working order. The If-Match header must carry the order's current ETag:
 * two clients editing the same order cannot silently overwrite one another.
 */
public synchronized Order amend(Caller caller, String accountId, String orderId,
    OrderAmendment change, String ifMatch) {
    Order order = order(caller, accountId, orderId);
    if (ifMatch == null) {
        throw new ApiException(ProblemType.PRECONDITION_REQUIRED,
            "Send If-Match with the order's current ETag to change it");
    }
    if (!ifMatch.equals(order.etag()) && !ifMatch.equals("*")) {
        throw new ApiException(ProblemType.PRECONDITION_FAILED,
            "The order has changed since you read it; fetch it again and reapply your change")
            .with("currentEtag", order.etag());
    }
    if (!order.status().isWorking()) {
        throw new ApiException(ProblemType.ORDER_NOT_REPLACEABLE,
            "Order " + orderId + " is " + order.status() + " and can no longer be changed");
    }
    BigDecimal quantity = orElse(change.getQuantity(), order.quantity());
    if (quantity.compareTo(order.filledQuantity()) <= 0) {
        throw new ApiException(ProblemType.INVALID_REQUEST, "quantity must be greater than the "
            + order.filledQuantity().toString() + " already filled");
    }
    OrderRequest amended = new OrderRequest(order.instrumentId(), order.side(), order.type(),
        orElse(change.getTimeInForce(), order.timeInForce()), quantity,
        orElse(change.getLimitPrice(), order.limitPrice()),
        orElse(change.getStopPrice(), order.stopPrice()), order.clientOrderId());

    Account account = ledger.account(caller.customerId(), accountId);
    orders.remove(orderId); // assess the new terms without this order's own reservation
    try {
        assess(caller, account, amended).issues().stream().findFirst().ifPresent(issue -> {
            throw issue;
        });
    } finally {
        orders.put(orderId, order);
    }
}
```

```

    Order updated = order.withTerms(amended.getQuantity(), amended.getLimitPrice(),
        amended.getStopPrice(), amended.getTimeInForce(), clock.instant());
    save(updated);
    events.publish("order.replaced", updated);
    return updated;
}

```

api/orders/OrderController.java

```

/** Changes a working order's quantity, prices or time in force. Requires If-Match. */
@PatchMapping(path = "/accounts/{accountId}/orders/{orderId}",
    consumes = { MERGE_PATCH_JSON, "application/json" })
public ResponseEntity<EntityModel<Order>> replaceOrder(@PathVariable String accountId,
    @PathVariable String orderId,
    @RequestHeader(name = HttpHeaders.IF_MATCH, required = false) String ifMatch,
    @Valid @RequestBody OrderAmendment change, @AuthenticationPrincipal Jwt jwt) {
    Caller caller = Caller.of(jwt);
    Order order = service.amend(caller, accountId, orderId, change, ifMatch);
    return ResponseEntity.ok().eTag(order.etag()).body(assembler.toModel(order, caller));
}

```

Polling cheaply with If-None-Match

The same tag makes polling almost free. A client that has an order sends its tag in `If-None-Match`. While nothing has changed, the answer is a bodiless `304 Not Modified`:

HTTP

```

GET /accounts/ACC-1001/orders/ORD-100002 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
If-None-Match: "ORD-100002-v1"
-----
HTTP/1.1 304 Not Modified
ETag: "ORD-100002-v1"

```

api/orders/OrderController.java

```

/** One order. Its ETag supports cheap polling (If-None-Match) and safe edits (If-Match). */
@GetMapping("/accounts/{accountId}/orders/{orderId}")
public ResponseEntity<EntityModel<Order>> order(@PathVariable String accountId,
    @PathVariable String orderId, WebRequest request, @AuthenticationPrincipal Jwt jwt) {
    Caller caller = Caller.of(jwt);
    Order order = service.order(caller, accountId, orderId);
    if (request.checkNotModified(order.etag())) {
        return null;
    }
    return ResponseEntity.ok().eTag(order.etag()).body(assembler.toModel(order, caller));
}

```

Canceling: 202 Accepted

On a real exchange, a cancel is a request, not a command. It travels to the market, and a fill may arrive first. The API models that honestly. A cancellation request is accepted at once with `202 Accepted`, the order moves to `PENDING_CANCEL`, and `Location` points at the order, whose status will settle:

HTTP

POST /accounts/ACC-1001/orders/ORD-100002/cancellation HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

API-Version: 2

HTTP/1.1 202 Accepted

Location: http://localhost:8080/accounts/ACC-1001/orders/ORD-100002

ETag: "ORD-100002-v3"

Content-Type: application/hal+json

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100002"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-MSFT"
    }
  },
  "id": "ORD-100002",
  "accountId": "ACC-1001",
  "clientOrderId": "rebalance-2026-09-28",
  "instrumentId": "EQ-MSFT",
  "symbol": "MSFT",
  "instrumentType": "EQUITY",
  "side": "BUY",
  "type": "LIMIT",
  "timeInForce": "GTC",
  "quantity": "10",
  "limitPrice": "455.00",
  "status": "PENDING_CANCEL",
  "filledQuantity": "0",
  "fills": [],
  "createdAt": "2026-09-29T10:45:28.998175Z",
  "updatedAt": "2026-09-29T10:45:29.072117Z",
  "remainingQuantity": "10"
}
```

Notice that the `edit` and `bk:cancel` links are already gone: a pending cancel cannot be changed or canceled again. On the matching engine's next pass, the order becomes `CANCELLED`, unless a fill wins the race, in which case it becomes `FILLED` and the cancel is moot. You saw the final state in Chapter 9.

```

/**
 * Requests cancellation. Like a real exchange, the cancel is asynchronous: the order
 * moves to PENDING_CANCEL now and to CANCELLED at the next matching pass, and it may
 * still fill in between.
 */
public synchronized Order cancel(Caller caller, String accountId, String orderId) {
    Order order = order(caller, accountId, orderId);
    if (order.status() == Status.PENDING_CANCEL) {
        return order;
    }
    if (!order.status().isWorking()) {
        throw new ApiException(ProblemType.ORDER_NOT_CANCELABLE,
            "Order " + orderId + " is " + order.status() + " and cannot be canceled");
    }
    Order pending = order.withStatus(Status.PENDING_CANCEL, clock.instant());
    save(pending);
    events.publish("order.cancel-requested", pending);
    return pending;
}

```

Asking twice is harmless: a second request while the cancel is pending returns the same pending order, so a client can retry a cancel without an idempotency key. Once the order is final, the request is refused with `409 Conflict`.

Why not DELETE?

`DELETE /orders/ORD-100002` reads naturally, but it promises the wrong thing: that the resource will be gone. A canceled order must remain readable for the customer's records, for regulators, and for the client that asked. Modeling the cancellation as its own resource keeps every guarantee true: the order is never deleted, the request is accepted rather than completed, and the final state is visible on the order itself.

THE STANDARD

1. Every mutable resource has a strong `ETag`; changes require `If-Match` (428 without, 412 when stale, with the current tag in the problem).
2. Partial updates are JSON merge patches with `application/merge-patch+json`.
3. Operations that complete later answer `202 Accepted` with a `Location` to watch.
4. Records that must be kept are never deleted; ending them is a state change.

Problem details: errors as RFC 9457 documents

An error response is an API's most important documentation, because it is the one a developer reads when something is already wrong. RFC 9457 gives every error the same shape, and a stable identifier that code can depend on.

Anatomy of a problem

RFC 9457, *Problem Details for HTTP APIs*, replaced RFC 7807 in 2023 and defines the media type `application/problem+json`. Every error from the brokerage API is one:

```
{
  "type": "https://docs../problems/
    insufficient-buying-power",
  "title": "The account does not have...",
  "status": 422,
  "detail": "This order needs 330000.00...",
  "instance": "/accounts/ACC-1001/orders",
  "requestId": "71e69cf7-...",
  "required": "330000.00",
  "available": "46241.10" }
```

type: the identity of the problem

stable URI; clients branch on it; it resolves to docs

title, status

a fixed summary per type; the HTTP status, repeated

detail, instance

this occurrence, for humans; the request path

extension members

machine-readable facts, defined per type

Figure 16.1 — A problem document. `type` is the contract; `title` and `detail` are for people; extension members carry the facts a program needs.

The most important rule is the division of labor. **Code branches on `type`**, never on `title` or `detail`, which are prose and may be reworded or translated. The type is a URI, so it is globally unique. In this API it also resolves to a page that documents the problem: what it means, when it happens, and what to do about it.

A catalog, not a free-for-all

Every problem the API can return is declared in one enumeration, with its type, status and title. There is no other way to produce an error, so the catalog is complete by construction:

`api/platform/ProblemType.java`

```
/**
 * The catalog of error types this API can return. Each becomes the "type" URI of an
 * RFC 9457 problem document, so clients branch on a stable identifier, never on the
 * human-readable title or detail.
 */
public enum ProblemType {

    INVALID_REQUEST("invalid-request", HttpStatus.BAD_REQUEST, "The request is not valid"),
    UNAUTHORIZED("unauthorized", HttpStatus.UNAUTHORIZED, "Authentication is required"),
```

```

INSUFFICIENT_SCOPE("insufficient-scope", HttpStatus.FORBIDDEN, "The access token lacks a required scope"),
NOT_FOUND("not-found", HttpStatus.NOT_FOUND, "The resource does not exist"),
IDEMPOTENCY_KEY_MISSING("idempotency-key-missing", HttpStatus.BAD_REQUEST,
    "An Idempotency-Key header is required"),
IDEMPOTENCY_KEY_REUSED("idempotency-key-reused", HttpStatus.UNPROCESSABLE_CONTENT,
    "The Idempotency-Key was already used for a different request"),
PRECONDITION_REQUIRED("precondition-required", HttpStatus.PRECONDITION_REQUIRED,
    "An If-Match header is required"),
PRECONDITION_FAILED("precondition-failed", HttpStatus.PRECONDITION_FAILED, "The resource has changed"),
ORDER_NOT_CANCELLABLE("order-not-cancellable", HttpStatus.CONFLICT,
    "The order can no longer be canceled"),
ORDER_NOT_REPLACEABLE("order-not-replaceable", HttpStatus.CONFLICT, "The order can no longer be changed"),
INSUFFICIENT_BUYING_POWER("insufficient-buying-power", HttpStatus.UNPROCESSABLE_CONTENT,
    "The account does not have enough buying power"),
INSUFFICIENT_POSITION("insufficient-position", HttpStatus.UNPROCESSABLE_CONTENT,
    "The account does not hold enough of the instrument"),
INSTRUMENT_NOT_TRADABLE("instrument-not-tradable", HttpStatus.UNPROCESSABLE_CONTENT,
    "The instrument cannot be traded this way"),
TRADING_LIMIT_EXCEEDED("trading-limit-exceeded", HttpStatus.FORBIDDEN,
    "The order exceeds the trading limit delegated to this client"),
UNSUPPORTED_API_VERSION("unsupported-api-version", HttpStatus.BAD_REQUEST,
    "The requested API version is not supported"),
RATE_LIMITED("rate-limited", HttpStatus.TOO_MANY_REQUESTS, "Too many requests");

```

Appendix B lists each type with what a client should do about it. Business code raises an `ApiException` with a type, a detail and any extension members:

`api/orders/OrderService.java`

raising a problem

```

throw new ApiException(ProblemType.PRECONDITION_FAILED,
    "The order has changed since you read it; fetch it again and reapply your change")
    .with("currentEtag", order.etag());

```

One handler for every error

A single `@RestControllerAdvice` turns every exception into a problem document, adding the request path as `instance` and the request ID for support:

`api/platform/ApiExceptionHandler.java`

```

@ExceptionHandler(ApiException.class)
ResponseBody<ProblemDetail> handleApi(ApiException ex, HttpServletRequest request) {
    ProblemDetail problem = problem(ex.type(), ex.getMessage(), request);
    ex.extensions().forEach(problem::setProperty);
    HttpHeaders headers = new HttpHeaders();
    if (ex instanceof RateLimitedException limited) {
        headers.set(HttpHeaders.RETRY_AFTER, String.valueOf(limited.retryAfterSeconds()));
    }
    return ResponseEntity.status(ex.type().status()).headers(headers).body(problem);
}

```

api/platform/ApiExceptionHandler.java

```
static ProblemDetail problem(ProblemType type, String detail, HttpServletRequest request) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(type.status(), detail);
    problem.setType(type.uri());
    problem.setTitle(type.title());
    problem.setInstance(URI.create(request.getRequestURI()));
    problem.setProperty("requestId", RequestIdFilter.currentId(request));
    return problem;
}
```

The class extends Spring's `ResponseEntityExceptionHandler`, which already renders Spring's own exceptions, such as 405, 406 and 415, as problem details. Two overrides give invalid input the API's consistent `errors` array, whether Bean Validation rejected a field or Jackson could not read it:

api/platform/ApiExceptionHandler.java

```
/** Bean Validation failures: one problem, with every invalid field listed. */
@Override
protected ResponseEntity<Object> handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
    HttpHeaders headers, HttpStatusCode status, WebRequest request) {
    ProblemDetail problem = problem(ProblemType.INVALID_REQUEST, "One or more fields are invalid",
        ((ServletWebRequest) request).getRequest());
    List<Map<String, String>> errors = ex.getBindingResult().getFieldErrors().stream()
        .sorted(Comparator.comparing(FieldError::getField))
        .map(error -> Map.of("field", error.getField(),
            "message", String.valueOf(error.getDefaultMessage())))
        .toList();
    problem.setProperty("errors", errors);
    return ResponseEntity.badRequest().body(problem);
}
```

Security errors are problems too

Authentication and authorization failures happen in Spring Security's filter chain, before any controller runs, and by default they produce an empty body. The API keeps what Spring Security does well, the standard `WWW-Authenticate` challenge of RFC 6750, and adds the problem document by handing the exception to the same MVC exception handling:

api/platform/SecurityConfig.java

```
public static final class ProblemResponses implements AuthenticationEntryPoint, AccessDeniedHandler {

    private final HandlerExceptionResolver resolver;
    private final BearerTokenAuthenticationEntryPoint challenge =
        new BearerTokenAuthenticationEntryPoint();
    private final BearerTokenAccessDeniedHandler denied = new BearerTokenAccessDeniedHandler();

    ProblemResponses(HandlerExceptionResolver resolver) {
        this.resolver = resolver;
    }

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException ex) {
        challenge.commence(request, response, ex);
        resolver.resolveException(request, response, null, ex);
    }

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
        AccessDeniedException ex) {
        denied.handle(request, response, ex);
        resolver.resolveException(request, response, null, ex);
    }
}
```

HTTP

no token: 401, a challenge, and where to get a token

GET /accounts HTTP/1.1

Host: localhost:8080

API-Version: 2

HTTP/1.1 401 Unauthorized

Content-Type: application/problem+json

WWW-Authenticate: Bearer resource_metadata="http://localhost:8080/.well-known/oauth-protected-resource"

```
{
  "detail": "A valid bearer token is required",
  "instance": "/accounts",
  "status": 401,
  "title": "Authentication is required",
  "type": "https://docs.brokerage.example/problems/unauthorized",
  "requestId": "05ab6bff-2c40-4a22-8ad8-bcb0fd95eddc"
}
```

```

HTTP/1.1 403 Forbidden
Content-Type: application/problem+json
WWW-Authenticate: Bearer error="insufficient_scope", error_description="The request requires higher privileges than provided by the access token.", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"

{
  "detail": "The access token does not grant the scope this operation requires",
  "instance": "/accounts",
  "status": 403,
  "title": "The access token lacks a required scope",
  "type": "https://docs.brokerage.example/problems/insufficient-scope",
  "requestId": "329ae2cf-4691-4de4-93d9-bdeb2c2d19ea"
}

```

Three statuses, three meanings. **401**: I do not know who you are; get a token (the challenge even says where, through the `resource_metadata` parameter of RFC 9728). **403**: I know who you are, and your token does not allow this. **404**: as far as you are concerned, it does not exist, which is also the answer for another customer's account.

Request IDs

Every response carries an `X-Request-Id`, and every problem repeats it as `requestId`. A client may supply its own ID so that its logs and the server's share one value. The server accepts only IDs that are short and plain, so the header cannot be used to inject text into logs:

api/platform/RequestIdFilter.java

```

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
    FilterChain chain) throws ServletException, IOException {
    String supplied = request.getHeader(HEADER);
    String id = supplied != null && SAFE.matcher(supplied).matches()
        ? supplied : UUID.randomUUID().toString();
    request.setAttribute(ATTRIBUTE, id);
    response.setHeader(HEADER, id);
    MDC.put("requestId", id);
    try {
        chain.doFilter(request, response);
    } finally {
        MDC.remove("requestId");
    }
}

```

What never goes in a problem

- Stack traces, exception class names, SQL, file paths or server names.
- Anything that confirms the existence of a resource the caller may not see.
- Secrets, tokens or the full contents of the request.

An unexpected exception becomes a generic **500** problem with a request ID, and the details go to the server's log, where the request ID finds them.

THE STANDARD

1. Every error is `application/problem+json` with `type`, `title`, `status`, `detail` and `instance`.
2. Problem types are stable URIs from one catalog, each documented; clients branch on `type` only.
3. Facts a client needs travel as typed extension members; invalid fields are listed in one `errors` array.
4. 401 carries a `WWW-Authenticate` challenge; 401, 403 and 404 are used precisely.
5. Every response and problem carries a request ID; no problem reveals internals.

Operating at Scale

An API that works for one developer on a laptop must also work for thousands of clients over years. Collections that never end, events that must not be missed, versions that must be retired, quotas that must be shared, and tokens that must be trusted.



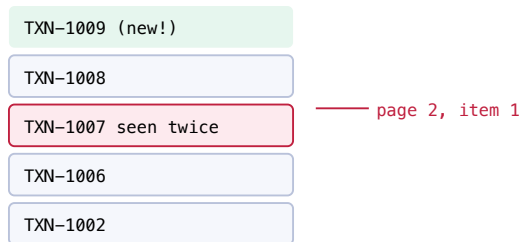
Collections: pagination, filtering and caching

A collection that can grow forever needs three things: a way to read it in pieces that never skip or repeat, a way to ask for less of it, and a way to avoid reading it again when nothing has changed.

Why not page=2?

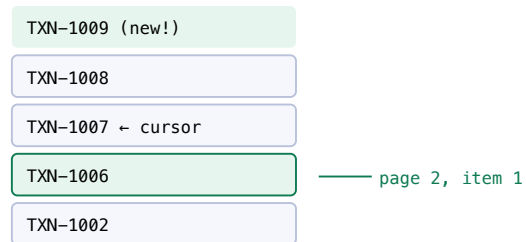
Offset pagination (`?page=2&size=25` , or `?offset=25`) is easy to build and wrong for any list that changes while it is read. A statement is newest first, so every new transaction is inserted at the top. If one arrives between a client's request for page 1 and its request for page 2, every item shifts down by one, and page 2 begins with an item the client has already seen. If an item is removed, one is skipped instead. Neither is visible to the client.

Offset: page 2 starts at position 3



Page 1 returned TXN-1008 and TXN-1007. Then TXN-1009 arrived at the top.

Cursor: page 2 starts after TXN-1007



The cursor names the last item seen, so new items cannot shift the next page.

Figure 17.1 — Offset pagination repeats (or skips) items when the list changes between pages. A cursor marks a position in the sort order itself, so it cannot.

Cursors

A **cursor** encodes the sort key of the last item returned: here, its timestamp and ID, which together are unique. The next page is “items older than that key,” a question with the same answer no matter what was inserted above. The cursor is base64url-encoded so that it looks, and is treated, as opaque:

`api/platform/CursorPage.java`

```
/**
 * One page of a newest-first list, with an opaque cursor for the next page.
 *
 * <p>The cursor encodes the sort key (timestamp and ID) of the last item returned, so a
 * page never skips or repeats items when new ones are inserted at the top: something
 * offset-based paging ("page=3") cannot promise. Clients never parse cursors; they
 * follow the "next" link that carries one.
 */
public record CursorPage<T>(List<T> items, String nextCursor) {

    public static final int DEFAULT_LIMIT = 25;
    public static final int MAX_LIMIT = 100;
```

api/platform/CursorPage.java

```
public static <T> CursorPage<T> of(List<T> all, Function<T, Instant> time, Function<T, String> id,
    String cursor, Integer requestedLimit) {
    int limit = requestedLimit == null ? DEFAULT_LIMIT : requestedLimit;
    if (limit < 1 || limit > MAX_LIMIT) {
        throw new ApiException(ProblemType.INVALID_REQUEST,
            "limit must be between 1 and " + MAX_LIMIT);
    }
    Comparator<T> newestFirst = Comparator.comparing(time).thenComparing(id).reversed();
    List<T> sorted = all.stream().sorted(newestFirst).toList();

    int start = 0;
    if (cursor != null) {
        Key after = Key.decode(cursor);
        while (start < sorted.size() && !isOlder(sorted.get(start), after, time, id)) {
            start++;
        }
    }
    int end = Math.min(start + limit, sorted.size());
    List<T> items = sorted.subList(start, end);
    String next = end < sorted.size()
        ? new Key(time.apply(items.getLast()), id.apply(items.getLast())).encode()
        : null;
    return new CursorPage<>(items, next);
}
```

Opacity is a promise the server keeps to itself: because no client parses cursors, the encoding can change at any time. A cursor that fails to decode, whether tampered with or from a retired format, is a 400, never a silent empty page. The page size is bounded too:

HTTP

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "detail": "limit must be between 1 and 100",
  "instance": "/accounts/ACC-1001/transactions",
  "status": 400,
  "title": "The request is not valid",
  "type": "https://docs.brokerage.example/problems/invalid-request",
  "requestId": "17e3a770-0ee5-4dd6-a480-6d4f58554dc7"
}
```

Following `next` from the first page of Chapter 7 gives the second, with its own `next`:

HTTP

```
GET /accounts/ACC-1001/transactions?cursor=MTc4ODk0MjMyNjg3NjUWE4tMTAwNw&limit=2 HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
API-Version: 2
-----
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "_embedded": {
    "bk:transactions": [
      {

```

```

    "id": "TXN-1006",
    "type": "DIVIDEND",
    "description": "Cash dividend AAPL",
    "amount": "13.00",
    "instrumentId": "EQ-AAPL",
    "symbol": "AAPL",
    "occurredAt": "2026-08-15T10:30:26.876245Z",
    "settlesOn": "2026-08-15"
  },
  {
    "id": "TXN-1002",
    "type": "DEPOSIT",
    "description": "ACH deposit",
    "amount": "10000.00",
    "occurredAt": "2026-06-01T06:45:26.876076Z",
    "settlesOn": "2026-06-02"
  }
]
},
"_links": {
  "self": {
    "href": "http://localhost:8080/accounts/ACC-1001/transactions?
cursor=MTc4ODk0MjMyNjg3NjpuWE4tMTAwNw&limit=2"
  },
  "bk:account": {
    "href": "http://localhost:8080/accounts/ACC-1001"
  },
  "next": {
    "href": "http://localhost:8080/accounts/ACC-1001/transactions?
cursor=MTc4MDI5NjMyNjg3NjpuWE4tMTAwMg&limit=2"
  }
}
}
}

```

There is no `prev`, no page number and no total count. Each would cost the server work, typically a count query over a growing table, and none is stable while the list changes. A statement screen needs “more,” and `next` is exactly that.

Filtering

Filters are optional query parameters that narrow a collection, and they are declared in the collection's link template, so clients discover them. The conventions are few and uniform:

- **Equality:** `?symbol=AAPL`, `?type=DIVIDEND`.
- **Any of several values:** repeat the parameter: `?status=OPEN&status=PARTIALLY_FILLED` (a comma-separated list is accepted too, which is what RFC 6570 expansion of a list produces).
- **Ranges:** `from` and `to`, inclusive, as ISO dates in UTC.
- **Order:** fixed, newest first. A stable, indexable order is what makes cursors cheap.

HTTP

working orders: two statuses, one request

```

GET /accounts/ACC-1001/orders?status=OPEN&status=PARTIALLY_FILLED HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQ0i0i3MTE1NmVl...
API-Version: 2

```

HTTP/1.1 200 OK

Content-Type: application/hal+json

```
{
  "_embedded": {
    "bk:orders": [
      {
        "_links": {
          "self": {
            "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100001"
          },
          "bk:account": {
            "href": "http://localhost:8080/accounts/ACC-1001"
          },
          "bk:instrument": {
            "href": "http://localhost:8080/instruments/EQ-MSFT"
          },
          "edit": {
            "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100001"
          },
          "bk:cancel": {
            "href": "http://localhost:8080/accounts/ACC-1001/orders/ORD-100001/cancellation"
          }
        },
        "id": "ORD-100001",
        "accountId": "ACC-1001",
        "clientOrderId": "rebalance-msft-01",
        "instrumentId": "EQ-MSFT",
        "symbol": "MSFT",
        "instrumentType": "EQUITY",
        "side": "BUY",
        "type": "LIMIT",
        "timeInForce": "GTC",
        "quantity": "10",
        "limitPrice": "480.00",
        "status": "OPEN",
        "filledQuantity": "0",
        "fills": [],
        "createdAt": "2026-09-28T10:45:26.882403Z",
        "updatedAt": "2026-09-28T10:45:26.882403Z",
        "remainingQuantity": "10"
      }
    ]
  },
  "_links": {
    "self": {
      "href": "http://localhost:8080/accounts/ACC-1001/orders?status=OPEN&status=PARTIALLY_FILLED"
    },
    "bk:account": {
      "href": "http://localhost:8080/accounts/ACC-1001"
    }
  }
}
```

Caching

HTTP caching (RFC 9111) is the cheapest performance work an API can do, because clients and proxies already implement it. The server's job is to say, for every response, whether it may be stored and for how long, with `Cache-Control`:

RESOURCE	CACHE-CONTROL	WHY
Accounts, balances, orders	no-cache, no-store, max-age=0, must-revalidate	Private and changing; Spring Security's default
Quotes	max-age=1, private	Changes every second; licensed per user
The OpenAPI document	max-age=3600	Public, and changes only with a release

Spring Security writes the strict `no-store` policy on every response unless the application sets its own header. That is a safe default for an API full of personal data, and even reference data such as an instrument inherits it:

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/hal+json
Cache-Control: no-cache, no-store, max-age=0, must-revalidate

{
  "_links": {
    "self": {
      "href": "http://localhost:8080/instruments/EQ-AAPL"
    },
    "bk:quote": {
      "href": "http://localhost:8080/instruments/EQ-AAPL/quote"
    },
    "bk:option-chain": {
      "href": "http://localhost:8080/instruments/EQ-AAPL/option-chain{?expiration}",
      "templated": true
    }
  },
  "id": "EQ-AAPL",
  "symbol": "AAPL",
  "name": "Apple Inc.",
  "type": "EQUITY",
  "exchange": "XNAS",
  "currency": "USD",
  "tradable": true,
  "fractionable": true,
  "multiplier": "1"
}

```

Where caching is safe, the controller opts in explicitly, as the quote endpoint does (Chapter 8) and as the contract does:

HTTP

```

HEAD /openapi.yaml HTTP/1.1
Host: localhost:8080
API-Version: 2
-----
HTTP/1.1 200 OK
Content-Type: application/openapi+yaml
Cache-Control: max-age=3600

```

Validation complements expiry. When a cached copy is stale, the client revalidates it by sending its `ETag` in `If-None-Match`, and a `304` costs almost nothing. Orders and quotes both support it; you saw both 304s in earlier chapters. Collections do not carry ETags in this API: a page of a changing list is cheap to regenerate and rarely identical twice.

THE STANDARD

1. Page unbounded collections with opaque cursors and a `next` link; bound the page size and reject bad cursors with 400.
2. Sort in one stable order; filter with optional query parameters declared in the link template.
3. Every response states its cacheability; personal data is `no-store`, licensed data is `private`.
4. Resources that clients poll carry an `ETag` and answer `If-None-Match` with 304.

Live order events with server-sent events

A trader wants to know the moment an order fills. Polling can tell them, one wasted request at a time. A stream tells them once, as it happens, and with the right design it never loses an event across a reconnect.

Choosing a push technology

OPTION	STRENGTHS	COSTS
Polling	Trivial; works everywhere; cheap with ETags	Latency equals the interval; wasted requests
Server-sent events	Plain HTTP; one-way push; built-in event IDs and reconnection with <code>Last-Event-ID</code>	Server to client only; one long-lived connection per client
WebSockets	Two-way, low overhead per message	A separate protocol; reconnection and replay are yours to build
Webhooks	No open connection; good for server-to-server	The client must host a public endpoint and verify signatures

Order updates flow one way, from the brokerage to the client, and each one matters. Server-sent events (SSE), defined in the WHATWG HTML standard, fit both needs. They run over ordinary HTTP, through the same authentication and proxies as the rest of the API, and they were designed with replay in mind.

The event stream

A client opens the stream with an ordinary `GET` and `Accept: text/event-stream`, and the response simply never ends. Each event is a few `field:value` lines followed by a blank line. Here is the stream while a market order for NVDA is placed and fills:

```
GET /accounts/ACC-1001/order-events HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJrawQiOiI3MTE1NmVl...
Accept: text/event-stream
```

```
-----
HTTP/1.1 200 OK
```

```
Content-Type: text/event-stream
```

```
id:7
event:order.created
data:{"orderId":"ORD-100004","status":"OPEN","filledQuantity":"0","remainingQuantity":"4","occurredAt":"2026-09-29T10:45:35.143433Z","_links":{"order":{"href":"http://localhost:8080/accounts/ACC-1001/orders/ORD-100004"}}}

id:8
event:order.filled
data:{"orderId":"ORD-100004","status":"FILLED","filledQuantity":"4","remainingQuantity":"0","averageFillPrice":"185.5800","occurredAt":"2026-09-29T10:45:35.249581Z","_links":{"order":{"href":"http://localhost:8080/accounts/ACC-1001/orders/ORD-100004"}}}
```

`id` numbers every event, `event` names its type, and `data` carries a JSON summary of the order. The summary is deliberately small, and it links to the full order, so the event stream is hypermedia too: a client that needs every detail follows `_links.order`.

EVENT TYPE	PUBLISHED WHEN
order.created	An order is placed.
order.replaced	A merge patch changed a working order.
order.partially-filled	An execution filled part of the order.
order.filled	The order is completely filled.
order.cancel-requested	A cancellation was accepted (202).
order.cancelled	The cancellation completed.

Never missing an event

Connections drop. When a browser's `EventSource` reconnects, it automatically sends the ID of the last event it received in a `Last-Event-ID` header, and any client can do the same. The server keeps a short history of each account's events and replays everything after that ID before streaming new ones:

```
GET /accounts/ACC-1001/order-events HTTP/1.1
Host: localhost:8080
Authorization: Bearer eyJraWQiOiI3MTE1NmVl...
Accept: text/event-stream
Last-Event-ID: 7
```

```
-----
HTTP/1.1 200 OK
```

```
Content-Type: text/event-stream
```

```
id:8
event:order.filled
data:{"orderId":"ORD-100004","status":"FILLED","filledQuantity":"4","remainingQuantity":"0","averageFillPrice":"185.5800","occurredAt":"2026-09-29T10:45:35.249581Z","_links":{"order":{"href":"http://localhost:8080/accounts/ACC-1001/orders/ORD-100004"}}}
```

api/orders/OrderEvents.java

```
/** Opens a stream for one account, first replaying anything after lastEventId. */
public SseEmitter subscribe(String accountId, Long lastEventId, Function<Event, Object> renderer) {
    Subscriber subscriber = new Subscriber(new SseEmitter(TIMEOUT_MILLIS), renderer);
    subscriber.emitter().onCompletion(() -> unsubscribe(accountId, subscriber));
    subscriber.emitter().onTimeout(() -> unsubscribe(accountId, subscriber));
    if (lastEventId != null) {
        replay(accountId, lastEventId).forEach(event -> send(accountId, subscriber, event));
    }
    subscribers.computeIfAbsent(accountId, id -> new CopyOnWriteArrayList<>()).add(subscriber);
    return subscriber.emitter();
}
```

api/orders/OrderEvents.java

```
/** Events for an account with an ID greater than lastEventId, oldest first. */
public List<Event> replay(String accountId, long lastEventId) {
    Deque<Event> events = history.getOrDefault(accountId, new ArrayDeque<>());
    synchronized (events) {
        return events.stream().filter(event -> event.id() > lastEventId).toList();
    }
}
```

The controller checks ownership exactly as every other account endpoint does, then hands the subscription to `OrderEvents`. Note the base URI captured at subscription time: events are sent later, outside the original request, where the current request is no longer available to build links from.

api/orders/OrderEventController.java

```
@GetMapping(path = "/accounts/{accountId}/order-events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter events(@PathVariable String accountId,
    @RequestHeader(name = "Last-Event-ID", required = false) Long lastEventId,
    @AuthenticationPrincipal Jwt jwt) {
    ledger.account(Caller.of(jwt).customerId(), accountId);
    // Events are sent later, outside this request, so capture the base URI now.
    String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
    return events.subscribe(accountId, lastEventId, event -> render(base, event));
}
```

Running streams in production

- **History has a horizon.** This implementation keeps the last 500 events per account. A client that has been away longer should re-read its orders, then resume the stream.
- **Proxies buffer.** Some reverse proxies hold responses until they end, which never happens here. Disable response buffering for the stream's path.
- **Idle connections die.** Load balancers close quiet connections. Streams that may be idle for long should send a periodic comment line (a line starting with a colon) as a heartbeat.
- **One process is not a cluster.** Here, events live in memory. With several instances, publish them through a message broker so every instance can serve every subscriber, and replay from a durable log.

THE STANDARD

1. Push state changes as server-sent events over the same authenticated HTTP as the rest of the API.
2. Every event has an increasing `id` and a typed `event` name; reconnects with `Last-Event-ID` replay what was missed.
3. Event data is a small summary that links to the full resource.
4. Document the replay horizon, and what a client should do beyond it.

Evolution: versions, deprecation and rate limits

An API that never changes is dead, and an API that breaks its clients is soon abandoned. The craft lies between: evolve without versions whenever you can, version precisely when you must, and tell clients well ahead of time what is going away.

Change without breaking

Most changes need no version at all, provided clients are tolerant readers (Chapter 6). These are safe:

- Adding a property to a representation, or a new link to its `_links`.
- Adding a resource, a link relation, an optional query parameter or a new problem type.
- Adding a value to an enumeration that clients were told to treat as open-ended.
- Moving a resource to a new URI, for clients that follow links. This is the change hypermedia makes free.

These break clients: removing or renaming a property, changing its type or shape, changing what a value means, adding a required input, or tightening validation. The brokerage has one breaking change in its history. Version 1 of a quote had flat `bid` and `bidSize` properties; version 2 groups each side into a price level and adds the day's change:

`api/market/Quotes.java`

```
/**
 * The two representations of a quote. Version 2 regroups price and size into bid and ask
 * objects and adds the day's change: a breaking change of shape, so it needs a new API
 * version rather than a new optional field.
 */
public final class Quotes {

    private Quotes() {}

    /** API version 1 (deprecated): flat fields. */
    @Relation(itemRelation = "quote")
    public record QuoteV1(String instrumentId, String symbol, BigDecimal last, BigDecimal bid,
        BigDecimal ask, BigDecimal bidSize, BigDecimal askSize, long volume, Instant asOf) {

        public static QuoteV1 of(Instrument instrument, MarketData.Tick tick) {
            return new QuoteV1(instrument.id(), instrument.symbol(), tick.last(), tick.bid(), tick.ask(),
                tick.bidSize(), tick.askSize(), tick.volume(), tick.asOf());
        }
    }

    /** API version 2: bid and ask as price levels, plus the change since the previous close. */
    @Relation(itemRelation = "quote")
    public record Quote(String instrumentId, String symbol, BigDecimal last, PriceLevel bid, PriceLevel ask,
        BigDecimal change, BigDecimal changePercent, long volume, Instant asOf) {

        public record PriceLevel(BigDecimal price, BigDecimal size) {}
    }
}
```

```

    public static Quote of(Instrument instrument, MarketData.Tick tick) {
        BigDecimal change = tick.last().subtract(tick.previousClose());
        BigDecimal percent = change.multiply(BigDecimal.valueOf(100))
            .divide(tick.previousClose(), 2, RoundingMode.HALF_EVEN);
        return new Quote(instrument.id(), instrument.symbol(), tick.last(),
            new PriceLevel(tick.bid(), tick.bidSize()), new PriceLevel(tick.ask(), tick.askSize()),
            change, percent, tick.volume(), tick.asOf());
    }
}

```

Where to put the version

STRATEGY	EXAMPLE	TRADE-OFF
URI path	/v2/instruments/...	Visible and simple, but every resource gets one URI per version, and every link must know the version.
Query parameter	?version=2	Easy to add, easy to lose when a client follows a link.
Request header	API-Version: 2	One URI per resource; links stay version-free; the client pins its version once.
Media type parameter	application/hal+json;v=2	Precise per representation, but awkward in tooling and caches.

For a hypermedia API the header wins, because links must work for every client regardless of version. The brokerage uses the API versioning support new in Spring Framework 7: one line chooses the header, and handler mappings declare the version they serve.

api/platform/WebConfig.java

```

@Override
public void configureApiVersioning(ApiVersionConfigurer versions) {
    StandardApiVersionDeprecationHandler deprecations = new StandardApiVersionDeprecationHandler();
    deprecations.configureVersion("1")
        .setDeprecationDate(ZonedDateTime.parse("2026-09-01T00:00:00Z"))
        .setSunsetDate(ZonedDateTime.parse("2027-03-01T00:00:00Z"))
        .setSunsetLink(URI.create("https://docs.brokerage.example/migrations/v2"));

    versions.useRequestHeader(VERSION_HEADER)
        .addSupportedVersions("1", "2")
        .setDefaultVersion("1")
        .setDeprecationHandler(deprecations);
}

```

api/market/InstrumentController.java

```

/** Version 1 of a quote (deprecated; see WebConfig). */
@GetMapping(path = "/instruments/{instrumentId}/quote", version = "1")
public ResponseEntity<EntityModel<QuoteV1>> quoteV1(@PathVariable String instrumentId,
    WebRequest request) {
    Instrument instrument = market.instrument(instrumentId);
    MarketData.Tick tick = market.tick(instrumentId);
    return quoteResponse(instrument, tick, QuoteV1.of(instrument, tick), request);
}

```

```
api/market/InstrumentController.java
```

```
@GetMapping(path = "/instruments/{instrumentId}/quote", version = "2")
public ResponseEntity<EntityModel<Quote>> quote(@PathVariable String instrumentId,
        WebRequest request) {
    Instrument instrument = market.instrument(instrumentId);
    MarketData.Tick tick = market.tick(instrumentId);
    return quoteResponse(instrument, tick, Quote.of(instrument, tick), request);
}
```

Mappings without a version, which is every other endpoint, serve all versions. A version the server does not know is refused, and the problem lists the ones it does:

HTTP

HTTP/1.1 400 Bad Request

Content-Type: application/problem+json

```
{
  "detail": "API version 9.0.0 is not supported",
  "instance": "/instruments/EQ-AAPL/quote",
  "status": 400,
  "title": "The requested API version is not supported",
  "type": "https://docs.brokerage.example/problems/unsupported-api-version",
  "requestId": "0c4b3419-6d67-4906-a629-d89d26468f8c",
  "supportedVersions": [
    "1",
    "2"
  ]
}
```

Deprecation and sunset

Version 1 is still the default, so that clients written before version 2 existed keep working. It is also deprecated, and every response in version 1 says so with three standard header fields:

HTTP

no API-Version header: version 1, with warnings

GET /instruments/EQ-AAPL/quote HTTP/1.1

Host: localhost:8080

Authorization: Bearer eyJraWQiOiI3MTE1NmVl...

HTTP/1.1 200 OK

ETag: "q3"

Content-Type: application/hal+json

Cache-Control: max-age=1, private

Deprecation: @1788220800

Sunset: Mon, 1 Mar 2027 00:00:00 GMT

Link: <https://docs.brokerage.example/migrations/v2>; rel="sunset"; type="text/html"

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/instruments/EQ-AAPL/quote"
    },
    "bk:instrument": {
      "href": "http://localhost:8080/instruments/EQ-AAPL"
    }
  },
}
```

```

"instrumentId": "EQ-AAPL",
"symbol": "AAPL",
"last": "232.78",
"bid": "232.76",
"ask": "232.80",
"bidSize": "500",
"askSize": "500",
"volume": 1250184,
"asOf": "2026-09-29T10:45:28.216609Z"
}

```

Deprecation: @1788220800 (RFC 9745)

The moment the version was deprecated, as a Unix timestamp: September 1, 2026. It still works; it will not improve.

Sunset: Mon, 1 Mar 2027 00:00:00 GMT (RFC 8594)

When it is expected to stop working.

Link: <.../migrations/v2>; rel="sunset"

Where to read about the move.

Because the version is a property of the whole contract, a version 1 client sees the warning on *every* response, not just on quotes. That is by design: the client, not a single endpoint, must move. Clients log and alert on `Deprecation` headers, and they pin a version on every request, as the book's client does, so that a change of default can never change what they receive:

client/HypermediaClient.java

```

public HypermediaClient(RestClient.Builder builder) {
    this.json = JsonMapper.builder()
        // Tolerant reader: new properties on the server must never break this client.
        .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
        // The API's convention: decimals are strings, and absent fields are omitted.
        .withConfigOverride(BigDecimal.class,
            o -> o.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)))
        .changeDefaultPropertyInclusion(i -> i.withValueInclusion(JsonInclude.Include.NON_NULL))
        .build();
    this.http = builder
        .defaultHeader(HttpHeaders.ACCEPT, HAL_JSON + ", application/problem+json")
        // Pin the API version on every request, so a new default can never change
        // what this client receives.
        .defaultHeader("API-Version", "2")
        .defaultStatusHandler(HttpStatusCode::isError, (request, response) -> {
            throw ApiProblem.from(response, json);
        })
        .build();
}

```

Rate limits

Quotas keep one client's bug from becoming every client's outage. The brokerage applies two per-client policies in fixed one-minute windows: `default`, for every request, and a stricter `orders` policy for requests that change orders.

```
brokerage-api/src/main/resources/application.yaml
```

```
rate-limit:
  requests-per-minute: 120
  order-changes-per-minute: 30
```

A quota is only fair if clients can see it before they hit it. Every response advertises the policy and what is left of it, using the `RateLimit-Policy` and `RateLimit` fields from the IETF HTTP APIs working group's draft: `q` is the quota, `w` the window in seconds, `r` the requests remaining and `t` the seconds until the window resets. To show a limit being reached, these captures come from a second API instance configured with a quota of two requests per minute:

HTTP

```
HTTP/1.1 200 OK
Content-Type: application/hal+json
RateLimit-Policy: "default";q=2;w=60
RateLimit: "default";r=1;t=14

{
  "_embedded": {
    "bk:accounts": [
      /* 2 more */
    ]
  },
  "_links": {
    "self": {
      "href": "http://localhost:8081/accounts"
    },
    "curies": [
      {
        "href": "https://docs.brokerage.example/rels/{rel}",
        "name": "bk",
        "templated": true
      }
    ]
  }
}
```

HTTP

the third request in the window

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/problem+json
Retry-After: 14
RateLimit-Policy: "default";q=2;w=60
RateLimit: "default";r=0;t=14

{
  "detail": "The default request quota for this client is used up; retry in 14 seconds",
  "instance": "/accounts",
  "status": 429,
  "title": "Too many requests",
  "type": "https://docs.brokerage.example/problems/rate-limited",
  "requestId": "62fcd984-eda4-4ac6-bf00-7f4d40732149"
}
```

```

@Override
public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
    Authentication caller = SecurityContextHolder.getContext().getAuthentication();
    if (caller == null || !caller.isAuthenticated()) {
        return true;
    }
    boolean changesOrders = !HttpMethod.GET.matches(request.getMethod())
        && request.getRequestURI().contains("/orders");
    String policy = changesOrders ? "orders" : "default";
    int quota = changesOrders ? quotas.orderChangesPerMinute() : quotas.requestsPerMinute();

    long now = clock.instant().getEpochSecond();
    long windowStart = now - (now % WINDOW_SECONDS);
    Window window = windows.compute(caller.getName() + "|" + policy,
        (key, existing) -> existing == null || existing.start() != windowStart
            ? new Window(windowStart, 1) : new Window(windowStart, existing.used() + 1));

    long resetIn = windowStart + WINDOW_SECONDS - now;
    int remaining = Math.max(0, quota - window.used());
    response.setHeader("RateLimit-Policy", "\"%s\";q=%d;w=%d".formatted(policy, quota, WINDOW_SECONDS));
    response.setHeader("RateLimit", "\"%s\";r=%d;t=%d".formatted(policy, remaining, resetIn));
    if (window.used() > quota) {
        throw new RateLimitedException(policy, resetIn);
    }
    return true;
}

```

A well-behaved client slows down as `r` approaches zero, waits out `Retry-After` when it does not, and retries with the same idempotency key (Chapter 14). A fixed window is the simplest fair algorithm; a production gateway would typically use a sliding window or token bucket, and share its counters across instances.

THE STANDARD

1. Prefer additive, backward-compatible change; clients must tolerate new properties, links and values.
2. Version the contract with a request header, never the path; unknown versions are 400 with the supported list.
3. Announce retirement with `Deprecation`, `Sunset` and a `rel="sunset"` link; clients pin their version.
4. Every response advertises its quota with `RateLimit-Policy` and `RateLimit`; exceeding it is 429 with `Retry-After`.

Securing the API with OAuth 2.1

A brokerage API must answer two questions on every request: who is this client acting for, and what has it been allowed to do? OAuth 2.1 answers both with a signed token, and never lets the API see a password.

Four roles

OAuth names four parties. The **resource owner** is the customer, Alice. The **client** is the software acting for her: a web app, her own script, an AI agent. The **authorization server** authenticates her, asks her consent, and issues tokens. The **resource server** is the brokerage API, which accepts tokens and enforces what they allow. OAuth 2.1, an IETF draft that consolidates a decade of OAuth 2.0 security guidance, removes the risky parts of the original: no implicit grant, no password grant, PKCE for every authorization code, exact redirect URI matching, and refresh tokens that are rotated or bound to a key.

Three clients, three patterns

CLIENT	GRANT	PATTERN AND LIMITS
brokerage-web	Authorization code + PKCE, with consent; refresh tokens	A web app's backend (a “backend for frontend”). A confidential client: tokens stay on the server, and the browser holds only a session cookie. Access tokens last 10 minutes; refresh tokens 8 hours, rotated on every use.
brokerage-cli	Client credentials	A customer's own script, registered to that customer, like a personal API key. Tokens last 15 minutes.
brokerage-agent	Client credentials	The AI agent (Chapter 25). Registered to the customer, with tokens that last 5 minutes and a <code>trading_limit</code> of 5,000 USD per order.

The development authorization server is Spring Authorization Server, part of Spring Security 7. Its clients are declared in configuration:

`auth-server/src/main/resources/application.yaml`

```
client:
  # A customer's own script or trading bot: the "personal API key" pattern.
  brokerage-cli:
    registration:
      client-id: brokerage-cli
      client-secret: "{noop}cli-secret"
      client-authentication-methods: [client_secret_basic]
      authorization-grant-types: [client_credentials]
      scopes: [accounts:read, market-data:read, orders:read, orders:write]
    token:
      access-token-time-to-live: 15m
  # The AI agent: same customer, but every order is capped by trading_limit.
  brokerage-agent:
    registration:
```

```

client-id: brokerage-agent
client-secret: "{noop}agent-secret"
client-authentication-methods: [client_secret_basic]
authorization-grant-types: [client_credentials]
scopes: [accounts:read, market-data:read, orders:read, orders:write]
token:
  access-token-time-to-live: 5m
# A web app's backend (a "backend for frontend"): a confidential client that signs
# the customer in with the authorization code grant, PKCE and consent. Tokens stay
# on the server; the browser holds only a session cookie. Spring Authorization
# Server issues refresh tokens to confidential clients only.
brokerage-web:
  registration:
    client-id: brokerage-web
    client-secret: "{noop}web-secret"
    client-authentication-methods: [client_secret_basic]
    authorization-grant-types: [authorization_code, refresh_token]
    redirect-uris: [http://127.0.0.1:3000/callback]
    scopes: [openid, accounts:read, market-data:read, orders:read, orders:write]
  require-proof-key: true
  require-authorization-consent: true
  token:
    access-token-time-to-live: 10m
    refresh-token-time-to-live: 8h
    reuse-refresh-tokens: false

```

Why the web app is a confidential client. A single-page app running only in the browser is a *public* client: it cannot keep a secret, and any token it holds is exposed to whatever script runs on the page. The IETF's guidance for browser-based apps recommends a backend-for-frontend instead: a small server that performs the OAuth flow as a confidential client and gives the browser only an `HttpOnly` session cookie. Spring Authorization Server agrees in its own way: it issues refresh tokens to confidential clients only.

What a token says

The API trusts only what the authorization server signed. Beyond the standard claims, the brokerage's tokens carry two of its own, added by a token customizer:

auth/AuthServerApplication.java

```
/** Stamps the audience, customer_id and (if configured) trading_limit into every access token. */
@Bean
OAuth2TokenCustomizer<JwtEncodingContext> brokerageClaims(Identities identities) {
    return context -> {
        if (!OAuth2TokenType.ACCESS_TOKEN.equals(context.getTokenType())) {
            return;
        }
        // Tokens are minted for one API, which checks that it is the intended audience.
        context.getClaims().audience(List.of("brokerage-api"));
        // The principal is the signed-in user, or the client itself for client_credentials.
        String principal = context.getPrincipal().getName();
        String customerId = identities.customers().get(principal);
        if (customerId != null) {
            context.getClaims().claim("customer_id", customerId);
        }
        String clientId = context.getRegisteredClient().getClientId();
        BigDecimal limit = identities.tradingLimits().get(clientId);
        if (limit != null) {
            context.getClaims().claim("trading_limit", limit.toPlainString());
        }
    };
}
```

auth-server/src/main/resources/application.yaml

```
brokerage:
  identities:
    customers:
      alice: cust-1001
      brokerage-cli: cust-1001
      brokerage-agent: cust-1001
    trading-limits:
      brokerage-agent: 5000
```

Alice, through brokerage-web

```
{
  "sub": "alice",
  "aud": "brokerage-api",
  "nbf": 1790678743,
  "scope": [
    "orders:read",
    "market-data:read",
    "openid",
    "accounts:read"
  ],
  "iss": "http://localhost:9000",
  "exp": 1790679343,
  "customer_id": "cust-1001",
  "iat": 1790678743,
  "jti": "9fc1ec32-9095-444f-8610-b029056e33db"
}
```

the agent, through brokerage-agent

```
{
  "sub": "brokerage-agent",
  "aud": "brokerage-api",
  "nbf": 1790678728,
  "scope": [
    "orders:read",
    "market-data:read",
    "orders:write",
    "accounts:read"
  ],
  "iss": "http://localhost:9000",
  "trading_limit": "5000",
  "exp": 1790679028,
  "customer_id": "cust-1001",
  "iat": 1790678728,
  "jti": "a925c8ce-4dfd-49c1-8a84-c943cb5bfb2b"
}
```

The two tokens act for the same customer, `cust-1001`, with different authority. Alice's browser session was granted read scopes only, because that is what the web app asked for and she consented to. The agent may trade, but only up to its `trading_limit`. Chapter 25 shows the API enforcing it.

The browser flow: authorization code with PKCE

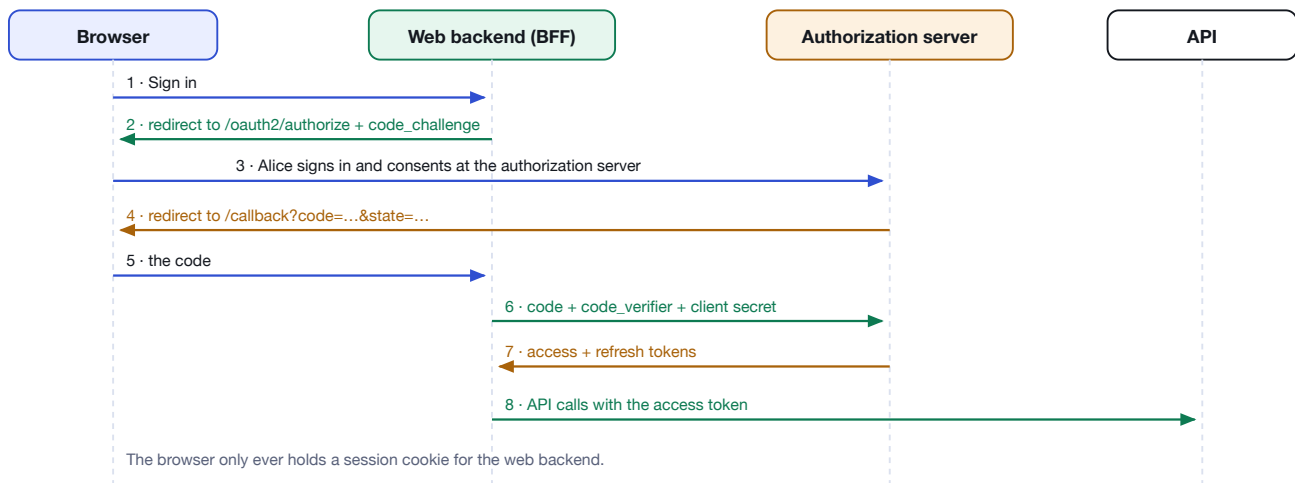


Figure 20.1 — The authorization code grant with PKCE, through a backend for frontend. The `code_verifier` is known only to the backend, so an intercepted code is useless without it.

PKCE (RFC 7636) binds the code to the client that asked for it. Before redirecting, the backend creates a random `code_verifier` and sends its SHA-256 hash as the `code_challenge`. Here is the authorization request, captured from the sample server:

HTTP

```
GET /oauth2/authorize?response_type=code&client_id=brokerage-web&scope=openid%20accounts:read%20market-
data:read%20orders:read&redirect_uri=http://127.0.0.1:3000/callback&state=af0ifjsldkj&code_challenge=bsRU3Yzij
izgUG3WIskNMLooqcx3_TCXps9JbAEFzPA&code_challenge_method=S256 HTTP/1.1
Host: localhost:9000
```

After Alice signs in and consents, the authorization server redirects back with a short-lived code and the `state` the backend chose, which it checks to defeat cross-site request forgery:

HTTP

```
HTTP/1.1 302 Found
Location: http://127.0.0.1:3000/callback?code=EEkhB9NTzLUBU2HHPgP85Rgh...tYp105&state=af0ifjsldkj
```

The backend exchanges the code, proving possession of the verifier and authenticating with its own secret:

HTTP

tokens shortened for print

```

POST /oauth2/token HTTP/1.1
Host: localhost:9000
Authorization: Basic YnJva2VyYWdlLXdlYjp3ZWItc2VjcmV0
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=EekhB9NTzL...
&redirect_uri=http://127.0.0.1:3000/callback&code_verifier=jNo6pGT_KQEd...
-----
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
Cache-Control: no-cache, no-store, max-age=0, must-revalidate

{
  "access_token": "eyJraWQiOiI3MTE1NmVlYi02...",
  "refresh_token": "wk_7mKDJwLgXXbM2JwMWLnJF...",
  "scope": "orders:read market-data:read openid accounts:read",
  "id_token": "eyJraWQiOiI3MTE1NmVlYi02...",
  "token_type": "Bearer",
  "expires_in": 599
}

```

When the access token expires, the backend uses the refresh token, and receives a *new* refresh token each time, because the server is configured not to reuse them. A stolen refresh token therefore works at most once, and its use by an attacker invalidates the legitimate chain:

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8

{
  "access_token": "eyJraWQiOiI3MTE1NmVlYi02...",
  "refresh_token": "Alt22befVQCIqaNDpF10gvmf...",
  "scope": "orders:read market-data:read openid accounts:read",
  "id_token": "eyJraWQiOiI3MTE1NmVlYi02...",
  "token_type": "Bearer",
  "expires_in": 599
}

```

The resource server

The API validates each token's signature against the authorization server's published keys, checks its issuer and expiry, and checks that it was minted for this API:

```
brokerage-api/src/main/resources/application.yaml
```

```

security:
  oauth2:
    resourceserver:
      jwt:
        issuer-uri: http://localhost:9000
        # Tokens must be minted for this API, not merely by a trusted issuer.
        audiences: brokerage-api

```

Then one configuration method decides, for every operation, which scope it requires. Keeping every rule in one place makes the security model reviewable at a glance:

api/platform/SecurityConfig.java

```
public static final String MARKET_DATA_READ = "SCOPE_market-data:read";
public static final String ACCOUNTS_READ = "SCOPE_accounts:read";
public static final String ORDERS_READ = "SCOPE_orders:read";
public static final String ORDERS_WRITE = "SCOPE_orders:write";

/** Discovery documents that anyone may read, with or without a token. */
private static final String[] PUBLIC = { "/", "/openapi.yaml", "/actuator/health/**",
    "/.well-known/oauth-protected-resource" };
```

api/platform/SecurityConfig.java

```
@Bean
SecurityFilterChain api(HttpSecurity http, ProblemResponses problems,
    @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer)
    throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, PUBLIC).permitAll()
            .requestMatchers(HttpMethod.HEAD, PUBLIC).permitAll()
            .requestMatchers(HttpMethod.GET, "/instruments/**").hasAuthority(MARKET_DATA_READ)
            .requestMatchers(HttpMethod.GET, "/accounts/*/orders/**", "/accounts/*/order-events")
                .hasAuthority(ORDERS_READ)
            .requestMatchers(HttpMethod.POST, "/accounts/*/order-previews")
                .hasAuthority(ORDERS_READ)
            .requestMatchers("/accounts/*/orders/**").hasAuthority(ORDERS_WRITE)
            .requestMatchers(HttpMethod.GET, "/accounts/**").hasAuthority(ACCOUNTS_READ)
            .anyRequest().denyAll())
        .oauth2ResourceServer(server -> server
            .jwt(Customizer.withDefaults())
            // RFC 9728: publish which authorization server and scopes this API uses.
            .protectedResourceMetadata(metadata -> metadata.protectedResourceMetadataCustomizer(
                resource -> resource.resourceName("Brokerage API")
                    .authorizationServer(issuer)
                    .tlsClientCertificateBoundAccessTokens(false)
                    .scopes(scopes -> scopes.addAll(List.of("market-data:read",
                        "accounts:read", "orders:read", "orders:write")))))
            .authenticationEntryPoint(problems)
            .accessDeniedHandler(problems))
        // Bearer tokens are sent explicitly, never automatically like cookies, so
        // there is no cross-site request forgery to defend against, and no session.
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .build();
}
```

Beyond scopes, controllers turn the token into a `Caller`, whose customer ID scopes every query and whose trading limit caps every order:

api/platform/Caller.java

```
/**
 * Who is calling, as established by the access token: the customer whose data may be
 * read, the client application acting for them, what it may do, and any trading limit
 * the customer delegated to that client.
 */
```

```

public record Caller(String customerId, String clientId, Collection<String> scopes,
    Optional<BigDecimal> tradingLimit) {

    public static Caller of(Jwt jwt) {
        // A valid token that is not bound to a customer can see no accounts at all.
        String customerId = Optional.ofNullable(jwt.getClaimAsString("customer_id")).orElse("none");
        // "scope" may be a space-delimited string (RFC 6749) or a JSON array.
        Collection<String> scopes = jwt.getClaim("scope") instanceof String text
            ? List.of(text.split(" "))
            : Optional.ofNullable(jwt.getClaimAsStringList("scope")).orElse(List.of());
        Optional<BigDecimal> limit = Optional.ofNullable(jwt.getClaimAsString("trading_limit"))
            .map(BigDecimal::new);
        return new Caller(customerId, jwt.getSubject(), scopes, limit);
    }

    public boolean canTrade() {
        return scopes.contains("orders:write");
    }

    public boolean canReadOrders() {
        return scopes.contains("orders:read");
    }
}

```

Discovery: RFC 9728

How does a client that has never seen this API find out where to get a token? RFC 9728, *OAuth 2.0 Protected Resource Metadata*, answers it. The API publishes a metadata document naming its authorization servers and scopes, and points to it from the challenge in every 401 (Chapter 16) and from the API root.

HTTP

```

HTTP/1.1 200 OK
Content-Type: application/json

{
  "resource": "http://localhost:8080",
  "bearer_methods_supported": [
    "header"
  ],
  "tls_client_certificate_bound_access_tokens": false,
  "resource_name": "Brokerage API",
  "authorization_servers": [
    "http://localhost:9000"
  ],
  "scopes_supported": [
    "market-data:read",
    "accounts:read",
    "orders:read",
    "orders:write"
  ]
}

```

A generic client, or an AI agent, can go from a 401 to a token without any documentation: read `resource_metadata` from the challenge, fetch the metadata, fetch the authorization server's own metadata (RFC 8414) from `/.well-known/oauth-authorization-server`, and start the right flow.

Before production

- **TLS everywhere.** The samples use plain HTTP on localhost only.
- **Durable, rotated keys.** The development server generates a new signing key at each start.
- **Sender-constrained tokens** with DPoP (RFC 9449) or mutual TLS (RFC 8705), so a stolen token is useless on another machine.
- **Secrets in a vault,** never in configuration files, and never tokens in URLs or logs.
- **Step-up and consent** for sensitive actions, and revocation when a device is lost.

THE STANDARD

1. Authenticate every request with an OAuth 2.1 bearer token; browsers go through a backend for frontend with PKCE.
2. Tokens are audience-restricted, short-lived and carry fine-grained scopes; refresh tokens rotate.
3. Each operation's required scope is declared in one place; every query is scoped to the token's customer.
4. Publish RFC 9728 metadata and point to it from every 401 challenge.

Contracts, Clients and Agents

The API described precisely, tested against its own description, consumed by a client that follows links, compared with a real RPC-style brokerage API, and finally handed to an AI agent, with a human keeping the final word on every trade.



The OpenAPI 3.2 contract

Hypermedia tells a client what it can do right now. A contract tells a developer, a code generator and a test suite what the API can do at all. A serious API needs both, and they must never disagree.

Links and contracts are complementary

An OpenAPI document describes every operation, parameter, header, schema and error an API offers. It powers documentation sites, mock servers, client generators, gateway configuration and contract tests. It does not replace hypermedia: the paths in the document are documentation of where resources currently live, not an invitation to build URLs. The brokerage's contract says so in its description, and links to it from the API root with the registered `service-desc` relation, so a client can find the contract the same way it finds everything else.

What is new in OpenAPI 3.2

OpenAPI 3.2.0 was published in September 2025, and 3.2.1 followed as a patch release. The brokerage's contract uses five of the version's additions:

FEATURE	WHAT IT DOES
<code>\$self</code>	The document's own URI, the base for resolving its references, even when copied elsewhere.
<code>tags: summary, parent, kind</code>	Hierarchical tags: <i>Orders</i> sits under <i>Trading</i> ; <code>kind</code> separates navigation from badges such as <i>Deprecated</i> .
<code>itemSchema</code>	Describes each item of a streaming response, such as each server-sent event.
<code>oauth2MetadataUrl</code>	Points an OAuth security scheme at the authorization server's RFC 8414 metadata.
<code>dataValue, serializedValue</code>	Examples that separate the data from its serialized form.

The version also adds support for the `QUERY` method, query-string parameters described as a whole, and the OAuth device authorization flow, none of which the brokerage needs.

A tour of the contract

The document opens with its identity and a summary of the conventions every operation follows, so a reader of the rendered documentation learns the rules before the details:

```
openapi/brokerage-api.yaml
```

```
openapi: 3.2.1
$self: https://api.brokerage.example/openapi.yaml

info:
  title: Brokerage API
```

```

version: "2.0"
summary: Accounts, positions, market data and trading for a retail brokerage, as hypermedia.
description: |
  A resource-oriented REST API that follows the standards in *REST API Standards with
  HATEOAS* (Smiroh Publishers).

  * **Hypermedia.** Every representation is HAL (`application/hal+json`) with a `_links`
  object. Ask for `application/prs.hal-forms+json` to also receive `_templates`: the
  state transitions available right now. Clients should bookmark only the API root and
  follow links from there. Custom link relations use the CURIE `bk:`, which expands to
  `https://docs.brokerage.example/rels/{rel}`.
  * **Money.** Prices, amounts and quantities are decimal strings, never JSON numbers.
  * **Errors.** Every error is an RFC 9457 problem document (`application/problem+json`)
  whose `type` URI identifies the error. Branch on `type`, never on `title` or `detail`.
  * **Versions.** Send `API-Version: 2` to opt in to version 2. Version 1 is the default and
  is deprecated: its responses carry `Deprecation` (RFC 9745) and `Sunset` (RFC 8594).
  * **Quotas.** Every response carries `RateLimit-Policy` and `RateLimit`. Exceeding a quota
  returns 429 with `Retry-After`.
  * **Tracing.** Every response echoes `X-Request-Id`; quote it in support requests.
contact:
  name: Brokerage API support
  url: https://docs.brokerage.example/support
license:
  name: Apache 2.0
  identifier: Apache-2.0

```

Hierarchical tags organize the operations for documentation tools:

```
openapi/brokerage-api.yaml
```

```

tags:
- name: discovery
  summary: Discovery
  description: The API root and the protected resource metadata, where every client starts.
  kind: nav
- name: accounts
  summary: Accounts
  description: A customer's brokerage accounts.
  kind: nav
- name: holdings
  summary: Balances and positions
  parent: accounts
  kind: nav
- name: transactions
  summary: Transaction history
  parent: accounts
  kind: nav
- name: trading
  summary: Trading
  description: Orders, previews and live order events.
  kind: nav
- name: orders
  summary: Orders
  parent: trading
  kind: nav
- name: order-events
  summary: Order events
  parent: trading
  kind: nav
- name: market-data
  summary: Market data
  description: Instruments, quotes and option chains.
  kind: nav

```

```
- name: deprecated
  summary: Deprecated
  description: Operations or representations scheduled for removal.
  kind: badge
```

Each operation states its required scope, its headers, its request schema with an example, and every response it can produce, including the problem responses, which are defined once and referenced everywhere. OpenAPI's own `links` describe, at design time, what HAL's links provide at run time: from a new order to reading or canceling it.

openapi/brokerage-api.yaml

```
post:
  operationId: placeOrder
  summary: Place an order
  description: >-
    Every new order needs a unique Idempotency-Key. Retrying with the same key and body
    returns the original order (Idempotent-Replayed: true); the same key with a different
    body is refused.
  tags: [orders]
  security: [{ oauth2: [orders:write] }]
  parameters:
    - $ref: "#/components/parameters/Idempotency-Key"
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: "#/components/schemas/OrderRequest" }
        examples:
          limitBuy:
            summary: Buy 10 AAPL at 230.00 or better
            dataValue:
              instrumentId: EQ-AAPL
              side: BUY
              type: LIMIT
              timeInForce: DAY
              quantity: "10"
              limitPrice: "230.00"
              clientOrderId: rebalance-2026-09
  responses:
    "201":
      description: The order was accepted and is now working.
      headers:
        Location: { $ref: "#/components/headers/Location" }
        ETag: { $ref: "#/components/headers/ETag" }
        Idempotent-Replayed: { $ref: "#/components/headers/Idempotent-Replayed" }
      content:
        application/hal+json:
          schema: { $ref: "#/components/schemas/Order" }
      links:
        getOrder:
          operationId: getOrder
          parameters:
            accountId: $request.path.accountId
            orderId: $response.body#/id
        cancelOrder:
          operationId: cancelOrder
          parameters:
            accountId: $request.path.accountId
            orderId: $response.body#/id
    "400": { $ref: "#/components/responses/BadRequest" }
```

```
"403": { $ref: "#/components/responses/Forbidden" }
"404": { $ref: "#/components/responses/NotFound" }
"422": { $ref: "#/components/responses/UnprocessableContent" }
"429": { $ref: "#/components/responses/TooManyRequests" }
```

The event stream is where 3.2 shines. `itemSchema` describes each event, and the JSON inside its `data` field is described with `contentMediaType` and `contentSchema`:

openapi/brokerage-api.yaml

```
get:
  operationId: streamOrderEvents
  summary: Stream order events
  description: >-
    Server-Sent Events for every order in the account. Reconnect with Last-Event-ID to
    receive only the events you missed.
  tags: [order-events]
  security: [{ oauth2: [orders:read] }]
  parameters:
    - name: Last-Event-ID
      in: header
      schema: { type: integer, minimum: 0 }
  responses:
    "200":
      description: An unending stream of events.
      content:
        text/event-stream:
          itemSchema:
            type: object
            required: [id, event, data]
            properties:
              id: { type: string }
              event:
                type: string
                enum: [order.created, order.replaced, order.partially-filled, order.filled,
                  order.cancel-requested, order.cancelled]
              data:
                type: string
                contentMediaType: application/json
                contentSchema: { $ref: "#/components/schemas/OrderEventData" }
```

Two shared schemas carry the conventions of Part II and Part IV: the decimal string, and the problem document with its `errors` array:

openapi/brokerage-api.yaml

```
Decimal:
  description: An exact decimal number, as a string.
  type: string
  pattern: '^~?[0-9]+(\.[0-9]+)?$'
  examples: ["187.50"]
```

openapi/brokerage-api.yaml

```
Problem:
description: An RFC 9457 problem document. Extension members depend on the type.
type: object
required: [type, title, status]
properties:
  type: { type: string, format: uri }
  title: { type: string }
  status: { type: integer }
  detail: { type: string }
  instance: { type: string, format: uri-reference }
  requestId: { type: string }
errors:
  type: array
  items:
    type: object
    required: [field, message]
    properties:
      field: { type: string }
      message: { type: string }
```

And the security scheme names both OAuth flows, and where their metadata lives:

openapi/brokerage-api.yaml

```
securitySchemes:
  oauth2:
    type: oauth2
    description: >-
      OAuth 2.1. Web apps sign customers in through their backend, a confidential client,
      with the authorization code grant and PKCE; a customer's own scripts and agents use
      client credentials registered to that customer. Tokens carry `customer_id` and, for
      delegated clients, `trading_limit`.
    oauth2MetadataUrl: http://localhost:9000/.well-known/oauth-authorization-server
    flows:
      authorizationCode:
        authorizationUrl: http://localhost:9000/oauth2/authorize
        tokenUrl: http://localhost:9000/oauth2/token
        refreshToken: http://localhost:9000/oauth2/token
```

Shipping the contract with the code

The contract lives in its own folder, reviewed like code, and the build copies it into the API's jar so the application serves the very document it was tested against:

brokerage-api/pom.xml

```
<!-- The OpenAPI contract ships inside the application and is served at /openapi.yaml -->
<resource>
  <directory>${project.basedir}/../openapi</directory>
  <targetPath>contract</targetPath>
</resource>
```

api/RootController.java

```
/** The OpenAPI contract, served by the application it describes so the two ship together. */
@GetMapping(path = "/openapi.yaml", produces = "application/openapi+yaml")
public ResponseEntity<Resource> contract() {
    return ResponseEntity.ok()
        .cacheControl(CacheControl.maxAge(Duration.ofHours(1)))
        .body(new ClassPathResource("contract/brokerage-api.yaml"));
}
```

Design-first or code-first? This contract was written by hand, beside the code, and is enforced by tests (Chapter 22). Generating the document from annotations is the popular alternative, and it guarantees the document matches the code, but only in the details the annotations capture. Hypermedia rules, problem types and link relations are easier to describe by hand. Whichever you choose, the test that compares real responses with the schemas is what keeps the contract honest.

THE STANDARD

1. Every API has an OpenAPI 3.2 document, served by the API and linked with `service-desc`.
2. The contract states each operation's scope, headers, examples and every problem response.
3. The contract is versioned with the code and tested against the running application.
4. Its paths document where resources live; clients still navigate by links.

Testing a hypermedia API

An API's tests should exercise what clients depend on: status codes, headers, links, problem types and the contract, through the real security layer. The brokerage's tests run the whole application, in memory, in a few seconds.

One annotation for the whole application

Every API test boots the complete Spring application with `MockMvc`, so requests pass through the real filters, security rules, controllers and exception handlers without opening a socket. One meta-annotation holds the setup, and it freezes the simulated market so every price is known:

test: `api/ApiTest.java`

```
/**
 * The whole application, with the simulated market frozen: prices never move by
 * themselves, and orders fill only when a test calls OrderService.match().
 */
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@SpringBootTest(properties = "brokerage.simulator.enabled=false")
@AutoConfigureMockMvc
public @interface ApiTest {
```

Tests need access tokens without an authorization server. Spring Security's test support builds an authenticated JWT directly, and a few factory methods give the tests the same claims the real server would issue:

test: api/ApiTest.java

```
/** Alice (customer cust-1001) through her own script, the brokerage-cli client. */
public static RequestPostProcessor alice(String... scopes) {
    return token("brokerage-cli", "cust-1001", null, scopes);
}

/** Alice's AI agent: same customer, orders capped at the given notional. */
public static RequestPostProcessor agent(String tradingLimit, String... scopes) {
    return token("brokerage-agent", "cust-1001", tradingLimit, scopes);
}

/** Bob (customer cust-2002), who must never see Alice's accounts. */
public static RequestPostProcessor bob(String... scopes) {
    return token("bob-app", "cust-2002", null, scopes);
}

private static RequestPostProcessor token(String client, String customer, String limit,
String... scopes) {
    return jwt().jwt(token -> {
        token.subject(client).audience(List.of("brokerage-api")).claim("customer_id", customer)
            .claim("scope", new ArrayList<>(List.of(scopes)));
        if (limit != null) {
            token.claim("trading_limit", limit);
        }
    });
}
```

Assertions that read like the protocol

`MockMvcTester`, new in Spring Framework 6.2 and refined in 7, pairs `MockMvc` with `AssertJ`, so a test states status codes, headers and JSON paths fluently. This test is Chapter 15 in miniature:

test: api/OrderLifecycleTest.java

```
@Test
void changingAnOrderRequiresItsCurrentEtag() {
    MvcTestResult placed = place(UUID.randomUUID().toString(), ""
        {"instrumentId": "EQ-NVDA", "side": "BUY", "type": "LIMIT", "quantity": "5",
        "limitPrice": "150.00"});
    String url = placed.getResponse().getHeader("Location");
    String etag = placed.getResponse().getHeader("ETag");

    // Polling an unchanged order costs a 304 and no body.
    assertThat(mvc.get().uri(url).with(alice(ALL)).header("If-None-Match", etag))
        .hasStatus(HttpStatus.NOT_MODIFIED);

    assertThat(amend(url, null, ""
        {"limitPrice": "151.00"})).hasStatus(HttpStatus.PRECONDITION_REQUIRED);

    MvcTestResult changed = amend(url, etag, ""
        {"limitPrice": "151.00"});
    assertThat(changed).hasStatusOk();
    assertThat(changed.bodyJson().extractingPath("$.limitPrice").isEqualTo("151.00"));
    assertThat(changed.getResponse().getHeader("ETag")).isNotEqualTo(etag);

    // The old ETag is now stale: a second writer holding it is refused, not merged.
    assertThat(amend(url, etag, ""
        {"limitPrice": "152.00"}))
```

```

        .hasStatus(HttpStatus.PRECONDITION_FAILED)
        .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "precondition-failed");
    }

```

test: api/ProtocolTest.java

```

@Test
void aValueOfTheWrongTypeIsReportedLikeAValidationError() {
    MvcTestResult result = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
        .contentType("application/json")
        .content("""
            {"instrumentId": "EQ-AAPL", "side": "HOLD", "type": "MARKET", "quantity": "1"}""")
        .exchange();
    assertThat(result).hasStatus(HttpStatus.BAD_REQUEST);
    assertThat(result).bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "invalid-request");
    assertThat(result).bodyJson().extractingPath("$.errors[0].field").isEqualTo("side");
    assertThat(result).bodyJson().extractingPath("$.errors[0].message")
        .isEqualTo("must be one of BUY, SELL");
}

```

Some behavior needs different configuration. The rate limit test starts the application with a quota of three requests per minute, rather than wait for the real quota of 120:

test: api/RateLimitTest.java

```

/**
 * Quotas are advertised on every response, and enforced with 429 and Retry-After.
 */
@SpringBootTest(properties = {
    "brokerage.simulator.enabled=false",
    "brokerage.rate-limit.requests-per-minute=3" })
@AutoConfigureMockMvc
class RateLimitTest {

    @Autowired
    MockMvcTester mvc;

    @Test
    void aClientOverItsQuotaIsToldWhenToComeBack() {
        MvcTestResult first = mvc.get().uri("/instruments").with(alice(ALL)).exchange();
        assertThat(first.getResponse().getHeader("RateLimit-Policy")).isEqualTo("\"default\";q=3;w=60");
        assertThat(first.getResponse().getHeader("RateLimit")).startsWith("\"default\";r=2;");

        mvc.get().uri("/instruments").with(alice(ALL)).exchange();
        mvc.get().uri("/instruments").with(alice(ALL)).exchange();
        MvcTestResult refused = mvc.get().uri("/instruments").with(alice(ALL)).exchange();

        assertThat(refused).hasStatus(HttpStatus.TOO_MANY_REQUESTS);
        assertThat(refused.getResponse().getHeader("Retry-After")).isNotBlank();
        assertThat(refused).bodyJson().extractingPath("$.type")
            .isEqualTo("https://docs.brokerage.example/problems/rate-limited");
    }
}

```

Contract tests

The OpenAPI document is code, so it is tested. `ContractTest` loads the contract and the official OpenAPI 3.2 meta-schema into a JSON Schema validator, and checks three things: the document is a valid OpenAPI 3.2 document, its examples match its own schemas, and real responses from the running application match the schemas the document promises.

test: api/ContractTest.java

```
@BeforeAll
static void loadTheContract() throws IOException {
    try (InputStream yaml = new ClassPathResource("contract/brokerage-api.yaml").getInputStream();
        InputStream meta = new ClassPathResource("oas-3.2-schema.json").getInputStream()) {
        contract = YAMLMapper.builder().build().readTree(yaml);
        String metaSchema = new String(meta.readAllBytes());
        String contractJson = JsonMapper.builder().build().writeValueAsString(contract);
        schemas = SchemaRegistry.withDefaultDialect(SpecificationVersion.DRAFT_2020_12,
            registry -> registry.schemas(Map.of(CONTRACT, contractJson, OAS_32, metaSchema)));
    }
}
```

test: api/ContractTest.java

```
@Test
void theContractIsAValidOpenApi32Document() {
    List<Error> errors = schemas.getSchema(SchemaLocation.of(OAS_32)).validate(contract);
    assertThat(errors).as("violations of the OpenAPI 3.2 meta-schema").isEmpty();
}

@Test
void theMetaSchemaCheckCatchesABrokenContract() {
    ObjectNode broken = (ObjectNode) contract.deepCopy();
    broken.remove("info");
    broken.put("openapi", "3.2");
    assertThat(schemas.getSchema(SchemaLocation.of(OAS_32)).validate(broken)).isNotEmpty();
}
```

The second test matters as much as the first: a validation that never fails proves nothing, so the test breaks a copy of the contract on purpose and checks that the validator notices. Then real responses are compared with their schemas:

test: api/ContractTest.java

```
@Test
void readOnlyResourcesMatchTheirSchemas() throws Exception {
    assertMatches("ApiRoot", get("/"));
    assertMatches("AccountCollection", get("/accounts"));
    assertMatches("Account", get("/accounts/ACC-1001"));
    assertMatches("Balances", get("/accounts/ACC-1001/balances"));
    assertMatches("PositionCollection", get("/accounts/ACC-1001/positions"));
    assertMatches("Position", get("/accounts/ACC-1001/positions/EQ-AAPL"));
    assertMatches("TransactionPage", get("/accounts/ACC-1001/transactions?limit=5"));
    assertMatches("InstrumentCollection", get("/instruments?type=EQUITY"));
    assertMatches("InstrumentCollection", get("/instruments/EQ-AAPL/option-chain"));
    assertMatches("QuoteV1", get("/instruments/EQ-AAPL/quote"));
    assertMatches("OrderPage", get("/accounts/ACC-1001/orders"));
    assertMatches("ProtectedResourceMetadata", get("/.well-known/oauth-protected-resource"));
}
```

test: api/ContractTest.java

```
private static void assertMatches(String schemaName, JsonNode instance) {
    SchemaLocation location = SchemaLocation.of(CONTRACT + "#/components/schemas/" + schemaName);
    List<Error> errors = schemas.getSchema(location).validate(instance);
    assertThat(errors).as("%s does not match the contract:%n%s", schemaName, instance).isEmpty();
}
```

It found real bugs. While this book was being written, the contract test caught two mismatches that every other test had missed. Jackson was serializing `Instrument.isOption()` as an undocumented boolean property named `option`, colliding with the real `option` object, and the schema for previews rejected the HAL curies entry that every response carries. Both were fixed in minutes, because the test named the schema, the path and the offending document.

Testing clients without a server

The client and agent modules test against `MockRestServiceServer`, which answers a real `RestClient` with scripted responses. The client's test uses deliberately odd URLs: a client that built paths would never find them, so passing proves it follows links.

test: client/BrokerageClientTest.java

```
// Deliberately odd URLs: a client that constructs paths would never find them.
static final String ROOT = ""
    {"_links": {"self": {"href": "https://api.test/"},
      "bk:accounts": {"href": "https://api.test/v9/acct-list"},
      "bk:instruments": {"href": "https://api.test/catalog{?symbol,type}", "templated": true}}}}"";
static final String ACCOUNTS = ""
    {"_embedded": {"bk:accounts": [{"id": "ACC-1", "nickname": "Main", "type": "MARGIN",
      "_links": {"self": {"href": "https://api.test/a/1"},
        "bk:balances": {"href": "https://api.test/money/for/ACC-1"},
        "bk:orders": {"href": "https://api.test/a/1/o{?status,cursor}",
          "templated": true}}}}},
      "_links": {"self": {"href": "https://api.test/v9/acct-list"}}}}"";

MockRestServiceServer server;
BrokerageClient brokerage;

@BeforeEach
void setUp() {
    RestClient.Builder builder = RestClient.builder();
    server = MockRestServiceServer.bindTo(builder).build();
    brokerage = new BrokerageClient(new HypermediaClient(builder), URI.create("https://api.test/"));
}

@Test
void itFindsBalancesByFollowingLinksFromTheRoot() {
    expectGet("https://api.test/", ROOT);
    expectGet("https://api.test/v9/acct-list", ACCOUNTS);
    expectGet("https://api.test/money/for/ACC-1", ""
        {"currency": "USD", "cash": "1000.10", "buyingPower": "2000.20", "futureField": "ignored",
          "_links": {"self": {"href": "https://api.test/money/for/ACC-1"}}}}""
    );

    var balances = brokerage.balances("ACC-1");

    assertThat(balances.cash()).isEqualToByComparingTo("1000.10");
}
```

```

    assertThat(balances.buyingPower()).isEqualToComparingTo("2000.20");
    server.verify();
}

```

The suite

TEST CLASS	TESTS	WHAT IT PROVES
HypermediaTest	8	Discovery, link relations, scope-dependent links and templates, 401/403/404
OrderLifecycleTest	8	Placement, replay, validation, limits, ETags, cancellation, previews
ProtocolTest	9	Cursors, versions, caching, type errors, HEAD, event replay
ContractTest	5	The contract is valid and the application honors it
RateLimitTest	1	Quota headers, 429 and <code>Retry-After</code>
BrokerageClientTest	4	Link following, retries with one key, typed problems, missing links
BrokerageToolsTest, GuardrailsTest	7	The agent's tools and its human-in-the-loop guarantees

`./mvnw verify` runs all 42 tests in well under a minute.

THE STANDARD

1. Test through the real security layer with realistic tokens; freeze anything random.
2. Assert on what clients see: status, headers, links, problem types.
3. Validate the contract against the OpenAPI meta-schema, and real responses against the contract.
4. Prove that each validation can fail, and prove that clients follow links by giving them odd URLs.

A hypermedia client in Java

A client that follows links is not harder to write than one that builds URLs. It is a different habit, and three small classes are enough to make it the easy one.

The rules the client lives by

- 1 **One configured URL:** the API root. Everything else comes from links.
- 2 **Relations, not paths:** every navigation step names a link relation.
- 3 **Tolerant reading:** unknown properties and values are ignored, never fatal.
- 4 **Problems are typed:** an error becomes an exception carrying the problem type.
- 5 **Safe retries:** transient failures are retried with the same idempotency key.
- 6 **A pinned version:** every request says which API version it expects.

HypermediaClient: HTTP without paths

The lowest layer wraps Spring's `RestClient`. There is no method that takes a path to append to a base URL. Every request targets a link the server provided. You saw its constructor in Chapter 19; its methods are equally short:

client/HypermediaClient.java

```
public HalResource post(Link target, Object body, Consumer<HttpHeaders> headers) {
    RestClient.RequestBodySpec request = http.post().uri(target.toUri()).headers(headers);
    if (body != null) {
        request.contentType(MediaType.APPLICATION_JSON).body(json.writeValueAsString(body));
    }
    return read(request.retrieve().toEntity(String.class));
}

/** A JSON merge patch, sent only if the resource still has the ETag the caller read. */
public HalResource patch(Link target, Object changes, String ifMatch) {
    return read(http.patch().uri(target.toUri())
        .contentType(MERGE_PATCH)
        .header(HttpHeaders.IF_MATCH, ifMatch)
        .body(json.writeValueAsString(changes))
        .retrieve().toEntity(String.class));
}
```

Errors never escape as raw HTTP. A status handler turns every error response into an `ApiProblem` built from the problem document:

client/ApiProblem.java

```
static ApiProblem from(ClientHttpResponse response, JsonMapper json) throws IOException {
    JsonNode body = json.readTree(response.getBody());
    int status = response.getStatusCode().value();
    Optional<Long> retryAfter = Optional.ofNullable(response.getHeaders().getFirst("Retry-After"))
        .map(Long::valueOf);
    return new ApiProblem(status, body.path("type").asString("about:blank"),
        body.path("title").asString(response.getStatusText()), body.path("detail").asString(null),
        body, retryAfter);
}

/** Worth retrying later with the same request (and the same Idempotency-Key). */
public boolean isTransient() {
    return status == 429 || status == 502 || status == 503 || status == 504;
}
```

HalResource: reading links

A response is wrapped in a `HalResource`, which answers the questions a client asks of hypermedia. `requiredLink` turns a missing link into a clear message rather than a null pointer, because a missing link *means* something:

client/HalResource.java

```
/** The first link with this relation, if the server offered one. */
public Optional<Link> link(String rel) {
    return links(rel).stream().findFirst();
}

/** The link with this relation; its absence means the action is not available now. */
public Link requiredLink(String rel) {
    return link(rel).orElseThrow(() -> new IllegalStateException(
        "The server offered no '" + rel + "' link, so that action is not available"));
}

/** Every link with this relation. HAL sends one link as an object, several as an array. */
public List<Link> links(String rel) {
    JsonNode node = body.path("_links").path(rel);
    List<Link> links = new ArrayList<>();
    for (JsonNode each : node.isArray() ? node : List.of(node)) {
        if (each.hasNonNull("href")) {
            Link link = Link.of(each.get("href").asString(), rel);
            links.add(each.hasNonNull("name") ? link.withName(each.get("name").asString()) : link);
        }
    }
    return links;
}
```

BrokerageClient: the domain, as navigation

The top layer offers the domain as Java methods, each a short walk through the link graph. Read the navigation helpers and notice what is absent: no path, no format string, no URL.

client/BrokerageClient.java

```
private List<HalResource> accountResources() {
    HalResource start = http.get(root);
    return follow(start, "bk:accounts").embedded("bk:accounts");
}

private HalResource account(String accountId) {
    return accountResources().stream()
        .filter(account -> accountId.equals(account.body().path("id").asString()))
        .findFirst()
        .orElseThrow(() -> new IllegalArgumentException(
            "No account " + accountId + " for this customer"));
}

private HalResource instrumentResource(String symbol) {
    Link search = http.get(root).requiredLink("bk:instruments");
    return http.get(search.expand(Map.of("symbol", symbol)).toUri()).embedded("bk:instruments").stream()
        .findFirst()
        .orElseThrow(() -> new IllegalArgumentException("Unknown symbol " + symbol));
}

private HalResource follow(HalResource from, String rel) {
    return http.get(from.requiredLink(rel).expand().toUri());
}
```

client/BrokerageClient.java

```
public List<Account> accounts() {
    return accountResources().stream().map(account -> account.as(Account.class)).toList();
}

public Balances balances(String accountId) {
    return follow(account(accountId), "bk:balances").as(Balances.class);
}

public List<Position> positions(String accountId) {
    return follow(account(accountId), "bk:positions").embedded("bk:positions").stream()
        .map(position -> position.as(Position.class)).toList();
}
```

Canceling shows the payoff. The client does not decide whether an order can be canceled. It looks for the link, and refuses without a round trip when the server has not offered one:

client/BrokerageClient.java

```
/** Cancels an order, if and only if the server still offers the cancel link. */
public Order cancel(Order order) {
    URI cancel = order.link("bk:cancel").orElseThrow(() -> new IllegalStateException(
        "Order " + order.id() + " is " + order.status() + " and can no longer be canceled"));
    return http.post(Link.of(cancel.toString()), null, headers -> { }).as(Order.class);
}
```

The model records are the client's own view of the API, and only the fields it uses. Statuses and types are strings, so the server can add values freely:

client/Model.java

```
/** An order, with the links the server offered for it at the time it was read. */
public record Order(String id, String symbol, String side, String type, BigDecimal quantity,
    BigDecimal limitPrice, String status, BigDecimal filledQuantity, BigDecimal averageFillPrice,
    Instant createdAt, @JsonProperty("_links") JsonNode links) {

    public URI self() {
        return link("self").orElseThrow();
    }

    /** Present only while the server allows the action it names. */
    public Optional<URI> link(String rel) {
        JsonNode href = links == null ? null : links.path(rel).path("href");
        return href == null || href.isMissingNode() ? Optional.empty()
            : Optional.of(URI.create(href.asString()));
    }

    public boolean isCancellable() {
        return link("bk:cancel").isPresent();
    }
}
```

Generated clients and hypermedia clients

OpenAPI generators produce a method per operation with typed models, and they are convenient. They also bake the URL templates into the client, which quietly converts a hypermedia API back into a fixed-URL one. A good compromise is to generate the *models* from the contract's schemas and write the navigation by hand, as this client does in a few dozen lines. For scripts and throwaway tools, a generated client is fine; for anything that must survive the API's evolution, follow the links.

THE STANDARD

1. Clients are configured with the API root only, and navigate by relation.
2. Clients read tolerantly and pin the API version on every request.
3. Errors surface as typed problems; transient ones are retried with the original idempotency key.
4. An action is attempted only when its link is present.

RPC or resources? Lessons from the Webull OpenAPI

Real brokerage APIs are often designed as remote procedure calls. Comparing one with this book's design, operation by operation, shows what each style gets right, and which standards cost nothing to adopt either way.

Webull publishes its OpenAPI for customers and partners at *developer.webull.com*. It is a capable, well-documented API that supports equities, options, futures and other asset classes, with market data over HTTP and streaming. It is also a clear example of the RPC style: operations named by paths, each with its own request and response shape. What follows describes the API as documented in September 2026, and it is a comparison of styles, not a review. Details change, so consult Webull's documentation before building against it.

The same operations, two designs

OPERATION	WEBULL OPENAPI (RPC STYLE)	THIS BOOK (RESOURCES)
List accounts	GET /trading/accounts/list	GET /accounts
Balances	GET /trading/assets/balances/get ?account_id=...	GET /accounts/{id}/balances
Positions	GET /trading/assets/positions/list ?account_id=...	GET /accounts/{id}/positions
Preview an order	POST /trading/orders/preview	POST /accounts/{id}/order-previews
Place an order	POST /trading/orders/place	POST /accounts/{id}/orders
Change an order	POST /trading/orders/replace	PATCH .../orders/{orderId}
Cancel an order	POST /trading/orders/cancel	POST .../orders/{orderId}/cancellation
One order	GET /trading/orders/get	GET .../orders/{orderId}
Open orders	GET /trading/orders/open-orders/list	GET .../orders?status=OPEN
Order history	GET /trading/orders/historical-orders/list	GET .../orders (paged)
Instruments	GET /trading/instruments/stocks/profiles/list	GET /instruments?symbol=...
Quotes	GET /market-data/stocks/snapshots/list	GET /instruments/{id}/quote

In the RPC style, the account is a parameter: a query parameter for reads and a body field (`account_id`) for writes. Operations are verbs at the end of the path (`place`, `replace`, `cancel`, `list`, `get`), and every change is a `POST`. In the resource style, the account is part of the address of everything it owns, and the

method carries the verb.

RPC: the path names the procedure

```
POST /trading/orders/place
{"account_id": "...", "new_orders": [ ... ]}

POST /trading/orders/cancel
{"account_id": "...", "client_order_id": "..."}

417 {"error_code": "...", "message": "..."}

next page: ?pagination_key=...
```

Resources: the path names the thing

```
POST /accounts/ACC-1001/orders
Idempotency-Key: ... → 201 + Location

POST .../orders/ORD-100002/cancellation
→ 202 Accepted, from the order's bk:cancel

422 application/problem+json

next page: follow _links.next
```

Figure 24.1 — The two styles side by side. Neither is wrong; they put the knowledge in different places. In RPC, it lives in documentation and SDKs; in hypermedia, the responses carry much of it.

Point by point

Authentication: request signing versus bearer tokens

Webull authenticates each request with an application key and an HMAC signature: the client signs the path, query, body and a set of headers (`x-app-key`, `x-timestamp`, `x-signature-nonce`, `x-signature-algorithm`, `x-signature-version`, and the host) with its app secret, which never travels. Requests made on a customer's behalf also carry an `x-access-token`. Signing gives something a plain bearer token does not: a captured request cannot be replayed after its timestamp and nonce are spent, or altered in transit. OAuth's answer to the same threat is sender-constrained tokens (DPoP or mutual TLS, Chapter 20). The OAuth route wins on ecosystem: standard libraries, delegated consent, scopes, and discovery through RFC 9728, with no custom signing code in every client.

Errors: 417 versus problem details

Webull reports business-rule failures with HTTP 417 and a body of `error_code` and `message`. The structured code is the valuable part, and it is the same idea as a problem type. The status is the questionable part: 417 *Expectation Failed* is defined by HTTP for failures of the `Expect` request header, so generic tools misread it. RFC 9457 over 400, 403, 409 and 422 says the same things in a standard shape.

Idempotency: client_order_id versus Idempotency-Key

Webull requires a `client_order_id` of up to 32 characters, unique per account, which makes a retried placement detectable. Chapter 14 compared the two approaches: the header generalizes the idea to any `POST` and defines what a replay returns.

Pagination: pagination_key versus next links

Order lists take a `pagination_key`, a cursor in all but name, which is the right algorithm. A `next` link carries the same cursor with nothing for the client to assemble.

Versions: x-version versus API-Version

Both put the version in a header rather than the path. The difference is only the name.

Naming: snake_case versus camelCase

Webull uses `account_id`; this book uses `accountId`. Either is fine, consistently applied.

What the RPC design does well

- **Composite orders.** One placement request carries a `new_orders` array and a `combo_type`, such as a bracket or one-cancels-other group, so linked orders are accepted or refused together. A resource design needs an explicit resource for this, such as an *order group* posted to `/accounts/{id}/order-groups` that links to its member orders. That resource is worth designing before you need it.
- **SDK ergonomics.** Operation-per-path maps one to one onto SDK methods, and Webull ships SDKs that also hide the signing.
- **Breadth.** One uniform envelope stretches across many asset classes and market-data products.

Standards cost nothing to adopt, in either style

The most useful lesson of the comparison is that most of this book's standards do not depend on the resource style at all. An RPC API can return RFC 9457 problems with accurate statuses, accept an `Idempotency-Key` on every `POST`, write money as decimal strings, send `Deprecation` and `RateLimit` headers, link to its next page, publish RFC 9728 metadata, and describe itself in OpenAPI 3.2. Hypermedia is the step that needs resources. The rest is available to everyone.

Integrating with an RPC upstream. If your firm routes orders to a broker whose API is RPC-style, you can still offer your own clients a resource API. Put the upstream behind an anti-corruption layer: your resources, links and problem types on the outside, translated to the upstream's procedures, error codes and signing on the inside. Clients never see the difference, and you can change brokers without changing clients.

THE STANDARD

1. Choose resources by default; use RPC-style operations only where a resource model is genuinely awkward.
2. Whatever the style, adopt the style-independent standards: problem details, idempotency keys, decimal strings, deprecation and rate-limit headers, OpenAPI.
3. Model composite actions, such as bracket orders, as resources of their own.
4. Hide a foreign API behind your own contract, not inside your clients.

Agentic AI: a trading assistant with Spring AI

An AI agent is a new kind of API client: capable, fast, and occasionally wrong with great confidence. Everything this book has built, from scopes to idempotency keys to problem types, is what lets such a client act for a customer without being trusted blindly.

What the agent does

The `brokerage-agent` module is a console assistant for one customer. Ask it about balances, positions, quotes or open orders, and it answers from live data. Ask it to buy or sell, and it checks the price, previews the order and *proposes* it, with an estimate and a confirmation code. Nothing happens until the customer types `/confirm` with that code.

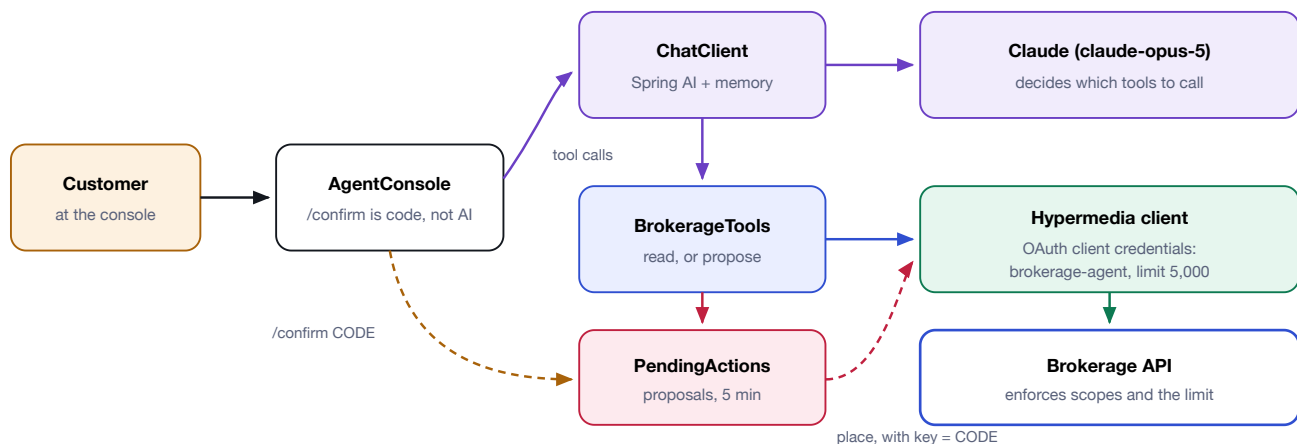


Figure 25.1 — The agent's architecture. The model can read and propose; only the console, driven by a person, can confirm. The API enforces the agent's scopes and trading limit whatever the model or the console does.

Configuration

Spring AI's Anthropic starter provides the chat model; the agent selects Claude Opus 5 and leaves room for tool use and thinking in each response. The agent is also an OAuth client in its own right, with its own registration:

brokerage-agent/src/main/resources/application.yaml

```
ai:
  anthropic:
    api-key: ${ANTHROPIC_API_KEY:}
  chat:
    options:
      model: claude-opus-5
      # Leave room for tool calls and thinking; Spring AI's default is 4096.
      max-tokens: 16000
security:
  oauth2:
    client:
      registration:
        brokerage:
          provider: brokerage-auth
          client-id: brokerage-agent
          client-secret: agent-secret
          authorization-grant-type: client_credentials
          scope: accounts:read, market-data:read, orders:read, orders:write
      provider:
        brokerage-auth:
          token-uri: http://localhost:9000/oauth2/token
```

Spring Security fetches, caches and renews the agent's client credentials token, and adds it to every request the hypermedia client makes:

agent/BrokerageAgentApplication.java

```
/**
 * The agent's own OAuth 2.1 client: Spring Security fetches, caches and renews a
 * client_credentials token for the "brokerage" registration and adds it to every request.
 */
@Bean
BrokerageClient brokerageClient(ClientRegistrationRepository registrations,
    @Value("${brokerage.api-root}") URI root) {
    var manager = new AuthorizedClientServiceOAuth2AuthorizedClientManager(registrations,
        new InMemoryOAuth2AuthorizedClientService(registrations));
    manager.setAuthorizedClientProvider(
        OAuth2AuthorizedClientProviderBuilder.builder().clientCredentials().build());
    var tokens = new OAuth2ClientHttpRequestInterceptor(manager);
    tokens.setClientRegistrationIdResolver(request -> "brokerage");
    RestClient.Builder http = RestClient.builder().requestInterceptor(tokens);
    return new BrokerageClient(new HypermediaClient(http), root);
}

@Bean
ChatClient chatClient(ChatClient.Builder builder, BrokerageTools tools) {
    return builder
        .defaultSystem(SYSTEM_PROMPT)
        .defaultTools(tools)
        .defaultAdvisors(
            MessageChatMemoryAdvisor.builder(MessageWindowChatMemory.builder().build()).build())
        .build();
}
```

The system prompt sets the agent's role and its limits in plain language. It explains *why* the agent cannot act alone, so the model can explain it to the customer:

agent/BrokerageAgentApplication.java

```
static final String SYSTEM_PROMPT = """"
    You help a customer of a retail brokerage with their own accounts. Use the tools to
    look up accounts, balances, positions, quotes and orders, and to propose orders or
    cancellations the customer asks for.

    You cannot place or cancel anything yourself. A proposal only takes effect when the
    customer confirms it, so after proposing, tell the customer what was proposed and
    the confirmation code. Before proposing a trade, check the quote and state the
    estimated total. Treat estimates as estimates.

    Carry out the customer's instructions and answer questions about their accounts; do
    not recommend what to buy or sell. When a tool reports an error, explain it plainly.
    Amounts are in US dollars. Keep answers short.""";
```

Tools: thin wrappers over the client

A Spring AI tool is an annotated method. Its description and parameter descriptions become the schema the model sees. Each brokerage tool is a one-line call to the hypermedia client, so the model sees exactly what the API allows, and no URL ever passes through the model:

agent/BrokerageTools.java

```
@Tool(description = "List the customer's brokerage accounts: ID, nickname, type and currency.")
public List<Account> listAccounts() {
    return brokerage.accounts();
}

@Tool(description = "Cash, cash reserved for open orders, buying power and total equity of one account.")
public Balances getBalances(@ToolParam(description = "Account ID, such as ACC-1001") String accountId) {
    return brokerage.balances(accountId);
}
```

Tools that would change anything only *propose*. `proposeOrder` resolves the symbol, previews the order through the API, and either returns the preview's issues or records a proposal:

agent/BrokerageTools.java

```
@Tool(description = ""
    Propose an order for the customer to confirm. The brokerage checks it first and the
    estimate is returned. Nothing is placed until the customer confirms the proposal.""")
public Object proposeOrder(@ToolParam(description = "Account ID") String accountId,
    @ToolParam(description = "Ticker symbol") String symbol,
    @ToolParam(description = "BUY or SELL") String side,
    @ToolParam(description = "Number of shares, as a decimal string such as \"10\"") String quantity,
    @ToolParam(description = "Limit price as a decimal string; omit for a market order",
        required = false) String limitPrice) {
    String instrumentId = brokerage.instrument(symbol).id();
    BigDecimal shares = new BigDecimal(quantity);
    OrderTicket ticket = limitPrice == null || limitPrice.isBlank()
        ? OrderTicket.market(instrumentId, side, shares)
        : OrderTicket.limit(instrumentId, side, shares, new BigDecimal(limitPrice));
    OrderPreview preview = brokerage.preview(accountId, ticket);
    if (!preview.acceptable()) {
        return preview; // the issues explain why; nothing to confirm
    }
    String price = ticket.limitPrice() == null ? "at market" : "limit " + ticket.limitPrice();
    String summary = "%s %s %s in %s, %s, estimated total %s USD".formatted(side, quantity, symbol,
        accountId, price, preview.estimatedTotal());
    return pending.propose(summary, idempotencyKey -> {
        Order order = brokerage.place(accountId, ticket, idempotencyKey);
        return "Placed order " + order.id() + ": " + order.status();
    });
}
```

The human in the loop

A proposal is a closure waiting in memory. The model receives only its code, summary and instructions, and has no tool that can confirm one. Confirmation happens in `PendingActions`, called only from the console:

agent/PendingActions.java

```
/** Records an action to run later. The action receives the code to use as its Idempotency-Key. */
public Proposal propose(String summary, Function<String, String> action) {
    String code = UUID.randomUUID().toString().substring(0, 8).toUpperCase(Locale.ROOT);
    Proposal proposal = new Proposal(code, summary, clock.instant().plus(LIFETIME),
        "Nothing has been done yet. Ask the customer to type /confirm " + code
        + " to go ahead, or /discard " + code + " to drop it. It expires in 5 minutes.");
    pending.put(code, new Pending(proposal, action));
    return proposal;
}

/** Called only from the console, when a person types /confirm CODE. */
public String confirm(String code) {
    Pending action = pending.remove(code.toUpperCase(Locale.ROOT));
    if (action == null) {
        return "There is no pending action " + code + ".";
    }
    if (clock.instant().isAfter(action.proposal().expiresAt())) {
        return "Proposal " + code + " expired; ask the assistant again.";
    }
    return action.action().apply(action.proposal().code());
}
```

```

@Override
public void run(String... args) throws IOException {
    System.out.println("Brokerage assistant. Ask about your accounts or ask for a trade.");
    System.out.println("Commands: /confirm CODE, /discard CODE, /status, /quit");
    BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
    for (String line; (line = prompt(in)) != null && !line.equals("/quit"); ) {
        if (line.startsWith("/confirm ")) {
            System.out.println(pending.confirm(line.substring(9).trim()));
        } else if (line.startsWith("/discard ")) {
            boolean discarded = pending.discard(line.substring(9).trim());
            System.out.println(discarded ? "Discarded." : "Nothing to discard.");
        } else if (line.equals("/status")) {
            // The tools are plain Java: call one directly to check the API and the token,
            // with no model involved.
            tools.listAccounts()
                .forEach(account -> System.out.println(account.id() + " " + account.nickname()));
        } else if (!line.isBlank()) {
            System.out.println(modelConfigured ? ask(line)
                : "Set ANTHROPIC_API_KEY to talk to the assistant.");
        }
    }
}

```

Three details make the gate robust. Commands starting with `/` are handled by code and never reach the model, so no prompt, however cleverly injected, can type `/confirm` for the customer. Proposals expire after five minutes, so a stale estimate cannot be confirmed tomorrow. And the confirmation code *is* the order's Idempotency-Key: confirming twice, or retrying after a timeout, can never place two orders.

Defense in depth: the API has the last word

Suppose everything above failed: a bug in the gate, or a confused model that proposes a trade far larger than asked. The API still refuses anything beyond the limit the customer delegated to this client, because the limit is in the agent's token (Chapter 20) and the API checks it on every order:

HTTP

HTTP/1.1 403 Forbidden

Content-Type: application/problem+json

```

{
  "detail": "This client may place orders up to 5000 USD; this one is 10261.80",
  "instance": "/accounts/ACC-1001/orders",
  "status": 403,
  "title": "The order exceeds the trading limit delegated to this client",
  "type": "https://docs.brokerage.example/problems/trading-limit-exceeded",
  "requestId": "8c2cb3c7-092d-4043-b7d4-f812552f5286",
  "tradingLimit": "5000"
}

```

The agent's scopes, its five-minute tokens, its trading limit and the customer's own accounts are all enforced by the resource server, not by the agent. The agent is trusted exactly as much as its token says, which is the only safe way to trust software that reasons.

Refusals and fallbacks

A model may decline a request. The console checks why the model stopped before reading its answer, and shows a clear message for a refusal rather than an empty reply:

agent/AgentConsole.java

```
private String ask(String question) {
    ChatResponse response = chat.prompt()
        .user(question)
        .advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversation))
        .call()
        .chatResponse();
    // Check why the model stopped before reading its text: a refusal has no answer to show.
    String finish = response.getResult().getMetadata().getFinishReason();
    if ("refusal".equalsIgnoreCase(finish)) {
        return "The assistant declined that request. Try rephrasing it.";
    }
    return response.getResult().getOutput().getText();
}
```

The sample also opts in to a Claude API feature called **server-side refusal fallbacks**, in beta as of this writing. When a safety classifier declines a request, the API re-runs it on the fallback model Anthropic recommends for that category, instead of returning the refusal. Spring AI has no option for this request field, so a small `OkHttp` interceptor adds the beta header and `"fallbacks": "default"` to each Messages API request. It is an explicit opt-in: remove the `RefusalFallbacks` component to turn it off.

agent/RefusalFallbacks.java

```
@Override
public Response intercept(Chain chain) throws IOException {
    Request request = chain.request();
    if (!"POST".equals(request.method()) || !request.url().encodedPath().endsWith("/v1/messages")
        || request.body() == null) {
        return chain.proceed(request);
    }
    Buffer body = new Buffer();
    request.body().writeTo(body);
    MediaType type = request.body().contentType();
    String existing = request.header("anthropic-beta");
    return chain.proceed(request.newBuilder()
        .header("anthropic-beta", existing == null ? BETA : existing + "," + BETA)
        .post(RequestBody.create(withFallbacks(body.readUtf8()), type))
        .build());
}

/** The request body with "fallbacks": "default" added. */
static String withFallbacks(String json) {
    ObjectNode request = (ObjectNode) JSON.readTree(json);
    request.put("fallbacks", "default");
    return JSON.writeValueAsString(request);
}
```

Testing an agent without a model

The tools are plain Java, so the most important guarantee, that proposing places nothing and confirming places exactly once, is tested without any model at all:

test: agent/BrokerageToolsTest.java

```
@Test
void proposingPlacesNothingAndConfirmingPlacesExactlyOnce() {
    expect("http://api/", ROOT); // symbol -> instrument
    expect("http://api/instruments?symbol=AAPL", AAPL);
    expect("http://api/", ROOT); // account -> its previews link
    expect("http://api/accounts", ACCOUNTS);
    api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/order-previews"))
        .andExpect(method(HttpMethod.POST))
        .andRespond(withSuccess("""
            {"symbol": "AAPL", "estimatedPrice": "232.42", "estimatedTotal": "2324.20",
             "buyingPower": "55241.10", "acceptable": true, "issues": []}""", HAL));

    Object result = tools.proposeOrder("ACC-1001", "AAPL", "BUY", "10", null);

    assertThat(result).assertInstanceOf(Proposal.class);
    Proposal proposal = (Proposal) result;
    assertThat(proposal.summary()).contains("BUY 10 AAPL").contains("2324.20");
    api.verify(); // a preview, and no order

    api.reset();
    expect("http://api/", ROOT);
    expect("http://api/accounts", ACCOUNTS);
    api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/orders"))
        .andExpect(method(HttpMethod.POST))
        .andExpect(header("Idempotency-Key", proposal.code()))
        .andRespond(withStatus(HttpStatus.CREATED).contentType(HAL).body("""
            {"id": "ORD-100009", "status": "OPEN",
             "_links": {"self": {"href": "http://api/accounts/ACC-1001/orders/ORD-100009"}}}"""));

    assertThat(pending.confirm(proposal.code())).isEqualTo("Placed order ORD-100009: OPEN");
    assertThat(pending.confirm(proposal.code())).startsWith("There is no pending action");
    api.verify();
}
```

Running the agent

With the authorization server and the API running, build the agent and start it with your Anthropic API key in the environment:

terminal

```
$ export ANTHROPIC_API_KEY=...
$ java -jar brokerage-agent/target/brokerage-agent-1.0.0.jar
```

Without a key, the console still starts and `/status` checks the API and the agent's token by calling a tool directly, with no model involved:

agent console

recorded from a real run, without an API key

```
Brokerage assistant. Ask about your accounts or ask for a trade.
Commands: /confirm CODE, /discard CODE, /status, /quit
```

```
> /status
ACC-1001 Individual Margin
ACC-1002 Roth IRA

> /quit
```

With a key, a request such as “buy 5 AAPL in my margin account” unfolds in a fixed sequence. The model calls `getQuote` for the price, then `proposeOrder`. The tool previews the order through the API and returns a proposal with its estimated total and code, and the model relays both. The customer types `/confirm` with the code, the console calls `PendingActions`, and the order is placed with the code as its idempotency key. The model never touched the order itself.

Hypermedia and agents. Agents benefit from hypermedia more than any other client. Links let an agent discover what it may do at run time instead of relying on stale training data about URL patterns; HAL-FORMS templates are already machine-readable descriptions of actions, close to the tool schemas models consume; and problem types give an agent a precise reason to explain to its user. An API designed by this book's standards is, almost by accident, an agent-ready API.

THE STANDARD

1. An agent is an OAuth client with its own registration, short-lived tokens and least-privilege scopes.
2. Delegated limits, such as a maximum order value, live in the token and are enforced by the API.
3. Agents propose; people confirm through a path the model cannot reach; confirmations are idempotent and expire.
4. Tools are thin wrappers over a hypermedia client; test their guarantees without a model.

The standard on one page

Every rule from the chapters, gathered as a checklist you can adopt as your own API style guide, adapt to your organization, or argue with in a design review.

Resources and URIs

- ☐ Everything important, including processes, is a noun with one canonical URI.
- ☐ Lowercase, hyphenated, plural collections, nested only for ownership.
- ☐ Filters in the query string; no verbs, no versions in paths.
- ☐ Identifiers are opaque strings.

Representations

- ☐ Money and quantities are decimal strings with a stated scale and currency.
- ☐ Moments are UTC ISO 8601; dates are ISO dates.
- ☐ camelCase properties, UPPER_SNAKE_CASE values.
- ☐ Absent values omitted; internals hidden; sensitive values masked.
- ☐ Clients read tolerantly.

Hypermedia

- ☐ HAL with `_links` and `_embedded`; one root URL.
- ☐ Registered relations first; extension relations as documented URIs via one CURIE.
- ☐ Transitions appear only when state and permissions allow; decided in one place.
- ☐ HAL-FORMS templates derived from request types.

HTTP semantics

- ☐ Methods by their guarantees; precise status codes; never an error with 2xx.
- ☐ 201 with `Location` for creation; 202 for work in progress.
- ☐ Registered media types and headers before invented ones.

Errors

- ☐ Every error is RFC 9457 problem details from one catalog of type URIs.
- ☐ Invalid fields listed in one `errors` array; facts as extension members.
- ☐ 401 with a challenge, 403 for scope, 404 for others' data.
- ☐ Request IDs everywhere; no internals revealed.

Safety

- ☐ An `Idempotency-Key` on every valuable `POST`; replays marked.
- ☐ `ETag` on mutable resources; `If-Match` required (428/412).
- ☐ Partial updates as JSON merge patch.
- ☐ Previews use the same rules as the real action.
- ☐ Records that must be kept are never deleted.

Collections, caching, events

- ☐ Opaque cursors and `next` links; bounded pages; one stable order.
- ☐ Explicit `Cache-Control`; `no-store` for personal data; 304 on `If-None-Match`.
- ☐ Server-sent events with IDs and `Last-Event-ID` replay; events link to resources.

Evolution and quotas

- ☐ Additive change first; version with a header when breaking.
- ☐ `Deprecation`, `Sunset` and a sunset link before removal; clients pin versions.
- ☐ `RateLimit` and `RateLimit-Policy` on every response; 429 with `Retry-After`.

Security

- ☐ OAuth 2.1 bearer tokens: audience-restricted, short-lived, scoped.
- ☐ Browsers through a backend for frontend with PKCE; refresh tokens rotate.
- ☐ Scopes decided in one place; every query scoped to the token's customer.
- ☐ RFC 9728 metadata, linked from every 401.

Contracts, clients and agents

- ☐ An OpenAPI 3.2 contract, served and tested against real responses.
- ☐ Clients configured with the root only, navigating by relation.
- ☐ Agents propose, people confirm, and the API enforces delegated limits.

Epilogue: the train, again

A TUESDAY, SOME MONTHS LATER · THE SAME TUNNEL

Maya taps *Buy*. The train enters the tunnel, and the app waits, gives up, and tries again, exactly as before. This time the second request carries the same idempotency key as the first. The brokerage recognizes it, returns the order it created a few seconds earlier, and marks the response as a replay. Maya owns 100 shares.

At the office, the app's developers barely notice the day. When an order is refused, the app shows the problem's detail and branches on its type. When the brokerage moved its order URLs last month, the app did not change, because it had never known them. When version 1 of the quotes endpoint began sending `Deprecation` headers, an alert opened a ticket, and the move to version 2 was a morning's work.

None of it is remarkable. That is the point of a standard: it turns the decisions nobody knew they were making into decisions somebody made on purpose, once, for everyone.

Quick reference

Everything you need at the keyboard: commands, clients, demo data and every endpoint with the scope it requires.

Commands

Run from the `brokerage-apis` folder, with JDK 27.

TASK	COMMAND
Build and run all tests	<code>./mvnw verify</code>
Build the jars only	<code>./mvnw -q -DskipTests package</code>
Start the authorization server	<code>java -jar auth-server/target/auth-server-1.0.0.jar</code>
Start the API	<code>java -jar brokerage-api/target/brokerage-api-1.0.0.jar</code>
Run the client demo	<code>./mvnw -q -pl brokerage-client compile exec:java -Dexec.mainClass=com.example.brokerage.client.Demo</code>
Run the agent	<code>ANTHROPIC_API_KEY=... java -jar brokerage-agent/target/brokerage-agent-1.0.0.jar</code>
Get a token	<code>curl -u brokerage-cli:cli-secret http://localhost:9000/oauth2/token -d grant_type=client_credentials -d scope="accounts:read market-data:read orders:read orders:write"</code>

Clients and the demo customer

CLIENT	SECRET	GRANT	TOKENS AND NOTES
brokerage-cli	cli-secret	client_credentials	15-minute access tokens; customer <code>cust-1001</code>
brokerage-agent	agent-secret	client_credentials	5-minute access tokens; <code>cust-1001</code> ; <code>trading_limit</code> 5000
brokerage-web	web-secret	authorization_code + PKCE; refresh_token	10-minute access tokens; 8-hour refresh tokens, rotated; consent; redirect <code>http://127.0.0.1:3000/callback</code>

The browser sign-in user is `alice`, password `wonderland`, customer `cust-1001`. Every client may request the scopes `accounts:read`, `market-data:read`, `orders:read` and `orders:write`; `brokerage-web` may also request `openid`. These credentials exist for local development only.

Demo data

ACCOUNT	CUSTOMER	DESCRIPTION
ACC-1001	cust-1001	Individual margin account: cash, AAPL, MSFT, SPY and AMZN positions, dividends, and a resting limit order for MSFT
ACC-1002	cust-1001	Roth IRA (a cash account) holding NVDA
ACC-2001	cust-2002	Another customer's account, invisible to <code>cust-1001</code> 's tokens

Instruments: `EQ-AAPL`, `EQ-MSFT`, `EQ-NVDA`, `EQ-AMZN`, `ETF-SPY`, `EQ-HALT` (not tradable), and AAPL options for the next two monthly expirations at strikes 220, 230 and 240.

Endpoints

METHOD	PATH	SCOPE	SUCCESS
GET	/	none	200
GET	/.well-known/oauth-protected-resource	none	200
GET	/openapi.yaml	none	200
GET	/accounts	accounts:read	200
GET	/accounts/{accountId}	accounts:read	200
GET	/accounts/{accountId}/balances	accounts:read	200
GET	/accounts/{accountId}/positions[/{instrumentId}]	accounts:read	200
GET	/accounts/{accountId}/transactions	accounts:read	200
POST	/accounts/{accountId}/order-previews	orders:read	200
GET	/accounts/{accountId}/orders	orders:read	200
POST	/accounts/{accountId}/orders	orders:write	201
GET	/accounts/{accountId}/orders/{orderId}	orders:read	200, 304
PATCH	/accounts/{accountId}/orders/{orderId}	orders:write	200
POST	.../orders/{orderId}/cancellation	orders:write	202
GET	/accounts/{accountId}/order-events	orders:read	200 (stream)
GET	/instruments	market-data:read	200
GET	/instruments/{instrumentId}	market-data:read	200
GET	/instruments/{instrumentId}/quote	market-data:read	200, 304
GET	/instruments/{instrumentId}/option-chain	market-data:read	200

Clients should reach each of these by following links from `/`; the paths are listed for reference and debugging.

Status codes and problem types

Every problem type the API can return, what it means, and what a well-written client does about it. Each type is `https://docs.brokerage.example/problems/` followed by the name in the first column.

TYPE	STATUS	MEANING	CLIENT ACTION
invalid-request	400	Malformed body, invalid fields (listed in <code>errors</code>), inconsistent prices, a bad cursor or limit	Fix the request; show field messages
idempotency-key-missing	400	A new order was sent without <code>Idempotency-Key</code>	Add a unique key per order
unsupported-api-version	400	<code>API-Version</code> names an unknown version; <code>supportedVersions</code> lists the valid ones	Pin a supported version
unauthorized	401	No valid access token	Get a token (see <code>WWW-Authenticate</code>), retry
insufficient-scope	403	The token lacks the operation's scope	Request the scope; do not retry as is
trading-limit-exceeded	403	The order's notional exceeds the client's delegated limit (<code>tradingLimit</code>)	Place a smaller order, or use a client with more authority
not-found	404	No such resource for this caller	Re-navigate from the root
method-not-allowed	405	The resource does not support the method	Use the method its link or template names
not-acceptable	406	No representation matches <code>Accept</code>	Accept HAL or HAL-FORMS
order-not-cancellable	409	The order is final and cannot be canceled	Re-read the order
order-not-replaceable	409	The order is final and cannot be changed	Re-read the order
precondition-failed	412	The order changed since it was read (<code>currentEtag</code>)	Re-read, reapply, resend with the new ETag
unsupported-media-type	415	The body's content type is not accepted here	Send JSON, or a merge patch for changes
idempotency-key-reused	422	The key was already used for a different request	Use a new key for a new order
insufficient-buying-power	422	The purchase needs more than is available (<code>required</code> , <code>available</code>)	Show the amounts to the user
insufficient-position	422	Not enough shares to sell (<code>available</code>); short selling is not supported	Reduce the quantity

TYPE	STATUS	MEANING	CLIENT ACTION
instrument-not-tradable	422	The instrument does not exist, is halted, or cannot trade this way	Check the instrument's properties
precondition-required	428	A change was sent without <code>If-Match</code>	Read the ETag first
rate-limited	429	The client's quota for the window is used up	Wait for <code>Retry-After</code> , retry with the same key

Every problem also carries `instance` (the request path) and `requestId`. An unexpected server error is a 500 problem with a request ID; quote it to support.

Glossary

Every term in one place, with the chapter that explains it.

Affordance

A description of an action a client can take, such as a HAL-FORMS template. (Ch. 12)

API root

The one URL a hypermedia client is configured with; everything else is reached by links. (Ch. 9)

Audience

The `aud` claim naming the API a token was issued for. (Ch. 20)

Backend for frontend (BFF)

A server-side component that runs OAuth for a browser app, so tokens never reach the browser. (Ch. 20)

Buying power

What a new purchase may spend: free cash, doubled in a margin account. (Ch. 7)

CURIE

A compact URI such as `bk:orders`, which expands to an extension relation's URI. (Ch. 10)

Cursor

An opaque marker of a position in a sorted collection, used for the next page. (Ch. 17)

Deprecation, Sunset

Response headers announcing that something is deprecated, and when it will stop working. (Ch. 19)

ETag

An entity tag identifying one version of a representation. (Ch. 8, 15)

Extension relation

A link relation defined by an API owner as a URI. (Ch. 10)

HAL

JSON Hypertext Application Language: JSON with `_links` and `_embedded`. (Ch. 10)

HAL-FORMS

HAL plus `_templates` describing the requests a client may send. (Ch. 12)

OCC symbol

The industry symbol for a listed option: root, expiration, C or P, strike times 1,000. (Ch. 8)

Optimistic concurrency

Allowing a change only if the resource is still the version the client saw. (Ch. 15)

PKCE

Proof Key for Code Exchange: binds an authorization code to the client that requested it. (Ch. 20)

Problem details

The RFC 9457 format for errors: `type`, `title`, `status`, `detail`, `instance`. (Ch. 16)

Protected resource metadata

An API's RFC 9728 document naming its authorization servers and scopes. (Ch. 20)

Representation

A document describing a resource's current state in some media type. (Ch. 2)

Resource

Anything important enough to have its own URI. (Ch. 2, 5)

Richardson maturity model

Four levels of REST adoption, from one endpoint to hypermedia. (Ch. 2)

Safe method

A method that does not change server state the client is responsible for, such as GET. (Ch. 3)

Scope

A named permission carried by a token, such as `orders:write`. (Ch. 20)

Server-sent events

A one-way stream of events over HTTP, with IDs for replay. (Ch. 18)

T+1

Settlement one business day after the trade date. (Ch. 6)

HATEOAS

Hypermedia as the engine of application state: clients advance by following links. (Ch. 9)

Idempotency key

A client-generated value that makes a retried request return the original result. (Ch. 14)

Idempotent method

A method whose effect is the same whether sent once or many times. (Ch. 3)

Link relation

The name of a link's meaning, such as `self`, `next` or `bk:cancel`. (Ch. 10)

Merge patch

A partial JSON document whose members replace the resource's (RFC 7396). (Ch. 15)

Tolerant reader

A client that ignores what it does not understand. (Ch. 6, 23)

Tool (AI)

A function an AI model may call, described by a name and a parameter schema. (Ch. 25)

Trading limit

The largest order a delegated client may place, carried in its token. (Ch. 20, 25)

URI template

An RFC 6570 pattern such as `{?status,symbol}` that a client expands. (Ch. 10)

Further reading

The standards behind each chapter, and the best places to go deeper. RFCs are at rfc-editor.org/rfc/rfcNNNN; Internet-Drafts at datatracker.ietf.org.

HTTP and the Web

RFC 9110	HTTP Semantics: methods, status codes, conditional requests, content negotiation
RFC 9111	HTTP Caching
RFC 6585	Additional HTTP Status Codes, including 428 and 429
RFC 8288	Web Linking: link relations and the IANA registry
RFC 6570	URI Template
RFC 8631	Link Relation Types for Web Services: <code>service-desc</code> , <code>service-doc</code>
RFC 9457	Problem Details for HTTP APIs
RFC 7396	JSON Merge Patch
RFC 9745, RFC 8594	The Deprecation and Sunset HTTP header fields
RFC 3339	Date and Time on the Internet: Timestamps
Drafts	The Idempotency-Key HTTP Header Field; RateLimit header fields for HTTP (IETF HTTP APIs working group)
WHATWG HTML	Server-sent events and <code>EventSource</code>

Hypermedia and REST

Roy T. Fielding	<i>Architectural Styles and the Design of Network-based Software Architectures</i> , doctoral dissertation, University of California, Irvine, 2000. Chapter 5 defines REST.
Martin Fowler	“Richardson Maturity Model” (2010) and “Tolerant Reader” (2011), martinfowler.com
HAL	JSON Hypertext Application Language, Internet-Draft <code>draft-kelly-json-hal</code>
HAL-FORMS	The HAL-FORMS media type specification, by Mike Amundsen, rwcbook.github.io/hal-forms
Spring HATEOAS	Reference documentation, docs.spring.io/spring-hateoas

Security

OAuth 2.1	The OAuth 2.1 Authorization Framework, Internet-Draft <code>draft-ietf-oauth-v2-1</code>
RFC 7636	Proof Key for Code Exchange (PKCE)
RFC 6750	Bearer Token Usage, including <code>WWW-Authenticate</code> errors
RFC 8414, RFC 9728	Authorization Server Metadata; Protected Resource Metadata

RFC 9449, RFC 8705	DPoP and mutual-TLS sender-constrained tokens
Browser-based apps	OAuth 2.0 for Browser-Based Applications, Internet-Draft draft-ietf-oauth-browser-based-apps
Spring Security	Reference documentation for the resource server and the authorization server

Contracts, clients and agents

OpenAPI 3.2	The OpenAPI Specification, spec.openapis.org/oas/v3.2.0.html , and its JSON Schema
Spring Framework 7	Reference documentation: API versioning, MockMvcTester, RestClient
Spring AI	Reference documentation: ChatClient, tool calling, chat memory, the Anthropic starter
Claude API	Anthropic's documentation for models, tool use and refusal handling, docs.claude.com
Webull OpenAPI	developer.webull.com/apis/docs , the RPC-style API compared in Chapter 24

Complete source code

Every file of the sample application, printed from the tested source when this book was built: 67 files and 6,424 lines. With this appendix, the book contains everything you need to rebuild the brokerage from nothing.

Two kinds of file are not reprinted. The OpenAPI 3.2 meta-schema that `ContractTest` validates against (`brokerage-api/src/test/resources/oas-3.2-schema.json`) is published by the OpenAPI Initiative at spec.openapis.org/oas/3.2/schema/2025-09-17. The Maven Wrapper scripts (`mvnw`, `mvnw.cmd` and `.mvn/wrapper`) are generated by `mvn wrapper:wrapper`.

Files in this appendix

E.1 The build

<code>pom.xml</code>	159
<code>auth-server/pom.xml</code>	160
<code>brokerage-api/pom.xml</code>	160
<code>brokerage-client/pom.xml</code>	161
<code>brokerage-agent/pom.xml</code>	162

E.2 The authorization server

<code>auth/AuthServerApplication.java</code>	163
<code>auth-server/src/main/resources/application.yaml</code>	164

E.3 The API

<code>api/BrokerageApiApplication.java</code>	165
<code>api/RootController.java</code>	165
<code>api/platform/ApiException.java</code>	166
<code>api/platform/ApiExceptionHandler.java</code>	167
<code>api/platform/Caller.java</code>	169
<code>api/platform/CursorPage.java</code>	170
<code>api/platform/HypermediaConfig.java</code>	171
<code>api/platform/JsonConfig.java</code>	172
<code>api/platform/ProblemType.java</code>	172
<code>api/platform/RateLimitedException.java</code>	173
<code>api/platform/RateLimiter.java</code>	174
<code>api/platform/RequestIdFilter.java</code>	175
<code>api/platform/SecurityConfig.java</code>	175
<code>api/platform/WebConfig.java</code>	177
<code>api/market/Instrument.java</code>	178
<code>api/market/InstrumentController.java</code>	179
<code>api/market/MarketData.java</code>	181
<code>api/market/Quotes.java</code>	183

api/accounts/Account.java	184
api/accounts/AccountController.java	184
api/accounts/Holdings.java	187
api/accounts/Ledger.java	187
api/accounts/Portfolio.java	191
api/accounts/Reservations.java	192
api/accounts/Transaction.java	192
api/orders/IdempotencyKeys.java	192
api/orders/Order.java	193
api/orders/OrderAmendment.java	195
api/orders/OrderController.java	195
api/orders/OrderEventController.java	198
api/orders/OrderEvents.java	198
api/orders/OrderModelAssembler.java	200
api/orders/OrderPreview.java	201
api/orders/OrderRequest.java	201
api/orders/OrderService.java	202
api/orders/TradingSession.java	208
brokerage-api/src/main/resources/application.yaml	209

E.4 The API's tests

test: api/ApiTest.java	209
test: api/ContractTest.java	210
test: api/HypermediaTest.java	213
test: api/OrderLifecycleTest.java	214
test: api/ProtocolTest.java	217
test: api/RateLimitTest.java	219

E.5 The hypermedia client

client/ApiProblem.java	219
client/BrokerageClient.java	220
client/ClientCredentials.java	222
client/Demo.java	223
client/HalResource.java	224
client/HypermediaClient.java	225
client/Model.java	227
test: client/BrokerageClientTest.java	228

E.6 The agent

agent/AgentConsole.java	230
agent/BrokerageAgentApplication.java	231
agent/BrokerageTools.java	232
agent/PendingActions.java	234
agent/RefusalFallbacks.java	235

brokerage-agent/src/main/resources/application.yaml	236
test: agent/BrokerageToolsTest.java	236
test: agent/GuardrailsTest.java	238
E.7 The OpenAPI contract	
openapi/brokerage-api.yaml	240

E.1 The build

The parent POM and each module's POM.

pom.xml

64 lines

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!--
3   Brokerage APIs: a hypermedia (HATEOAS) brokerage REST API on Spring Boot.
4
5   auth-server      OAuth 2.1 authorization server (Spring Authorization Server)
6   brokerage-api    the REST API: HAL + HAL-FORMS, RFC 9457 errors, OAuth 2.1 scopes
7   brokerage-client a small hypermedia client that follows links instead of building URLs
8   brokerage-agent  an AI agent (Spring AI) that uses the client as its tools
9
10  Build and test everything: ./mvnw verify
11  -->
12  <project xmlns="http://maven.apache.org/POM/4.0.0"
13          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
14          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
15    <modelVersion>4.0.0</modelVersion>
16
17    <parent>
18      <groupId>org.springframework.boot</groupId>
19      <artifactId>spring-boot-starter-parent</artifactId>
20      <version>4.1.1</version>
21      <relativePath/>
22    </parent>
23
24    <groupId>com.example.brokerage</groupId>
25    <artifactId>brokerage-apis</artifactId>
26    <version>1.0.0</version>
27    <packaging>pom</packaging>
28    <name>Brokerage APIs</name>
29
30    <modules>
31      <module>auth-server</module>
32      <module>brokerage-api</module>
33      <module>brokerage-client</module>
34      <module>brokerage-agent</module>
35    </modules>
36
37    <properties>
38      <java.version>27</java.version>
39      <spring-ai.version>2.0.1</spring-ai.version>
40      <json-schema-validator.version>3.0.7</json-schema-validator.version>
41    </properties>
42
43    <dependencyManagement>
44      <dependencies>
45        <dependency>
46          <groupId>org.springframework.ai</groupId>
47          <artifactId>spring-ai-bom</artifactId>
48          <version>${spring-ai.version}</version>
49          <type>pom</type>
50          <scope>import</scope>
51        </dependency>
52        <dependency>
53          <groupId>com.example.brokerage</groupId>
54          <artifactId>brokerage-client</artifactId>
55          <version>${project.version}</version>
56        </dependency>
57        <dependency>
58          <groupId>com.networknt</groupId>
59          <artifactId>json-schema-validator</artifactId>
60          <version>${json-schema-validator.version}</version>
```

```

61     </dependency>
62 </dependencies>
63 </dependencyManagement>
64 </project>

```

auth-server/pom.xml

38 lines

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <parent>
8         <groupId>com.example.brokerage</groupId>
9         <artifactId>brokerage-apis</artifactId>
10        <version>1.0.0</version>
11    </parent>
12
13    <artifactId>auth-server</artifactId>
14    <name>Brokerage Authorization Server</name>
15    <description>OAuth 2.1 authorization server for local development</description>
16
17    <dependencies>
18        <dependency>
19            <groupId>org.springframework.boot</groupId>
20            <artifactId>spring-boot-starter-security-oauth2-authorization-server</artifactId>
21        </dependency>
22
23        <dependency>
24            <groupId>org.springframework.boot</groupId>
25            <artifactId>spring-boot-starter-webmvc-test</artifactId>
26            <scope>test</scope>
27        </dependency>
28    </dependencies>
29
30    <build>
31        <plugins>
32            <plugin>
33                <groupId>org.springframework.boot</groupId>
34                <artifactId>spring-boot-maven-plugin</artifactId>
35            </plugin>
36        </plugins>
37    </build>
38 </project>

```

brokerage-api/pom.xml

75 lines

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <parent>
8         <groupId>com.example.brokerage</groupId>
9         <artifactId>brokerage-apis</artifactId>
10        <version>1.0.0</version>
11    </parent>
12
13    <artifactId>brokerage-api</artifactId>
14    <name>Brokerage API</name>
15    <description>The hypermedia brokerage REST API</description>
16
17    <dependencies>
18        <dependency>
19            <groupId>org.springframework.boot</groupId>
20            <artifactId>spring-boot-starter-hateoas</artifactId>
21        </dependency>

```

```

22     <dependency>
23         <groupId>org.springframework.boot</groupId>
24         <artifactId>spring-boot-starter-validation</artifactId>
25     </dependency>
26     <dependency>
27         <groupId>org.springframework.boot</groupId>
28         <artifactId>spring-boot-starter-security-oauth2-resource-server</artifactId>
29     </dependency>
30     <dependency>
31         <groupId>org.springframework.boot</groupId>
32         <artifactId>spring-boot-starter-actuator</artifactId>
33     </dependency>
34
35     <dependency>
36         <groupId>org.springframework.boot</groupId>
37         <artifactId>spring-boot-starter-hateoas-test</artifactId>
38         <scope>test</scope>
39     </dependency>
40     <dependency>
41         <groupId>org.springframework.boot</groupId>
42         <artifactId>spring-boot-starter-security-oauth2-resource-server-test</artifactId>
43         <scope>test</scope>
44     </dependency>
45     <dependency>
46         <groupId>com.networknt</groupId>
47         <artifactId>json-schema-validator</artifactId>
48         <scope>test</scope>
49     </dependency>
50     <dependency>
51         <groupId>tools.jackson.dataformat</groupId>
52         <artifactId>jackson-dataformat-yaml</artifactId>
53         <scope>test</scope>
54     </dependency>
55 </dependencies>
56
57 <build>
58     <resources>
59         <resource>
60             <directory>src/main/resources</directory>
61         </resource>
62         <!-- The OpenAPI contract ships inside the application and is served at /openapi.yaml -->
63         <resource>
64             <directory>${project.basedir}/../openapi</directory>
65             <targetPath>contract</targetPath>
66         </resource>
67     </resources>
68     <plugins>
69         <plugin>
70             <groupId>org.springframework.boot</groupId>
71             <artifactId>spring-boot-maven-plugin</artifactId>
72         </plugin>
73     </plugins>
74 </build>
75 </project>

```

brokerage-client/pom.xml

38 lines

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <modelVersion>4.0.0</modelVersion>
6
7      <parent>
8          <groupId>com.example.brokerage</groupId>
9          <artifactId>brokerage-apis</artifactId>
10         <version>1.0.0</version>
11     </parent>
12

```

```

13 <artifactId>brokerage-client</artifactId>
14 <name>Brokerage Hypermedia Client</name>
15 <description>A client that navigates the API by following links</description>
16
17 <dependencies>
18   <dependency>
19     <groupId>org.springframework.boot</groupId>
20     <artifactId>spring-boot-starter-restclient</artifactId>
21   </dependency>
22   <dependency>
23     <groupId>org.springframework.boot</groupId>
24     <artifactId>spring-boot-starter-jackson</artifactId>
25   </dependency>
26   <!-- Only for Link and RFC 6570 URI templates such as {?symbol,type} -->
27   <dependency>
28     <groupId>org.springframework.hateoas</groupId>
29     <artifactId>spring-hateoas</artifactId>
30   </dependency>
31
32   <dependency>
33     <groupId>org.springframework.boot</groupId>
34     <artifactId>spring-boot-starter-restclient-test</artifactId>
35     <scope>test</scope>
36   </dependency>
37 </dependencies>
38 </project>

```

brokerage-agent/pom.xml

46 lines

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6
7   <parent>
8     <groupId>com.example.brokerage</groupId>
9     <artifactId>brokerage-apis</artifactId>
10    <version>1.0.0</version>
11  </parent>
12
13  <artifactId>brokerage-agent</artifactId>
14  <name>Brokerage AI Agent</name>
15  <description>A Spring AI agent that uses the brokerage API through tools</description>
16
17  <dependencies>
18    <dependency>
19      <groupId>com.example.brokerage</groupId>
20      <artifactId>brokerage-client</artifactId>
21    </dependency>
22    <dependency>
23      <groupId>org.springframework.ai</groupId>
24      <artifactId>spring-ai-starter-model-anthropic</artifactId>
25    </dependency>
26    <dependency>
27      <groupId>org.springframework.boot</groupId>
28      <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
29    </dependency>
30
31    <dependency>
32      <groupId>org.springframework.boot</groupId>
33      <artifactId>spring-boot-starter-restclient-test</artifactId>
34      <scope>test</scope>
35    </dependency>
36  </dependencies>
37
38  <build>
39    <plugins>
40      <plugin>

```

```

41     <groupId>org.springframework.boot</groupId>
42     <artifactId>spring-boot-maven-plugin</artifactId>
43 </plugin>
44 </plugins>
45 </build>
46 </project>

```

E.2 The authorization server

auth-server: Spring Authorization Server, configured for three clients and one demo customer.

auth-server/src/main/java/com/example/brokerage/auth/AuthServerApplication.java

64 lines

```

1  package com.example.brokerage.auth;
2
3  import java.math.BigDecimal;
4  import java.util.List;
5  import java.util.Map;
6
7  import org.springframework.boot.SpringApplication;
8  import org.springframework.boot.autoconfigure.SpringBootApplication;
9  import org.springframework.boot.context.properties.ConfigurationProperties;
10 import org.springframework.boot.context.properties.EnableConfigurationProperties;
11 import org.springframework.context.annotation.Bean;
12 import org.springframework.security.oauth2.server.authorization.OAuth2TokenType;
13 import org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
14 import org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
15
16 /**
17  * A development-only OAuth 2.1 authorization server. Clients, scopes and the demo user are
18  * declared in application.yaml; this class adds the two brokerage-specific claims that the
19  * API relies on.
20  */
21 @SpringBootApplication
22 @EnableConfigurationProperties(AuthServerApplication.Identities.class)
23 public class AuthServerApplication {
24
25     public static void main(String[] args) {
26         SpringApplication.run(AuthServerApplication.class, args);
27     }
28
29     /**
30      * Who each token is for. A user signs in as themselves; a confidential client that uses
31      * the client_credentials grant is registered to one customer, like a personal API key.
32      * An optional per-client trading limit caps the notional value of any single order.
33      */
34     @ConfigurationProperties("brokerage.identities")
35     record Identities(Map<String, String> customers, Map<String, BigDecimal> tradingLimits) {
36         Identities {
37             customers = customers == null ? Map.of() : customers;
38             tradingLimits = tradingLimits == null ? Map.of() : tradingLimits;
39         }
40     }
41
42     /** Stamps the audience, customer_id and (if configured) trading_limit into every access token. */
43     @Bean
44     OAuth2TokenCustomizer<JwtEncodingContext> brokerageClaims(Identities identities) {
45         return context -> {
46             if (!OAuth2TokenType.ACCESS_TOKEN.equals(context.getTokenType())) {
47                 return;
48             }
49             // Tokens are minted for one API, which checks that it is the intended audience.
50             context.getClaims().audience(List.of("brokerage-api"));
51             // The principal is the signed-in user, or the client itself for client_credentials.
52             String principal = context.getPrincipal().getName();
53             String customerId = identities.customers().get(principal);
54             if (customerId != null) {

```

```

55         context.getClaims().claim("customer_id", customerId);
56     }
57     String clientId = context.getRegisteredClient().getClientId();
58     BigDecimal limit = identities.tradingLimits().get(clientId);
59     if (limit != null) {
60         context.getClaims().claim("trading_limit", limit.toPlainString());
61     }
62 };
63 }
64 }

```

auth-server/src/main/resources/application.yaml

64 lines

```

1  # Development-only authorization server. Plain HTTP, in-memory keys and clients:
2  # never deploy this configuration.
3  server:
4      port: 9000
5
6  spring:
7      application:
8          name: auth-server
9      security:
10         user:
11             # The demo customer, for the browser-based authorization code flow.
12             name: alice
13             password: wonderland
14         oauth2:
15             authorizationserver:
16                 issuer: http://localhost:9000
17                 client:
18                     # A customer's own script or trading bot: the "personal API key" pattern.
19                     brokerage-cli:
20                         registration:
21                             client-id: brokerage-cli
22                             client-secret: "{noop}cli-secret"
23                             client-authentication-methods: [client_secret_basic]
24                             authorization-grant-types: [client_credentials]
25                             scopes: [accounts:read, market-data:read, orders:read, orders:write]
26                         token:
27                             access-token-time-to-live: 15m
28                     # The AI agent: same customer, but every order is capped by trading_limit.
29                     brokerage-agent:
30                         registration:
31                             client-id: brokerage-agent
32                             client-secret: "{noop}agent-secret"
33                             client-authentication-methods: [client_secret_basic]
34                             authorization-grant-types: [client_credentials]
35                             scopes: [accounts:read, market-data:read, orders:read, orders:write]
36                         token:
37                             access-token-time-to-live: 5m
38                     # A web app's backend (a "backend for frontend"): a confidential client that signs
39                     # the customer in with the authorization code grant, PKCE and consent. Tokens stay
40                     # on the server; the browser holds only a session cookie. Spring Authorization
41                     # Server issues refresh tokens to confidential clients only.
42                     brokerage-web:
43                         registration:
44                             client-id: brokerage-web
45                             client-secret: "{noop}web-secret"
46                             client-authentication-methods: [client_secret_basic]
47                             authorization-grant-types: [authorization_code, refresh_token]
48                             redirect-uri: [http://127.0.0.1:3000/callback]
49                             scopes: [openid, accounts:read, market-data:read, orders:read, orders:write]
50                         require-proof-key: true
51                         require-authorization-consent: true
52                         token:
53                             access-token-time-to-live: 10m
54                             refresh-token-time-to-live: 8h
55                             reuse-refresh-tokens: false
56

```

```

57 brokerage:
58   identities:
59     customers:
60       alice: cust-1001
61       brokerage-cli: cust-1001
62       brokerage-agent: cust-1001
63   trading-limits:
64     brokerage-agent: 5000

```

E.3 The API

brokerage-api: the hypermedia REST API, package by package.

brokerage-api/src/main/java/com/example/brokerage/api/BrokerageApiApplication.java

29 lines

```

1  package com.example.brokerage.api;
2
3  import java.time.Clock;
4
5  import org.springframework.boot.SpringApplication;
6  import org.springframework.boot.autoconfigure.SpringBootApplication;
7  import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.scheduling.annotation.EnableScheduling;
10
11 /**
12  * The brokerage REST API: accounts, positions, transactions, instruments, quotes and
13  * orders, served as HAL (and HAL-FORMS) hypermedia and protected by OAuth 2.1 scopes.
14  */
15 @SpringBootApplication
16 @EnableScheduling
17 @ConfigurationPropertiesScan
18 public class BrokerageApiApplication {
19
20     public static void main(String[] args) {
21         SpringApplication.run(BrokerageApiApplication.class, args);
22     }
23
24     /** One clock for the whole application, so time can be controlled in tests. */
25     @Bean
26     Clock clock() {
27         return Clock.systemUTC();
28     }
29 }

```

brokerage-api/src/main/java/com/example/brokerage/api/RootController.java

57 lines

```

1  package com.example.brokerage.api;
2
3  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6  import java.time.Duration;
7
8  import com.example.brokerage.api.accounts.AccountController;
9  import com.example.brokerage.api.market.InstrumentController;
10
11 import org.springframework.core.io.ClassPathResource;
12 import org.springframework.core.io.Resource;
13 import org.springframework.hateoas.Link;
14 import org.springframework.hateoas.RepresentationModel;
15 import org.springframework.http.CacheControl;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.web.bind.annotation.GetMapping;
18 import org.springframework.web.bind.annotation.RestController;
19 import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
20

```

```

21 /**
22  * The API root: the one URL a client needs to know. Everything else is discovered by
23  * following links from here, so the server is free to reorganize every other URI.
24  */
25 @RestController
26 public class RootController {
27
28     /** The root's own properties, beside its links. */
29     public static class ApiRoot extends RepresentationModel<ApiRoot> {
30
31         public final String name = "Brokerage API";
32         public final String description = "Accounts, positions, market data and trading, as hypermedia";
33     }
34
35     @GetMapping("/")
36     public ApiRoot root() {
37         String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
38         return new ApiRoot()
39             .add(linkTo(methodOn(RootController.class).root()).withSelfRel())
40             .add(linkTo(methodOn(AccountController.class).accounts(null)).withRel("accounts"))
41             .add(linkTo(methodOn(InstrumentController.class).search(null, null)).withRel("instruments"))
42             // RFC 8631: the machine-readable description of this API ...
43             .add(Link.of(base + "/openapi.yaml", "service-desc").withType("application/openapi+yaml"))
44             // ... and its human-readable documentation.
45             .add(Link.of("https://docs.brokerage.example", "service-doc").withType("text/html"))
46             // RFC 9728: where clients learn which authorization server issues tokens for this API.
47             .add(Link.of(base + "/.well-known/oauth-protected-resource", "oauth-protected-resource"));
48     }
49
50     /** The OpenAPI contract, served by the application it describes so the two ship together. */
51     @GetMapping(path = "/openapi.yaml", produces = "application/openapi+yaml")
52     public ResponseEntity<Resource> contract() {
53         return ResponseEntity.ok()
54             .cacheControl(CacheControl.maxAge(Duration.ofHours(1)))
55             .body(new ClassPathResource("contract/brokerage-api.yaml"));
56     }
57 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ApiException.java

36 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.util.LinkedHashMap;
4 import java.util.Map;
5
6 /**
7  * An error the API reports on purpose. It carries its problem type, a detail message for
8  * the developer, and optional extension members (for example the buying power available).
9  */
10 public class ApiException extends RuntimeException {
11
12     private final ProblemType type;
13     private final Map<String, Object> extensions = new LinkedHashMap<>();
14
15     public ApiException(ProblemType type, String detail) {
16         super(detail);
17         this.type = type;
18     }
19
20     public ApiException with(String name, Object value) {
21         extensions.put(name, value);
22         return this;
23     }
24
25     public ProblemType type() {
26         return type;
27     }
28
29     public Map<String, Object> extensions() {

```

```

30     return extensions;
31 }
32
33 public static ApiException notFound(String what, String id) {
34     return new ApiException(ProblemType.NOT_FOUND, what + " " + id + " was not found");
35 }
36 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ApiExceptionHandler.java

162 lines

```

1 package com.example.brokerage.api.platform;
2
3 import java.math.BigDecimal;
4 import java.net.URI;
5 import java.util.Arrays;
6 import java.util.Comparator;
7 import java.util.List;
8 import java.util.Map;
9 import java.util.stream.Collectors;
10
11 import jakarta.servlet.http.HttpServletRequest;
12
13 import org.springframework.http.HttpHeaders;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.ProblemDetail;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.http.converter.HttpMessageNotReadableException;
18 import org.springframework.security.access.AccessDeniedException;
19 import org.springframework.security.core.AuthenticationException;
20 import org.springframework.web.accept.InvalidApiVersionException;
21 import org.springframework.validation.FieldError;
22 import org.springframework.web.bind.MethodArgumentNotValidException;
23 import org.springframework.web.bind.annotation.ExceptionHandler;
24 import org.springframework.web.bind.annotation.RestControllerAdvice;
25 import org.springframework.web.context.request.ServletWebRequest;
26 import org.springframework.web.context.request.WebRequest;
27 import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;
28 import tools.jackson.databind.exc.MismatchedInputException;
29
30 /**
31  * Turns every error into an RFC 9457 problem document (application/problem+json), with a
32  * stable type URI, the request path as "instance", and the request ID for support.
33  * Spring's own exceptions (bad JSON, 405, 406, unsupported versions...) are handled by the
34  * base class; this class adds the API's catalog and a consistent shape for all of them.
35  */
36 @RestControllerAdvice
37 public class ApiExceptionHandler extends ResponseEntityExceptionHandler {
38
39     @ExceptionHandler(ApiException.class)
40     ResponseEntity<ProblemDetail> handleApi(ApiException ex, HttpServletRequest request) {
41         ProblemDetail problem = problem(ex.type(), ex.getMessage(), request);
42         ex.extensions().forEach(problem::setProperty);
43         HttpHeaders headers = new HttpHeaders();
44         if (ex instanceof RateLimitedException limited) {
45             headers.set(HttpHeaders.RETRY_AFTER, String.valueOf(limited.retryAfterSeconds()));
46         }
47         return ResponseEntity.status(ex.type().status()).headers(headers).body(problem);
48     }
49
50     /** Raised by the security filter chain when no valid token is present. */
51     @ExceptionHandler(AuthenticationException.class)
52     ResponseEntity<ProblemDetail> handleUnauthenticated(AuthenticationException ex,
53         HttpServletRequest request) {
54         return ResponseEntity.status(ProblemType.UNAUTHORIZED.status())
55             .body(problem(ProblemType.UNAUTHORIZED, "A valid bearer token is required", request));
56     }
57
58     /** Raised when the token is valid but does not carry the scope the operation needs. */
59     @ExceptionHandler(AccessDeniedException.class)

```

```

60     ResponseEntity<ProblemDetail> handleForbidden(AccessDeniedException ex, HttpServletRequest request) {
61         return ResponseEntity.status(ProblemType.INSUFFICIENT_SCOPE.status())
62             .body(problem(ProblemType.INSUFFICIENT_SCOPE,
63                 "The access token does not grant the scope this operation requires", request));
64     }
65
66     /** An API-Version header naming a version this server does not offer. */
67     @ExceptionHandler(InvalidApiVersionException.class)
68     ResponseEntity<ProblemDetail> handleVersion(InvalidApiVersionException ex, HttpServletRequest request) {
69         ProblemDetail problem = problem(ProblemType.UNSUPPORTED_API_VERSION,
70             "API version " + ex.getVersion() + " is not supported", request);
71         problem.setProperty("supportedVersions", List.of("1", "2"));
72         return ResponseEntity.badRequest().body(problem);
73     }
74
75     /** Bean Validation failures: one problem, with every invalid field listed. */
76     @Override
77     protected ResponseEntity<Object> handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
78         HttpHeaders headers, HttpStatusCode status, WebRequest request) {
79         ProblemDetail problem = problem(ProblemType.INVALID_REQUEST, "One or more fields are invalid",
80             ((ServletWebRequest) request).getRequest());
81         List<Map<String, String>> errors = ex.getBindingResult().getFieldErrors().stream()
82             .sorted(Comparator.comparing(FieldError::getField))
83             .map(error -> Map.of("field", error.getField(),
84                 "message", String.valueOf(error.getDefaultMessage())))
85             .toList();
86         problem.setProperty("errors", errors);
87         return ResponseEntity.badRequest().body(problem);
88     }
89
90     /**
91      * A body that cannot be read: malformed JSON, or a value of the wrong type such as an
92      * unknown enum constant. A wrong value is reported in the same "errors" shape as a
93      * Bean Validation failure, so clients handle one format for every invalid field.
94      */
95     @Override
96     protected ResponseEntity<Object> handleHttpMessageNotReadable(HttpMessageNotReadableException ex,
97         HttpHeaders headers, HttpStatusCode status, WebRequest request) {
98         HttpServletRequest servletRequest = ((ServletWebRequest) request).getRequest();
99         for (Throwable cause = ex; cause != null; cause = cause.getCause()) {
100             if (cause instanceof MismatchedInputException mismatch && !mismatch.getPath().isEmpty()) {
101                 ProblemDetail problem = problem(ProblemType.INVALID_REQUEST, "One or more fields are invalid",
102                     servletRequest);
103                 problem.setProperty("errors", List.of(Map.of("field", fieldPath(mismatch),
104                     "message", expected(mismatch.getTargetType()))));
105                 return ResponseEntity.badRequest().body(problem);
106             }
107         }
108         return ResponseEntity.badRequest().body(problem(ProblemType.INVALID_REQUEST,
109             "The request body is not valid JSON", servletRequest));
110     }
111
112     private static String fieldPath(MismatchedInputException mismatch) {
113         return mismatch.getPath().stream()
114             .map(ref -> ref.getPropertyName() != null ? ref.getPropertyName()
115                 : "[" + ref.getIndex() + "]")
116             .collect(Collectors.joining("."));
117     }
118
119     private static String expected(Class<?> type) {
120         if (type != null && type.isEnum()) {
121             return "must be one of " + Arrays.stream(type.getEnumConstants()).map(Object::toString)
122                 .collect(Collectors.joining(", "));
123         }
124         return type == BigDecimal.class ? "must be a decimal number written as a string, such as \"10.25\""
125             : "has the wrong type";
126     }
127
128     /** Everything Spring raises itself gets the same instance and requestId members. */

```

```

129  @Override
130  protected ResponseEntity<Object> createResponseEntity(Object body, HttpHeaders headers,
131      HttpStatusCode status, WebRequest request) {
132      if (body instanceof ProblemDetail problem) {
133          HttpServletRequest servletRequest = ((ServletWebRequest) request).getRequest();
134          if (problem.getType() == null || "about:blank".equals(problem.getType().toString())) {
135              problem.setType(URI.create(ProblemType.BASE + slugFor(status)));
136          }
137          problem.setInstance(URI.create(servletRequest.getRequestURI()));
138          problem.setProperty("requestId", RequestIdFilter.currentId(servletRequest));
139      }
140      return super.createResponseEntity(body, headers, status, request);
141  }
142
143  static ProblemDetail problem(ProblemType type, String detail, HttpServletRequest request) {
144      ProblemDetail problem = ProblemDetail.forStatusAndDetail(type.status(), detail);
145      problem.setType(type.uri());
146      problem.setTitle(type.title());
147      problem.setInstance(URI.create(request.getRequestURI()));
148      problem.setProperty("requestId", RequestIdFilter.currentId(request));
149      return problem;
150  }
151
152  private static String slugFor(HttpStatusCode status) {
153      return switch (status.value()) {
154          case 400 -> "invalid-request";
155          case 404 -> "not-found";
156          case 405 -> "method-not-allowed";
157          case 406 -> "not-acceptable";
158          case 415 -> "unsupported-media-type";
159          default -> "http-" + status.value();
160      };
161  }
162 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/Caller.java

37 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.math.BigDecimal;
4  import java.util.Collection;
5  import java.util.List;
6  import java.util.Optional;
7
8  import org.springframework.security.oauth2.jwt.Jwt;
9
10 /**
11  * Who is calling, as established by the access token: the customer whose data may be
12  * read, the client application acting for them, what it may do, and any trading limit
13  * the customer delegated to that client.
14  */
15 public record Caller(String customerId, String clientId, Collection<String> scopes,
16     Optional<BigDecimal> tradingLimit) {
17
18     public static Caller of(Jwt jwt) {
19         // A valid token that is not bound to a customer can see no accounts at all.
20         String customerId = Optional.ofNullable(jwt.getClaimAsString("customer_id")).orElse("none");
21         // "scope" may be a space-delimited string (RFC 6749) or a JSON array.
22         Collection<String> scopes = jwt.getClaim("scope") instanceof String text
23             ? List.of(text.split(" "))
24             : Optional.ofNullable(jwt.getClaimAsStringList("scope")).orElse(List.of());
25         Optional<BigDecimal> limit = Optional.ofNullable(jwt.getClaimAsString("trading_limit"))
26             .map(BigDecimal::new);
27         return new Caller(customerId, jwt.getSubject(), scopes, limit);
28     }
29
30     public boolean canTrade() {
31         return scopes.contains("orders:write");
32     }
33 }

```

```

33
34     public boolean canReadOrders() {
35         return scopes.contains("orders:read");
36     }
37 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/CursorPage.java

76 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.nio.charset.StandardCharsets;
4  import java.time.Instant;
5  import java.util.Base64;
6  import java.util.Comparator;
7  import java.util.List;
8  import java.util.function.Function;
9
10 /**
11  * One page of a newest-first list, with an opaque cursor for the next page.
12  *
13  * <p>The cursor encodes the sort key (timestamp and ID) of the last item returned, so a
14  * page never skips or repeats items when new ones are inserted at the top: something
15  * offset-based paging ("page=3") cannot promise. Clients never parse cursors; they
16  * follow the "next" link that carries one.
17  */
18 public record CursorPage<T>(List<T> items, String nextCursor) {
19
20     public static final int DEFAULT_LIMIT = 25;
21     public static final int MAX_LIMIT = 100;
22
23     public boolean hasNext() {
24         return nextCursor != null;
25     }
26
27     public static <T> CursorPage<T> of(List<T> all, Function<T, Instant> time, Function<T, String> id,
28         String cursor, Integer requestedLimit) {
29         int limit = requestedLimit == null ? DEFAULT_LIMIT : requestedLimit;
30         if (limit < 1 || limit > MAX_LIMIT) {
31             throw new ApiException(ProblemType.INVALID_REQUEST,
32                 "limit must be between 1 and " + MAX_LIMIT);
33         }
34         Comparator<T> newestFirst = Comparator.comparing(time).thenComparing(id).reversed();
35         List<T> sorted = all.stream().sorted(newestFirst).toList();
36
37         int start = 0;
38         if (cursor != null) {
39             Key after = Key.decode(cursor);
40             while (start < sorted.size() && !isOlder(sorted.get(start), after, time, id)) {
41                 start++;
42             }
43         }
44         int end = Math.min(start + limit, sorted.size());
45         List<T> items = sorted.subList(start, end);
46         String next = end < sorted.size()
47             ? new Key(time.apply(items.getLast()), id.apply(items.getLast())).encode()
48             : null;
49         return new CursorPage<>(items, next);
50     }
51
52     private static <T> boolean isOlder(T item, Key after, Function<T, Instant> time, Function<T, String> id) {
53         int byTime = time.apply(item).compareTo(after.time());
54         return byTime < 0 || (byTime == 0 && id.apply(item).compareTo(after.id()) < 0);
55     }
56
57     record Key(Instant time, String id) {
58
59         String encode() {
60             String raw = time.toEpochMilli() + ":" + id;
61             return Base64.getUrlEncoder().withoutPadding().

```

```

62         .encodeToString(raw.getBytes(StandardCharsets.UTF_8));
63     }
64
65     static Key decode(String cursor) {
66         try {
67             String raw = new String(Base64.getUrlDecoder().decode(cursor), StandardCharsets.UTF_8);
68             int colon = raw.indexOf(':');
69             return new Key(Instant.ofEpochMilli(Long.parseLong(raw.substring(0, colon))),
70                 raw.substring(colon + 1));
71         } catch (RuntimeException ex) {
72             throw new ApiException(ProblemType.INVALID_REQUEST, "The cursor is malformed or expired");
73         }
74     }
75 }
76 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/HypermediaConfig.java

68 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.math.BigDecimal;
4  import java.util.Set;
5
6  import com.example.brokerage.api.orders.Order;
7  import com.example.brokerage.api.orders.OrderRequest;
8
9  import org.springframework.context.annotation.Bean;
10 import org.springframework.context.annotation.Configuration;
11 import org.springframework.hateoas.LinkRelation;
12 import org.springframework.hateoas.UriTemplate;
13 import org.springframework.hateoas.config.EnableHypermediaSupport;
14 import org.springframework.hateoas.config.EnableHypermediaSupport.HypermediaType;
15 import org.springframework.hateoas.mediatype.hal.CurieProvider;
16 import org.springframework.hateoas.mediatype.hal.DefaultCurieProvider;
17 import org.springframework.hateoas.mediatype.hal.HalLinkRelation;
18 import org.springframework.hateoas.mediatype.hal.forms.HalFormsConfiguration;
19 import org.springframework.hateoas.mediatype.hal.forms.HalFormsOptions;
20
21 /**
22  * Hypermedia formats and link relations.
23  *
24  * <p>HAL (application/hal+json) is the default. Clients that ask for HAL-FORMS
25  * (application/prs.hal-forms+json) also receive "_templates": machine-readable
26  * descriptions of the state transitions available right now, such as placing,
27  * changing or canceling an order.
28  */
29 @Configuration(proxyBeanMethods = false)
30 @EnableHypermediaSupport(type = { HypermediaType.HAL, HypermediaType.HAL_FORMS })
31 public class HypermediaConfig {
32
33     /** Documentation for every custom link relation lives at this URI template. */
34     public static final String RELS = "https://docs.brokerage.example/rels/{rel}";
35
36     /**
37      * Relations that are not registered with IANA (like "orders") are custom, and HAL
38      * requires custom relations to be URIs. A CURIE keeps them short on the wire:
39      * "bk:orders" expands to https://docs.brokerage.example/rels/orders.
40      */
41     @Bean
42     CurieProvider curieProvider() {
43         return new DefaultCurieProvider("bk", UriTemplate.of(RELS)) {
44             @Override
45             public HalLinkRelation getNamespacedRelFor(LinkRelation relation) {
46                 // Registered by RFC 8631, but missing from Spring HATEOAS's list of IANA relations.
47                 return RFC_8631.contains(relation.value()) ? HalLinkRelation.uncuried(relation.value())
48                     : super.getNamespacedRelFor(relation);
49             }
50         };
51     }
52 }

```

```

52
53     private static final Set<String> RFC_8631 = Set.of("service-desc", "service-doc", "service-meta");
54
55     /** Offer the allowed values of enum fields inside HAL-FORMS templates. */
56     @Bean
57     HalFormsConfiguration halFormsConfiguration() {
58         return new HalFormsConfiguration()
59             // Decimals travel as strings: describe them as text that must look like a number.
60             .withPattern(BigDecimal.class, "[0-9]+(\\.[0-9]+)?")
61             .withOptions(OrderRequest.class, "side",
62                 metadata -> HalFormsOptions.inline(Order.Side.values()))
63             .withOptions(OrderRequest.class, "type",
64                 metadata -> HalFormsOptions.inline(Order.Type.values()))
65             .withOptions(OrderRequest.class, "timeInForce",
66                 metadata -> HalFormsOptions.inline(Order.TimeInForce.values()));
67     }
68 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/JsonConfig.java

27 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.math.BigDecimal;
4
5  import com.fasterxml.jackson.annotation.JsonFormat;
6
7  import org.springframework.boot.jackson.autoconfigure.JsonMapperBuilderCustomizer;
8  import org.springframework.context.annotation.Bean;
9  import org.springframework.context.annotation.Configuration;
10
11  /**
12   * JSON conventions that apply to every representation.
13   */
14  @Configuration(proxyBeanMethods = false)
15  public class JsonConfig {
16
17      /**
18       * Money, prices and quantities are written as decimal strings ("187.50"), never as
19       * JSON numbers. Many JSON parsers read numbers into binary floating point, where
20       * 0.1 + 0.2 is not 0.3; a string survives every parser intact.
21       */
22      @Bean
23      JsonMapperBuilderCustomizer decimalsAsStrings() {
24          return builder -> builder.withConfigOverride(BigDecimal.class,
25              override -> override.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)));
26      }
27  }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/ProblemType.java

64 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.net.URI;
4
5  import org.springframework.http.HttpStatus;
6
7  /**
8   * The catalog of error types this API can return. Each becomes the "type" URI of an
9   * RFC 9457 problem document, so clients branch on a stable identifier, never on the
10  * human-readable title or detail.
11  */
12 public enum ProblemType {
13
14     INVALID_REQUEST("invalid-request", HttpStatus.BAD_REQUEST, "The request is not valid"),
15     UNAUTHORIZED("unauthorized", HttpStatus.UNAUTHORIZED, "Authentication is required"),
16     INSUFFICIENT_SCOPE("insufficient-scope", HttpStatus.FORBIDDEN, "The access token lacks a required scope"),
17     NOT_FOUND("not-found", HttpStatus.NOT_FOUND, "The resource does not exist"),
18     IDEMPOTENCY_KEY_MISSING("idempotency-key-missing", HttpStatus.BAD_REQUEST,
19         "An Idempotency-Key header is required"),

```

```

20 IDEMPOTENCY_KEY_REUSED("idempotency-key-reused", HttpStatus.UNPROCESSABLE_CONTENT,
21     "The Idempotency-Key was already used for a different request"),
22 PRECONDITION_REQUIRED("precondition-required", HttpStatus.PRECONDITION_REQUIRED,
23     "An If-Match header is required"),
24 PRECONDITION_FAILED("precondition-failed", HttpStatus.PRECONDITION_FAILED, "The resource has changed"),
25 ORDER_NOT_CANCELLABLE("order-not-cancellable", HttpStatus.CONFLICT,
26     "The order can no longer be canceled"),
27 ORDER_NOT_REPLACEABLE("order-not-replaceable", HttpStatus.CONFLICT, "The order can no longer be changed"),
28 INSUFFICIENT_BUYING_POWER("insufficient-buying-power", HttpStatus.UNPROCESSABLE_CONTENT,
29     "The account does not have enough buying power"),
30 INSUFFICIENT_POSITION("insufficient-position", HttpStatus.UNPROCESSABLE_CONTENT,
31     "The account does not hold enough of the instrument"),
32 INSTRUMENT_NOT_TRADABLE("instrument-not-tradable", HttpStatus.UNPROCESSABLE_CONTENT,
33     "The instrument cannot be traded this way"),
34 TRADING_LIMIT_EXCEEDED("trading-limit-exceeded", HttpStatus.FORBIDDEN,
35     "The order exceeds the trading limit delegated to this client"),
36 UNSUPPORTED_API_VERSION("unsupported-api-version", HttpStatus.BAD_REQUEST,
37     "The requested API version is not supported"),
38 RATE_LIMITED("rate-limited", HttpStatus.TOO_MANY_REQUESTS, "Too many requests");
39
40 /** Problem type URIs resolve to human-readable documentation for each error. */
41 public static final String BASE = "https://docs.brokerage.example/problems/";
42
43 private final String slug;
44 private final HttpStatus status;
45 private final String title;
46
47 ProblemType(String slug, HttpStatus status, String title) {
48     this.slug = slug;
49     this.status = status;
50     this.title = title;
51 }
52
53 public URI uri() {
54     return URI.create(BASE + slug);
55 }
56
57 public HttpStatus status() {
58     return status;
59 }
60
61 public String title() {
62     return title;
63 }
64 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/RateLimitedException.java

17 lines

```

1 package com.example.brokerage.api.platform;
2
3 /** Raised when a client has used up its request quota for the current window. */
4 public class RateLimitedException extends ApiException {
5
6     private final long retryAfterSeconds;
7
8     public RateLimitedException(String policy, long retryAfterSeconds) {
9         super(ProblemType.RATE_LIMITED, "The " + policy
10             + " request quota for this client is used up; retry in " + retryAfterSeconds + " seconds");
11         this.retryAfterSeconds = retryAfterSeconds;
12     }
13
14     public long retryAfterSeconds() {
15         return retryAfterSeconds;
16     }
17 }

```

```

1 package com.example.brokerage.api.platform;
2
3 import java.time.Clock;
4 import java.util.Map;
5 import java.util.concurrent.ConcurrentHashMap;
6
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9
10 import org.springframework.boot.context.properties.ConfigurationProperties;
11 import org.springframework.http.HttpMethod;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.security.core.context.SecurityContextHolder;
14 import org.springframework.stereotype.Component;
15 import org.springframework.web.servlet.HandlerInterceptor;
16
17 /**
18  * A fixed-window rate limiter per client, with two policies: "default" for every request
19  * and a stricter "orders" policy for requests that change orders. Every response
20  * advertises the policy and what is left of it, using the RateLimit-Policy and RateLimit
21  * fields from the IETF httpapi working group's draft, so well-behaved clients can slow
22  * down before they are refused.
23  */
24 @Component
25 public class RateLimiter implements HandlerInterceptor {
26
27     @ConfigurationProperties("brokerage.rate-limit")
28     public record Quotas(int requestsPerMinute, int orderChangesPerMinute) {
29         public Quotas {
30             requestsPerMinute = requestsPerMinute > 0 ? requestsPerMinute : 120;
31             orderChangesPerMinute = orderChangesPerMinute > 0 ? orderChangesPerMinute : 30;
32         }
33     }
34
35     private static final int WINDOW_SECONDS = 60;
36
37     private final Quotas quotas;
38     private final Clock clock;
39     private final Map<String, Window> windows = new ConcurrentHashMap<>();
40
41     public RateLimiter(Quotas quotas) {
42         this.quotas = quotas;
43         this.clock = Clock.systemUTC();
44     }
45
46     @Override
47     public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
48         Authentication caller = SecurityContextHolder.getContext().getAuthentication();
49         if (caller == null || !caller.isAuthenticated()) {
50             return true;
51         }
52         boolean changesOrders = !HttpMethod.GET.matches(request.getMethod())
53             && request.getRequestURI().contains("/orders");
54         String policy = changesOrders ? "orders" : "default";
55         int quota = changesOrders ? quotas.orderChangesPerMinute() : quotas.requestsPerMinute();
56
57         long now = clock.instant().getEpochSecond();
58         long windowStart = now - (now % WINDOW_SECONDS);
59         Window window = windows.compute(caller.getName() + "|" + policy,
60             (key, existing) -> existing == null || existing.start() != windowStart
61                 ? new Window(windowStart, 1) : new Window(windowStart, existing.used() + 1));
62
63         long resetIn = windowStart + WINDOW_SECONDS - now;
64         int remaining = Math.max(0, quota - window.used());
65         response.setHeader("RateLimit-Policy", "\"%s\";q=%d;w=%d".formatted(policy, quota, WINDOW_SECONDS));
66         response.setHeader("RateLimit", "\"%s\";r=%d;t=%d".formatted(policy, remaining, resetIn));
67         if (window.used() > quota) {

```

```

68         throw new RateLimitedException(policy, resetIn);
69     }
70     return true;
71 }
72
73 private record Window(long start, int used) {
74 }
75 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/RequestIdFilter.java

51 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.io.IOException;
4  import java.util.UUID;
5  import java.util.regex.Pattern;
6
7  import jakarta.servlet.FilterChain;
8  import jakarta.servlet.ServletException;
9  import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11
12 import org.slf4j.MDC;
13 import org.springframework.core.Ordered;
14 import org.springframework.core.annotation.Order;
15 import org.springframework.stereotype.Component;
16 import org.springframework.web.filter.OncePerRequestFilter;
17
18 /**
19  * Gives every request an ID: the caller's X-Request-Id if it is well formed, otherwise a
20  * new UUID. The ID is echoed in the response, written to every log line through the MDC,
21  * and included in every problem document, so a support ticket can quote one value.
22  */
23 @Component
24 @Order(Ordered.HIGHEST_PRECEDENCE)
25 public class RequestIdFilter extends OncePerRequestFilter {
26
27     public static final String HEADER = "X-Request-Id";
28     private static final String ATTRIBUTE = RequestIdFilter.class.getName();
29     private static final Pattern SAFE = Pattern.compile("[A-Za-z0-9._-]{1,64}");
30
31     @Override
32     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
33         FilterChain chain) throws ServletException, IOException {
34         String supplied = request.getHeader(HEADER);
35         String id = supplied != null && SAFE.matcher(supplied).matches()
36             ? supplied : UUID.randomUUID().toString();
37         request.setAttribute(ATTRIBUTE, id);
38         response.setHeader(HEADER, id);
39         MDC.put("requestId", id);
40         try {
41             chain.doFilter(request, response);
42         } finally {
43             MDC.remove("requestId");
44         }
45     }
46
47     public static String currentId(HttpServletRequest request) {
48         Object id = request.getAttribute(ATTRIBUTE);
49         return id != null ? id.toString() : null;
50     }
51 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/SecurityConfig.java

119 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.util.List;
4

```

```

5 import jakarta.servlet.http.HttpServletRequest;
6 import jakarta.servlet.http.HttpServletResponse;
7
8 import org.springframework.beans.factory.annotation.Qualifier;
9 import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.context.annotation.Bean;
11 import org.springframework.context.annotation.Configuration;
12 import org.springframework.http.HttpMethod;
13 import org.springframework.security.access.AccessDeniedException;
14 import org.springframework.security.config.Customizer;
15 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
16 import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
17 import org.springframework.security.config.http.SessionCreationPolicy;
18 import org.springframework.security.core.AuthenticationException;
19 import org.springframework.security.oauth2.server.resource.web.BearerTokenAuthenticationEntryPoint;
20 import org.springframework.security.oauth2.server.resource.web.access.BearerTokenAccessDeniedHandler;
21 import org.springframework.security.web.AuthenticationEntryPoint;
22 import org.springframework.security.web.SecurityFilterChain;
23 import org.springframework.security.web.access.AccessDeniedHandler;
24 import org.springframework.web.servlet.HandlerExceptionResolver;
25
26 /**
27  * OAuth 2.1 resource server. Every operation requires one scope; the rules below are the
28  * single place where an operation's required scope is decided.
29  *
30  * <pre>
31  * market-data:read  instruments, quotes, option chains
32  * accounts:read     accounts, balances, positions, transactions
33  * orders:read       orders, order previews, the order event stream
34  * orders:write      placing, changing and canceling orders
35  * </pre>
36  */
37 @Configuration(proxyBeanMethods = false)
38 public class SecurityConfig {
39
40     public static final String MARKET_DATA_READ = "SCOPE_market-data:read";
41     public static final String ACCOUNTS_READ = "SCOPE_accounts:read";
42     public static final String ORDERS_READ = "SCOPE_orders:read";
43     public static final String ORDERS_WRITE = "SCOPE_orders:write";
44
45     /** Discovery documents that anyone may read, with or without a token. */
46     private static final String[] PUBLIC = { "/", "/openapi.yaml", "/actuator/health/**",
47         "/.well-known/oauth-protected-resource" };
48
49     @Bean
50     SecurityFilterChain api(HttpSecurity http, ProblemResponses problems,
51         @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}") String issuer)
52         throws Exception {
53         return http
54             .authorizeHttpRequests(auth -> auth
55                 .requestMatchers(HttpMethod.GET, PUBLIC).permitAll()
56                 .requestMatchers(HttpMethod.HEAD, PUBLIC).permitAll()
57                 .requestMatchers(HttpMethod.GET, "/instruments/**").hasAuthority(MARKET_DATA_READ)
58                 .requestMatchers(HttpMethod.GET, "/accounts/*/orders/**", "/accounts/*/order-events")
59                     .hasAuthority(ORDERS_READ)
60                 .requestMatchers(HttpMethod.POST, "/accounts/*/order-previews")
61                     .hasAuthority(ORDERS_READ)
62                 .requestMatchers("/accounts/*/orders/**").hasAuthority(ORDERS_WRITE)
63                 .requestMatchers(HttpMethod.GET, "/accounts/**").hasAuthority(ACCOUNTS_READ)
64                 .anyRequest().denyAll())
65             .oauth2ResourceServer(server -> server
66                 .jwt(Customizer.withDefaults())
67                 // RFC 9728: publish which authorization server and scopes this API uses.
68                 .protectedResourceMetadata(metadata -> metadata.protectedResourceMetadataCustomizer(
69                     resource -> resource.resourceName("Brokerage API")
70                         .authorizationServer(issuer)
71                         .tlsClientCertificateBoundAccessTokens(false)
72                         .scopes(scopes -> scopes.addAll(List.of("market-data:read",
73                             "accounts:read", "orders:read", "orders:write")))))

```

```

74         .authenticationEntryPoint(problems)
75         .accessDeniedHandler(problems))
76         // Bearer tokens are sent explicitly, never automatically like cookies, so
77         // there is no cross-site request forgery to defend against, and no session.
78         .csrf(AbstractHttpConfigurer::disable)
79         .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
80         .build();
81     }
82
83     /**
84      * 401 and 403 responses keep the standard WWW-Authenticate header (RFC 6750) and get the
85      * same problem-document body as every other error, by handing the exception to the MVC
86      * exception handling in {@link ApiExceptionHandler}.
87      */
88     @Bean
89     ProblemResponses problemResponses(
90         @Qualifier("handlerExceptionHandler") HandlerExceptionResolver resolver) {
91         return new ProblemResponses(resolver);
92     }
93
94     public static final class ProblemResponses implements AuthenticationEntryPoint, AccessDeniedHandler {
95
96         private final HandlerExceptionResolver resolver;
97         private final BearerTokenAuthenticationEntryPoint challenge =
98             new BearerTokenAuthenticationEntryPoint();
99         private final BearerTokenAccessDeniedHandler denied = new BearerTokenAccessDeniedHandler();
100
101         ProblemResponses(HandlerExceptionResolver resolver) {
102             this.resolver = resolver;
103         }
104
105         @Override
106         public void commence(HttpServletRequest request, HttpServletResponse response,
107             AuthenticationException ex) {
108             challenge.commence(request, response, ex);
109             resolver.resolveException(request, response, null, ex);
110         }
111
112         @Override
113         public void handle(HttpServletRequest request, HttpServletResponse response,
114             AccessDeniedException ex) {
115             denied.handle(request, response, ex);
116             resolver.resolveException(request, response, null, ex);
117         }
118     }
119 }

```

brokerage-api/src/main/java/com/example/brokerage/api/platform/WebConfig.java

49 lines

```

1  package com.example.brokerage.api.platform;
2
3  import java.net.URI;
4  import java.time.ZonedDateTime;
5
6  import org.springframework.context.annotation.Configuration;
7  import org.springframework.web.accept.StandardApiVersionDeprecationHandler;
8  import org.springframework.web.servlet.config.annotation.ApiVersionConfigurer;
9  import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
10 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
11
12 /**
13  * API versioning and request interceptors.
14  *
15  * <p>Clients choose a version with the API-Version request header. Version 1 is the
16  * default so that existing clients keep working, but it is deprecated: every version 1
17  * response carries Deprecation (RFC 9745) and Sunset (RFC 8594) headers, plus a link to
18  * the migration guide. Version 2 differs only in the shape of a quote.
19  */
20 @Configuration(proxyBeanMethods = false)

```

```

21 public class WebConfig implements WebMvcConfigurer {
22
23     public static final String VERSION_HEADER = "API-Version";
24
25     private final RateLimiter rateLimiter;
26
27     public WebConfig(RateLimiter rateLimiter) {
28         this.rateLimiter = rateLimiter;
29     }
30
31     @Override
32     public void configureApiVersioning(ApiVersionConfigurer versions) {
33         StandardApiVersionDeprecationHandler deprecations = new StandardApiVersionDeprecationHandler();
34         deprecations.configureVersion("1")
35             .setDeprecationDate(ZonedDateTime.parse("2026-09-01T00:00:00Z"))
36             .setSunsetDate(ZonedDateTime.parse("2027-03-01T00:00:00Z"))
37             .setSunsetLink(URI.create("https://docs.brokerage.example/migrations/v2"));
38
39         versions.useRequestHeader(VERSION_HEADER)
40             .addSupportedVersions("1", "2")
41             .setDefaultVersion("1")
42             .setDeprecationHandler(deprecations);
43     }
44
45     @Override
46     public void addInterceptors(InterceptorRegistry registry) {
47         registry.addInterceptor(rateLimiter).addPathPatterns("/accounts/**", "/instruments/**");
48     }
49 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/Instrument.java

34 lines

```

1 package com.example.brokerage.api.market;
2
3 import java.math.BigDecimal;
4 import java.time.LocalDate;
5
6 import com.fasterxml.jackson.annotation.JsonIgnore;
7
8 import org.springframework.hateoas.server.core.Relation;
9
10 /**
11  * Something that can be traded: a stock, an ETF, or an option contract. The ID is
12  * opaque: clients store it and send it back, but never take it apart.
13  *
14  * @param multiplier shares controlled by one unit: 1 for stocks, 100 for a US equity option
15  * @param option present only for option contracts
16  */
17 @Relation(collectionRelation = "instruments", itemRelation = "instrument")
18 public record Instrument(String id, String symbol, String name, Type type, String exchange,
19     String currency, boolean tradable, boolean fractionable, BigDecimal multiplier, OptionTerms option) {
20
21     public enum Type { EQUITY, ETF, OPTION }
22
23     public enum Right { CALL, PUT }
24
25     /** The contract terms of an option. */
26     public record OptionTerms(String underlyingId, String underlyingSymbol, Right right,
27         BigDecimal strike, LocalDate expiration, String style) {
28     }
29
30     @JsonIgnore // otherwise Jackson would also write a boolean "option" property
31     public boolean isOption() {
32         return type == Type.OPTION;
33     }
34 }

```

```

1 package com.example.brokerage.api.market;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6 import java.time.LocalDate;
7 import java.util.List;
8 import java.util.concurrent.TimeUnit;
9
10 import com.example.brokerage.api.market.Instrument.Type;
11 import com.example.brokerage.api.market.Quotes.Quote;
12 import com.example.brokerage.api.market.Quotes.QuoteV1;
13
14 import org.springframework.format.annotation.DateTimeFormat;
15 import org.springframework.hateoas.CollectionModel;
16 import org.springframework.hateoas.EntityModel;
17 import org.springframework.http.CacheControl;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.web.bind.annotation.GetMapping;
20 import org.springframework.web.bind.annotation.PathVariable;
21 import org.springframework.web.bind.annotation.RequestParam;
22 import org.springframework.web.bind.annotation.RestController;
23 import org.springframework.web.context.request.WebRequest;
24
25 /**
26  * Instruments, quotes and option chains. Scope: market-data:read.
27  */
28 @RestController
29 public class InstrumentController {
30
31     private final MarketData market;
32
33     public InstrumentController(MarketData market) {
34         this.market = market;
35     }
36
37     @GetMapping("/instruments")
38     public CollectionModel<EntityModel<Instrument>> search(
39         @RequestParam(required = false) String symbol, @RequestParam(required = false) Type type) {
40         List<EntityModel<Instrument>> found = market.search(symbol, type).stream()
41             .map(this::toModel)
42             .toList();
43         return CollectionModel.of(found,
44             linkTo(methodOn(InstrumentController.class).search(symbol, type)).withSelfRel().expand());
45     }
46
47     @GetMapping("/instruments/{instrumentId}")
48     public EntityModel<Instrument> instrument(@PathVariable String instrumentId) {
49         return toModel(market.instrument(instrumentId));
50     }
51
52     /** Version 1 of a quote (deprecated; see WebConfig). */
53     @GetMapping(path = "/instruments/{instrumentId}/quote", version = "1")
54     public ResponseEntity<EntityModel<QuoteV1>> quoteV1(@PathVariable String instrumentId,
55         WebRequest request) {
56         Instrument instrument = market.instrument(instrumentId);
57         MarketData.Tick tick = market.tick(instrumentId);
58         return quoteResponse(instrument, tick, QuoteV1.of(instrument, tick), request);
59     }
60
61     @GetMapping(path = "/instruments/{instrumentId}/quote", version = "2")
62     public ResponseEntity<EntityModel<Quote>> quote(@PathVariable String instrumentId,
63         WebRequest request) {
64         Instrument instrument = market.instrument(instrumentId);
65         MarketData.Tick tick = market.tick(instrumentId);
66         return quoteResponse(instrument, tick, Quote.of(instrument, tick), request);
67     }

```

```

68
69 /**
70  * Quotes change every second, so they may be cached only briefly and only privately
71  * (market data is licensed per user). The ETag lets a client poll cheaply: an unchanged
72  * quote costs a 304 with no body.
73  */
74 private <T> ResponseEntity<EntityModel<T>> quoteResponse(Instrument instrument, MarketData.Tick tick,
75     T quote, WebRequest request) {
76     String etag = "\"q\" + tick.sequence() + \"\"";
77     if (request.checkNotModified(etag)) {
78         return null;
79     }
80     EntityModel<T> model = EntityModel.of(quote,
81         linkTo(methodOn(InstrumentController.class).quote(instrument.id(), null)).withSelfRel(),
82         linkTo(methodOn(InstrumentController.class).instrument(instrument.id()))
83             .withRel("instrument"));
84     return ResponseEntity.ok()
85         .cacheControl(CacheControl.maxAge(1, TimeUnit.SECONDS).cachePrivate())
86         .eTag(etag)
87         .body(model);
88 }
89
90 /**
91  * The option chain for one expiration. Every available expiration is offered as a
92  * link with the same relation, told apart by its "name": the client picks one instead
93  * of guessing dates.
94  */
95 @GetMapping("/instruments/{instrumentId}/option-chain")
96 public CollectionModel<EntityModel<Instrument>> optionChain(@PathVariable String instrumentId,
97     @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
98     LocalDate expiration) {
99     Instrument underlying = market.instrument(instrumentId);
100     List<LocalDate> expirations = market.expirations(underlying.id());
101     LocalDate chosen = expiration != null ? expiration
102         : expirations.isEmpty() ? null : expirations.getFirst();
103     List<EntityModel<Instrument>> contracts = chosen == null ? List.of()
104         : market.optionChain(underlying.id(), chosen).stream().map(this::toModel).toList();
105
106     CollectionModel<EntityModel<Instrument>> chain = CollectionModel.of(contracts,
107         linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), chosen))
108             .withSelfRel(),
109         linkTo(methodOn(InstrumentController.class).instrument(underlying.id()))
110             .withRel("underlying"));
111     for (LocalDate date : expirations) {
112         chain.add(linkTo(methodOn(InstrumentController.class).optionChain(underlying.id(), date))
113             .withRel("expiration").withName(date.toString()));
114     }
115     return chain;
116 }
117
118 private EntityModel<Instrument> toModel(Instrument instrument) {
119     EntityModel<Instrument> model = EntityModel.of(instrument,
120         linkTo(methodOn(InstrumentController.class).instrument(instrument.id())).withSelfRel(),
121         linkTo(methodOn(InstrumentController.class).quote(instrument.id(), null))
122             .withRel("quote"));
123     if (instrument.isOption()) {
124         String underlyingId = instrument.option().underlyingId();
125         model.add(linkTo(methodOn(InstrumentController.class).instrument(underlyingId))
126             .withRel("underlying"));
127     } else if (!market.expirations(instrument.id()).isEmpty()) {
128         model.add(linkTo(methodOn(InstrumentController.class).optionChain(instrument.id(), null))
129             .withRel("option-chain"));
130     }
131     return model;
132 }
133 }

```

```

1 package com.example.brokerage.api.market;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Clock;
6 import java.time.DayOfWeek;
7 import java.time.Instant;
8 import java.time.LocalDate;
9 import java.time.format.DateTimeFormatter;
10 import java.time.temporal.ChronoUnit;
11 import java.time.temporal.TemporalAdjusters;
12 import java.util.LinkedHashMap;
13 import java.util.List;
14 import java.util.Locale;
15 import java.util.Map;
16 import java.util.Random;
17 import java.util.concurrent.ConcurrentHashMap;
18
19 import com.example.brokerage.api.market.Instrument.OptionTerms;
20 import com.example.brokerage.api.market.Instrument.Right;
21 import com.example.brokerage.api.market.Instrument.Type;
22 import com.example.brokerage.api.platform.ApiException;
23
24 import org.springframework.stereotype.Component;
25
26 /**
27  * The instrument catalog and a simulated market. Prices are fictional: each stock takes a
28  * small random step every tick, and option prices follow their underlying. Option
29  * expirations are the third Friday of the next two months, so the demo never serves
30  * contracts that have already expired.
31  */
32 @Component
33 public class MarketData {
34
35     /** The market state of one instrument at one moment. */
36     public record Tick(BigDecimal last, BigDecimal bid, BigDecimal ask, BigDecimal bidSize,
37         BigDecimal askSize, BigDecimal previousClose, long volume, long sequence, Instant asOf) {
38     }
39
40     private static final DateTimeFormatter OPTION_DATE = DateTimeFormatter.ofPattern("yyMMdd", Locale.US);
41     private static final BigDecimal CENT = new BigDecimal("0.01");
42
43     private final Map<String, Instrument> instruments = new LinkedHashMap<>();
44     private final Map<String, Tick> ticks = new ConcurrentHashMap<>();
45     private final Random random = new Random(7);
46     private final Clock clock;
47
48     public MarketData(Clock clock) {
49         this.clock = clock;
50         stock("EQ-AAPL", "AAPL", "Apple Inc.", Type.EQUITY, "XNAS", "232.40", "230.10", true);
51         stock("EQ-MSFT", "MSFT", "Microsoft Corporation", Type.EQUITY, "XNAS", "512.75", "509.90", true);
52         stock("EQ-NVDA", "NVDA", "NVIDIA Corporation", Type.EQUITY, "XNAS", "186.20", "188.45", true);
53         stock("EQ-AMZN", "AMZN", "Amazon.com, Inc.", Type.EQUITY, "XNAS", "228.30", "226.00", true);
54         stock("ETF-SPY", "SPY", "SPDR S&P 500 ETF Trust", Type.ETF, "ARCX", "668.10", "664.80", true);
55         stock("EQ-HALT", "HALT", "Halted Holdings (suspended)", Type.EQUITY, "XNYS", "12.00", "12.00", false);
56
57         LocalDate today = LocalDate.now(clock);
58         for (int monthsAhead = 1; monthsAhead <= 2; monthsAhead++) {
59             LocalDate expiration = today.plusMonths(monthsAhead).withDayOfMonth(1)
60                 .with(TemporalAdjusters.dayOfWeekInMonth(3, DayOfWeek.FRIDAY));
61             for (int strike : new int[] { 220, 230, 240 }) {
62                 option(instruments.get("EQ-AAPL"), expiration, Right.CALL, strike);
63                 option(instruments.get("EQ-AAPL"), expiration, Right.PUT, strike);
64             }
65         }
66     }
67

```

```

68 // ----- catalog
69
70 public Instrument instrument(String id) {
71     Instrument instrument = instruments.get(id);
72     if (instrument == null) {
73         throw ApiException.notFound("Instrument", id);
74     }
75     return instrument;
76 }
77
78 public List<Instrument> search(String symbol, Type type) {
79     return instruments.values().stream()
80         .filter(i -> symbol == null || i.symbol().equalsIgnoreCase(symbol))
81         .filter(i -> type == null || i.type() == type)
82         .toList();
83 }
84
85 public List<LocalDate> expirations(String underlyingId) {
86     return options(underlyingId).stream()
87         .map(i -> i.option().expiration())
88         .distinct().sorted().toList();
89 }
90
91 public List<Instrument> optionChain(String underlyingId, LocalDate expiration) {
92     return options(underlyingId).stream()
93         .filter(i -> i.option().expiration().equals(expiration))
94         .toList();
95 }
96
97 private List<Instrument> options(String underlyingId) {
98     return instruments.values().stream()
99         .filter(i -> i.isOption() && i.option().underlyingId().equals(underlyingId))
100         .toList();
101 }
102
103 // ----- prices
104
105 public Tick tick(String instrumentId) {
106     return ticks.get(instrument(instrumentId).id());
107 }
108
109 /** One step of the simulated market. Called on a schedule by the order matcher. */
110 public synchronized void move() {
111     for (Instrument instrument : instruments.values()) {
112         if (instrument.isOption() || !instrument.tradable()) {
113             continue;
114         }
115         Tick tick = ticks.get(instrument.id());
116         double step = (random.nextGaussian() * 0.0008) * tick.last().doubleValue();
117         BigDecimal last = tick.last().add(BigDecimal.valueOf(step))
118             .setScale(2, RoundingMode.HALF_EVEN);
119         long volume = tick.volume() + random.nextInt(400);
120         ticks.put(instrument.id(), stockTick(last, tick.previousClose(), volume, tick.sequence() + 1));
121     }
122     instruments.values().stream().filter(Instrument::isOption).forEach(this::priceOption);
123 }
124
125 // ----- seeding
126
127 private void stock(String id, String symbol, String name, Type type, String exchange,
128     String last, String previousClose, boolean tradable) {
129     instruments.put(id, new Instrument(id, symbol, name, type, exchange, "USD", tradable, tradable,
130         BigDecimal.ONE, null));
131     ticks.put(id, stockTick(new BigDecimal(last), new BigDecimal(previousClose), 1_250_000, 1));
132 }
133
134 private void option(Instrument underlying, LocalDate expiration, Right right, int strike) {
135     String callOrPut = right == Right.CALL ? "C" : "P";
136     String id = "OPT-%s-%s-%s-%d".formatted(underlying.symbol(), expiration.format(OPTION_DATE),

```

```

137         callOrPut, strike);
138         // The OCC option symbol: root, expiration (yyMMdd), C or P, and the strike times 1,000.
139         String symbol = "%s%s%08d".formatted(underlying.symbol(), expiration.format(OPTION_DATE),
140             callOrPut, strike * 1000);
141         String name = "%s %s %d %s".formatted(underlying.symbol(), expiration, strike, right);
142         OptionTerms terms = new OptionTerms(underlying.id(), underlying.symbol(), right,
143             BigDecimal.valueOf(strike), expiration, "AMERICAN");
144         instruments.put(id, new Instrument(id, symbol, name, Type.OPTION, "OPRA", "USD",
145             true, false, BigDecimal.valueOf(100), terms));
146         priceOption(instruments.get(id));
147     }
148
149     private Tick stockTick(BigDecimal last, BigDecimal previousClose, long volume, long sequence) {
150         BigDecimal halfSpread = last.multiply(new BigDecimal("0.0001")).max(CENT)
151             .setScale(2, RoundingMode.HALF_EVEN);
152         return new Tick(last, last.subtract(halfSpread), last.add(halfSpread), BigDecimal.valueOf(500),
153             BigDecimal.valueOf(500), previousClose, volume, sequence, clock.instant());
154     }
155
156     /** Intrinsic value plus a simple time value: enough to look plausible, not a pricing model. */
157     private void priceOption(Instrument option) {
158         OptionTerms terms = option.option();
159         BigDecimal underlying = ticks.get(terms.underlyingId()).last();
160         BigDecimal intrinsic = (terms.right() == Right.CALL
161             ? underlying.subtract(terms.strike())
162             : terms.strike().subtract(underlying)).max(BigDecimal.ZERO);
163         long days = Math.max(1, ChronoUnit.DAYS.between(LocalDate.now(clock), terms.expiration()));
164         BigDecimal timeValue = underlying.multiply(BigDecimal.valueOf(0.012 * Math.sqrt(days / 30.0)));
165         BigDecimal mid = intrinsic.add(timeValue).setScale(2, RoundingMode.HALF_EVEN);
166         Tick previous = ticks.get(option.id());
167         long sequence = previous == null ? 1 : previous.sequence() + 1;
168         BigDecimal previousClose = previous == null ? mid : previous.previousClose();
169         BigDecimal nickel = new BigDecimal("0.05");
170         ticks.put(option.id(), new Tick(mid, mid.subtract(nickel), mid.add(nickel), BigDecimal.valueOf(50),
171             BigDecimal.valueOf(50), previousClose, 3_400, sequence, clock.instant()));
172     }
173 }

```

brokerage-api/src/main/java/com/example/brokerage/api/market/Quotes.java

47 lines

```

1 package com.example.brokerage.api.market;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Instant;
6
7 import org.springframework.hateoas.server.core.Relation;
8
9 /**
10  * The two representations of a quote. Version 2 regroups price and size into bid and ask
11  * objects and adds the day's change: a breaking change of shape, so it needs a new API
12  * version rather than a new optional field.
13  */
14 public final class Quotes {
15
16     private Quotes() {
17     }
18
19     /** API version 1 (deprecated): flat fields. */
20     @Relation(itemRelation = "quote")
21     public record QuoteV1(String instrumentId, String symbol, BigDecimal last, BigDecimal bid,
22         BigDecimal ask, BigDecimal bidSize, BigDecimal askSize, long volume, Instant asOf) {
23
24         public static QuoteV1 of(Instrument instrument, MarketData.Tick tick) {
25             return new QuoteV1(instrument.id(), instrument.symbol(), tick.last(), tick.bid(), tick.ask(),
26                 tick.bidSize(), tick.askSize(), tick.volume(), tick.asOf());
27         }
28     }
29 }

```

```

30  /** API version 2: bid and ask as price levels, plus the change since the previous close. */
31  @Relation(itemRelation = "quote")
32  public record Quote(String instrumentId, String symbol, BigDecimal last, PriceLevel bid, PriceLevel ask,
33      BigDecimal change, BigDecimal changePercent, long volume, Instant asOf) {
34
35      public record PriceLevel(BigDecimal price, BigDecimal size) {
36      }
37
38      public static Quote of(Instrument instrument, MarketData.Tick tick) {
39          BigDecimal change = tick.last().subtract(tick.previousClose());
40          BigDecimal percent = change.multiply(BigDecimal.valueOf(100))
41              .divide(tick.previousClose(), 2, RoundingMode.HALF_EVEN);
42          return new Quote(instrument.id(), instrument.symbol(), tick.last(),
43              new PriceLevel(tick.bid(), tick.bidSize()), new PriceLevel(tick.ask(), tick.askSize()),
44              change, percent, tick.volume(), tick.asOf());
45      }
46  }
47  }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Account.java

24 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.time.LocalDate;
4
5  import com.fasterxml.jackson.annotation.JsonIgnore;
6
7  import org.springframework.hateoas.server.core.Relation;
8
9  /**
10   * A brokerage account. A customer can own several (an individual margin account and a
11   * retirement account, say); every other resource hangs off one of them.
12   *
13   * @param number the account number, masked: only the last four digits are ever sent
14   */
15  @Relation(collectionRelation = "accounts", itemRelation = "account")
16  public record Account(String id, @JsonIgnore String customerId, String number, String nickname, Type type,
17      Registration registration, String currency, LocalDate openedOn) {
18
19      /** How purchases are funded: settled cash only, or cash plus a margin loan. */
20      public enum Type { CASH, MARGIN }
21
22      /** The legal form of the account. */
23      public enum Registration { INDIVIDUAL, JOINT, ROTH_IRA, TRADITIONAL_IRA }
24  }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/AccountController.java

169 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.afford;
4  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
5  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
6
7  import java.time.LocalDate;
8  import java.time.ZoneOffset;
9  import java.util.List;
10
11  import com.example.brokerage.api.accounts.Holdings.Balances;
12  import com.example.brokerage.api.accounts.Holdings.Position;
13  import com.example.brokerage.api.market.InstrumentController;
14  import com.example.brokerage.api.orders.OrderController;
15  import com.example.brokerage.api.orders.OrderEventController;
16  import com.example.brokerage.api.platform.Caller;
17  import com.example.brokerage.api.platform.CursorPage;
18
19  import org.springframework.format.annotation.DateTimeFormat;
20  import org.springframework.hateoas.CollectionModel;
21  import org.springframework.hateoas.EntityModel;

```

```

22 import org.springframework.hateoas.Link;
23 import org.springframework.security.core.annotation.AuthenticationPrincipal;
24 import org.springframework.security.oauth2.jwt.Jwt;
25 import org.springframework.web.bind.annotation.GetMapping;
26 import org.springframework.web.bind.annotation.PathVariable;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29
30 /**
31  * Accounts and everything read-only beneath them. Scope: accounts:read.
32  */
33 @RestController
34 public class AccountController {
35
36     private final Ledger ledger;
37     private final Portfolio portfolio;
38
39     public AccountController(Ledger ledger, Portfolio portfolio) {
40         this.ledger = ledger;
41         this.portfolio = portfolio;
42     }
43
44     @GetMapping("/accounts")
45     public CollectionModel<EntityModel<Account>> accounts(@AuthenticationPrincipal Jwt jwt) {
46         Caller caller = Caller.of(jwt);
47         List<EntityModel<Account>> accounts = ledger.accounts(caller.customerId()).stream()
48             .map(account -> toModel(account, caller))
49             .toList();
50         return CollectionModel.of(accounts,
51             linkTo(methodOn(AccountController.class).accounts(null)).withSelfRel());
52     }
53
54     @GetMapping("/accounts/{accountId}")
55     public EntityModel<Account> account(@PathVariable String accountId,
56         @AuthenticationPrincipal Jwt jwt) {
57         Caller caller = Caller.of(jwt);
58         return toModel(ledger.account(caller.customerId(), accountId), caller);
59     }
60
61     @GetMapping("/accounts/{accountId}/balances")
62     public EntityModel<Balances> balances(@PathVariable String accountId,
63         @AuthenticationPrincipal Jwt jwt) {
64         Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
65         return EntityModel.of(portfolio.balances(account),
66             linkTo(methodOn(AccountController.class).balances(accountId, null)).withSelfRel(),
67             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"),
68             linkTo(methodOn(AccountController.class).positions(accountId, null)).withRel("positions"));
69     }
70
71     @GetMapping("/accounts/{accountId}/positions")
72     public CollectionModel<EntityModel<Position>> positions(@PathVariable String accountId,
73         @AuthenticationPrincipal Jwt jwt) {
74         Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
75         List<EntityModel<Position>> positions = portfolio.positions(account).stream()
76             .map(position -> toModel(accountId, position))
77             .toList();
78         return CollectionModel.of(positions,
79             linkTo(methodOn(AccountController.class).positions(accountId, null)).withSelfRel(),
80             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
81     }
82
83     @GetMapping("/accounts/{accountId}/positions/{instrumentId}")
84     public EntityModel<Position> position(@PathVariable String accountId,
85         @PathVariable String instrumentId, @AuthenticationPrincipal Jwt jwt) {
86         Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
87         return toModel(accountId, portfolio.position(account, instrumentId));
88     }
89
90 /**

```

```

91     * Transaction history, newest first, filterable by type and by an inclusive date range
92     * (UTC), paged with an opaque cursor.
93     */
94     @GetMapping("/accounts/{accountId}/transactions")
95     public CollectionModel<EntityModel<Transaction>> transactions(@PathVariable String accountId,
96         @RequestParam(required = false) Transaction.Type type,
97         @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate from,
98         @RequestParam(required = false) @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate to,
99         @RequestParam(required = false) String cursor, @RequestParam(required = false) Integer limit,
100         @AuthenticationPrincipal Jwt jwt) {
101         Account account = ledger.account(Caller.of(jwt).customerId(), accountId);
102         List<Transaction> matching = ledger.transactions(account.id()).stream()
103             .filter(t -> type == null || t.type() == type)
104             .filter(t -> from == null || !utcDate(t).isBefore(from))
105             .filter(t -> to == null || !utcDate(t).isAfter(to))
106             .toList();
107         CursorPage<Transaction> page = CursorPage.of(matching, Transaction::occurredAt, Transaction::id,
108             cursor, limit);
109
110         CollectionModel<EntityModel<Transaction>> model = CollectionModel.of(
111             page.items().stream().map(EntityModel::of).toList(),
112             linkTo(methodOn(AccountController.class)
113                 .transactions(accountId, type, from, to, cursor, limit, null))
114                 .withSelfRel().expand(),
115             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
116         if (page.hasNext()) {
117             // expand(): "next" is a concrete URL, not a template with the unused filters left open
118             model.add(linkTo(methodOn(AccountController.class)
119                 .transactions(accountId, type, from, to, page.nextCursor(), limit, null))
120                 .withRel("next").expand());
121         }
122         return model;
123     }
124
125     private static LocalDate utcDate(Transaction transaction) {
126         return transaction.occurredAt().atZone(ZoneOffset.UTC).toLocalDate();
127     }
128
129     // ----- representations
130
131     /**
132     * An account's links depend on what the caller may do: a token without orders:read is
133     * not shown the orders at all, and only a token with orders:write is offered the
134     * "placeOrder" affordance. Clients discover their capabilities instead of hard-coding them.
135     */
136     EntityModel<Account> toModel(Account account, Caller caller) {
137         String id = account.id();
138         EntityModel<Account> model = EntityModel.of(account,
139             linkTo(methodOn(AccountController.class).account(id, null)).withSelfRel(),
140             linkTo(methodOn(AccountController.class).balances(id, null)).withRel("balances"),
141             linkTo(methodOn(AccountController.class).positions(id, null)).withRel("positions"),
142             linkTo(methodOn(AccountController.class).transactions(id, null, null, null, null, null, null))
143                 .withRel("transactions"));
144         if (caller.canReadOrders()) {
145             Link orders = linkTo(methodOn(OrderController.class).orders(id, null, null, null, null, null))
146                 .withRel("orders");
147             if (caller.canTrade()) {
148                 orders = orders.andAffordance(
149                     afford(methodOn(OrderController.class).placeOrder(id, null, null, null));
150             }
151             model.add(orders,
152                 linkTo(methodOn(OrderController.class).previewOrder(id, null, null))
153                     .withRel("order-previews"),
154                 linkTo(methodOn(OrderEventController.class).events(id, null, null))
155                     .withRel("order-events"));
156         }
157         return model;
158     }
159 }

```

```

160     private EntityModel<Position> toModel(String accountId, Position position) {
161         return EntityModel.of(position,
162             linkTo(methodOn(AccountController.class).position(accountId, position.instrumentId(), null))
163                 .withSelfRel(),
164             linkTo(methodOn(InstrumentController.class).instrument(position.instrumentId()))
165                 .withRel("instrument"),
166             linkTo(methodOn(InstrumentController.class).quote(position.instrumentId(), null))
167                 .withRel("quote"));
168     }
169 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Holdings.java

47 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4  import java.time.Instant;
5
6  import com.example.brokerage.api.market.Instrument;
7
8  import org.springframework.hateoas.server.core.Relation;
9
10 /**
11  * The read models derived from the ledger and the market: positions and balances.
12  */
13 public final class Holdings {
14
15     private Holdings() {
16     }
17
18     /**
19      * A holding valued at the current market price.
20      *
21      * @param averageCost per share, or per contract share for options
22      */
23     @Relation(collectionRelation = "positions", itemRelation = "position")
24     public record Position(String instrumentId, String symbol, Instrument.Type instrumentType,
25         BigDecimal quantity, BigDecimal averageCost, BigDecimal costBasis, BigDecimal lastPrice,
26         BigDecimal marketValue, BigDecimal unrealizedPnl, BigDecimal unrealizedPnlPercent) {
27     }
28
29     /**
30      * What the account is worth and what it can buy.
31      *
32      * @param reservedForOrders cash held back for open buy orders that have not filled yet
33      */
34     public record Balances(String accountId, String currency, BigDecimal cash, BigDecimal reservedForOrders,
35         BigDecimal buyingPower, BigDecimal positionsMarketValue, BigDecimal totalEquity,
36         BigDecimal unrealizedPnl, Instant asOf) {
37     }
38
39     /**
40      * Buying power: unreserved cash in a cash account, twice that in a margin account
41      * (the 50% initial margin of Regulation T). A deliberate simplification.
42      */
43     public static BigDecimal buyingPower(Account.Type type, BigDecimal cash, BigDecimal reserved) {
44         BigDecimal free = cash.subtract(reserved).max(BigDecimal.ZERO);
45         return type == Account.Type.MARGIN ? free.multiply(BigDecimal.TWO) : free;
46     }
47 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Ledger.java

214 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4  import java.math.RoundingMode;
5  import java.time.Clock;
6  import java.time.DayOfWeek;

```

```

7 import java.time.Duration;
8 import java.time.Instant;
9 import java.time.LocalDate;
10 import java.time.ZoneOffset;
11 import java.util.ArrayList;
12 import java.util.Comparator;
13 import java.util.LinkedHashMap;
14 import java.util.List;
15 import java.util.Map;
16 import java.util.concurrent.atomic.AtomicInteger;
17
18 import com.example.brokerage.api.accounts.Account.Registration;
19 import com.example.brokerage.api.accounts.Transaction.Type;
20 import com.example.brokerage.api.market.Instrument;
21 import com.example.brokerage.api.market.MarketData;
22 import com.example.brokerage.api.platform.ApiException;
23
24 import org.springframework.stereotype.Component;
25
26 /**
27  * The books of record: accounts, cash, holdings and transaction history, held in memory
28  * and seeded with two demo customers. Every mutation goes through {@link #recordTrade},
29  * which keeps cash, holdings and history consistent with one another.
30  */
31 @Component
32 public class Ledger {
33
34     /** One holding: how much, and at what average cost per unit. */
35     public record Holding(String instrumentId, BigDecimal quantity, BigDecimal averageCost) {
36     }
37
38     private static final class Book {
39         BigDecimal cash;
40         final Map<String, Holding> holdings = new LinkedHashMap<>();
41         final List<Transaction> transactions = new ArrayList<>();
42
43         Book(BigDecimal cash) {
44             this.cash = cash;
45         }
46     }
47
48     private final Map<String, Account> accounts = new LinkedHashMap<>();
49     private final Map<String, Book> books = new LinkedHashMap<>();
50     private final AtomicInteger transactionIds = new AtomicInteger(1000);
51     private final MarketData market;
52     private final Clock clock;
53
54     public Ledger(MarketData market, Clock clock) {
55         this.market = market;
56         this.clock = clock;
57         seed();
58     }
59
60     // ----- reads
61
62     public List<Account> accounts(String customerId) {
63         return accounts.values().stream().filter(a -> a.customerId().equals(customerId)).toList();
64     }
65
66     /**
67      * An account the customer owns. Someone else's account is reported as not found,
68      * not as forbidden: a 403 would confirm that the account exists.
69      */
70     public Account account(String customerId, String accountId) {
71         Account account = accounts.get(accountId);
72         if (account == null || !account.customerId().equals(customerId)) {
73             throw ApiException.notFound("Account", accountId);
74         }
75         return account;

```

```

76     }
77
78     public synchronized BigDecimal cash(String accountId) {
79         return books.get(accountId).cash;
80     }
81
82     public synchronized List<Holding> holdings(String accountId) {
83         return List.copyOf(books.get(accountId).holdings.values());
84     }
85
86     public synchronized BigDecimal quantityHeld(String accountId, String instrumentId) {
87         Holding holding = books.get(accountId).holdings.get(instrumentId);
88         return holding == null ? BigDecimal.ZERO : holding.quantity();
89     }
90
91     public synchronized List<Transaction> transactions(String accountId) {
92         return List.copyOf(books.get(accountId).transactions);
93     }
94
95     // ----- writes
96
97     /**
98      * Books one execution. A positive quantity is a purchase, a negative one a sale.
99      * Cash moves by quantity x price x multiplier, plus or minus commission.
100     */
101     public synchronized Transaction recordTrade(String accountId, Instrument instrument,
102         BigDecimal signedQuantity, BigDecimal price, BigDecimal commission, String orderId,
103         Instant at) {
104         Book book = books.get(accountId);
105         BigDecimal quantity = signedQuantity.abs();
106         BigDecimal gross = quantity.multiply(price).multiply(instrument.multiplier());
107         boolean buy = signedQuantity.signum() > 0;
108         BigDecimal cashChange = (buy ? gross.negate() : gross).subtract(commission)
109             .setScale(2, RoundingMode.HALF_EVEN);
110         book.cash = book.cash.add(cashChange);
111
112         Holding old = book.holdings.get(instrument.id());
113         BigDecimal oldQuantity = old == null ? BigDecimal.ZERO : old.quantity();
114         BigDecimal newQuantity = oldQuantity.add(signedQuantity);
115         if (newQuantity.signum() == 0) {
116             book.holdings.remove(instrument.id());
117         } else if (buy) {
118             BigDecimal oldCost = old == null ? BigDecimal.ZERO
119                 : old.quantity().multiply(old.averageCost());
120             BigDecimal averageCost = oldCost.add(quantity.multiply(price))
121                 .divide(newQuantity, 4, RoundingMode.HALF_EVEN);
122             book.holdings.put(instrument.id(), new Holding(instrument.id(), newQuantity, averageCost));
123         } else {
124             BigDecimal averageCost = old == null ? price : old.averageCost();
125             book.holdings.put(instrument.id(), new Holding(instrument.id(), newQuantity, averageCost));
126         }
127
128         String what = instrument.symbol() + (instrument.isOption() ? " option" : "");
129         String description = "%s %s %s @ %s".formatted(buy ? "Bought" : "Sold",
130             quantity.stripTrailingZeros().toPlainString(), what, price.toPlainString());
131         Transaction transaction = new Transaction(nextTransactionId(), Type.TRADE, description, cashChange,
132             instrument.id(), instrument.symbol(), quantity, price, orderId, at, settlementDate(at));
133         book.transactions.add(transaction);
134         return transaction;
135     }
136
137     /** US equity and option trades settle one business day after the trade date (T+1). */
138     private static LocalDate settlementDate(Instant tradeTime) {
139         LocalDate date = tradeTime.atZone(ZoneOffset.UTC).toLocalDate().plusDays(1);
140         while (date.getDayOfWeek() == DayOfWeek.SATURDAY || date.getDayOfWeek() == DayOfWeek.SUNDAY) {
141             date = date.plusDays(1);
142         }
143         return date;
144     }

```

```

145
146 private String nextTransactionId() {
147     return "TXN-" + transactionIds.incrementAndGet();
148 }
149
150 // ----- seed data
151
152 private void seed() {
153     open(new Account("ACC-1001", "cust-1001", "****4821", "Individual Margin", Account.Type.MARGIN,
154         Registration.INDIVIDUAL, "USD", LocalDate.of(2021, 3, 15)));
155     open(new Account("ACC-1002", "cust-1001", "****7304", "Roth IRA", Account.Type.CASH,
156         Registration.ROTH_IRA, "USD", LocalDate.of(2023, 1, 9)));
157     open(new Account("ACC-2001", "cust-2002", "****5518", "Individual Cash", Account.Type.CASH,
158         Registration.INDIVIDUAL, "USD", LocalDate.of(2024, 6, 2)));
159
160     // Every cash movement is a transaction, so balances reconcile with the history.
161     deposit("ACC-1001", "50000.00", 400);
162     deposit("ACC-1001", "10000.00", 120);
163     historicalBuy("ACC-1001", "EQ-AAPL", "50", "180.25", 360);
164     historicalBuy("ACC-1001", "EQ-MSFT", "20", "402.10", 300);
165     historicalBuy("ACC-1001", "ETF-SPY", "15", "540.00", 200);
166     dividend("ACC-1001", "EQ-AAPL", "13.00", 45);
167     dividend("ACC-1001", "ETF-SPY", "26.85", 20);
168     historicalBuy("ACC-1001", "EQ-AMZN", "12", "205.40", 9);
169
170     deposit("ACC-1002", "12000.00", 600);
171     historicalBuy("ACC-1002", "EQ-NVDA", "40", "120.50", 500);
172     dividend("ACC-1002", "EQ-NVDA", "0.40", 60);
173
174     deposit("ACC-2001", "5000.00", 90);
175     historicalBuy("ACC-2001", "EQ-AMZN", "10", "190.00", 80);
176     books.values().forEach(book -> book.transactions.sort(Comparator.comparing(Transaction::occurredAt)));
177 }
178
179 private void open(Account account) {
180     accounts.put(account.id(), account);
181     books.put(account.id(), new Book(BigDecimal.ZERO));
182 }
183
184 private Instant daysAgo(int days) {
185     return clock.instant().minus(Duration.ofDays(days)).minus(Duration.ofMinutes(days * 7L % 300));
186 }
187
188 private void deposit(String accountId, String amount, int daysAgo) {
189     Book book = books.get(accountId);
190     BigDecimal cash = new BigDecimal(amount);
191     book.cash = book.cash.add(cash);
192     Instant at = daysAgo(daysAgo);
193     book.transactions.add(new Transaction(nextTransactionId(), Type.DEPOSIT, "ACH deposit",
194         cash, null, null, null, null, null, at, settlementDate(at)));
195 }
196
197 private void dividend(String accountId, String instrumentId, String amount, int daysAgo) {
198     Instrument instrument = market.instrument(instrumentId);
199     Book book = books.get(accountId);
200     BigDecimal cash = new BigDecimal(amount);
201     book.cash = book.cash.add(cash);
202     Instant at = daysAgo(daysAgo);
203     LocalDate paid = at.atZone(ZoneOffset.UTC).toLocalDate(); // a dividend settles when it is paid
204     book.transactions.add(new Transaction(nextTransactionId(), Type.DIVIDEND,
205         "Cash dividend " + instrument.symbol(), cash, instrument.id(), instrument.symbol(),
206         null, null, null, at, paid));
207 }
208
209 private void historicalBuy(String accountId, String instrumentId, String quantity, String price,
210     int daysAgo) {
211     recordTrade(accountId, market.instrument(instrumentId), new BigDecimal(quantity),

```

```

212         new BigDecimal(price), BigDecimal.ZERO, null, daysAgo(daysAgo));
213     }
214 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Portfolio.java

77 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4  import java.math.RoundingMode;
5  import java.time.Clock;
6  import java.util.List;
7
8  import com.example.brokerage.api.accounts.Holdings.Balances;
9  import com.example.brokerage.api.accounts.Holdings.Position;
10 import com.example.brokerage.api.market.Instrument;
11 import com.example.brokerage.api.market.MarketData;
12 import com.example.brokerage.api.platform.ApiException;
13
14 import org.springframework.stereotype.Service;
15
16 /**
17  * Values the ledger at current market prices: positions and balances are computed on
18  * every request, never stored, so they can never drift from the books of record.
19  */
20 @Service
21 public class Portfolio {
22
23     private final Ledger ledger;
24     private final MarketData market;
25     private final Reservations reservations;
26     private final Clock clock;
27
28     public Portfolio(Ledger ledger, MarketData market, Reservations reservations, Clock clock) {
29         this.ledger = ledger;
30         this.market = market;
31         this.reservations = reservations;
32         this.clock = clock;
33     }
34
35     public List<Position> positions(Account account) {
36         return ledger.holdings(account.id()).stream().map(this::value).toList();
37     }
38
39     public Position position(Account account, String instrumentId) {
40         return ledger.holdings(account.id()).stream()
41             .filter(h -> h.instrumentId().equals(instrumentId))
42             .findFirst()
43             .map(this::value)
44             .orElseThrow(() -> ApiException.notFound("Position", instrumentId));
45     }
46
47     public Balances balances(Account account) {
48         List<Position> positions = positions(account);
49         BigDecimal cash = ledger.cash(account.id());
50         BigDecimal reserved = reservations.reservedCash(account.id());
51         BigDecimal marketValue = positions.stream().map(Position::marketValue)
52             .reduce(BigDecimal.ZERO, BigDecimal::add);
53         BigDecimal unrealized = positions.stream().map(Position::unrealizedPnl)
54             .reduce(BigDecimal.ZERO, BigDecimal::add);
55         return new Balances(account.id(), account.currency(), cash, reserved,
56             Holdings.buyingPower(account.type(), cash, reserved), marketValue, cash.add(marketValue),
57             unrealized, clock.instant());
58     }
59
60     private Position value(Ledger.Holding holding) {
61         Instrument instrument = market.instrument(holding.instrumentId());
62         BigDecimal last = market.tick(instrument.id()).last();
63         BigDecimal multiplier = instrument.multiplier();

```

```

64         BigDecimal costBasis = money(holding.quantity().multiply(holding.averageCost()).multiply(multiplier));
65         BigDecimal marketValue = money(holding.quantity().multiply(last).multiply(multiplier));
66         BigDecimal pnl = marketValue.subtract(costBasis);
67         BigDecimal pnlPercent = costBasis.signum() == 0 ? BigDecimal.ZERO
68             : pnl.multiply(BigDecimal.valueOf(100)).divide(costBasis, 2, RoundingMode.HALF_EVEN);
69         return new Position(instrument.id(), instrument.symbol(), instrument.type(), holding.quantity(),
70             holding.averageCost().setScale(2, RoundingMode.HALF_EVEN), costBasis, last, marketValue, pnl,
71             pnlPercent);
72     }
73
74     private static BigDecimal money(BigDecimal amount) {
75         return amount.setScale(2, RoundingMode.HALF_EVEN);
76     }
77 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Reservations.java

16 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4
5  /**
6   * What open orders have already claimed. Implemented by the order service; declared here
7   * so that the accounts package does not depend on the orders package.
8   */
9  public interface Reservations {
10
11      /** Cash held back for open buy orders. */
12      BigDecimal reservedCash(String accountId);
13
14      /** Units of an instrument already promised to open sell orders. */
15      BigDecimal reservedQuantity(String accountId, String instrumentId);
16 }

```

brokerage-api/src/main/java/com/example/brokerage/api/accounts/Transaction.java

22 lines

```

1  package com.example.brokerage.api.accounts;
2
3  import java.math.BigDecimal;
4  import java.time.Instant;
5  import java.time.LocalDate;
6
7  import org.springframework.hateoas.server.core.Relation;
8
9  /**
10   * One entry in an account's history. The amount is signed: money into the account is
11   * positive, money out is negative, so a client can sum a statement without a lookup table.
12   *
13   * @param settlesOn when the cash actually moves: US equity trades settle one business day
14   *                   after the trade (T+1)
15   */
16  @Relation(collectionRelation = "transactions", itemRelation = "transaction")
17  public record Transaction(String id, Type type, String description, BigDecimal amount, String instrumentId,
18      String symbol, BigDecimal quantity, BigDecimal price, String orderId, Instant occurredAt,
19      LocalDate settlesOn) {
20
21      public enum Type { DEPOSIT, WITHDRAWAL, TRADE, DIVIDEND, FEE }
22 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/IdempotencyKeys.java

56 lines

```

1  package com.example.brokerage.api.orders;
2
3  import java.time.Clock;
4  import java.time.Duration;
5  import java.time.Instant;
6  import java.util.Map;
7  import java.util.Optional;

```

```

8 import java.util.concurrent.ConcurrentHashMap;
9
10 import com.example.brokerage.api.platform.ApiException;
11 import com.example.brokerage.api.platform.ProblemType;
12
13 import org.springframework.stereotype.Component;
14
15 /**
16  * Remembers which order each Idempotency-Key created, for 24 hours.
17  *
18  * <p>A network timeout leaves a client unsure whether its order was placed. With a key,
19  * retrying is always safe: the same key and the same request return the original order
20  * instead of buying twice. The same key with a different request is a client bug, and is
21  * refused rather than guessed at.
22  */
23 @Component
24 public class IdempotencyKeys {
25
26     static final Duration RETENTION = Duration.ofHours(24);
27
28     private record Entry(String fingerprint, String orderId, Instant createdAt) {
29     }
30
31     private final Map<String, Entry> entries = new ConcurrentHashMap<>();
32     private final Clock clock;
33
34     public IdempotencyKeys(Clock clock) {
35         this.clock = clock;
36     }
37
38     /** The order already created with this key, if any. */
39     Optional<String> existingOrder(String scope, String key, String fingerprint) {
40         Instant now = clock.instant();
41         entries.values().removeIf(entry -> entry.createdAt().plus(RETENTION).isBefore(now));
42         Entry entry = entries.get(scope + "|" + key);
43         if (entry == null) {
44             return Optional.empty();
45         }
46         if (!entry.fingerprint().equals(fingerprint)) {
47             throw new ApiException(ProblemType.IDEMPOTENCY_KEY_REUSED,
48                 "Idempotency-Key " + key + " was already used for a different order request");
49         }
50         return Optional.of(entry.orderId());
51     }
52
53     void remember(String scope, String key, String fingerprint, String orderId) {
54         entries.put(scope + "|" + key, new Entry(fingerprint, orderId, clock.instant()));
55     }
56 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/Order.java

105 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Instant;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 import com.example.brokerage.api.market.Instrument;
10 import com.fasterxml.jackson.annotation.JsonIgnore;
11 import com.fasterxml.jackson.annotation.JsonProperty;
12
13 import org.springframework.hateoas.server.core.Relation;
14
15 /**
16  * An order and its lifecycle. Orders are never deleted: a canceled or filled order stays
17  * readable, which is why canceling is a state change (POST .../cancellation), not DELETE.

```

```

18 *
19 * <pre>
20 *   OPEN —fill→ PARTIALLY_FILLED —fill→ FILLED
21 *   |
22 *   |—cancel→ PENDING_CANCEL ←cancel—| → CANCELLED
23 * </pre>
24 *
25 * @param version incremented on every change; exposed only as the ETag header
26 */
27 @Relation(collectionRelation = "orders", itemRelation = "order")
28 public record Order(String id, String accountId, String clientOrderId, String instrumentId, String symbol,
29     Instrument.Type instrumentType, Side side, Type type, TimeInForce timeInForce, BigDecimal quantity,
30     BigDecimal limitPrice, BigDecimal stopPrice, Status status, BigDecimal filledQuantity,
31     BigDecimal averageFillPrice, List<Fill> fills, Instant createdAt, Instant updatedAt,
32     @JsonIgnore boolean stopTriggered, @JsonIgnore long version) {
33
34     public enum Side { BUY, SELL }
35
36     public enum Type { MARKET, LIMIT, STOP, STOP_LIMIT }
37
38     /** DAY orders expire at the close; GTC ("good 'til canceled") orders stay working. */
39     public enum TimeInForce { DAY, GTC }
40
41     public enum Status {
42         OPEN, PARTIALLY_FILLED, FILLED, PENDING_CANCEL, CANCELLED, REJECTED;
43
44         /** Can still fill, and so can still be changed or canceled. */
45         public boolean isWorking() {
46             return this == OPEN || this == PARTIALLY_FILLED;
47         }
48
49         /** Not yet in a final state. */
50         public boolean isOpen() {
51             return isWorking() || this == PENDING_CANCEL;
52         }
53     }
54
55     /** One execution against the order. */
56     public record Fill(String id, BigDecimal quantity, BigDecimal price, Instant executedAt) {
57     }
58
59     @JsonProperty
60     public BigDecimal remainingQuantity() {
61         return quantity.subtract(filledQuantity);
62     }
63
64     /** A strong entity tag: changes whenever the order changes. */
65     @JsonIgnore
66     public String etag() {
67         return "\"" + id + "-v" + version + "\"";
68     }
69
70     // ----- state changes
71
72     Order withStatus(Status newStatus, Instant at) {
73         return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
74             timeInForce, quantity, limitPrice, stopPrice, newStatus, filledQuantity,
75             averageFillPrice, fills, createdAt, at, stopTriggered, version + 1);
76     }
77
78     Order withStopTriggered() {
79         return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
80             timeInForce, quantity, limitPrice, stopPrice, status, filledQuantity, averageFillPrice, fills,
81             createdAt, updatedAt, true, version);
82     }
83
84     Order withFill(Fill fill) {
85         BigDecimal filled = filledQuantity.add(fill.quantity());
86         BigDecimal cost = averageFillPrice == null ? BigDecimal.ZERO

```

```

87         : averageFillPrice.multiply(filledQuantity);
88     BigDecimal average = cost.add(fill.price().multiply(fill.quantity()))
89         .divide(filled, 4, RoundingMode.HALF_EVEN);
90     List<Fill> allFills = new ArrayList<>(fills);
91     allFills.add(fill);
92     Status newStatus = filled.compareTo(quantity) == 0 ? Status.FILLED
93         : status == Status.PENDING_CANCEL ? Status.PENDING_CANCEL : Status.PARTIALLY_FILLED;
94     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
95         timeInForce, quantity, limitPrice, stopPrice, newStatus, filled, average,
96         List.copyOf(allFills), createdAt, fill.executedAt(), stopTriggered, version + 1);
97 }
98
99 Order withTerms(BigDecimal newQuantity, BigDecimal newLimit, BigDecimal newStop, TimeInForce newTif,
100     Instant at) {
101     return new Order(id, accountId, clientOrderId, instrumentId, symbol, instrumentType, side, type,
102         newTif, newQuantity, newLimit, newStop, status, filledQuantity, averageFillPrice, fills,
103         createdAt, at, stopTriggered, version + 1);
104 }
105 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderAmendment.java

33 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4
5 import jakarta.validation.constraints.Digits;
6 import jakarta.validation.constraints.Positive;
7
8 import org.springframework.hateoas.InputType;
9
10 /**
11  * The body of PATCH ../orders/{orderId}, sent as application/merge-patch+json (RFC 7396):
12  * only the fields present are changed. The instrument, side and type of a working order
13  * cannot change; cancel it and place a new one instead.
14  */
15 public class OrderAmendment {
16
17     @InputType("text") @Positive @Digits(integer = 9, fraction = 6)
18     private BigDecimal quantity;
19     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
20     private BigDecimal limitPrice;
21     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
22     private BigDecimal stopPrice;
23     private Order.TimeInForce timeInForce;
24
25     public BigDecimal getQuantity() { return quantity; }
26     public void setQuantity(BigDecimal quantity) { this.quantity = quantity; }
27     public BigDecimal getLimitPrice() { return limitPrice; }
28     public void setLimitPrice(BigDecimal limitPrice) { this.limitPrice = limitPrice; }
29     public BigDecimal getStopPrice() { return stopPrice; }
30     public void setStopPrice(BigDecimal stopPrice) { this.stopPrice = stopPrice; }
31     public Order.TimeInForce getTimeInForce() { return timeInForce; }
32     public void setTimeInForce(Order.TimeInForce timeInForce) { this.timeInForce = timeInForce; }
33 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderController.java

146 lines

```

1 package com.example.brokerage.api.orders;
2
3 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
4 import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
5
6 import java.util.List;
7
8 import jakarta.validation.Valid;
9
10 import com.example.brokerage.api.accounts.AccountController;
11 import com.example.brokerage.api.platform.Caller;

```

```

12 import com.example.brokerage.api.platform.CursorPage;
13
14 import org.springframework.hateoas.CollectionModel;
15 import org.springframework.hateoas.EntityModel;
16 import org.springframework.hateoas.IanaLinkRelations;
17 import org.springframework.http.HttpHeaders;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.security.core.annotation.AuthenticationPrincipal;
20 import org.springframework.security.oauth2.jwt.Jwt;
21 import org.springframework.web.bind.annotation.GetMapping;
22 import org.springframework.web.bind.annotation.PatchMapping;
23 import org.springframework.web.bind.annotation.PathVariable;
24 import org.springframework.web.bind.annotation.PostMapping;
25 import org.springframework.web.bind.annotation.RequestBody;
26 import org.springframework.web.bind.annotation.RequestHeader;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29 import org.springframework.web.context.request.WebRequest;
30
31 /**
32  * Orders. Reading needs orders:read; placing, changing and canceling need orders:write.
33  */
34 @RestController
35 public class OrderController {
36
37     public static final String IDEMPOTENCY_KEY = "Idempotency-Key";
38     public static final String MERGE_PATCH_JSON = "application/merge-patch+json";
39
40     private final OrderService service;
41     private final OrderModelAssembler assembler;
42
43     public OrderController(OrderService service, OrderModelAssembler assembler) {
44         this.service = service;
45         this.assembler = assembler;
46     }
47
48     /** Orders, newest first, optionally filtered by status (repeatable) and symbol. */
49     @GetMapping("/accounts/{accountId}/orders")
50     public CollectionModel<EntityModel<Order>> orders(@PathVariable String accountId,
51         @RequestParam(required = false) List<Order.Status> status,
52         @RequestParam(required = false) String symbol,
53         @RequestParam(required = false) String cursor, @RequestParam(required = false) Integer limit,
54         @AuthenticationPrincipal Jwt jwt) {
55         Caller caller = Caller.of(jwt);
56         CursorPage<Order> page = service.orders(caller, accountId, status, symbol, cursor, limit);
57         CollectionModel<EntityModel<Order>> model = CollectionModel.of(
58             page.items().stream().map(order -> assembler.toModel(order, caller)).toList(),
59             linkTo(methodOn(OrderController.class).orders(accountId, status, symbol, cursor, limit, null))
60                 .withSelfRel().expand(),
61             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
62         if (page.hasNext()) {
63             model.add(linkTo(methodOn(OrderController.class)
64                 .orders(accountId, status, symbol, page.nextCursor(), limit, null))
65                 .withRel(IanaLinkRelations.NEXT).expand());
66         }
67         return model;
68     }
69
70     /**
71      * Places an order: 201 Created with a Location header and the order's ETag. Retrying
72      * with the same Idempotency-Key returns the original order, marked Idempotent-Replayed.
73      */
74     @PostMapping("/accounts/{accountId}/orders")
75     public ResponseEntity<EntityModel<Order>> placeOrder(@PathVariable String accountId,
76         @RequestHeader(name = IDEMPOTENCY_KEY, required = false) String idempotencyKey,
77         @Valid @RequestBody OrderRequest request, @AuthenticationPrincipal Jwt jwt) {
78         Caller caller = Caller.of(jwt);
79         OrderService.Placement placement = service.place(caller, accountId, request, idempotencyKey);
80         EntityModel<Order> model = assembler.toModel(placement.order(), caller);

```

```

81         return ResponseEntity.created(model.getRequiredLink(IanaLinkRelations.SELF).toUri())
82             .eTag(placement.order().etag())
83             .header("Idempotent-Replayed", Boolean.toString(placement.replayed()))
84             .body(model);
85     }
86
87     /** One order. Its ETag supports cheap polling (If-None-Match) and safe edits (If-Match). */
88     @GetMapping("/accounts/{accountId}/orders/{orderId}")
89     public ResponseEntity<EntityModel<Order>> order(@PathVariable String accountId,
90         @PathVariable String orderId, WebRequest request, @AuthenticationPrincipal Jwt jwt) {
91         Caller caller = Caller.of(jwt);
92         Order order = service.order(caller, accountId, orderId);
93         if (request.checkNotModified(order.etag())) {
94             return null;
95         }
96         return ResponseEntity.ok().eTag(order.etag()).body(assembler.toModel(order, caller));
97     }
98
99     /** Changes a working order's quantity, prices or time in force. Requires If-Match. */
100    @PatchMapping(path = "/accounts/{accountId}/orders/{orderId}",
101        consumes = { MERGE_PATCH_JSON, "application/json" })
102    public ResponseEntity<EntityModel<Order>> replaceOrder(@PathVariable String accountId,
103        @PathVariable String orderId,
104        @RequestHeader(name = HttpHeaders.IF_MATCH, required = false) String ifMatch,
105        @Valid @RequestBody OrderAmendment change, @AuthenticationPrincipal Jwt jwt) {
106        Caller caller = Caller.of(jwt);
107        Order order = service.amend(caller, accountId, orderId, change, ifMatch);
108        return ResponseEntity.ok().eTag(order.etag()).body(assembler.toModel(order, caller));
109    }
110
111    /**
112     * Requests cancellation. 202 Accepted: the cancel is in progress, and the Location
113     * header points at the order, whose status will become CANCELLED.
114     */
115    @PostMapping("/accounts/{accountId}/orders/{orderId}/cancellation")
116    public ResponseEntity<EntityModel<Order>> cancelOrder(@PathVariable String accountId,
117        @PathVariable String orderId, @AuthenticationPrincipal Jwt jwt) {
118        Caller caller = Caller.of(jwt);
119        Order order = service.cancel(caller, accountId, orderId);
120        EntityModel<Order> model = assembler.toModel(order, caller);
121        return ResponseEntity.accepted()
122            .location(linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
123                .toUri())
124            .eTag(order.etag())
125            .body(model);
126    }
127
128    /**
129     * Estimates an order without placing it. Safe to call as often as a user edits a
130     * ticket; when the order would be accepted, a link to place it is included.
131     */
132    @PostMapping("/accounts/{accountId}/order-previews")
133    public EntityModel<OrderPreview> previewOrder(@PathVariable String accountId,
134        @Valid @RequestBody OrderRequest request, @AuthenticationPrincipal Jwt jwt) {
135        Caller caller = Caller.of(jwt);
136        OrderPreview preview = service.preview(caller, accountId, request);
137        EntityModel<OrderPreview> model = EntityModel.of(preview,
138            linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"));
139        if (preview.acceptable() && caller.canTrade()) {
140            // expand(): this link is a POST target, so the listing's query parameters do not apply
141            model.add(linkTo(methodOn(OrderController.class).orders(accountId, null, null, null, null))
142                .withRel("place-order").expand());
143        }
144        return model;
145    }
146 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.time.Instant;
5 import java.util.Map;
6
7 import com.example.brokerage.api.accounts.Ledger;
8 import com.example.brokerage.api.platform.Caller;
9 import com.fasterxml.jackson.annotation.JsonProperty;
10
11 import org.springframework.http.MediaType;
12 import org.springframework.security.core.annotation.AuthenticationPrincipal;
13 import org.springframework.security.oauth2.jwt.Jwt;
14 import org.springframework.web.bind.annotation.GetMapping;
15 import org.springframework.web.bind.annotation.PathVariable;
16 import org.springframework.web.bind.annotation.RequestHeader;
17 import org.springframework.web.bind.annotation.RestController;
18 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
19 import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
20
21 /**
22  * A live stream of an account's order events (text/event-stream). Scope: orders:read.
23  */
24 @RestController
25 public class OrderEventController {
26
27     /** The data of one event: a summary, and a link to the full order. */
28     public record OrderEventData(String orderId, Order.Status status, BigDecimal filledQuantity,
29         BigDecimal remainingQuantity, BigDecimal averageFillPrice, Instant occurredAt,
30         @JsonProperty("_links") Map<String, Map<String, String>> links) {
31     }
32
33     private final OrderEvents events;
34     private final Ledger ledger;
35
36     public OrderEventController(OrderEvents events, Ledger ledger) {
37         this.events = events;
38         this.ledger = ledger;
39     }
40
41     @GetMapping(path = "/accounts/{accountId}/order-events", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
42     public SseEmitter events(@PathVariable String accountId,
43         @RequestHeader(name = "Last-Event-ID", required = false) Long lastEventId,
44         @AuthenticationPrincipal Jwt jwt) {
45         ledger.account(Caller.of(jwt).customerId(), accountId);
46         // Events are sent later, outside this request, so capture the base URI now.
47         String base = ServletUriComponentsBuilder.fromCurrentContextPath().toUriString();
48         return events.subscribe(accountId, lastEventId, event -> render(base, event));
49     }
50
51     static OrderEventData render(String base, OrderEvents.Event event) {
52         Order order = event.order();
53         String href = base + "/accounts/" + order.accountId() + "/orders/" + order.id();
54         return new OrderEventData(order.id(), order.status(), order.filledQuantity(),
55             order.remainingQuantity(), order.averageFillPrice(), event.at(),
56             Map.of("order", Map.of("href", href)));
57     }
58 }

```

```

1 package com.example.brokerage.api.orders;
2
3 import java.io.IOException;
4 import java.time.Clock;
5 import java.time.Instant;
6 import java.util.ArrayDeque;

```

```

7 import java.util.Deque;
8 import java.util.List;
9 import java.util.Map;
10 import java.util.concurrent.ConcurrentHashMap;
11 import java.util.concurrent.CopyOnWriteArrayList;
12 import java.util.concurrent.atomic.AtomicLong;
13 import java.util.function.Function;
14
15 import org.springframework.http.MediaType;
16 import org.springframework.stereotype.Component;
17 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
18
19 /**
20  * Order lifecycle events, pushed to subscribers as Server-Sent Events.
21  *
22  * <p>Every event gets an increasing ID, and the last 500 events of each account are kept.
23  * A client that loses its connection reconnects with a Last-Event-ID header and receives
24  * exactly the events it missed: no polling, and no gaps.
25  */
26 @Component
27 public class OrderEvents {
28
29     /** Something happened to an order. The type is order.created, order.filled, etc. */
30     public record Event(long id, String type, Order order, Instant at) {
31     }
32
33     private record Subscriber(SseEmitter emitter, Function<Event, Object> renderer) {
34     }
35
36     private static final int HISTORY = 500;
37     private static final long TIMEOUT_MILLIS = 30 * 60 * 1000L;
38
39     private final AtomicLong sequence = new AtomicLong();
40     private final Map<String, Deque<Event>> history = new ConcurrentHashMap<>();
41     private final Map<String, List<Subscriber>> subscribers = new ConcurrentHashMap<>();
42     private final Clock clock;
43
44     public OrderEvents(Clock clock) {
45         this.clock = clock;
46     }
47
48     void publish(String type, Order order) {
49         Event event = new Event(sequence.incrementAndGet(), type, order, clock.instant());
50         Deque<Event> events = history.computeIfAbsent(order.accountId(), id -> new ArrayDeque<>());
51         synchronized (events) {
52             events.addLast(event);
53             if (events.size() > HISTORY) {
54                 events.removeFirst();
55             }
56         }
57         for (Subscriber subscriber : subscribers.getOrDefault(order.accountId(), List.of())) {
58             send(order.accountId(), subscriber, event);
59         }
60     }
61
62     /** Opens a stream for one account, first replaying anything after lastEventId. */
63     public SseEmitter subscribe(String accountId, Long lastEventId, Function<Event, Object> renderer) {
64         Subscriber subscriber = new Subscriber(new SseEmitter(TIMEOUT_MILLIS), renderer);
65         subscriber.emitter().onCompletion(() -> unsubscribe(accountId, subscriber));
66         subscriber.emitter().onTimeout(() -> unsubscribe(accountId, subscriber));
67         if (lastEventId != null) {
68             replay(accountId, lastEventId).forEach(event -> send(accountId, subscriber, event));
69         }
70         subscribers.computeIfAbsent(accountId, id -> new CopyOnWriteArrayList<>()).add(subscriber);
71         return subscriber.emitter();
72     }
73
74     /** Events for an account with an ID greater than lastEventId, oldest first. */
75     public List<Event> replay(String accountId, long lastEventId) {

```

```

76     Deque<Event> events = history.getDefault(accountId, new ArrayDeque<>());
77     synchronized (events) {
78         return events.stream().filter(event -> event.id() > lastEventId).toList();
79     }
80 }
81
82 private void send(String accountId, Subscriber subscriber, Event event) {
83     try {
84         subscriber.emitter().send(SseEmitter.event()
85             .id(Long.toString(event.id()))
86             .name(event.type())
87             .data(subscriber.renderer().apply(event), MediaType.APPLICATION_JSON));
88     } catch (IOException | IllegalStateException gone) {
89         unsubscribe(accountId, subscriber);
90     }
91 }
92
93 private void unsubscribe(String accountId, Subscriber subscriber) {
94     subscribers.getDefault(accountId, List.of()).remove(subscriber);
95 }
96 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderModelAssembler.java

57 lines

```

1  package com.example.brokerage.api.orders;
2
3  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.afford;
4  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
5  import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;
6
7  import com.example.brokerage.api.accounts.AccountController;
8  import com.example.brokerage.api.market.InstrumentController;
9  import com.example.brokerage.api.platform.Caller;
10
11 import org.springframework.hateoas.EntityModel;
12 import org.springframework.hateoas.IanaLinkRelations;
13 import org.springframework.hateoas.Link;
14 import org.springframework.stereotype.Component;
15
16 /**
17  * Builds an order's representation. This is where hypermedia earns its keep: the links
18  * an order carries depend on its state and on the caller's token.
19  *
20  * <ul>
21  * <li>A working order, seen by a token with orders:write, offers "edit" (change it with
22  *   PATCH) and "bk:cancel"; in HAL-FORMS both come with a template describing the
23  *   request.</li>
24  * <li>A filled or canceled order offers neither. A client that shows buttons only for
25  *   links that are present can never offer an action the server would refuse.</li>
26  * </ul>
27  */
28 @Component
29 public class OrderModelAssembler {
30
31     public EntityModel<Order> toModel(Order order, Caller caller) {
32         String accountId = order.accountId();
33         String orderId = order.id();
34         boolean changeable = order.status().isWorking() && caller.canTrade();
35
36         Link self = linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
37             .withSelfRel();
38         if (changeable) {
39             self = self.andAffordance(afford(methodOn(OrderController.class)
40                 .replaceOrder(accountId, orderId, null, null, null)));
41         }
42         EntityModel<Order> model = EntityModel.of(order, self,
43             linkTo(methodOn(AccountController.class).account(accountId, null)).withRel("account"),
44             linkTo(methodOn(InstrumentController.class).instrument(order.instrumentId()))
45                 .withRel("instrument"));
46     }
47 }

```

```

46
47     if (changeable) {
48         model.add(linkTo(methodOn(OrderController.class).order(accountId, orderId, null, null))
49             .withRel(IanaLinkRelations.EDIT));
50         model.add(linkTo(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))
51             .withRel("cancel")
52             .andAffordance(
53                 afford(methodOn(OrderController.class).cancelOrder(accountId, orderId, null))));
54     }
55     return model;
56 }
57 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderPreview.java

22 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.net.URI;
5 import java.util.List;
6
7 /**
8  * What an order would cost and whether it would be accepted, without placing it. A preview
9  * that fails a business rule is still a successful response: "acceptable" is false and each
10 * issue carries the same problem type URI that placing the order would return.
11 *
12 * @param estimatedTotal for a purchase, the cash needed; for a sale, the expected proceeds
13 */
14 public record OrderPreview(String accountId, String instrumentId, String symbol, Order.Side side,
15     Order.Type type, BigDecimal quantity, BigDecimal estimatedPrice, BigDecimal estimatedNotional,
16     BigDecimal estimatedCommission, BigDecimal estimatedTotal, BigDecimal buyingPower,
17     boolean acceptable, List<Issue> issues) {
18
19     /** A reason the order would be refused. */
20     public record Issue(URI type, String detail) {
21     }
22 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderRequest.java

88 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4
5 import jakarta.validation.constraints.Digits;
6 import jakarta.validation.constraints.NotBlank;
7 import jakarta.validation.constraints.NotNull;
8 import jakarta.validation.constraints.Pattern;
9 import jakarta.validation.constraints.Positive;
10 import jakarta.validation.constraints.Size;
11
12 import org.springframework.hateoas.InputType;
13
14 /**
15  * The body of POST .../orders and POST .../order-previews. Field-level rules are declared
16  * here and checked by Bean Validation; rules that involve several fields or the account
17  * (a limit order needs a limit price, a sale needs shares to sell) live in OrderService.
18  *
19  * <p>This is a JavaBean rather than a record on purpose: Spring HATEOAS derives HAL-FORMS
20  * templates from bean properties, and marks a property without a setter as read-only.
21  */
22 public class OrderRequest {
23
24     @NotBlank
25     private String instrumentId;
26     @NotNull
27     private Order.Side side;
28     @NotNull
29     private Order.Type type;

```

```

30     private Order.TimeInForce timeInForce;
31     @InputType("text") @NotNull @Positive @Digits(integer = 9, fraction = 6)
32     private BigDecimal quantity;
33     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
34     private BigDecimal limitPrice;
35     @InputType("text") @Positive @Digits(integer = 9, fraction = 4)
36     private BigDecimal stopPrice;
37     /** The caller's own reference for the order, echoed back unchanged. */
38     @InputType("text") @Size(max = 64) @Pattern(regexp = "[A-Za-z0-9._]*")
39     private String clientId;
40
41     public OrderRequest() {
42     }
43
44     public OrderRequest(String instrumentId, Order.Side side, Order.Type type, Order.TimeInForce timeInForce,
45         BigDecimal quantity, BigDecimal limitPrice, BigDecimal stopPrice, String clientId) {
46         this.instrumentId = instrumentId;
47         this.side = side;
48         this.type = type;
49         this.timeInForce = timeInForce;
50         this.quantity = quantity;
51         this.limitPrice = limitPrice;
52         this.stopPrice = stopPrice;
53         this.clientId = clientId;
54     }
55
56     public Order.TimeInForce timeInForceOrDefault() {
57         return timeInForce == null ? Order.TimeInForce.DAY : timeInForce;
58     }
59
60     /** A stable fingerprint, so a retried request can be recognized as the same request. */
61     String fingerprint() {
62         return String.join("|", instrumentId, side.name(), type.name(), timeInForceOrDefault().name(),
63             plain(quantity), plain(limitPrice), plain(stopPrice), String.valueOf(clientId));
64     }
65
66     private static String plain(BigDecimal value) {
67         return value == null ? "" : value.stripTrailingZeros().toPlainString();
68     }
69
70     // ----- accessors
71
72     public String getInstrumentId() { return instrumentId; }
73     public void setInstrumentId(String instrumentId) { this.instrumentId = instrumentId; }
74     public Order.Side getSide() { return side; }
75     public void setSide(Order.Side side) { this.side = side; }
76     public Order.Type getType() { return type; }
77     public void setType(Order.Type type) { this.type = type; }
78     public Order.TimeInForce getTimeInForce() { return timeInForce; }
79     public void setTimeInForce(Order.TimeInForce timeInForce) { this.timeInForce = timeInForce; }
80     public BigDecimal getQuantity() { return quantity; }
81     public void setQuantity(BigDecimal quantity) { this.quantity = quantity; }
82     public BigDecimal getLimitPrice() { return limitPrice; }
83     public void setLimitPrice(BigDecimal limitPrice) { this.limitPrice = limitPrice; }
84     public BigDecimal getStopPrice() { return stopPrice; }
85     public void setStopPrice(BigDecimal stopPrice) { this.stopPrice = stopPrice; }
86     public String getClientId() { return clientId; }
87     public void setClientId(String clientId) { this.clientId = clientId; }
88 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/OrderService.java

402 lines

```

1 package com.example.brokerage.api.orders;
2
3 import java.math.BigDecimal;
4 import java.math.RoundingMode;
5 import java.time.Clock;
6 import java.time.Instant;
7 import java.util.ArrayList;

```

```

8 import java.util.Collection;
9 import java.util.LinkedHashMap;
10 import java.util.List;
11 import java.util.Map;
12 import java.util.Optional;
13 import java.util.concurrent.atomic.AtomicInteger;
14
15 import com.example.brokerage.api.accounts.Account;
16 import com.example.brokerage.api.accounts.Holdings;
17 import com.example.brokerage.api.accounts.Ledger;
18 import com.example.brokerage.api.accounts.Reservations;
19 import com.example.brokerage.api.market.Instrument;
20 import com.example.brokerage.api.market.MarketData;
21 import com.example.brokerage.api.orders.Order.Side;
22 import com.example.brokerage.api.orders.Order.Status;
23 import com.example.brokerage.api.orders.Order.TimeInForce;
24 import com.example.brokerage.api.orders.Order.Type;
25 import com.example.brokerage.api.platform.ApiException;
26 import com.example.brokerage.api.platform.Caller;
27 import com.example.brokerage.api.platform.CursorPage;
28 import com.example.brokerage.api.platform.ProblemType;
29
30 import org.springframework.stereotype.Service;
31
32 /**
33  * Order placement, preview, amendment, cancellation and matching.
34  *
35  * <p>All state changes are serialized by one lock: simple and obviously correct, which
36  * is what an example should be. A real system would partition by account.
37  */
38 @Service
39 public class OrderService implements Reservations {
40
41     /** Placement result: the order, and whether this was a replay of an earlier request. */
42     public record Placement(Order order, boolean replayed) {
43     }
44
45     private static final BigDecimal OPTION_COMMISSION = new BigDecimal("0.65");
46
47     private final Map<String, Order> orders = new LinkedHashMap<>();
48     private final AtomicInteger orderIds = new AtomicInteger(100_000);
49     private final AtomicInteger fillIds = new AtomicInteger(500_000);
50     private final Ledger ledger;
51     private final MarketData market;
52     private final IdempotencyKeys idempotencyKeys;
53     private final OrderEvents events;
54     private final Clock clock;
55
56     public OrderService(Ledger ledger, MarketData market, IdempotencyKeys idempotencyKeys,
57         OrderEvents events, Clock clock) {
58         this.ledger = ledger;
59         this.market = market;
60         this.idempotencyKeys = idempotencyKeys;
61         this.events = events;
62         this.clock = clock;
63         // A resting limit order, so that there is something to change and cancel.
64         Instant yesterday = clock.instant().minusSeconds(86_400);
65         save(new Order(nextOrderId(), "ACC-1001", "rebalance-msft-01", "EQ-MSFT", "MSFT",
66             Instrument.Type.EQUITY, Side.BUY, Type.LIMIT, TimeInForce.GTC, new BigDecimal("10"),
67             new BigDecimal("480.00"), null, Status.OPEN, BigDecimal.ZERO, null, List.of(),
68             yesterday, yesterday, false, 1));
69     }
70
71     // ----- queries
72
73     public synchronized Order order(Caller caller, String accountId, String orderId) {
74         ledger.account(caller.customerId(), accountId);
75         Order order = orders.get(orderId);
76         if (order == null || !order.accountId().equals(accountId)) {

```

```

77         throw ApiException.notFound("Order", orderId);
78     }
79     return order;
80 }
81
82 public synchronized CursorPage<Order> orders(Caller caller, String accountId,
83     Collection<Status> statuses, String symbol, String cursor, Integer limit) {
84     ledger.account(caller.customerId(), accountId);
85     List<Order> matching = orders.values().stream()
86         .filter(o -> o.accountId().equals(accountId))
87         .filter(o -> statuses == null || statuses.isEmpty() || statuses.contains(o.status()))
88         .filter(o -> symbol == null || o.symbol().equalsIgnoreCase(symbol))
89         .toList();
90     return CursorPage.of(matching, Order::createdAt, Order::id, cursor, limit);
91 }
92
93 // ----- commands
94
95 public synchronized Placement place(Caller caller, String accountId, OrderRequest request,
96     String idempotencyKey) {
97     Account account = ledger.account(caller.customerId(), accountId);
98     if (idempotencyKey == null || idempotencyKey.isBlank() || idempotencyKey.length() > 255) {
99         throw new ApiException(ProblemType.IDEMPOTENCY_KEY_MISSING,
100             "Send a unique Idempotency-Key header (up to 255 characters) with every new order");
101     }
102     String scope = caller.clientId() + "|" + accountId;
103     String fingerprint = request.fingerprint();
104     Optional<String> earlier = idempotencyKeys.existingOrder(scope, idempotencyKey, fingerprint);
105     if (earlier.isPresent()) {
106         return new Placement(orders.get(earlier.get()), true);
107     }
108
109     Assessment assessment = assess(caller, account, request);
110     assessment.issues().stream().findFirst().ifPresent(issue -> {
111         throw issue;
112     });
113
114     Instant now = clock.instant();
115     Instrument instrument = assessment.instrument();
116     Order order = new Order(nextOrderId(), accountId, request.getClientOrderId(), instrument.id(),
117         instrument.symbol(), instrument.type(), request.getSide(), request.getType(),
118         request.timeInForceOrDefault(), request.getQuantity(), request.getLimitPrice(),
119         request.getStopPrice(), Status.OPEN, BigDecimal.ZERO, null, List.of(), now, now,
120         false, 1);
121     save(order);
122     idempotencyKeys.remember(scope, idempotencyKey, fingerprint, order.id());
123     events.publish("order.created", order);
124     return new Placement(order, false);
125 }
126
127 public synchronized OrderPreview preview(Caller caller, String accountId, OrderRequest request) {
128     Account account = ledger.account(caller.customerId(), accountId);
129     Assessment assessment = assess(caller, account, request);
130     List<OrderPreview.Issue> issues = assessment.issues().stream()
131         .map(issue -> new OrderPreview.Issue(issue.type().uri(), issue.getMessage()))
132         .toList();
133     Instrument instrument = assessment.instrument();
134     return new OrderPreview(accountId, request.getInstrumentId(),
135         instrument == null ? null : instrument.symbol(), request.getSide(), request.getType(),
136         request.getQuantity(), assessment.price(), assessment.notional(), assessment.commission(),
137         assessment.total(), assessment.buyingPower(), issues.isEmpty(), issues);
138 }
139
140 /**
141  * Changes a working order. The If-Match header must carry the order's current ETag:
142  * two clients editing the same order cannot silently overwrite one another.
143  */
144 public synchronized Order amend(Caller caller, String accountId, String orderId,
145     OrderAmendment change, String ifMatch) {

```

```

146     Order order = order(caller, accountId, orderId);
147     if (ifMatch == null) {
148         throw new ApiException(ProblemType.PRECONDITION_REQUIRED,
149             "Send If-Match with the order's current ETag to change it");
150     }
151     if (!ifMatch.equals(order.etag()) && !ifMatch.equals("*")) {
152         throw new ApiException(ProblemType.PRECONDITION_FAILED,
153             "The order has changed since you read it; fetch it again and reapply your change")
154         .with("currentEtag", order.etag());
155     }
156     if (!order.status().isWorking()) {
157         throw new ApiException(ProblemType.ORDER_NOT_REPLACEABLE,
158             "Order " + orderId + " is " + order.status() + " and can no longer be changed");
159     }
160     BigDecimal quantity = orElse(change.getQuantity(), order.quantity());
161     if (quantity.compareTo(order.filledQuantity()) <= 0) {
162         throw new ApiException(ProblemType.INVALID_REQUEST, "quantity must be greater than the "
163             + order.filledQuantity().toString() + " already filled");
164     }
165     OrderRequest amended = new OrderRequest(order.instrumentId(), order.side(), order.type(),
166         orElse(change.getTimeInForce(), order.timeInForce()), quantity,
167         orElse(change.getLimitPrice(), order.limitPrice()),
168         orElse(change.getStopPrice(), order.stopPrice()), order.clientOrderId());
169
170     Account account = ledger.account(caller.customerId(), accountId);
171     orders.remove(orderId); // assess the new terms without this order's own reservation
172     try {
173         assess(caller, account, amended).issues().stream().findFirst().ifPresent(issue -> {
174             throw issue;
175         });
176     } finally {
177         orders.put(orderId, order);
178     }
179     Order updated = order.withTerms(amended.getQuantity(), amended.getLimitPrice(),
180         amended.getStopPrice(), amended.getTimeInForce(), clock.instant());
181     save(updated);
182     events.publish("order.replaced", updated);
183     return updated;
184 }
185
186 /**
187  * Requests cancellation. Like a real exchange, the cancel is asynchronous: the order
188  * moves to PENDING_CANCEL now and to CANCELLED at the next matching pass, and it may
189  * still fill in between.
190  */
191 public synchronized Order cancel(Caller caller, String accountId, String orderId) {
192     Order order = order(caller, accountId, orderId);
193     if (order.status() == Status.PENDING_CANCEL) {
194         return order;
195     }
196     if (!order.status().isWorking()) {
197         throw new ApiException(ProblemType.ORDER_NOT_CANCELLABLE,
198             "Order " + orderId + " is " + order.status() + " and cannot be canceled");
199     }
200     Order pending = order.withStatus(Status.PENDING_CANCEL, clock.instant());
201     save(pending);
202     events.publish("order.cancel-requested", pending);
203     return pending;
204 }
205
206 // ----- matching
207
208 /** One pass of the simulated exchange over every open order. */
209 public synchronized void match() {
210     for (Order order : List.copyOf(orders.values())) {
211         if (order.status() == Status.PENDING_CANCEL) {
212             Order cancelled = order.withStatus(Status.CANCELLED, clock.instant());
213             save(cancelled);
214             events.publish("order.cancelled", cancelled);

```

```

215         } else if (order.status().isWorking()) {
216             tryToFill(order);
217         }
218     }
219 }
220
221 private void tryToFill(Order order) {
222     Instrument instrument = market.instrument(order.instrumentId());
223     MarketData.Tick tick = market.tick(instrument.id());
224     boolean buy = order.side() == Side.BUY;
225     Order current = order;
226
227     if ((order.type() == Type.STOP || order.type() == Type.STOP_LIMIT) && !order.stopTriggered()) {
228         boolean triggered = buy ? tick.last().compareTo(order.stopPrice()) >= 0
229             : tick.last().compareTo(order.stopPrice()) <= 0;
230         if (!triggered) {
231             return;
232         }
233         current = current.withStopTriggered();
234         save(current);
235     }
236
237     BigDecimal price = buy ? tick.ask() : tick.bid();
238     if (order.type() == Type.LIMIT || order.type() == Type.STOP_LIMIT) {
239         boolean marketable = buy ? price.compareTo(order.limitPrice()) <= 0
240             : price.compareTo(order.limitPrice()) >= 0;
241         if (!marketable) {
242             return;
243         }
244     }
245
246     BigDecimal displayed = buy ? tick.askSize() : tick.bidSize();
247     BigDecimal quantity = current.remainingQuantity().min(displayed);
248     Instant now = clock.instant();
249     ledger.recordTrade(order.accountId(), instrument, buy ? quantity : quantity.negate(), price,
250         commission(instrument, quantity), order.id(), now);
251     String fillId = "FIL-" + fillIds.incrementAndGet();
252     Order filled = current.withFill(new Order.Fill(fillId, quantity, price, now));
253     save(filled);
254     boolean complete = filled.status() == Status.FILLED;
255     events.publish(complete ? "order.filled" : "order.partially-filled", filled);
256 }
257
258 // ----- reservations
259
260 @Override
261 public synchronized BigDecimal reservedCash(String accountId) {
262     return orders.values().stream()
263         .filter(o -> o.accountId().equals(accountId))
264         .filter(o -> o.side() == Side.BUY && o.status().isOpen())
265         .map(o -> {
266             Instrument instrument = market.instrument(o.instrumentId());
267             BigDecimal price = referencePrice(o.type(), Side.BUY, o.limitPrice(),
268                 o.stopPrice(), instrument);
269             return price.multiply(o.remainingQuantity()).multiply(instrument.multiplier())
270                 .add(commission(instrument, o.remainingQuantity()));
271         })
272         .reduce(BigDecimal.ZERO, BigDecimal::add)
273         .setScale(2, RoundingMode.HALF_EVEN);
274 }
275
276 @Override
277 public synchronized BigDecimal reservedQuantity(String accountId, String instrumentId) {
278     return orders.values().stream()
279         .filter(o -> o.accountId().equals(accountId) && o.instrumentId().equals(instrumentId))
280         .filter(o -> o.side() == Side.SELL && o.status().isOpen())
281         .map(Order::remainingQuantity)
282         .reduce(BigDecimal.ZERO, BigDecimal::add);
283 }

```

```

284
285 // ----- the rules
286
287 /** The outcome of checking an order request against the rules, without acting on it. */
288 private record Assessment(Instrument instrument, BigDecimal price, BigDecimal notional,
289     BigDecimal commission, BigDecimal total, BigDecimal buyingPower,
290     List<ApiException> issues) {
291
292     static Assessment failed(Instrument instrument, List<ApiException> issues) {
293         return new Assessment(instrument, null, null, null, null, null, issues);
294     }
295 }
296
297 private Assessment assess(Caller caller, Account account, OrderRequest request) {
298     List<ApiException> issues = new ArrayList<>();
299     Instrument instrument;
300     try {
301         instrument = market.instrument(request.getInstrumentId());
302     } catch (ApiException notFound) {
303         issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,
304             "Instrument " + request.getInstrumentId() + " does not exist"));
305         return Assessment.failed(null, issues);
306     }
307
308     checkPrices(request, issues);
309     if (!instrument.tradable()) {
310         issues.add(new ApiException(ProblemType.INSTRUMENT_NOT_TRADABLE,
311             instrument.symbol() + " is not tradable"));
312     }
313     boolean wholeUnits = request.getQuantity().stripTrailingZeros().scale() <= 0;
314     if (!wholeUnits && (!instrument.fractionable() || request.getType() != Type.MARKET)) {
315         issues.add(new ApiException(ProblemType.INVALID_REQUEST,
316             "Fractional quantities are allowed only in market orders for fractionable stocks"));
317     }
318     if (!issues.isEmpty()) {
319         return Assessment.failed(instrument, issues);
320     }
321
322     BigDecimal price = referencePrice(request.getType(), request.getSide(),
323         request.getLimitPrice(), request.getStopPrice(), instrument);
324     BigDecimal notional = price.multiply(request.getQuantity())
325         .multiply(instrument.multiplier())
326         .setScale(2, RoundingMode.HALF_EVEN);
327     BigDecimal commission = commission(instrument, request.getQuantity());
328     boolean buy = request.getSide() == Side.BUY;
329     BigDecimal total = buy ? notional.add(commission) : notional.subtract(commission);
330     BigDecimal buyingPower = Holdings.buyingPower(account.type(), ledger.cash(account.id()),
331         reservedCash(account.id()));
332
333     if (buy && total.compareTo(buyingPower) > 0) {
334         issues.add(new ApiException(ProblemType.INSUFFICIENT_BUYING_POWER,
335             "This order needs " + total.toPlainString() + " but only "
336             + buyingPower.toPlainString() + " is available")
337             .with("required", total).with("available", buyingPower));
338     }
339     if (!buy) {
340         BigDecimal available = ledger.quantityHeld(account.id(), instrument.id())
341             .subtract(reservedQuantity(account.id(), instrument.id()));
342         if (request.getQuantity().compareTo(available) > 0) {
343             issues.add(new ApiException(ProblemType.INSUFFICIENT_POSITION,
344                 "Only " + available.toPlainString() + " " + instrument.symbol()
345                 + " are available to sell; short selling is not supported")
346                 .with("available", available));
347         }
348     }
349     caller.tradingLimit().filter(limit -> notional.compareTo(limit) > 0).ifPresent(limit ->
350         issues.add(new ApiException(ProblemType.TRADING_LIMIT_EXCEEDED,
351             "This client may place orders up to " + limit.toPlainString()
352             + " USD; this one is " + notional.toPlainString())

```

```

353         .with("tradingLimit", limit));
354     return new Assessment(instrument, price, notional, commission, total, buyingPower, issues);
355 }
356
357 private static void checkPrices(OrderRequest request, List<ApiException> issues) {
358     Type type = request.getType();
359     boolean needsLimit = type == Type.LIMIT || type == Type.STOP_LIMIT;
360     boolean needsStop = type == Type.STOP || type == Type.STOP_LIMIT;
361     if (needsLimit != (request.getLimitPrice() != null)) {
362         issues.add(new ApiException(ProblemType.INVALID_REQUEST, needsLimit
363             ? "limitPrice is required for " + type + " orders"
364             : "limitPrice is not allowed for " + type + " orders"));
365     }
366     if (needsStop != (request.getStopPrice() != null)) {
367         issues.add(new ApiException(ProblemType.INVALID_REQUEST, needsStop
368             ? "stopPrice is required for " + type + " orders"
369             : "stopPrice is not allowed for " + type + " orders"));
370     }
371 }
372
373 /** The price used to estimate cost and reserve cash: the worst price the order can get. */
374 private BigDecimal referencePrice(Type type, Side side, BigDecimal limit, BigDecimal stop,
375     Instrument instrument) {
376     return switch (type) {
377         case LIMIT, STOP_LIMIT -> limit;
378         case STOP -> stop;
379         case MARKET -> {
380             MarketData.Tick tick = market.tick(instrument.id());
381             yield side == Side.BUY ? tick.ask() : tick.bid();
382         }
383     };
384 }
385
386 private static BigDecimal commission(Instrument instrument, BigDecimal quantity) {
387     BigDecimal perUnit = instrument.isOption() ? OPTION_COMMISSION : BigDecimal.ZERO;
388     return perUnit.multiply(quantity).setScale(2, RoundingMode.HALF_EVEN);
389 }
390
391 private static <T> T orElse(T value, T fallback) {
392     return value != null ? value : fallback;
393 }
394
395 private void save(Order order) {
396     orders.put(order.id(), order);
397 }
398
399 private String nextOrderId() {
400     return "ORD-" + orderIds.incrementAndGet();
401 }
402 }

```

brokerage-api/src/main/java/com/example/brokerage/api/orders/TradingSession.java

31 lines

```

1 package com.example.brokerage.api.orders;
2
3 import com.example.brokerage.api.market.MarketData;
4
5 import org.springframework.boot.autoconfigure.condition.ConditionalOnBooleanProperty;
6 import org.springframework.scheduling.annotation.Scheduled;
7 import org.springframework.stereotype.Component;
8
9 /**
10  * Drives the simulated market: every tick, prices move and open orders are matched.
11  * Tests switch this off (brokerage.simulator.enabled=false) and call match() directly,
12  * so that every price is known in advance.
13  */
14 @Component
15 @ConditionalOnBooleanProperty(name = "brokerage.simulator.enabled", matchIfMissing = true)
16 class TradingSession {

```

```

17
18     private final MarketData market;
19     private final OrderService orders;
20
21     TradingSession(MarketData market, OrderService orders) {
22         this.market = market;
23         this.orders = orders;
24     }
25
26     @Scheduled(fixedDelayString = "${brokerage.simulator.tick-millis:1000}")
27     void tick() {
28         market.move();
29         orders.match();
30     }
31 }

```

brokerage-api/src/main/resources/application.yaml

34 lines

```

1  server:
2    port: 8080
3
4  spring:
5    application:
6      name: brokerage-api
7    jackson:
8      # Absent optional fields are omitted rather than sent as null.
9      default-property-inclusion: non_null
10   security:
11     oauth2:
12       resourceserver:
13         jwt:
14           issuer-uri: http://localhost:9000
15           # Tokens must be minted for this API, not merely by a trusted issuer.
16           audiences: brokerage-api
17
18   brokerage:
19     simulator:
20       enabled: true
21       tick-millis: 1000
22     rate-limit:
23       requests-per-minute: 120
24       order-changes-per-minute: 30
25
26   management:
27     endpoints:
28       web:
29         exposure:
30           include: health
31
32   logging:
33     pattern:
34       level: "%5p [%X{requestId:-}]"

```

E.4 The API's tests

brokerage-api/src/test: protocol, hypermedia, lifecycle, rate limit and contract tests.

brokerage-api/src/test/java/com/example/brokerage/api/ApiTest.java

64 lines

```

1  package com.example.brokerage.api;
2
3  import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPostProcessors.jwt;
4
5  import java.lang.annotation.ElementType;
6  import java.lang.annotation.Retention;
7  import java.lang.annotation.RetentionPolicy;
8  import java.lang.annotation.Target;

```

```

9 import java.util.ArrayList;
10 import java.util.List;
11
12 import org.springframework.boot.test.context.SpringBootTest;
13 import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
14 import org.springframework.test.web.servlet.request.RequestPostProcessor;
15
16 /**
17  * The whole application, with the simulated market frozen: prices never move by
18  * themselves, and orders fill only when a test calls OrderService.match().
19  */
20 @Target(ElementType.TYPE)
21 @Retention(RetentionPolicy.RUNTIME)
22 @SpringBootTest(properties = "brokerage.simulator.enabled=false")
23 @AutoConfigureMockMvc
24 public @interface ApiTest {
25
26     /** Access tokens for tests, as the authorization server would issue them. */
27     final class Tokens {
28
29         public static final String MARKET_DATA = "market-data:read";
30         public static final String ACCOUNTS = "accounts:read";
31         public static final String ORDERS_READ = "orders:read";
32         public static final String ORDERS_WRITE = "orders:write";
33         public static final String[] ALL = { MARKET_DATA, ACCOUNTS, ORDERS_READ, ORDERS_WRITE };
34
35         private Tokens() {
36         }
37
38         /** Alice (customer cust-1001) through her own script, the brokerage-cli client. */
39         public static RequestPostProcessor alice(String... scopes) {
40             return token("brokerage-cli", "cust-1001", null, scopes);
41         }
42
43         /** Alice's AI agent: same customer, orders capped at the given notional. */
44         public static RequestPostProcessor agent(String tradingLimit, String... scopes) {
45             return token("brokerage-agent", "cust-1001", tradingLimit, scopes);
46         }
47
48         /** Bob (customer cust-2002), who must never see Alice's accounts. */
49         public static RequestPostProcessor bob(String... scopes) {
50             return token("bob-app", "cust-2002", null, scopes);
51         }
52
53         private static RequestPostProcessor token(String client, String customer, String limit,
54             String... scopes) {
55             return jwt().jwt(token -> {
56                 token.subject(client).audience(List.of("brokerage-api")).claim("customer_id", customer)
57                     .claim("scope", new ArrayList<>(List.of(scopes)));
58                 if (limit != null) {
59                     token.claim("trading_limit", limit);
60                 }
61             });
62         }
63     }
64 }

```

brokerage-api/src/test/java/com/example/brokerage/api/ContractTest.java

149 lines

```

1 package com.example.brokerage.api;
2
3 import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4 import static com.example.brokerage.api.ApiTest.Tokens.alice;
5 import static org.assertj.core.api.Assertions.assertThat;
6
7 import java.io.IOException;
8 import java.io.InputStream;
9 import java.util.List;
10 import java.util.Map;

```

```

11 import java.util.UUID;
12
13 import com.networknt.schema.Error;
14 import com.networknt.schema.SchemaLocation;
15 import com.networknt.schema.SchemaRegistry;
16 import com.networknt.schema.SpecificationVersion;
17 import tools.jackson.databind.JsonNode;
18 import tools.jackson.databind.json.JsonMapper;
19 import tools.jackson.databind.node.ObjectNode;
20 import tools.jackson.dataformat.yaml.YAMLMapper;
21
22 import org.junit.jupiter.api.BeforeAll;
23 import org.junit.jupiter.api.Test;
24 import org.springframework.beans.factory.annotation.Autowired;
25 import org.springframework.core.io.ClassPathResource;
26 import org.springframework.http.MediaType;
27 import org.springframework.test.web.servlet.assertj.MockMvcTester;
28 import org.springframework.test.web.servlet.assertj.MvcTestResult;
29
30 /**
31  * The OpenAPI document is a contract, so it is tested like code: it must be a valid
32  * OpenAPI 3.2 document, its examples must match its own schemas, and real responses from
33  * the running application must match the schemas it promises.
34  */
35 @ApiTest
36 class ContractTest {
37
38     static final String CONTRACT = "https://api.brokerage.example/openapi.yaml";
39     static final String OAS_32 = "https://spec.openapis.org/oas/3.2/schema/2025-09-17";
40
41     static JsonNode contract;
42     static SchemaRegistry schemas;
43
44     @Autowired
45     MockMvcTester mvc;
46
47     @BeforeAll
48     static void loadTheContract() throws IOException {
49         try (InputStream yaml = new ClassPathResource("contract/brokerage-api.yaml").getInputStream();
50              InputStream meta = new ClassPathResource("oas-3.2-schema.json").getInputStream()) {
51             contract = YAMLMapper.builder().build().readTree(yaml);
52             String metaSchema = new String(meta.readAllBytes());
53             String contractJson = JsonMapper.builder().build().writeValueAsString(contract);
54             schemas = SchemaRegistry.withDefaultDialect(SpecificationVersion.DRAFT_2020_12,
55                                                         registry -> registry.schemas(Map.of(CONTRACT, contractJson, OAS_32, metaSchema)));
56         }
57     }
58
59     @Test
60     void theContractIsAValidOpenApi32Document() {
61         List<Error> errors = schemas.getSchema(SchemaLocation.of(OAS_32)).validate(contract);
62         assertThat(errors).as("violations of the OpenAPI 3.2 meta-schema").isEmpty();
63     }
64
65     @Test
66     void theMetaSchemaCheckCatchesABrokenContract() {
67         ObjectNode broken = (ObjectNode) contract.deepCopy();
68         broken.remove("info");
69         broken.put("openapi", "3.2");
70         assertThat(schemas.getSchema(SchemaLocation.of(OAS_32)).validate(broken)).isNotEmpty();
71     }
72
73     @Test
74     void everyExampleMatchesItsOwnSchema() {
75         JsonNode placeOrder = contract.at("/paths/~1accounts~1{accountId}~1orders/post/requestBody/content/"
76                                           + "application~1json/examples/limitBuy/dataValue");
77         assertMatches("OrderRequest", placeOrder);
78         for (String response : List.of("BadRequest", "UnprocessableContent")) {
79             contract.at("/components/responses/" + response + "/content/application~1problem+json/examples")

```

```

80         .forEach(example -> assertMatches("Problem", example.get("dataValue")));
81     }
82 }
83
84 @Test
85 void readOnlyResourcesMatchTheirSchemas() throws Exception {
86     assertMatches("ApiRoot", get("/"));
87     assertMatches("AccountCollection", get("/accounts"));
88     assertMatches("Account", get("/accounts/ACC-1001"));
89     assertMatches("Balances", get("/accounts/ACC-1001/balances"));
90     assertMatches("PositionCollection", get("/accounts/ACC-1001/positions"));
91     assertMatches("Position", get("/accounts/ACC-1001/positions/EQ-AAPL"));
92     assertMatches("TransactionPage", get("/accounts/ACC-1001/transactions?limit=5"));
93     assertMatches("InstrumentCollection", get("/instruments?type=EQUITY"));
94     assertMatches("InstrumentCollection", get("/instruments/EQ-AAPL/option-chain"));
95     assertMatches("QuoteV1", get("/instruments/EQ-AAPL/quote"));
96     assertMatches("OrderPage", get("/accounts/ACC-1001/orders"));
97     assertMatches("ProtectedResourceMetadata", get("/.well-known/oauth-protected-resource"));
98 }
99
100 @Test
101 void ordersPreviewsAndProblemsMatchTheirSchemas() throws Exception {
102     MvcTestResult placed = mvc.post().uri("/accounts/ACC-1001/orders").with(alice(ALL))
103         .header("Idempotency-Key", UUID.randomUUID().toString())
104         .contentType(MediaType.APPLICATION_JSON)
105         .content("""
106             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "LIMIT", "quantity": "1",
107              "limitPrice": "200.00", "clientOrderId": "contract-test"}""")
108         .exchange();
109     assertMatches("Order", json(placed));
110
111     MvcTestResult preview = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
112         .contentType(MediaType.APPLICATION_JSON)
113         .content("""
114             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "3"}""")
115         .exchange();
116     assertMatches("OrderPreview", json(preview));
117
118     MvcTestResult problem = mvc.post().uri("/accounts/ACC-1001/orders").with(alice(ALL))
119         .header("Idempotency-Key", UUID.randomUUID().toString())
120         .contentType(MediaType.APPLICATION_JSON)
121         .content("""
122             {"instrumentId": "EQ-HALT", "side": "BUY", "type": "MARKET", "quantity": "1"}""")
123         .exchange();
124     assertThat(problem).hasStatus(422);
125     assertMatches("Problem", json(problem));
126
127     MvcTestResult v2 = mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "2")
128         .with(alice(ALL)).exchange();
129     assertMatches("Quote", json(v2));
130 }
131
132 // ----- helpers
133
134 private JsonNode get(String uri) throws Exception {
135     MvcTestResult result = mvc.get().uri(uri).with(alice(ALL)).exchange();
136     assertThat(result).as(uri).hasStatusOk();
137     return json(result);
138 }
139
140 private static JsonNode json(MvcTestResult result) throws Exception {
141     return JsonMapper.builder().build().readTree(result.getResponse().getContentAsString());
142 }
143
144 private static void assertMatches(String schemaName, JsonNode instance) {
145     SchemaLocation location = SchemaLocation.of(CONTRACT + "#/components/schemas/" + schemaName);
146     List<Error> errors = schemas.getSchema(location).validate(instance);

```

```

147         assertThat(errors).as("%s does not match the contract:%n%s", schemaName, instance).isEmpty();
148     }
149 }

```

brokerage-api/src/test/java/com/example/brokerage/api/HypermediaTest.java

98 lines

```

1 package com.example.brokerage.api;
2
3 import static com.example.brokerage.api.ApiTest.Tokens.ACCOUNTS;
4 import static com.example.brokerage.api.ApiTest.Tokens.ALL;
5 import static com.example.brokerage.api.ApiTest.Tokens.ORDERS_READ;
6 import static com.example.brokerage.api.ApiTest.Tokens.alice;
7 import static com.example.brokerage.api.ApiTest.Tokens.bob;
8 import static org.assertj.core.api.Assertions.assertThat;
9
10 import org.junit.jupiter.api.Test;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.hateoas.MediaTypes;
13 import org.springframework.http.HttpStatus;
14 import org.springframework.test.web.servlet.assertj.MockMvcTester;
15 import org.springframework.test.web.servlet.assertj.MvcTestResult;
16
17 /**
18  * Discovery, link relations and the security boundary: what a client can learn, and
19  * what it is allowed to see.
20  */
21 @ApiTest
22 class HypermediaTest {
23
24     static final String PROBLEMS = "https://docs.brokerage.example/problems/";
25
26     @Autowired
27     MockMvcTester mvc;
28
29     @Test
30     void theRootIsPublicAndLinksToEverything() {
31         MvcTestResult root = mvc.get().uri("/").exchange();
32         assertThat(root).hasStatusOk().hasContentType(MediaTypes.HAL_JSON);
33         assertThat(root).bodyJson().extractingPath("$. _links['bk:accounts'].href").asString()
34             .endsWith("/accounts");
35         assertThat(root).bodyJson().extractingPath("$. _links['bk:instruments'].templated").isEqualTo(true);
36         assertThat(root).bodyJson().extractingPath("$. _links['service-desc'].href").asString()
37             .endsWith("/openapi.yaml");
38     }
39
40     @Test
41     void theContractIsServedBesideTheApi() {
42         assertThat(mvc.get().uri("/openapi.yaml")).hasStatusOk().bodyText().contains("openapi: 3.2.1");
43     }
44
45     @Test
46     void protectedResourceMetadataNamesTheAuthorizationServer() {
47         assertThat(mvc.get().uri("/.well-known/oauth-protected-resource")).hasStatusOk()
48             .bodyJson().extractingPath("$.authorization_servers[0]").isEqualTo("http://localhost:9000");
49     }
50
51     @Test
52     void aRequestWithoutATokenGetsAProblemAndAChallenge() {
53         MvcTestResult result = mvc.get().uri("/accounts").exchange();
54         assertThat(result).hasStatus(HttpStatus.UNAUTHORIZED).hasContentType("application/problem+json");
55         assertThat(result.getResponse().getHeader("WWW-Authenticate")).startsWith("Bearer")
56             .contains("resource_metadata=");
57         assertThat(result).bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "unauthorized");
58     }
59
60     @Test
61     void aTokenWithoutTheScopeIsForbidden() {
62         assertThat(mvc.get().uri("/accounts").with(alice(ORDERS_READ)))
63             .hasStatus(HttpStatus.FORBIDDEN)

```

```

64         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "insufficient-scope");
65     }
66
67     @Test
68     void anotherCustomersAccountDoesNotExist() {
69         assertThat(mvc.get().uri("/accounts/ACC-1001").with(bob(ALL)))
70             .hasStatus(HttpStatus.NOT_FOUND)
71             .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "not-found");
72     }
73
74     @Test
75     void linksReflectWhatTheTokenMayDo() {
76         // Read-only: accounts are visible, but orders are not even mentioned.
77         MvcTestResult readOnly = mvc.get().uri("/accounts/ACC-1001").with(alice(ACCOUNTS)).exchange();
78         assertThat(readOnly).hasStatusOk();
79         assertThat(readOnly).bodyJson()
80             .hasPath("$. _links['bk:positions']")
81             .doesNotHavePath("$. _links['bk:orders']");
82
83         // A full trading token, asking for HAL-FORMS: the placeOrder template appears.
84         MvcTestResult trading = mvc.get().uri("/accounts/ACC-1001").with(alice(ALL))
85             .accept(MediaType.HAL_FORMS_JSON).exchange();
86         assertThat(trading).hasStatusOk();
87         assertThat(trading).bodyJson().hasPath("$. _links['bk:orders']");
88         assertThat(trading).bodyJson().extractingPath("$. _templates.placeOrder.method").isEqualTo("POST");
89     }
90
91     @Test
92     void anOptionChainOffersEveryExpirationAsANamedLink() {
93         MvcTestResult chain = mvc.get().uri("/instruments/EQ-AAPL/option-chain").with(alice(ALL)).exchange();
94         assertThat(chain).hasStatusOk();
95         assertThat(chain).bodyJson().extractingPath("$. _links['bk:expiration']").asArray().hasSize(2);
96         assertThat(chain).bodyJson().extractingPath("$. _embedded['bk:instruments']").asArray().hasSize(6);
97     }
98 }

```

brokerage-api/src/test/java/com/example/brokerage/api/OrderLifecycleTest.java

183 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4  import static com.example.brokerage.api.ApiTest.Tokens.agent;
5  import static com.example.brokerage.api.ApiTest.Tokens.alice;
6  import static org.assertj.core.api.Assertions.assertThat;
7
8  import java.util.UUID;
9
10 import com.example.brokerage.api.orders.OrderService;
11
12 import org.junit.jupiter.api.Test;
13 import org.springframework.beans.factory.annotation.Autowired;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.MediaType;
16 import org.springframework.test.web.servlet.assertj.MockMvcTester;
17 import org.springframework.test.web.servlet.assertj.MvcTestResult;
18
19 /**
20  * Placing, retrying, changing and canceling orders, and the links that follow each state.
21  */
22 @ApiTest
23 class OrderLifecycleTest {
24
25     static final String ORDERS = "/accounts/ACC-1001/orders";
26     static final String PROBLEMS = "https://docs.brokerage.example/problems/";
27     static final MediaType MERGE_PATCH = MediaType.parseMediaType("application/merge-patch+json");
28
29     @Autowired
30     MockMvcTester mvc;
31

```

```

32 @Autowired
33 OrderService orders;
34
35 @Test
36 void aMarketOrderIsCreatedThenFilledAndLosesItsCancelLink() {
37     MvcTestResult placed = place(UUID.randomUUID().toString(), ""
38         {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "2"}""");
39     assertThat(placed).hasStatus(HttpStatus.CREATED);
40     assertThat(placed.getResponse().getHeader("Location")).contains(ORDERS + "/ORD-");
41     assertThat(placed.getResponse().getHeader("ETag")).isNotBlank();
42     assertThat(placed.bodyJson().extractingPath("$.status").isEqualTo("OPEN"));
43     assertThat(placed.bodyJson().hasPath("$. _links['bk:cancel']")).hasPath("$. _links.edit");
44
45     orders.match();
46
47     String url = placed.getResponse().getHeader("Location");
48     MvcTestResult filled = mvc.get().uri(url).with(alice(ALL)).exchange();
49     assertThat(filled.bodyJson().extractingPath("$.status").isEqualTo("FILLED"));
50     // AAPL is frozen at 232.40, so the ask is 232.42.
51     assertThat(filled.bodyJson().extractingPath("$.averageFillPrice").isEqualTo("232.4200"));
52     assertThat(filled.bodyJson()
53         .doesNotHavePath("$. _links['bk:cancel']")
54         .doesNotHavePath("$. _links.edit"));
55 }
56
57 @Test
58 void retryingWithTheSameKeyReturnsTheSameOrder() {
59     String key = UUID.randomUUID().toString();
60     String body = ""
61         {"instrumentId": "EQ-MSFT", "side": "BUY", "type": "LIMIT", "quantity": "1",
62         "limitPrice": "400.00"}""";
63     MvcTestResult first = place(key, body);
64     MvcTestResult retry = place(key, body);
65
66     assertThat(retry).hasStatus(HttpStatus.CREATED);
67     assertThat(retry.getResponse().getHeader("Location"))
68         .isEqualTo(first.getResponse().getHeader("Location"));
69     assertThat(retry.getResponse().getHeader("Idempotent-Replayed")).isEqualTo("true");
70
71     MvcTestResult different = place(key, body.replace("\1", "\2"));
72     assertThat(different).hasStatus(HttpStatus.UNPROCESSABLE_CONTENT)
73         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "idempotency-key-reused");
74 }
75
76 @Test
77 void anOrderWithoutAnIdempotencyKeyIsRefused() {
78     assertThat(mvc.post().uri(ORDERS).with(alice(ALL)).contentType(MediaType.APPLICATION_JSON)
79         .content("""
80         {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "1"}"""))
81         .hasStatus(HttpStatus.BAD_REQUEST)
82         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "idempotency-key-missing");
83 }
84
85 @Test
86 void businessRuleViolationsAre422WithTheDetailsAClientNeeds() {
87     MvcTestResult tooBig = place(UUID.randomUUID().toString(), ""
88         {"instrumentId": "EQ-MSFT", "side": "BUY", "type": "LIMIT", "quantity": "1000",
89         "limitPrice": "500"}""");
90     assertThat(tooBig).hasStatus(HttpStatus.UNPROCESSABLE_CONTENT);
91     assertThat(tooBig.bodyJson().extractingPath("$.type")
92         .isEqualTo(PROBLEMS + "insufficient-buying-power"));
93     assertThat(tooBig.bodyJson().extractingPath("$.required").isEqualTo("500000.00"));
94     assertThat(tooBig.bodyJson().hasPath("$.available"));
95 }
96
97 @Test
98 void theAgentsDelegatedTradingLimitIsEnforcedByTheApi() {
99     // 30 x 232.42 = 6,972.60, above the agent's 5,000 limit.
100     assertThat(mvc.post().uri(ORDERS).with(agent("5000", ALL))

```

```

101         .header("Idempotency-Key", UUID.randomUUID().toString())
102         .contentType(MediaType.APPLICATION_JSON)
103         .content("""
104             {"instrumentId": "EQ-AAPL", "side": "BUY", "type": "MARKET", "quantity": "30"}""")
105         .hasStatus(HttpStatus.FORBIDDEN)
106         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "trading-limit-exceeded");
107     }
108
109     @Test
110     void changingAnOrderRequiresItsCurrentEtag() {
111         MvcTestResult placed = place(UUID.randomUUID().toString(), """
112             {"instrumentId": "EQ-NVDA", "side": "BUY", "type": "LIMIT", "quantity": "5",
113             "limitPrice": "150.00"}""");
114         String url = placed.getResponse().getHeader("Location");
115         String etag = placed.getResponse().getHeader("ETag");
116
117         // Polling an unchanged order costs a 304 and no body.
118         assertThat(mvc.get().uri(url).with(alice(ALL)).header("If-None-Match", etag))
119             .hasStatus(HttpStatus.NOT_MODIFIED);
120
121         assertThat(amend(url, null, """
122             {"limitPrice": "151.00"}"""))
123             .hasStatus(HttpStatus.PRECONDITION_REQUIRED);
124
125         MvcTestResult changed = amend(url, etag, """
126             {"limitPrice": "151.00"}""");
127         assertThat(changed).hasStatusOk();
128         assertThat(changed.bodyJson().extractingPath("$.limitPrice").isEqualTo("151.00");
129         assertThat(changed.getResponse().getHeader("ETag")).isNotEqualTo(etag);
130
131         // The old ETag is now stale: a second writer holding it is refused, not merged.
132         assertThat(amend(url, etag, """
133             {"limitPrice": "152.00"}"""))
134             .hasStatus(HttpStatus.PRECONDITION_FAILED)
135             .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "precondition-failed");
136     }
137
138     @Test
139     void cancellationIsAcceptedThenCompletes() {
140         MvcTestResult placed = place(UUID.randomUUID().toString(), """
141             {"instrumentId": "ETF-SPY", "side": "SELL", "type": "LIMIT", "quantity": "1",
142             "limitPrice": "900.00"}""");
143         String url = placed.getResponse().getHeader("Location");
144
145         MvcTestResult cancel = mvc.post().uri(url + "/cancellation").with(alice(ALL)).exchange();
146         assertThat(cancel).hasStatus(HttpStatus.ACCEPTED);
147         assertThat(cancel.getResponse().getHeader("Location")).isEqualTo(url);
148         assertThat(cancel.bodyJson().extractingPath("$.status").isEqualTo("PENDING_CANCEL");
149
150         orders.match();
151
152         assertThat(mvc.get().uri(url).with(alice(ALL)))
153             .bodyJson().extractingPath("$.status").isEqualTo("CANCELLED");
154         assertThat(mvc.post().uri(url + "/cancellation").with(alice(ALL)))
155             .hasStatus(HttpStatus.CONFLICT)
156             .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "order-not-cancellable");
157     }
158
159     @Test
160     void aPreviewReportsProblemsWithoutPlacingAnything() {
161         MvcTestResult preview = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
162             .contentType(MediaType.APPLICATION_JSON)
163             .content("""
164                 {"instrumentId": "EQ-AMZN", "side": "SELL", "type": "MARKET", "quantity": "500"}""")
165             .exchange();
166         assertThat(preview).hasStatusOk();
167         assertThat(preview.bodyJson().extractingPath("$.acceptable").isEqualTo(false);
168         assertThat(preview.bodyJson().extractingPath("$.issues[0].type")
169             .isEqualTo(PROBLEMS + "insufficient-position");
170         assertThat(preview.bodyJson().doesNotHavePath("$.links['bk:place-order']"));

```

```

170     }
171
172     // ----- helpers
173
174     private MvcTestResult place(String idempotencyKey, String json) {
175         return mvc.post().uri(ORDERS).with(alice(ALL)).header("Idempotency-Key", idempotencyKey)
176             .contentType(MediaType.APPLICATION_JSON).content(json).exchange();
177     }
178
179     private MvcTestResult amend(String url, String ifMatch, String json) {
180         var request = mvc.patch().uri(url).with(alice(ALL)).contentType(MERGE_PATCH).content(json);
181         return (ifMatch == null ? request : request.header("If-Match", ifMatch)).exchange();
182     }
183 }

```

brokerage-api/src/test/java/com/example/brokerage/api/ProtocolTest.java

126 lines

```

1  package com.example.brokerage.api;
2
3  import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4  import static com.example.brokerage.api.ApiTest.Tokens.alice;
5  import static org.assertj.core.api.Assertions.assertThat;
6
7  import java.util.List;
8
9  import com.example.brokerage.api.orders.OrderEvents;
10 import com.jayway.jsonpath.JsonPath;
11
12 import org.junit.jupiter.api.Test;
13 import org.springframework.beans.factory.annotation.Autowired;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.test.web.servlet.assertj.MockMvcTester;
16 import org.springframework.test.web.servlet.assertj.MvcTestResult;
17
18 /**
19  * Pagination, versioning, caching and event replay.
20  */
21 @ApiTest
22 class ProtocolTest {
23
24     static final String PROBLEMS = "https://docs.brokerage.example/problems/";
25
26     @Autowired
27     MockMvcTester mvc;
28
29     @Autowired
30     OrderEvents events;
31
32     @Test
33     void transactionsArePagedWithAnOpaqueCursor() throws Exception {
34         MvcTestResult first = mvc.get().uri("/accounts/ACC-1001/transactions?type=TRADE&limit=2")
35             .with(alice(ALL)).exchange();
36         assertThat(first).hasStatusOk();
37         List<String> firstIds = ids(first);
38         assertThat(firstIds).hasSize(2);
39
40         String next = JsonPath.read(first.getResponse().getContentAsString(), "$._links.next.href");
41         assertThat(next).contains("cursor=").doesNotContain("{}");
42
43         MvcTestResult second = mvc.get().uri(next).with(alice(ALL)).exchange();
44         assertThat(ids(second)).doesNotContainAnyElementsOf(firstIds);
45     }
46
47     @Test
48     void aTamperedCursorIsABadRequest() {
49         assertThat(mvc.get().uri("/accounts/ACC-1001/transactions?cursor=not-a-cursor").with(alice(ALL)))
50             .hasStatus(HttpStatus.BAD_REQUEST)
51             .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "invalid-request");
52     }

```

```

53
54 @Test
55 void aValueOfTheWrongTypeIsReportedLikeAValidationError() {
56     MvcTestResult result = mvc.post().uri("/accounts/ACC-1001/order-previews").with(alice(ALL))
57         .contentType("application/json")
58         .content("""
59             {"instrumentId": "EQ-AAPL", "side": "HOLD", "type": "MARKET", "quantity": "1"}""")
60         .exchange();
61     assertThat(result).hasStatus(HttpStatus.BAD_REQUEST);
62     assertThat(result).bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "invalid-request");
63     assertThat(result).bodyJson().extractingPath("$.errors[0].field").isEqualTo("side");
64     assertThat(result).bodyJson().extractingPath("$.errors[0].message")
65         .isEqualTo("must be one of BUY, SELL");
66 }
67
68 @Test
69 void theContractAnswersHeadRequestsWithoutAToken() {
70     assertThat(mvc.head().uri("/openapi.yaml")).hasStatusOk();
71 }
72
73 @Test
74 void versionOneQuotesAreFlatAndDeprecated() {
75     MvcTestResult v1 = mvc.get().uri("/instruments/EQ-AAPL/quote").with(alice(ALL)).exchange();
76     assertThat(v1).hasStatusOk();
77     assertThat(v1).bodyJson().extractingPath("$.bid").isEqualTo("232.38");
78     assertThat(v1.getResponse().getHeader("Deprecation")).startsWith("@");
79     assertThat(v1.getResponse().getHeader("Sunset")).contains("2027");
80     assertThat(v1.getResponse().getHeader("Cache-Control")).contains("max-age=1").contains("private");
81 }
82
83 @Test
84 void versionTwoQuotesNestPriceLevels() {
85     MvcTestResult v2 = mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "2")
86         .with(alice(ALL)).exchange();
87     assertThat(v2).hasStatusOk();
88     assertThat(v2).bodyJson().extractingPath("$.bid.price").isEqualTo("232.38");
89     assertThat(v2).bodyJson().extractingPath("$.changePercent").isEqualTo("1.00");
90     assertThat(v2.getResponse().getHeader("Deprecation")).isNull();
91 }
92
93 @Test
94 void anUnknownVersionIsRefusedWithTheSupportedOnes() {
95     assertThat(mvc.get().uri("/instruments/EQ-AAPL/quote").header("API-Version", "7").with(alice(ALL)))
96         .hasStatus(HttpStatus.BAD_REQUEST)
97         .bodyJson().extractingPath("$.type").isEqualTo(PROBLEMS + "unsupported-api-version");
98 }
99
100 @Test
101 void anUnchangedQuoteCostsA304() {
102     MvcTestResult quote = mvc.get().uri("/instruments/EQ-MSFT/quote").with(alice(ALL)).exchange();
103     assertThat(mvc.get().uri("/instruments/EQ-MSFT/quote").with(alice(ALL))
104         .header("If-None-Match", quote.getResponse().getHeader("ETag")))
105         .hasStatus(HttpStatus.NOT_MODIFIED);
106 }
107
108 @Test
109 void aReconnectingClientReceivesOnlyTheEventsItMissed() {
110     mvc.post().uri("/accounts/ACC-1002/orders").with(alice(ALL)).header("Idempotency-Key", "events-1")
111         .contentType("application/json")
112         .content("""
113             {"instrumentId": "EQ-NVDA", "side": "SELL", "type": "LIMIT", "quantity": "1",
114              "limitPrice": "999"}""")
115         .exchange();
116     List<OrderEvents.Event> all = events.replay("ACC-1002", 0);
117     assertThat(all).isNotEmpty();
118     long last = all.getLast().id();
119     assertThat(events.replay("ACC-1002", last)).isEmpty();
120     assertThat(events.replay("ACC-1002", last - 1)).hasSize(1);
121 }

```

```

122
123     private static List<String> ids(MvcTestResult page) throws Exception {
124         return JsonPath.read(page.getResponse().getContentAsString(), "$._embedded['bk:transactions'][*].id");
125     }
126 }

```

brokerage-api/src/test/java/com/example/brokerage/api/RateLimitTest.java

42 lines

```

1 package com.example.brokerage.api;
2
3 import static com.example.brokerage.api.ApiTest.Tokens.ALL;
4 import static com.example.brokerage.api.ApiTest.Tokens.alice;
5 import static org.assertj.core.api.Assertions.assertThat;
6
7 import org.junit.jupiter.api.Test;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.boot.test.context.SpringBootTest;
10 import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
11 import org.springframework.http.HttpStatus;
12 import org.springframework.test.web.servlet.assertj.MockMvcTester;
13 import org.springframework.test.web.servlet.assertj.MvcTestResult;
14
15 /**
16  * Quotas are advertised on every response, and enforced with 429 and Retry-After.
17  */
18 @SpringBootTest(properties = {
19     "brokerage.simulator.enabled=false",
20     "brokerage.rate-limit.requests-per-minute=3" })
21 @AutoConfigureMockMvc
22 class RateLimitTest {
23
24     @Autowired
25     MockMvcTester mvc;
26
27     @Test
28     void aClientOverItsQuotaIsToldWhenToComeBack() {
29         MvcTestResult first = mvc.get().uri("/instruments").with(alice(ALL)).exchange();
30         assertThat(first.getResponse().getHeader("RateLimit-Policy")).isEqualTo("\"default\";q=3;w=60");
31         assertThat(first.getResponse().getHeader("RateLimit")).startsWith("\"default\";r=2;");
32
33         mvc.get().uri("/instruments").with(alice(ALL)).exchange();
34         mvc.get().uri("/instruments").with(alice(ALL)).exchange();
35         MvcTestResult refused = mvc.get().uri("/instruments").with(alice(ALL)).exchange();
36
37         assertThat(refused).hasStatus(HttpStatus.TOO_MANY_REQUESTS);
38         assertThat(refused.getResponse().getHeader("Retry-After")).isNotBlank();
39         assertThat(refused.bodyJson().extractingPath("$.type")
40             .isEqualTo("https://docs.brokerage.example/problems/rate-limited");
41     }
42 }

```

E.5 The hypermedia client

brokerage-client: a Java client that follows links.

brokerage-client/src/main/java/com/example/brokerage/client/ApiProblem.java

59 lines

```

1 package com.example.brokerage.client;
2
3 import java.io.IOException;
4 import java.util.Optional;
5
6 import tools.jackson.databind.JsonNode;
7 import tools.jackson.databind.json.JsonMapper;
8
9 import org.springframework.http.client.ClientHttpResponse;
10

```

```

11 /**
12  * An RFC 9457 problem returned by the API, as an exception. Callers branch on {@link #type()},
13  * the stable identifier, and may show {@link #detail()} to a person.
14  */
15 public class ApiProblem extends RuntimeException {
16
17     private final int status;
18     private final String type;
19     private final String title;
20     private final JsonNode body;
21     private final Optional<Long> retryAfterSeconds;
22
23     ApiProblem(int status, String type, String title, String detail, JsonNode body,
24         Optional<Long> retryAfter) {
25         super(status + " " + title + (detail == null ? "" : ": " + detail));
26         this.status = status;
27         this.type = type;
28         this.title = title;
29         this.body = body;
30         this.retryAfterSeconds = retryAfter;
31     }
32
33     static ApiProblem from(ClientHttpResponse response, JsonMapper json) throws IOException {
34         JsonNode body = json.readTree(response.getBody());
35         int status = response.getStatusCode().value();
36         Optional<Long> retryAfter = Optional.ofNullable(response.getHeaders().getFirst("Retry-After"))
37             .map(Long::valueOf);
38         return new ApiProblem(status, body.path("type").asString("about:blank"),
39             body.path("title").asString(response.getStatusText()), body.path("detail").asString(null),
40             body, retryAfter);
41     }
42
43     /** Worth retrying later with the same request (and the same Idempotency-Key). */
44     public boolean isTransient() {
45         return status == 429 || status == 502 || status == 503 || status == 504;
46     }
47
48     /** The problem type's short name, such as "insufficient-buying-power". */
49     public String typeName() {
50         return type.substring(type.lastIndexOf('/') + 1);
51     }
52
53     public int status() { return status; }
54     public String type() { return type; }
55     public String title() { return title; }
56     public String detail() { return body.path("detail").asString(null); }
57     public JsonNode body() { return body; }
58     public Optional<Long> retryAfterSeconds() { return retryAfterSeconds; }
59 }

```

brokerage-client/src/main/java/com/example/brokerage/client/BrokerageClient.java

145 lines

```

1 package com.example.brokerage.client;
2
3 import java.net.URI;
4 import java.time.Duration;
5 import java.util.List;
6 import java.util.Map;
7
8 import com.example.brokerage.client.Model.Account;
9 import com.example.brokerage.client.Model.Balances;
10 import com.example.brokerage.client.Model.Instrument;
11 import com.example.brokerage.client.Model.Order;
12 import com.example.brokerage.client.Model.OrderPreview;
13 import com.example.brokerage.client.Model.OrderTicket;
14 import com.example.brokerage.client.Model.Position;
15 import com.example.brokerage.client.Model.Quote;
16
17 import org.springframework.hateoas.Link;

```

```

18 import org.springframework.web.client.ResourceAccessException;
19
20 /**
21  * The brokerage, as a Java API. Every method starts from the API root and follows link
22  * relations; not one URL is built here. If the server moves a resource, this class keeps
23  * working, and if the server withholds a link, the corresponding action is unavailable.
24  */
25 public class BrokerageClient {
26
27     private static final int ATTEMPTS = 3;
28
29     private final HypermediaClient http;
30     private final URI root;
31
32     public BrokerageClient(HypermediaClient http, URI root) {
33         this.http = http;
34         this.root = root;
35     }
36
37     // ----- accounts
38
39     public List<Account> accounts() {
40         return accountResources().stream().map(account -> account.as(Account.class)).toList();
41     }
42
43     public Balances balances(String accountId) {
44         return follow(account(accountId), "bk:balances").as(Balances.class);
45     }
46
47     public List<Position> positions(String accountId) {
48         return follow(account(accountId), "bk:positions").embedded("bk:positions").stream()
49             .map(position -> position.as(Position.class)).toList();
50     }
51
52     // ----- market data
53
54     public Instrument instrument(String symbol) {
55         return instrumentResource(symbol).as(Instrument.class);
56     }
57
58     /** The latest quote, in the version 2 shape that the client pins on every request. */
59     public Quote quote(String symbol) {
60         Link quote = instrumentResource(symbol).requiredLink("bk:quote");
61         return http.get(quote.toUri()).as(Quote.class);
62     }
63
64     // ----- trading
65
66     public OrderPreview preview(String accountId, OrderTicket ticket) {
67         Link previews = account(accountId).requiredLink("bk:order-previews");
68         return http.post(previews, ticket, headers -> { }).as(OrderPreview.class);
69     }
70
71     /**
72      * Places an order. On a timeout or a transient error the request is retried with the
73      * same Idempotency-Key, so a retry can never place a second order.
74      */
75     public Order place(String accountId, OrderTicket ticket, String idempotencyKey) {
76         Link orders = account(accountId).requiredLink("bk:orders").expand();
77         for (int attempt = 1; ; attempt++) {
78             try {
79                 return http.post(orders, ticket, headers -> headers.set("Idempotency-Key", idempotencyKey))
80                     .as(Order.class);
81             } catch (ResourceAccessException | ApiProblem failure) {
82                 boolean retryable = !(failure instanceof ApiProblem problem) || problem.isTransient();
83                 if (!retryable || attempt == ATTEMPTS) {
84                     throw failure;
85                 }
86                 long seconds = failure instanceof ApiProblem problem

```

```

87         ? problem.retryAfterSeconds().orElse(1L) : attempt;
88         pause(seconds);
89     }
90 }
91 }
92
93 public List<Order> openOrders(String accountId) {
94     Link orders = account(accountId).requiredLink("bk:orders");
95     URI open = orders.expand(Map.of("status", List.of("OPEN", "PARTIALLY_FILLED"))).toUri();
96     return http.get(open).embedded("bk:orders").stream().map(order -> order.as(Order.class)).toList();
97 }
98
99 /** Re-reads an order through its own self link. */
100 public Order refresh(Order order) {
101     return http.get(order.self()).as(Order.class);
102 }
103
104 /** Cancels an order, if and only if the server still offers the cancel link. */
105 public Order cancel(Order order) {
106     URI cancel = order.link("bk:cancel").orElseThrow(() -> new IllegalStateException(
107         "Order " + order.id() + " is " + order.status() + " and can no longer be canceled"));
108     return http.post(Link.of(cancel.toString()), null, headers -> { }).as(Order.class);
109 }
110
111 // ----- navigation
112
113 private List<HalResource> accountResources() {
114     HalResource start = http.get(root);
115     return follow(start, "bk:accounts").embedded("bk:accounts");
116 }
117
118 private HalResource account(String accountId) {
119     return accountResources().stream()
120         .filter(account -> accountId.equals(account.body().path("id").asString()))
121         .findFirst()
122         .orElseThrow(() -> new IllegalArgumentException(
123             "No account " + accountId + " for this customer"));
124 }
125
126 private HalResource instrumentResource(String symbol) {
127     Link search = http.get(root).requiredLink("bk:instruments");
128     return http.get(search.expand(Map.of("symbol", symbol)).toUri()).embedded("bk:instruments").stream()
129         .findFirst()
130         .orElseThrow(() -> new IllegalArgumentException("Unknown symbol " + symbol));
131 }
132
133 private HalResource follow(HalResource from, String rel) {
134     return http.get(from.requiredLink(rel).expand().toUri());
135 }
136
137 private static void pause(long seconds) {
138     try {
139         Thread.sleep(Duration.ofSeconds(Math.min(seconds, 5)));
140     } catch (InterruptedException interrupted) {
141         Thread.currentThread().interrupt();
142         throw new IllegalStateException(interrupted);
143     }
144 }
145 }

```

brokerage-client/src/main/java/com/example/brokerage/client/ClientCredentials.java

53 lines

```

1 package com.example.brokerage.client;
2
3 import java.net.URI;
4 import java.time.Instant;
5 import java.util.function.Supplier;
6
7 import tools.jackson.databind.JsonNode;

```

```

8
9 import org.springframework.http.MediaType;
10 import org.springframework.util.LinkedMultiValueMap;
11 import org.springframework.util.MultiValueMap;
12 import org.springframework.web.client.RestClient;
13
14 /**
15  * Access tokens by the OAuth 2.1 client credentials grant, cached until shortly before they
16  * expire. Enough for scripts and the demo; a Spring application should use Spring
17  * Security's OAuth 2.0 client instead, as the agent does.
18  */
19 public class ClientCredentials implements Supplier<String> {
20
21     private final RestClient http = RestClient.create();
22     private final URI tokenEndpoint;
23     private final String clientId;
24     private final String clientSecret;
25     private final String scope;
26     private String token;
27     private Instant expiresAt = Instant.EPOCH;
28
29     public ClientCredentials(URI tokenEndpoint, String clientId, String clientSecret, String scope) {
30         this.tokenEndpoint = tokenEndpoint;
31         this.clientId = clientId;
32         this.clientSecret = clientSecret;
33         this.scope = scope;
34     }
35
36     @Override
37     public synchronized String get() {
38         if (token == null || Instant.now().isAfter(expiresAt.minusSeconds(30))) {
39             MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
40             form.add("grant_type", "client_credentials");
41             form.add("scope", scope);
42             JsonNode response = http.post().uri(tokenEndpoint)
43                 .headers(headers -> headers.setBasicAuth(clientId, clientSecret))
44                 .contentType(MediaType.APPLICATION_FORM_URLENCODED)
45                 .body(form)
46                 .retrieve()
47                 .body(JsonNode.class);
48             token = response.get("access_token").asString();
49             expiresAt = Instant.now().plusSeconds(response.path("expires_in").asLong(60));
50         }
51         return token;
52     }
53 }

```

brokerage-client/src/main/java/com/example/brokerage/client/Demo.java

71 lines

```

1 package com.example.brokerage.client;
2
3 import java.math.BigDecimal;
4 import java.net.URI;
5 import java.util.UUID;
6
7 import com.example.brokerage.client.Model.Order;
8 import com.example.brokerage.client.Model.OrderTicket;
9
10 import org.springframework.web.client.RestClient;
11
12 /**
13  * A guided tour of the API through the hypermedia client. Start the authorization server
14  * and the API first, then run:
15  *
16  * <pre>
17  * ./mvnw -q -pl brokerage-client compile exec:java -Dexec.mainClass=com.example.brokerage.client.Demo
18  * </pre>
19  */
20 public final class Demo {

```

```

21
22 public static void main(String[] args) throws InterruptedException {
23     ClientCredentials tokens = new ClientCredentials(URI.create("http://localhost:9000/oauth2/token"),
24         "brokerage-cli", "cli-secret", "accounts:read market-data:read orders:read orders:write");
25     HypermediaClient http = new HypermediaClient(RestClient.builder()
26         .requestInterceptor((request, body, execution) -> {
27             request.getHeaders().setBearerAuth(tokens.get());
28             return execution.execute(request, body);
29         }));
30     BrokerageClient brokerage = new BrokerageClient(http, URI.create("http://localhost:8080/"));
31
32     System.out.println("Accounts");
33     brokerage.accounts().forEach(account -> System.out.printf(" %9s %-18s %s\n",
34         account.id(), account.nickname(), account.type()));
35
36     var balances = brokerage.balances("ACC-1001");
37     System.out.printf("%nACC-1001: cash %s, buying power %s, equity %s\n",
38         balances.cash(), balances.buyingPower(), balances.totalEquity());
39
40     System.out.println("\nPositions");
41     brokerage.positions("ACC-1001").forEach(p -> System.out.printf(" %5s %6s @ %9s P&L %s\n",
42         p.symbol(), p.quantity(), p.lastPrice(), p.unrealizedPnL()));
43
44     var quote = brokerage.quote("AAPL");
45     System.out.printf("%nAAPL %s bid %s ask %s (%s%%)\n",
46         quote.last(), quote.bid().price(), quote.ask().price(), quote.changePercent());
47
48     // A limit order well below the market, so that it rests and can be canceled.
49     BigDecimal price = quote.last().multiply(new BigDecimal("0.90"))
50         .setScale(2, java.math.RoundingMode.DOWN);
51     OrderTicket ticket = OrderTicket.limit("EQ-AAPL", "BUY", new BigDecimal("5"), price);
52     var preview = brokerage.preview("ACC-1001", ticket);
53     System.out.printf("%nPreview: 5 AAPL at %s costs %s (acceptable: %s)\n",
54         price, preview.estimatedTotal(), preview.acceptable());
55
56     Order order = brokerage.place("ACC-1001", ticket, UUID.randomUUID().toString());
57     System.out.printf("Placed %s: %s, cancelable: %s\n", order.id(), order.status(),
58         order.isCancellable());
59
60     order = brokerage.cancel(order);
61     System.out.printf("Cancel requested: %s\n", order.status());
62     while (order.isCancellable() || "PENDING_CANCEL".equals(order.status())) {
63         Thread.sleep(500);
64         order = brokerage.refresh(order);
65     }
66     System.out.printf("Now %s, cancelable: %s\n", order.status(), order.isCancellable());
67 }
68
69 private Demo() {
70 }
71 }

```

brokerage-client/src/main/java/com/example/brokerage/client/HalResource.java

77 lines

```

1 package com.example.brokerage.client;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.Optional;
6
7 import tools.jackson.databind.JsonNode;
8 import tools.jackson.databind.json.JsonMapper;
9
10 import org.springframework.hateoas.Link;
11 import org.springframework.http.HttpHeaders;
12
13 /**
14  * A HAL representation as a client sees it: properties, links by relation, embedded
15  * resources, and the response headers that came with it.

```

```

16  */
17  public final class HalResource {
18
19      private final JsonNode body;
20      private final HttpHeaders headers;
21      private final ObjectMapper json;
22
23      HalResource(JsonNode body, HttpHeaders headers, ObjectMapper json) {
24          this.body = body;
25          this.headers = headers;
26          this.json = json;
27      }
28
29      /** The first link with this relation, if the server offered one. */
30      public Optional<Link> link(String rel) {
31          return links(rel).stream().findFirst();
32      }
33
34      /** The link with this relation; its absence means the action is not available now. */
35      public Link requiredLink(String rel) {
36          return link(rel).orElseThrow(() -> new IllegalStateException(
37              "The server offered no '" + rel + "' link, so that action is not available"));
38      }
39
40      /** Every link with this relation. HAL sends one link as an object, several as an array. */
41      public List<Link> links(String rel) {
42          JsonNode node = body.path("_links").path(rel);
43          List<Link> links = new ArrayList<>();
44          for (JsonNode each : node.isArray() ? node : List.of(node)) {
45              if (each.hasNonNull("href")) {
46                  Link link = Link.of(each.get("href").asString(), rel);
47                  links.add(each.hasNonNull("name") ? link.withName(each.get("name").asString()) : link);
48              }
49          }
50          return links;
51      }
52
53      /** Resources embedded under a relation, such as the items of a collection. */
54      public List<HalResource> embedded(String rel) {
55          List<HalResource> items = new ArrayList<>();
56          body.path("_embedded").path(rel)
57              .forEach(item -> items.add(new HalResource(item, new HttpHeaders(), json)));
58          return items;
59      }
60
61      /** The properties as a Java type. Unknown properties are ignored: a tolerant reader. */
62      public <T> T as(Class<T> type) {
63          return json.treeToValue(body, type);
64      }
65
66      public String etag() {
67          return headers.getETag();
68      }
69
70      public HttpHeaders headers() {
71          return headers;
72      }
73
74      public JsonNode body() {
75          return body;
76      }
77  }

```

brokerage-client/src/main/java/com/example/brokerage/client/HypermediaClient.java

80 lines

```

1  package com.example.brokerage.client;
2
3  import java.math.BigDecimal;
4  import java.net.URI;

```

```

5 import java.util.function.Consumer;
6
7 import com.fasterxml.jackson.annotation.JsonFormat;
8 import com.fasterxml.jackson.annotation.JsonInclude;
9 import tools.jackson.databind.DeserializationFeature;
10 import tools.jackson.databind.json.JsonMapper;
11
12 import org.springframework.hateoas.Link;
13 import org.springframework.http.HttpHeaders;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.MediaType;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.web.client.RestClient;
18
19 /**
20  * HTTP for hypermedia: every request targets a link the server provided. There is no
21  * method that takes a path to append to a base URL, by design.
22  */
23 public class HypermediaClient {
24
25     public static final MediaType HAL_JSON = MediaType.parseMediaType("application/hal+json");
26     public static final MediaType MERGE_PATCH = MediaType.parseMediaType("application/merge-patch+json");
27
28     private final RestClient http;
29     private final JsonMapper json;
30
31     public HypermediaClient(RestClient.Builder builder) {
32         this.json = JsonMapper.builder()
33             // Tolerant reader: new properties on the server must never break this client.
34             .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
35             // The API's convention: decimals are strings, and absent fields are omitted.
36             .withConfigOverride(BigDecimal.class,
37                 o -> o.setFormat(JsonFormat.Value.forShape(JsonFormat.Shape.STRING)))
38             .changeDefaultPropertyInclusion(i -> i.withValueInclusion(JsonInclude.Include.NON_NULL))
39             .build();
40         this.http = builder
41             .defaultHeader(HttpHeaders.ACCEPT, HAL_JSON + ", application/problem+json")
42             // Pin the API version on every request, so a new default can never change
43             // what this client receives.
44             .defaultHeader("API-Version", "2")
45             .defaultStatusHandler(HttpStatus::isError, (request, response) -> {
46                 throw ApiProblem.from(response, json);
47             })
48             .build();
49     }
50
51     public HalResource get(URI target) {
52         return get(target, headers -> { });
53     }
54
55     public HalResource get(URI target, Consumer<HttpHeaders> headers) {
56         return read(http.get().uri(target).headers(headers).retrieve().toEntity(String.class));
57     }
58
59     public HalResource post(Link target, Object body, Consumer<HttpHeaders> headers) {
60         RestClient.RequestBodySpec request = http.post().uri(target.toUri()).headers(headers);
61         if (body != null) {
62             request.contentType(MediaType.APPLICATION_JSON).body(json.writeValueAsString(body));
63         }
64         return read(request.retrieve().toEntity(String.class));
65     }
66
67     /** A JSON merge patch, sent only if the resource still has the ETag the caller read. */
68     public HalResource patch(Link target, Object changes, String ifMatch) {
69         return read(http.patch().uri(target.toUri())
70             .contentType(MERGE_PATCH)
71             .header(HttpHeaders.IF_MATCH, ifMatch)
72             .body(json.writeValueAsString(changes))
73             .retrieve().toEntity(String.class));

```

```

74     }
75
76     private HalResource read(ResponseEntity<String> response) {
77         String body = response.getBody();
78         return new HalResource(json.readTree(body == null ? "{}" : body), response.getHeaders(), json);
79     }
80 }

```

brokerage-client/src/main/java/com/example/brokerage/client/Model.java

86 lines

```

1  package com.example.brokerage.client;
2
3  import java.math.BigDecimal;
4  import java.net.URI;
5  import java.time.Instant;
6  import java.util.List;
7  import java.util.Optional;
8
9  import com.fasterxml.jackson.annotation.JsonProperty;
10 import tools.jackson.databind.JsonNode;
11
12 /**
13  * The client's view of the API's representations: only the fields it uses. Enumerations
14  * are read as strings, so a status the server adds next year cannot break this client.
15  */
16 public final class Model {
17
18     private Model() {
19     }
20
21     public record Account(String id, String nickname, String type, String registration, String currency) {
22     }
23
24     public record Balances(String currency, BigDecimal cash, BigDecimal reservedForOrders,
25         BigDecimal buyingPower, BigDecimal positionsMarketValue, BigDecimal totalEquity,
26         BigDecimal unrealizedPnl) {
27     }
28
29     public record Position(String instrumentId, String symbol, BigDecimal quantity, BigDecimal averageCost,
30         BigDecimal lastPrice, BigDecimal marketValue, BigDecimal unrealizedPnl,
31         BigDecimal unrealizedPnlPercent) {
32     }
33
34     public record Instrument(String id, String symbol, String name, String type, boolean tradable) {
35     }
36
37     /** A quote, as served by API version 2. */
38     public record Quote(String symbol, BigDecimal last, PriceLevel bid, PriceLevel ask, BigDecimal change,
39         BigDecimal changePercent, Instant asOf) {
40
41         public record PriceLevel(BigDecimal price, BigDecimal size) {
42         }
43     }
44
45     /** What to trade: the body of a preview or a new order. */
46     public record OrderTicket(String instrumentId, String side, String type, BigDecimal quantity,
47         BigDecimal limitPrice, String timeInForce, String clientOrderId) {
48
49         public static OrderTicket market(String instrumentId, String side, BigDecimal quantity) {
50             return new OrderTicket(instrumentId, side, "MARKET", quantity, null, "DAY", null);
51         }
52
53         public static OrderTicket limit(String instrumentId, String side, BigDecimal quantity,
54             BigDecimal price) {
55             return new OrderTicket(instrumentId, side, "LIMIT", quantity, price, "DAY", null);
56         }
57     }
58
59     public record OrderPreview(String symbol, BigDecimal estimatedPrice, BigDecimal estimatedTotal,

```

```

60         BigDecimal buyingPower, boolean acceptable, List<Issue> issues) {
61
62         public record Issue(String type, String detail) {
63         }
64     }
65
66     /** An order, with the links the server offered for it at the time it was read. */
67     public record Order(String id, String symbol, String side, String type, BigDecimal quantity,
68         BigDecimal limitPrice, String status, BigDecimal filledQuantity, BigDecimal averageFillPrice,
69         Instant createdAt, @JsonProperty("_links") JsonNode links) {
70
71         public URI self() {
72             return link("self").orElseThrow();
73         }
74
75         /** Present only while the server allows the action it names. */
76         public Optional<URI> link(String rel) {
77             JsonNode href = links == null ? null : links.path(rel).path("href");
78             return href == null || href.isMissingNode() ? Optional.empty()
79                 : Optional.of(URI.create(href.asString()));
80         }
81
82         public boolean isCancellable() {
83             return link("bk:cancel").isPresent();
84         }
85     }
86 }

```

brokerage-client/src/test/java/com/example/brokerage/client/BrokerageClientTest.java

134 lines

```

1  package com.example.brokerage.client;
2
3  import static org.assertj.core.api.Assertions.assertThat;
4  import static org.assertj.core.api.Assertions.assertThatThrownBy;
5  import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
6  import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
7  import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
8  import static org.springframework.test.web.client.response.MockRestResponseCreators.withStatus;
9  import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;
10
11  import java.math.BigDecimal;
12  import java.net.URI;
13
14  import com.example.brokerage.client.Model.Order;
15  import com.example.brokerage.client.Model.OrderTicket;
16
17  import org.junit.jupiter.api.BeforeEach;
18  import org.junit.jupiter.api.Test;
19  import org.springframework.http.HttpMethod;
20  import org.springframework.http.HttpStatus;
21  import org.springframework.http.MediaType;
22  import org.springframework.test.web.client.ExpectedCount;
23  import org.springframework.test.web.client.MockRestServiceServer;
24  import org.springframework.web.client.RestClient;
25
26  /**
27   * The client follows whatever links the server sends, including unusual ones, and never
28   * builds a URL of its own.
29   */
30  class BrokerageClientTest {
31
32      static final MediaType HAL = MediaType.parseMediaType("application/hal+json");
33      static final MediaType PROBLEM = MediaType.parseMediaType("application/problem+json");
34
35      // Deliberately odd URLs: a client that constructs paths would never find them.
36      static final String ROOT = ""
37          {"_links": {"self": {"href": "https://api.test/"},
38              "bk:accounts": {"href": "https://api.test/v9/acct-list"},
39              "bk:instruments": {"href": "https://api.test/catalog{?symbol,type}", "templated": true}}};

```

```

40 static final String ACCOUNTS = ""
41     {"_embedded": {"bk:accounts": [{"id": "ACC-1", "nickname": "Main", "type": "MARGIN",
42         "_links": {"self": {"href": "https://api.test/a/1"},
43             "bk:balances": {"href": "https://api.test/money/for/ACC-1"},
44             "bk:orders": {"href": "https://api.test/a/1/o/{status,cursor}",
45                 "templated": true}}}}},
46     "_links": {"self": {"href": "https://api.test/v9/acct-list"}}}"";
47
48 MockRestServiceServer server;
49 BrokerageClient brokerage;
50
51 @BeforeEach
52 void setUp() {
53     RestClient.Builder builder = RestClient.builder();
54     server = MockRestServiceServer.bindTo(builder).build();
55     brokerage = new BrokerageClient(new HypermediaClient(builder), URI.create("https://api.test/"));
56 }
57
58 @Test
59 void itFindsBalancesByFollowingLinksFromTheRoot() {
60     expectGet("https://api.test/", ROOT);
61     expectGet("https://api.test/v9/acct-list", ACCOUNTS);
62     expectGet("https://api.test/money/for/ACC-1", ""
63         {"currency": "USD", "cash": "1000.10", "buyingPower": "2000.20", "futureField": "ignored",
64         "_links": {"self": {"href": "https://api.test/money/for/ACC-1"}}}""");
65
66     var balances = brokerage.balances("ACC-1");
67
68     assertThat(balances.cash()).isEqualToByComparingTo("1000.10");
69     assertThat(balances.buyingPower()).isEqualToByComparingTo("2000.20");
70     server.verify();
71 }
72
73 @Test
74 void aTransientFailureIsRetriedWithTheSameIdempotencyKey() {
75     expectGet("https://api.test/", ROOT);
76     expectGet("https://api.test/v9/acct-list", ACCOUNTS);
77     server.expect(ExpectedCount.once(), requestTo("https://api.test/a/1/o"))
78         .andExpect(method(HttpMethod.POST))
79         .andExpect(header("Idempotency-Key", "key-42"))
80         .andRespond(withStatus(HttpStatus.SERVICE_UNAVAILABLE).contentType(PROBLEM)
81             .header("Retry-After", "0"))
82         .body("""
83             {"type": "https://docs.test/problems/unavailable", "title": "Busy",
84             "status": 503}""");
85     server.expect(ExpectedCount.once(), requestTo("https://api.test/a/1/o"))
86         .andExpect(method(HttpMethod.POST))
87         .andExpect(header("Idempotency-Key", "key-42"))
88         .andRespond(withStatus(HttpStatus.CREATED).contentType(HAL).body("""
89             {"id": "ORD-9", "status": "OPEN", "quantity": "5",
90             "_links": {"self": {"href": "https://api.test/a/1/o/9"}}}"""));
91
92     OrderTicket ticket = OrderTicket.market("EQ-X", "BUY", new BigDecimal("5"));
93     Order order = brokerage.place("ACC-1", ticket, "key-42");
94
95     assertThat(order.id()).isEqualTo("ORD-9");
96     server.verify();
97 }
98
99 @Test
100 void aBusinessRuleViolationArrivesAsATypedProblem() {
101     expectGet("https://api.test/", ROOT);
102     expectGet("https://api.test/v9/acct-list", ACCOUNTS);
103     server.expect(requestTo("https://api.test/a/1/o")).andRespond(
104         withStatus(HttpStatus.UNPROCESSABLE_CONTENT).contentType(PROBLEM).body("""
105             {"type": "https://docs.brokerage.example/problems/insufficient-buying-power",
106             "title": "The account does not have enough buying power", "status": 422,
107             "detail": "This order needs 900.00 but only 10.00 is available",
108             "available": "10.00"}"""));

```

```

109
110     OrderTicket ticket = OrderTicket.market("EQ-X", "BUY", BigDecimal.TEN);
111     assertThatThrownBy(() -> brokerage.place("ACC-1", ticket, "k"))
112         .isInstanceOfSatisfying(ApiProblem.class, problem -> {
113             assertThat(problem.typeName()).isEqualTo("insufficient-buying-power");
114             assertThat(problem.isTransient()).isFalse();
115             assertThat(problem.body().path("available").asString()).isEqualTo("10.00");
116         });
117 }
118
119 @Test
120 void anOrderWithoutACancelLinkIsNotCancelledAndNoRequestIsSent() {
121     Order filled = new Order("ORD-7", "AAPL", "BUY", "MARKET", BigDecimal.ONE, null, "FILLED",
122         BigDecimal.ONE, new BigDecimal("232.42"), null, null);
123
124     assertThatThrownBy(() -> brokerage.cancel(filled))
125         .isInstanceOf(IllegalStateException.class)
126         .hasMessageContaining("can no longer be canceled");
127     server.verify();
128 }
129
130 private void expectGet(String url, String body) {
131     server.expect(ExpectedCount.once(), requestTo(url)).andExpect(method(HttpMethod.GET))
132         .andRespond(withSuccess(body, HAL));
133 }
134 }

```

E.6 The agent

brokerage-agent: the Spring AI assistant, its tools and its guardrails.

brokerage-agent/src/main/java/com/example/brokerage/agent/AgentConsole.java

80 lines

```

1 package com.example.brokerage.agent;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.util.UUID;
7
8 import org.springframework.ai.chat.client.ChatClient;
9 import org.springframework.ai.chat.memory.ChatMemory;
10 import org.springframework.ai.chat.model.ChatResponse;
11 import org.springframework.beans.factory.annotation.Value;
12 import org.springframework.boot.CommandLineRunner;
13 import org.springframework.boot.autoconfigure.condition.ConditionalOnBooleanProperty;
14 import org.springframework.stereotype.Component;
15
16 /**
17  * A console conversation. Lines starting with "/" are commands handled here, by code, and
18  * never reach the model: that is what makes "/confirm" a human decision.
19  */
20 @Component
21 @ConditionalOnBooleanProperty(name = "brokerage.console.enabled", matchIfMissing = true)
22 class AgentConsole implements CommandLineRunner {
23
24     private final ChatClient chat;
25     private final PendingActions pending;
26     private final BrokerageTools tools;
27     private final boolean modelConfigured;
28     private final String conversation = UUID.randomUUID().toString();
29
30     AgentConsole(ChatClient chat, PendingActions pending, BrokerageTools tools,
31         @Value("${spring.ai.anthropic.api-key:}") String apiKey) {
32         this.chat = chat;
33         this.pending = pending;
34         this.tools = tools;

```

```

35         this.modelConfigured = !apiKey.isBlank();
36     }
37
38     @Override
39     public void run(String... args) throws IOException {
40         System.out.println("Brokerage assistant. Ask about your accounts or ask for a trade.");
41         System.out.println("Commands: /confirm CODE, /discard CODE, /status, /quit");
42         BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
43         for (String line; (line = prompt(in)) != null && !line.equals("/quit"); ) {
44             if (line.startsWith("/confirm ")) {
45                 System.out.println(pending.confirm(line.substring(9).trim()));
46             } else if (line.startsWith("/discard ")) {
47                 boolean discarded = pending.discard(line.substring(9).trim());
48                 System.out.println(discarded ? "Discarded." : "Nothing to discard.");
49             } else if (line.equals("/status")) {
50                 // The tools are plain Java: call one directly to check the API and the token,
51                 // with no model involved.
52                 tools.listAccounts()
53                     .forEach(account -> System.out.println(account.id() + " " + account.nickname()));
54             } else if (!line.isBlank()) {
55                 System.out.println(modelConfigured ? ask(line)
56                     : "Set ANTHROPIC_API_KEY to talk to the assistant.");
57             }
58         }
59     }
60
61     private String ask(String question) {
62         ChatResponse response = chat.prompt()
63             .user(question)
64             .advisors(advisor -> advisor.param(ChatMemory.CONVERSATION_ID, conversation))
65             .call()
66             .chatResponse();
67         // Check why the model stopped before reading its text: a refusal has no answer to show.
68         String finish = response.getResult().getMetadata().getFinishReason();
69         if ("refusal".equalsIgnoreCase(finish)) {
70             return "The assistant declined that request. Try rephrasing it.";
71         }
72         return response.getResult().getOutput().getText();
73     }
74
75     private static String prompt(BufferedReader in) throws IOException {
76         System.out.print("\n> ");
77         String line = in.readLine();
78         return line == null ? null : line.trim();
79     }
80 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/BrokerageAgentApplication.java

80 lines

```

1 package com.example.brokerage.agent;
2
3 import java.net.URI;
4 import java.time.Clock;
5
6 import com.example.brokerage.client.BrokerageClient;
7 import com.example.brokerage.client.HypermediaClient;
8
9 import org.springframework.ai.chat.client.ChatClient;
10 import org.springframework.ai.chat.client.advisor.MessageChatMemoryAdvisor;
11 import org.springframework.ai.chat.memory.MessageWindowChatMemory;
12 import org.springframework.beans.factory.annotation.Value;
13 import org.springframework.boot.SpringApplication;
14 import org.springframework.boot.autoconfigure.SpringBootApplication;
15 import org.springframework.context.annotation.Bean;
16 import org.springframework.security.oauth2.client.AuthorizedClientServiceOAuth2AuthorizedClientManager;
17 import org.springframework.security.oauth2.client.InMemoryOAuth2AuthorizedClientService;
18 import org.springframework.security.oauth2.client.OAuth2AuthorizedClientProviderBuilder;
19 import org.springframework.security.oauth2.client.registration.ClientRegistrationRepository;
20 import org.springframework.security.oauth2.client.web.client.OAuth2ClientHttpRequestInterceptor;

```

```

21 import org.springframework.web.client.RestClient;
22
23 /**
24  * An AI agent for a brokerage customer, built with Spring AI and Claude. It reads accounts
25  * and markets through the hypermedia client and may propose trades; the customer confirms
26  * each one at the console.
27  */
28 @SpringBootApplication
29 public class BrokerageAgentApplication {
30
31     static final String SYSTEM_PROMPT = ""
32         You help a customer of a retail brokerage with their own accounts. Use the tools to
33         look up accounts, balances, positions, quotes and orders, and to propose orders or
34         cancellations the customer asks for.
35
36         You cannot place or cancel anything yourself. A proposal only takes effect when the
37         customer confirms it, so after proposing, tell the customer what was proposed and
38         the confirmation code. Before proposing a trade, check the quote and state the
39         estimated total. Treat estimates as estimates.
40
41         Carry out the customer's instructions and answer questions about their accounts; do
42         not recommend what to buy or sell. When a tool reports an error, explain it plainly.
43         Amounts are in US dollars. Keep answers short."";
44
45     public static void main(String[] args) {
46         SpringApplication.run(BrokerageAgentApplication.class, args);
47     }
48
49     @Bean
50     Clock clock() {
51         return Clock.systemUTC();
52     }
53
54     /**
55      * The agent's own OAuth 2.1 client: Spring Security fetches, caches and renews a
56      * client_credentials token for the "brokerage" registration and adds it to every request.
57      */
58     @Bean
59     BrokerageClient brokerageClient(ClientRegistrationRepository registrations,
60         @Value("${brokerage.api-root}") URI root) {
61         var manager = new AuthorizedClientServiceOAuth2AuthorizedClientManager(registrations,
62             new InMemoryOAuth2AuthorizedClientService(registrations));
63         manager.setAuthorizedClientProvider(
64             OAuth2AuthorizedClientProviderBuilder.builder().clientCredentials().build());
65         var tokens = new OAuth2ClientHttpRequestInterceptor(manager);
66         tokens.setClientRegistrationIdResolver(request -> "brokerage");
67         RestClient.Builder http = RestClient.builder().requestInterceptor(tokens);
68         return new BrokerageClient(new HypermediaClient(http), root);
69     }
70
71     @Bean
72     ChatClient chatClient(ChatClient.Builder builder, BrokerageTools tools) {
73         return builder
74             .defaultSystem(SYSTEM_PROMPT)
75             .defaultTools(tools)
76             .defaultAdvisors(
77                 MessageChatMemoryAdvisor.builder(MessageWindowChatMemory.builder().build()).build())
78             .build();
79     }
80 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/BrokerageTools.java

103 lines

```

1 package com.example.brokerage.agent;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5
6 import com.example.brokerage.agent.PendingActions.Proposal;

```

```

7 import com.example.brokerage.client.BrokerageClient;
8 import com.example.brokerage.client.Model.Account;
9 import com.example.brokerage.client.Model.Balances;
10 import com.example.brokerage.client.Model.Order;
11 import com.example.brokerage.client.Model.OrderPreview;
12 import com.example.brokerage.client.Model.OrderTicket;
13 import com.example.brokerage.client.Model.Position;
14 import com.example.brokerage.client.Model.Quote;
15
16 import org.springframework.ai.tool.annotation.Tool;
17 import org.springframework.ai.tool.annotation.ToolParam;
18 import org.springframework.stereotype.Component;
19
20 /**
21  * The agent's tools. Each is a thin wrapper over the hypermedia client, so the model sees
22  * exactly what the API allows, and every rule is still enforced by the API itself.
23  * Read tools act immediately; tools that would change anything only create proposals.
24  */
25 @Component
26 public class BrokerageTools {
27
28     private final BrokerageClient brokerage;
29     private final PendingActions pending;
30
31     public BrokerageTools(BrokerageClient brokerage, PendingActions pending) {
32         this.brokerage = brokerage;
33         this.pending = pending;
34     }
35
36     @Tool(description = "List the customer's brokerage accounts: ID, nickname, type and currency.")
37     public List<Account> listAccounts() {
38         return brokerage.accounts();
39     }
40
41     @Tool(description = "Cash, cash reserved for open orders, buying power and total equity of one account.")
42     public Balances getBalances(@ToolParam(description = "Account ID, such as ACC-1001") String accountId) {
43         return brokerage.balances(accountId);
44     }
45
46     @Tool(description = "Holdings of one account with quantity, average cost, market value and "
47         + "unrealized P&L.")
48     public List<Position> getPositions(@ToolParam(description = "Account ID") String accountId) {
49         return brokerage.positions(accountId);
50     }
51
52     @Tool(description = "The latest quote for a stock or ETF: last price, bid, ask and change on the day.")
53     public Quote getQuote(@ToolParam(description = "Ticker symbol, such as AAPL") String symbol) {
54         return brokerage.quote(symbol);
55     }
56
57     @Tool(description = "Orders in one account that are still working (open or partially filled).")
58     public List<Order> getOpenOrders(@ToolParam(description = "Account ID") String accountId) {
59         return brokerage.openOrders(accountId);
60     }
61
62     @Tool(description = ""
63         + "Propose an order for the customer to confirm. The brokerage checks it first and the "
64         + "estimate is returned. Nothing is placed until the customer confirms the proposal.""")
65     public Object proposeOrder(@ToolParam(description = "Account ID") String accountId,
66         @ToolParam(description = "Ticker symbol") String symbol,
67         @ToolParam(description = "BUY or SELL") String side,
68         @ToolParam(description = "Number of shares, as a decimal string such as \"10\"") String quantity,
69         @ToolParam(description = "Limit price as a decimal string; omit for a market order",
70             required = false) String limitPrice) {
71         String instrumentId = brokerage.instrument(symbol).id();
72         BigDecimal shares = new BigDecimal(quantity);
73         OrderTicket ticket = limitPrice == null || limitPrice.isBlank()
74             ? OrderTicket.market(instrumentId, side, shares)
75             : OrderTicket.limit(instrumentId, side, shares, new BigDecimal(limitPrice));

```

```

76     OrderPreview preview = brokerage.preview(accountId, ticket);
77     if (!preview.acceptable()) {
78         return preview; // the issues explain why; nothing to confirm
79     }
80     String price = ticket.limitPrice() == null ? "at market" : "limit " + ticket.limitPrice();
81     String summary = "%s %s %s in %s, %s, estimated total %s USD".formatted(side, quantity, symbol,
82         accountId, price, preview.estimatedTotal());
83     return pending.propose(summary, idempotencyKey -> {
84         Order order = brokerage.place(accountId, ticket, idempotencyKey);
85         return "Placed order " + order.id() + ": " + order.status();
86     });
87 }
88
89 @Tool(description = "Propose canceling a working order. "
90     + "Nothing is canceled until the customer confirms.")
91 public Proposal proposeCancellation(@ToolParam(description = "Account ID") String accountId,
92     @ToolParam(description = "Order ID, such as ORD-100002") String orderId) {
93     Order order = brokerage.openOrders(accountId).stream()
94         .filter(open -> open.id().equals(orderId))
95         .findFirst()
96         .orElseThrow(() -> new IllegalArgumentException(
97             "No working order " + orderId + " in " + accountId));
98     String summary = "Cancel " + order.side() + " " + order.quantity() + " " + order.symbol()
99         + " (" + orderId + ")";
100     return pending.propose(summary,
101         ignored -> "Cancellation requested: " + brokerage.cancel(order).status());
102 }
103 }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/PendingActions.java

65 lines

```

1  package com.example.brokerage.agent;
2
3  import java.time.Clock;
4  import java.time.Duration;
5  import java.time.Instant;
6  import java.util.Locale;
7  import java.util.Map;
8  import java.util.UUID;
9  import java.util.concurrent.ConcurrentHashMap;
10 import java.util.function.Function;
11
12 import org.springframework.stereotype.Component;
13
14 /**
15  * The human-in-the-loop gate. The model can only propose a trade; carrying it out takes a
16  * person typing "/confirm CODE" at the console, a path that no tool can reach.
17  *
18  * <p>The confirmation code doubles as the order's Idempotency-Key, so confirming the same
19  * proposal twice, or retrying after a timeout, can never place a second order.
20  */
21 @Component
22 public class PendingActions {
23
24     /** What the model is told about a proposal: enough to ask the customer, nothing to act on. */
25     public record Proposal(String code, String summary, Instant expiresAt, String instructions) {
26     }
27
28     private record Pending(Proposal proposal, Function<String, String> action) {
29     }
30
31     static final Duration LIFETIME = Duration.ofMinutes(5);
32
33     private final Map<String, Pending> pending = new ConcurrentHashMap<>();
34     private final Clock clock;
35
36     public PendingActions(Clock clock) {
37         this.clock = clock;
38     }

```

```

39
40  /** Records an action to run later. The action receives the code to use as its Idempotency-Key. */
41  public Proposal propose(String summary, Function<String, String> action) {
42      String code = UUID.randomUUID().toString().substring(0, 8).toUpperCase(Locale.ROOT);
43      Proposal proposal = new Proposal(code, summary, clock.instant().plus(LIFETIME),
44          "Nothing has been done yet. Ask the customer to type /confirm " + code
45              + " to go ahead, or /discard " + code + " to drop it. It expires in 5 minutes.");
46      pending.put(code, new Pending(proposal, action));
47      return proposal;
48  }
49
50  /** Called only from the console, when a person types /confirm CODE. */
51  public String confirm(String code) {
52      Pending action = pending.remove(code.toUpperCase(Locale.ROOT));
53      if (action == null) {
54          return "There is no pending action " + code + ".";
55      }
56      if (clock.instant().isAfter(action.proposal().expiresAt())) {
57          return "Proposal " + code + " expired; ask the assistant again.";
58      }
59      return action.action().apply(action.proposal().code());
60  }
61
62  public boolean discard(String code) {
63      return pending.remove(code.toUpperCase(Locale.ROOT)) != null;
64  }
65  }

```

brokerage-agent/src/main/java/com/example/brokerage/agent/RefusalFallbacks.java

58 lines

```

1  package com.example.brokerage.agent;
2
3  import java.io.IOException;
4
5  import okhttp3.Interceptor;
6  import okhttp3.MediaType;
7  import okhttp3.Request;
8  import okhttp3.RequestBody;
9  import okhttp3.Response;
10 import okio.Buffer;
11 import tools.jackson.databind.json.JsonMapper;
12 import tools.jackson.databind.node.ObjectNode;
13
14 import org.springframework.ai.anthropic.http.okhttp.AnthropicHttpClientBuilderCustomizer;
15 import org.springframework.ai.anthropic.http.okhttp.SpringAiAnthropicHttpClient;
16 import org.springframework.stereotype.Component;
17
18 /**
19  * Opts every Messages API request into server-side refusal fallbacks: if a safety
20  * classifier declines a request, the API re-runs it on the fallback model Anthropic
21  * recommends for that category instead of returning the refusal. Spring AI has no option
22  * for this request field, so it is added at the HTTP layer.
23  */
24 @Component
25 class RefusalFallbacks implements AnthropicHttpClientBuilderCustomizer, Interceptor {
26
27     static final String BETA = "server-side-fallback-2026-07-01";
28     private static final JsonMapper JSON = JsonMapper.builder().build();
29
30     @Override
31     public void customize(SpringAiAnthropicHttpClient.Builder builder) {
32         builder.interceptor(this);
33     }
34
35     @Override
36     public Response intercept(Chain chain) throws IOException {
37         Request request = chain.request();
38         if (!"POST".equals(request.method()) || !request.url().encodedPath().endsWith("/v1/messages"))
39             || request.body() == null) {

```

```

40         return chain.proceed(request);
41     }
42     Buffer body = new Buffer();
43     request.body().writeTo(body);
44     MediaType type = request.body().contentType();
45     String existing = request.header("anthropic-beta");
46     return chain.proceed(request.newBuilder()
47         .header("anthropic-beta", existing == null ? BETA : existing + "," + BETA)
48         .post(RequestBody.create(withFallbacks(body.readUtf8()), type))
49         .build());
50 }
51
52 /** The request body with "fallbacks": "default" added. */
53 static String withFallbacks(String json) {
54     ObjectNode request = (ObjectNode) JSON.readTree(json);
55     request.put("fallbacks", "default");
56     return JSON.writeValueAsString(request);
57 }
58 }

```

brokerage-agent/src/main/resources/application.yaml

34 lines

```

1 spring:
2   application:
3     name: brokerage-agent
4   main:
5     web-application-type: none
6     banner-mode: off
7   ai:
8     anthropic:
9       api-key: ${ANTHROPIC_API_KEY:}
10    chat:
11      options:
12        model: claude-opus-5
13        # Leave room for tool calls and thinking; Spring AI's default is 4096.
14        max-tokens: 16000
15    security:
16      oauth2:
17        client:
18          registration:
19            brokerage:
20              provider: brokerage-auth
21              client-id: brokerage-agent
22              client-secret: agent-secret
23              authorization-grant-type: client_credentials
24              scope: accounts:read, market-data:read, orders:read, orders:write
25          provider:
26            brokerage-auth:
27              token-uri: http://localhost:9000/oauth2/token
28
29 brokerage:
30   api-root: http://localhost:8080/
31
32 logging:
33   level:
34     root: warn

```

brokerage-agent/src/test/java/com/example/brokerage/agent/BrokerageToolsTest.java

130 lines

```

1 package com.example.brokerage.agent;
2
3 import static org.assertj.core.api.Assertions.assertThat;
4 import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
5 import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
6 import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
7 import static org.springframework.test.web.client.response.MockRestResponseCreators.withStatus;
8 import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;
9
10 import java.net.URI;

```

```

11 import java.time.Clock;
12 import java.util.Arrays;
13
14 import com.example.brokerage.agent.PendingActions.Proposal;
15 import com.example.brokerage.client.BrokerageClient;
16 import com.example.brokerage.client.HypermediaClient;
17
18 import org.junit.jupiter.api.BeforeEach;
19 import org.junit.jupiter.api.Test;
20 import org.springframework.ai.support.ToolCallbacks;
21 import org.springframework.ai.tool.ToolCallback;
22 import org.springframework.http.HttpMethod;
23 import org.springframework.http.HttpStatus;
24 import org.springframework.http.MediaType;
25 import org.springframework.test.web.client.ExpectedCount;
26 import org.springframework.test.web.client.MockRestServiceServer;
27 import org.springframework.web.client.RestClient;
28
29 /**
30  * The tools as the model will see them, and the guarantee that matters most: a proposal
31  * places nothing, and only a human confirmation does.
32  */
33 class BrokerageToolsTest {
34
35     static final MediaType HAL = MediaType.parseMediaType("application/hal+json");
36     static final String ROOT = ""
37         {"_links": {"self": {"href": "http://api/"/}, "bk:accounts": {"href": "http://api/accounts"/},
38         "bk:instruments": {"href": "http://api/instruments?symbol,type"/}, "templated": true}}"";
39     static final String ACCOUNTS = ""
40         {"_embedded": {"bk:accounts": [{"id": "ACC-1001", "nickname": "Main", "type": "MARGIN",
41         "_links": {"self": {"href": "http://api/accounts/ACC-1001"/},
42         "bk:orders": {"href": "http://api/accounts/ACC-1001/orders?status,symbol,cursor,limit"/},
43         "templated": true},
44         "bk:order-previews": {"href": "http://api/accounts/ACC-1001/order-previews"}}}}"";
45     static final String AAPL = ""
46         {"_embedded": {"bk:instruments": [{"id": "EQ-AAPL", "symbol": "AAPL", "name": "Apple Inc.",
47         "type": "EQUITY", "tradable": true,
48         "_links": {"self": {"href": "http://api/instruments/EQ-AAPL"/}}}}"";
49
50     MockRestServiceServer api;
51     PendingActions pending;
52     BrokerageTools tools;
53
54     @BeforeEach
55     void setUp() {
56         RestClient.Builder builder = RestClient.builder();
57         api = MockRestServiceServer.bindTo(builder).build();
58         pending = new PendingActions(Clock.systemUTC());
59         var brokerage = new BrokerageClient(new HypermediaClient(builder), URI.create("http://api/"));
60         tools = new BrokerageTools(brokerage, pending);
61     }
62
63     @Test
64     void theModelSeesSevenToolsWithDescribedParameters() {
65         ToolCallback[] callbacks = ToolCallbacks.from(tools);
66         assertThat(Arrays.stream(callbacks).map(callback -> callback.getToolDefinition().name()))
67             .containsExactlyInAnyOrder("listAccounts", "getBalances", "getPositions", "getQuote",
68             "getOpenOrders", "proposeOrder", "proposeCancellation");
69         String proposeOrder = Arrays.stream(callbacks)
70             .filter(callback -> callback.getToolDefinition().name().equals("proposeOrder"))
71             .findFirst().orElseThrow().getToolDefinition().inputSchema();
72         assertThat(proposeOrder).contains("\\required\\").contains("\\quantity\\").contains("decimal string");
73     }
74
75     @Test
76     void proposingPlacesNothingAndConfirmingPlacesExactlyOnce() {
77         expect("http://api/", ROOT); // symbol -> instrument
78         expect("http://api/instruments?symbol=AAPL", AAPL);
79         expect("http://api/", ROOT); // account -> its previews link

```

```

80     expect("http://api/accounts", ACCOUNTS);
81     api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/order-previews"))
82         .andExpect(method(HttpMethod.POST))
83         .andRespond(withSuccess("""
84             {"symbol": "AAPL", "estimatedPrice": "232.42", "estimatedTotal": "2324.20",
85              "buyingPower": "55241.10", "acceptable": true, "issues": []}""", HAL));
86
87     Object result = tools.proposeOrder("ACC-1001", "AAPL", "BUY", "10", null);
88
89     assertThat(result).isInstanceOf(Proposal.class);
90     Proposal proposal = (Proposal) result;
91     assertThat(proposal.summary()).contains("BUY 10 AAPL").contains("2324.20");
92     api.verify(); // a preview, and no order
93
94     api.reset();
95     expect("http://api/", ROOT);
96     expect("http://api/accounts", ACCOUNTS);
97     api.expect(ExpectedCount.once(), requestTo("http://api/accounts/ACC-1001/orders"))
98         .andExpect(method(HttpMethod.POST))
99         .andExpect(header("Idempotency-Key", proposal.code()))
100        .andRespond(withStatus(HttpStatus.CREATED).contentType(HAL).body("""
101            {"id": "ORD-100009", "status": "OPEN",
102             "_links": {"self": {"href": "http://api/accounts/ACC-1001/orders/ORD-100009"}}}"""));
103
104     assertThat(pending.confirm(proposal.code())).isEqualTo("Placed order ORD-100009: OPEN");
105     assertThat(pending.confirm(proposal.code())).startsWith("There is no pending action");
106     api.verify();
107 }
108
109 @Test
110 void anUnacceptablePreviewIsReturnedWithoutAProposal() {
111     expect("http://api/", ROOT); // symbol -> instrument
112     expect("http://api/instruments?symbol=AAPL", AAPL);
113     expect("http://api/", ROOT); // account -> its previews link
114     expect("http://api/accounts", ACCOUNTS);
115     api.expect(requestTo("http://api/accounts/ACC-1001/order-previews")).andRespond(withSuccess("""
116         {"symbol": "AAPL", "acceptable": false, "issues": [{"type":
117             "https://docs.brokerage.example/problems/insufficient-buying-power",
118             "detail": "This order needs 2324200.00 but only 55241.10 is available"}]}""", HAL));
119
120     Object result = tools.proposeOrder("ACC-1001", "AAPL", "BUY", "10000", null);
121
122     assertThat(result).isNotInstanceOf(Proposal.class);
123     assertThat(result.toString()).contains("insufficient-buying-power");
124 }
125
126 private void expect(String url, String body) {
127     api.expect(ExpectedCount.once(), requestTo(url)).andExpect(method(HttpMethod.GET))
128         .andRespond(withSuccess(body, HAL));
129 }
130 }

```

brokerage-agent/src/test/java/com/example/brokerage/agent/GuardrailsTest.java

81 lines

```

1 package com.example.brokerage.agent;
2
3 import static org.assertj.core.api.Assertions.assertThat;
4
5 import java.time.Clock;
6 import java.time.Instant;
7 import java.time.ZoneOffset;
8 import java.util.concurrent.atomic.AtomicInteger;
9
10 import org.junit.jupiter.api.Test;
11
12 /**
13  * Proposals expire, can be discarded, and run at most once; every Messages API request is
14  * opted into server-side refusal fallbacks.
15  */

```

```

16 class GuardrailsTest {
17
18     @Test
19     void anExpiredProposalIsNotCarriedOut() {
20         MutableClock clock = new MutableClock();
21         PendingActions pending = new PendingActions(clock);
22         AtomicInteger runs = new AtomicInteger();
23         var proposal = pending.propose("Buy 1 AAPL", key -> "ran " + runs.incrementAndGet());
24
25         clock.advanceSeconds(301);
26
27         assertThat(pending.confirm(proposal.code()).contains("expired"));
28         assertThat(runs).hasValue(0);
29     }
30
31     @Test
32     void aDiscardedProposalCannotBeConfirmed() {
33         PendingActions pending = new PendingActions(Clock.systemUTC());
34         var proposal = pending.propose("Sell 1 MSFT", key -> "ran");
35
36         assertThat(pending.discard(proposal.code())).isTrue();
37         assertThat(pending.confirm(proposal.code()).startsWith("There is no pending action"));
38     }
39
40     @Test
41     void theConfirmationCodeIsTheIdempotencyKey() {
42         PendingActions pending = new PendingActions(Clock.systemUTC());
43         var proposal = pending.propose("Buy 1 AAPL", key -> key);
44
45         assertThat(pending.confirm(proposal.code().toLowerCase()).isEqualTo(proposal.code()));
46     }
47
48     @Test
49     void everyMessagesRequestAsksForServerSideFallbacks() {
50         String request = "{\"model\":\"claude-opus-5\",\"max_tokens\":16000,\"messages\":[]}";
51
52         assertThat(RefusalFallbacks.withFallbacks(request))
53             .contains("{\"fallbacks\":\"default\"")
54             .contains("{\"model\":\"claude-opus-5\"")
55             .contains("{\"max_tokens\":16000");
56     }
57
58     /** A clock that tests can move forward. */
59     static final class MutableClock extends Clock {
60         private Instant now = Instant.parse("2026-09-28T15:00:00Z");
61
62         void advanceSeconds(long seconds) {
63             now = now.plusSeconds(seconds);
64         }
65
66         @Override
67         public Instant instant() {
68             return now;
69         }
70
71         @Override
72         public ZoneOffset getZone() {
73             return ZoneOffset.UTC;
74         }
75
76         @Override
77         public Clock withZone(java.time.ZoneId zone) {
78             return this;
79         }
80     }
81 }

```

E.7 The OpenAPI contract

openapi/brokerage-api.yaml: the API's OpenAPI 3.2 description.

openapi/brokerage-api.yaml

1086 lines

```
1 openapi: 3.2.1
2 $self: https://api.brokerage.example/openapi.yaml
3
4 info:
5   title: Brokerage API
6   version: "2.0"
7   summary: Accounts, positions, market data and trading for a retail brokerage, as hypermedia.
8   description: |
9     A resource-oriented REST API that follows the standards in *REST API Standards with
10     HATEOAS* (Smiroh Publishers).
11
12   * **Hypermedia.** Every representation is HAL (`application/hal+json`) with a `_links`
13     object. Ask for `application/prs.hal-forms+json` to also receive `_templates`: the
14     state transitions available right now. Clients should bookmark only the API root and
15     follow links from there. Custom link relations use the CURIE `bk:`, which expands to
16     `https://docs.brokerage.example/rels/{rel}`.
17   * **Money.** Prices, amounts and quantities are decimal strings, never JSON numbers.
18   * **Errors.** Every error is an RFC 9457 problem document (`application/problem+json`)
19     whose `type` URI identifies the error. Branch on `type`, never on `title` or `detail`.
20   * **Versions.** Send `API-Version: 2` to opt in to version 2. Version 1 is the default and
21     is deprecated: its responses carry `Deprecation` (RFC 9745) and `Sunset` (RFC 8594).
22   * **Quotas.** Every response carries `RateLimit-Policy` and `RateLimit`. Exceeding a quota
23     returns 429 with `Retry-After`.
24   * **Tracing.** Every response echoes `X-Request-Id`; quote it in support requests.
25   contact:
26     name: Brokerage API support
27     url: https://docs.brokerage.example/support
28   license:
29     name: Apache 2.0
30     identifier: Apache-2.0
31
32 servers:
33   - url: http://localhost:8080
34     description: Local development (see brokerage-apis/README.md)
35   - url: https://api.brokerage.example
36     description: Production (illustrative)
37
38 tags:
39   - name: discovery
40     summary: Discovery
41     description: The API root and the protected resource metadata, where every client starts.
42     kind: nav
43   - name: accounts
44     summary: Accounts
45     description: A customer's brokerage accounts.
46     kind: nav
47   - name: holdings
48     summary: Balances and positions
49     parent: accounts
50     kind: nav
51   - name: transactions
52     summary: Transaction history
53     parent: accounts
54     kind: nav
55   - name: trading
56     summary: Trading
57     description: Orders, previews and live order events.
58     kind: nav
59   - name: orders
60     summary: Orders
61     parent: trading
62     kind: nav
```

```

63 - name: order-events
64   summary: Order events
65   parent: trading
66   kind: nav
67 - name: market-data
68   summary: Market data
69   description: Instruments, quotes and option chains.
70   kind: nav
71 - name: deprecated
72   summary: Deprecated
73   description: Operations or representations scheduled for removal.
74   kind: badge
75
76 paths:
77   /:
78     get:
79       operationId: getApiRoot
80       summary: The API root
81       description: The one URL a client needs to know. Everything else is discovered from its links.
82       tags: [discovery]
83       security: []
84       responses:
85         "200":
86           description: Links to the top-level resources and to this document.
87           headers:
88             X-Request-Id: { $ref: "#/components/headers/X-Request-Id" }
89           content:
90             application/hal+json:
91               schema: { $ref: "#/components/schemas/ApiRoot" }
92
93   /.well-known/oauth-protected-resource:
94     get:
95       operationId: getProtectedResourceMetadata
96       summary: OAuth 2.0 protected resource metadata (RFC 9728)
97       description: Which authorization server issues tokens for this API, and which scopes it uses.
98       tags: [discovery]
99       security: []
100      responses:
101        "200":
102          description: The metadata document.
103          content:
104            application/json:
105              schema: { $ref: "#/components/schemas/ProtectedResourceMetadata" }
106
107   /accounts:
108     get:
109       operationId: listAccounts
110       summary: List accounts
111       tags: [accounts]
112       security: [{ oauth2: [accounts:read] }]
113       responses:
114         "200":
115           description: The caller's accounts. Links depend on the token's scopes.
116           headers:
117             RateLimit: { $ref: "#/components/headers/RateLimit" }
118             RateLimit-Policy: { $ref: "#/components/headers/RateLimit-Policy" }
119           content:
120             application/hal+json:
121               schema: { $ref: "#/components/schemas/AccountCollection" }
122         "401": { $ref: "#/components/responses/Unauthorized" }
123         "403": { $ref: "#/components/responses/Forbidden" }
124         "429": { $ref: "#/components/responses/TooManyRequests" }
125
126   /accounts/{accountId}:
127     parameters:
128       - $ref: "#/components/parameters/accountId"
129     get:
130       operationId: getAccount
131       summary: Get an account

```

```

132     tags: [accounts]
133     security: [{ oauth2: [accounts:read] }]
134     responses:
135       "200":
136         description: >-
137           The account. With orders:read the links include orders, previews and events; with
138           orders:write, HAL-FORMS clients also receive a placeOrder template.
139         content:
140           application/hal+json:
141             schema: { $ref: "#/components/schemas/Account" }
142           application/prs.hal-forms+json:
143             schema: { $ref: "#/components/schemas/Account" }
144       "401": { $ref: "#/components/responses/Unauthorized" }
145       "403": { $ref: "#/components/responses/Forbidden" }
146       "404": { $ref: "#/components/responses/NotFound" }
147
148 /accounts/{accountId}/balances:
149   parameters:
150     - $ref: "#/components/parameters/accountId"
151   get:
152     operationId: getBalances
153     summary: Get balances and buying power
154     tags: [holdings]
155     security: [{ oauth2: [accounts:read] }]
156     responses:
157       "200":
158         description: Cash, reservations for open orders, buying power and market value.
159         content:
160           application/hal+json:
161             schema: { $ref: "#/components/schemas/Balances" }
162       "401": { $ref: "#/components/responses/Unauthorized" }
163       "404": { $ref: "#/components/responses/NotFound" }
164
165 /accounts/{accountId}/positions:
166   parameters:
167     - $ref: "#/components/parameters/accountId"
168   get:
169     operationId: listPositions
170     summary: List positions
171     tags: [holdings]
172     security: [{ oauth2: [accounts:read] }]
173     responses:
174       "200":
175         description: Every holding, valued at the current market price.
176         content:
177           application/hal+json:
178             schema: { $ref: "#/components/schemas/PositionCollection" }
179       "401": { $ref: "#/components/responses/Unauthorized" }
180       "404": { $ref: "#/components/responses/NotFound" }
181
182 /accounts/{accountId}/positions/{instrumentId}:
183   parameters:
184     - $ref: "#/components/parameters/accountId"
185     - $ref: "#/components/parameters/instrumentId"
186   get:
187     operationId: getPosition
188     summary: Get one position
189     tags: [holdings]
190     security: [{ oauth2: [accounts:read] }]
191     responses:
192       "200":
193         description: The position in one instrument.
194         content:
195           application/hal+json:
196             schema: { $ref: "#/components/schemas/Position" }
197       "404": { $ref: "#/components/responses/NotFound" }
198
199 /accounts/{accountId}/transactions:
200   parameters:

```

```

201     - $ref: "#/components/parameters/accountId"
202 get:
203   operationId: listTransactions
204   summary: List transactions
205   description: Newest first. Follow the `next` link for older entries.
206   tags: [transactions]
207   security: [{ oauth2: [accounts:read] }]
208   parameters:
209     - name: type
210       in: query
211       schema: { $ref: "#/components/schemas/TransactionType" }
212     - name: from
213       in: query
214       description: Earliest date to include (UTC), inclusive.
215       schema: { type: string, format: date }
216     - name: to
217       in: query
218       description: Latest date to include (UTC), inclusive.
219       schema: { type: string, format: date }
220     - $ref: "#/components/parameters/cursor"
221     - $ref: "#/components/parameters/limit"
222   responses:
223     "200":
224       description: One page of transactions.
225       content:
226         application/hal+json:
227           schema: { $ref: "#/components/schemas/TransactionPage" }
228     "400": { $ref: "#/components/responses/BadRequest" }
229     "404": { $ref: "#/components/responses/NotFound" }
230
231 /accounts/{accountId}/orders:
232   parameters:
233     - $ref: "#/components/parameters/accountId"
234   get:
235     operationId: listOrders
236     summary: List orders
237     description: Newest first, optionally filtered. `status` may be repeated.
238     tags: [orders]
239     security: [{ oauth2: [orders:read] }]
240     parameters:
241       - name: status
242         in: query
243         style: form
244         explode: true
245         schema:
246           type: array
247           items: { $ref: "#/components/schemas/OrderStatus" }
248       - name: symbol
249         in: query
250         schema: { type: string, examples:[AAPL] }
251       - $ref: "#/components/parameters/cursor"
252       - $ref: "#/components/parameters/limit"
253     responses:
254       "200":
255         description: One page of orders.
256         content:
257           application/hal+json:
258             schema: { $ref: "#/components/schemas/OrderPage" }
259       "400": { $ref: "#/components/responses/BadRequest" }
260       "403": { $ref: "#/components/responses/Forbidden" }
261       "404": { $ref: "#/components/responses/NotFound" }
262   post:
263     operationId: placeOrder
264     summary: Place an order
265     description: >-
266       Every new order needs a unique Idempotency-Key. Retrying with the same key and body
267       returns the original order (Idempotent-Replayed: true); the same key with a different
268       body is refused.
269     tags: [orders]

```

```

270 security: [{ oauth2: [orders:write] }]
271 parameters:
272   - $ref: "#/components/parameters/Idempotency-Key"
273 requestBody:
274   required: true
275   content:
276     application/json:
277       schema: { $ref: "#/components/schemas/OrderRequest" }
278     examples:
279       limitBuy:
280         summary: Buy 10 AAPL at 230.00 or better
281         dataValue:
282           instrumentId: EQ-AAPL
283           side: BUY
284           type: LIMIT
285           timeInForce: DAY
286           quantity: "10"
287           limitPrice: "230.00"
288           clientOrderId: rebalance-2026-09
289 responses:
290   "201":
291     description: The order was accepted and is now working.
292     headers:
293       Location: { $ref: "#/components/headers/Location" }
294       ETag: { $ref: "#/components/headers/ETag" }
295       Idempotent-Replayed: { $ref: "#/components/headers/Idempotent-Replayed" }
296     content:
297       application/hal+json:
298         schema: { $ref: "#/components/schemas/Order" }
299     links:
300       getOrder:
301         operationId: getOrder
302         parameters:
303           accountId: $request.path.accountId
304           orderId: $response.body#/id
305       cancelOrder:
306         operationId: cancelOrder
307         parameters:
308           accountId: $request.path.accountId
309           orderId: $response.body#/id
310   "400": { $ref: "#/components/responses/BadRequest" }
311   "403": { $ref: "#/components/responses/Forbidden" }
312   "404": { $ref: "#/components/responses/NotFound" }
313   "422": { $ref: "#/components/responses/UnprocessableContent" }
314   "429": { $ref: "#/components/responses/TooManyRequests" }
315
316 /accounts/{accountId}/orders/{orderId}:
317 parameters:
318   - $ref: "#/components/parameters/accountId"
319   - $ref: "#/components/parameters/orderId"
320 get:
321   operationId: getOrder
322   summary: Get an order
323   tags: [orders]
324   security: [{ oauth2: [orders:read] }]
325   parameters:
326     - $ref: "#/components/parameters/If-None-Match"
327   responses:
328     "200":
329       description: >-
330         The order. A working order links to `edit` and `bk:cancel`; a filled or canceled
331         order does not.
332       headers:
333         ETag: { $ref: "#/components/headers/ETag" }
334       content:
335         application/hal+json:
336           schema: { $ref: "#/components/schemas/Order" }
337         application/prs.hal-forms+json:
338           schema: { $ref: "#/components/schemas/Order" }

```

```

339     "304":
340         description: The order has not changed since the ETag in If-None-Match.
341     "404": { $ref: "#/components/responses/NotFound" }
342 patch:
343     operationId: replaceOrder
344     summary: Change a working order
345     description: >-
346         A JSON merge patch (RFC 7396): only the fields sent are changed. If-Match with the
347         order's current ETag is required, so concurrent edits cannot overwrite one another.
348     tags: [orders]
349     security: [{ oauth2: [orders:write] }]
350     parameters:
351         - $ref: "#/components/parameters/If-Match"
352     requestBody:
353         required: true
354         content:
355             application/merge-patch+json:
356                 schema: { $ref: "#/components/schemas/OrderAmendment" }
357     responses:
358         "200":
359             description: The changed order.
360             headers:
361                 ETag: { $ref: "#/components/headers/ETag" }
362             content:
363                 application/hal+json:
364                     schema: { $ref: "#/components/schemas/Order" }
365         "404": { $ref: "#/components/responses/NotFound" }
366         "409": { $ref: "#/components/responses/Conflict" }
367         "412": { $ref: "#/components/responses/PreconditionFailed" }
368         "422": { $ref: "#/components/responses/UnprocessableContent" }
369         "428": { $ref: "#/components/responses/PreconditionRequired" }
370
371 /accounts/{accountId}/orders/{orderId}/cancellation:
372     parameters:
373         - $ref: "#/components/parameters/accountId"
374         - $ref: "#/components/parameters/orderId"
375     post:
376         operationId: cancelOrder
377         summary: Request cancellation
378         description: >-
379             Canceling is asynchronous: the order moves to PENDING_CANCEL at once and to CANCELLED
380             shortly after, and it may still fill in between. Orders are never deleted.
381     tags: [orders]
382     security: [{ oauth2: [orders:write] }]
383     responses:
384         "202":
385             description: Cancellation requested. Location points at the order.
386             headers:
387                 Location: { $ref: "#/components/headers/Location" }
388             content:
389                 application/hal+json:
390                     schema: { $ref: "#/components/schemas/Order" }
391         "404": { $ref: "#/components/responses/NotFound" }
392         "409": { $ref: "#/components/responses/Conflict" }
393
394 /accounts/{accountId}/order-previews:
395     parameters:
396         - $ref: "#/components/parameters/accountId"
397     post:
398         operationId: previewOrder
399         summary: Preview an order
400         description: >-
401             Estimates cost and checks every rule without placing anything. A preview that would be
402             refused is still a 200, with `acceptable: false` and the reasons.
403     tags: [orders]
404     security: [{ oauth2: [orders:read] }]
405     requestBody:
406         required: true
407         content:

```

```

408     application/json:
409       schema: { $ref: "#/components/schemas/OrderRequest" }
410   responses:
411     "200":
412       description: The estimate.
413       content:
414         application/hal+json:
415           schema: { $ref: "#/components/schemas/OrderPreview" }
416     "400": { $ref: "#/components/responses/BadRequest" }
417     "404": { $ref: "#/components/responses/NotFound" }
418
419 /accounts/{accountId}/order-events:
420   parameters:
421     - $ref: "#/components/parameters/accountId"
422   get:
423     operationId: streamOrderEvents
424     summary: Stream order events
425     description: >-
426       Server-Sent Events for every order in the account. Reconnect with Last-Event-ID to
427       receive only the events you missed.
428     tags: [order-events]
429     security: [{ oauth2: [orders:read] }]
430     parameters:
431       - name: Last-Event-ID
432         in: header
433         schema: { type: integer, minimum: 0 }
434     responses:
435       "200":
436         description: An unending stream of events.
437         content:
438           text/event-stream:
439             itemSchema:
440               type: object
441               required: [id, event, data]
442             properties:
443               id: { type: string }
444             event:
445               type: string
446               enum: [order.created, order.replaced, order.partially-filled, order.filled,
447                 order.cancel-requested, order.cancelled]
448             data:
449               type: string
450               contentMediaType: application/json
451               contentSchema: { $ref: "#/components/schemas/OrderEventData" }
452       "404": { $ref: "#/components/responses/NotFound" }
453
454 /instruments:
455   get:
456     operationId: searchInstruments
457     summary: Search instruments
458     tags: [market-data]
459     security: [{ oauth2: [market-data:read] }]
460     parameters:
461       - name: symbol
462         in: query
463         schema: { type: string, examples: [AAPL] }
464       - name: type
465         in: query
466         schema: { $ref: "#/components/schemas/InstrumentType" }
467     responses:
468       "200":
469         description: Matching instruments.
470         content:
471           application/hal+json:
472             schema: { $ref: "#/components/schemas/InstrumentCollection" }
473
474 /instruments/{instrumentId}:
475   parameters:
476     - $ref: "#/components/parameters/instrumentId"

```

```

477 get:
478   operationId: getInstrument
479   summary: Get an instrument
480   tags: [market-data]
481   security: [{ oauth2: [market-data:read] }]
482   responses:
483     "200":
484       description: The instrument, with links to its quote and option chain or underlying.
485       content:
486         application/hal+json:
487           schema: { $ref: "#/components/schemas/Instrument" }
488     "404": { $ref: "#/components/responses/NotFound" }
489
490 /instruments/{instrumentId}/quote:
491   parameters:
492     - $ref: "#/components/parameters/instrumentId"
493   get:
494     operationId: getQuote
495     summary: Get a quote
496     description: >–
497       The shape depends on the API version: version 1 (the deprecated default) is flat;
498       version 2 groups price and size into bid and ask levels and adds the day's change.
499     tags: [market-data]
500     security: [{ oauth2: [market-data:read] }]
501     parameters:
502       - $ref: "#/components/parameters/API-Version"
503       - $ref: "#/components/parameters/If-None-Match"
504     responses:
505       "200":
506         description: The latest quote. Cacheable privately for one second.
507         headers:
508           Cache-Control:
509             schema: { type: string, examples: ["max-age=1, private"] }
510           ETag: { $ref: "#/components/headers/ETag" }
511           Deprecation: { $ref: "#/components/headers/Deprecation" }
512           Sunset: { $ref: "#/components/headers/Sunset" }
513         content:
514           application/hal+json:
515             schema:
516               oneOf:
517                 - $ref: "#/components/schemas/Quote"
518                 - $ref: "#/components/schemas/QuoteV1"
519       "304":
520         description: The quote has not changed.
521       "400": { $ref: "#/components/responses/BadRequest" }
522       "404": { $ref: "#/components/responses/NotFound" }
523
524 /instruments/{instrumentId}/option-chain:
525   parameters:
526     - $ref: "#/components/parameters/instrumentId"
527   get:
528     operationId: getOptionChain
529     summary: Get an option chain
530     description: >–
531       Contracts for one expiration (the nearest by default). Every available expiration is
532       linked with the relation `bk:expiration`, named by its date.
533     tags: [market-data]
534     security: [{ oauth2: [market-data:read] }]
535     parameters:
536       - name: expiration
537         in: query
538         schema: { type: string, format: date }
539     responses:
540       "200":
541         description: The option contracts.
542         content:
543           application/hal+json:
544             schema: { $ref: "#/components/schemas/InstrumentCollection" }
545       "404": { $ref: "#/components/responses/NotFound" }

```

```

546
547 components:
548   securitySchemes:
549     oauth2:
550       type: oauth2
551       description: >-
552         OAuth 2.1. Web apps sign customers in through their backend, a confidential client,
553         with the authorization code grant and PKCE; a customer's own scripts and agents use
554         client credentials registered to that customer. Tokens carry `customer_id` and, for
555         delegated clients, `trading_limit`.
556       oauth2MetadataUrl: http://localhost:9000/.well-known/oauth-authorization-server
557       flows:
558         authorizationCode:
559           authorizationUrl: http://localhost:9000/oauth2/authorize
560           tokenUrl: http://localhost:9000/oauth2/token
561           refreshToken: http://localhost:9000/oauth2/token
562           scopes:
563             market-data:read: Instruments, quotes and option chains
564             accounts:read: Accounts, balances, positions and transactions
565             orders:read: Orders, previews and order events
566             orders:write: Place, change and cancel orders
567         clientCredentials:
568           tokenUrl: http://localhost:9000/oauth2/token
569           scopes:
570             market-data:read: Instruments, quotes and option chains
571             accounts:read: Accounts, balances, positions and transactions
572             orders:read: Orders, previews and order events
573             orders:write: Place, change and cancel orders
574
575 parameters:
576   accountId:
577     name: accountId
578     in: path
579     required: true
580     schema: { type: string, examples: [ACC-1001] }
581   orderId:
582     name: orderId
583     in: path
584     required: true
585     schema: { type: string, examples: [ORD-100002] }
586   instrumentId:
587     name: instrumentId
588     in: path
589     required: true
590     schema: { type: string, examples: [EQ-AAPL] }
591   cursor:
592     name: cursor
593     in: query
594     description: Opaque. Take it from a `next` link; never build one.
595     schema: { type: string }
596   limit:
597     name: limit
598     in: query
599     schema: { type: integer, minimum: 1, maximum: 100, default: 25 }
600   Idempotency-Key:
601     name: Idempotency-Key
602     in: header
603     required: true
604     description: A unique value per new order (a UUID works well). Retries reuse it.
605     schema: { type: string, minLength: 1, maxLength: 255 }
606   If-Match:
607     name: If-Match
608     in: header
609     required: true
610     description: The order's current ETag.
611     schema: { type: string }
612   If-None-Match:
613     name: If-None-Match
614     in: header

```

```

615     description: An ETag from an earlier response; an unchanged resource returns 304.
616     schema: { type: string }
617 API-Version:
618     name: API-Version
619     in: header
620     description: The API version to use. Defaults to 1 (deprecated).
621     schema: { type: string, enum: ["1", "2"], default: "1" }
622
623 headers:
624   X-Request-Id:
625     description: The request's ID, echoed from the request or generated.
626     schema: { type: string }
627   Location:
628     description: The URL of the resource created or affected.
629     schema: { type: string, format: uri }
630   ETag:
631     description: The representation's entity tag.
632     schema: { type: string }
633   Idempotent-Replayed:
634     description: true when the response replays an earlier request with the same Idempotency-Key.
635     schema: { type: string, enum: ["true", "false"] }
636   RateLimit:
637     description: 'Remaining quota and seconds until reset, e.g. "default";r=117;t=42.'
638     schema: { type: string }
639   RateLimit-Policy:
640     description: 'The quota policy, e.g. "default";q=120;w=60.'
641     schema: { type: string }
642   Retry-After:
643     description: Seconds to wait before retrying.
644     schema: { type: integer }
645   Deprecation:
646     description: When this version was deprecated (RFC 9745), e.g. @1788220800.
647     schema: { type: string }
648   Sunset:
649     description: When this version stops working (RFC 8594).
650     schema: { type: string }
651
652 responses:
653   BadRequest:
654     description: The request is malformed or fails validation.
655     content:
656       application/problem+json:
657         schema: { $ref: "#/components/schemas/Problem" }
658         examples:
659           invalidFields:
660             dataValue:
661               type: https://docs.brokerage.example/problems/invalid-request
662               title: The request is not valid
663               status: 400
664               detail: One or more fields are invalid
665               instance: /accounts/ACC-1001/orders
666               requestId: 7f3c2a90-4a1e-4c1b-9d3e-2b8f1c6e0a11
667               errors:
668                 - { field: quantity, message: must be greater than 0 }
669   Unauthorized:
670     description: No valid bearer token. See WWW-Authenticate.
671     headers:
672       WWW-Authenticate:
673         schema: { type: string }
674     content:
675       application/problem+json:
676         schema: { $ref: "#/components/schemas/Problem" }
677   Forbidden:
678     description: The token lacks the scope, or the order exceeds the client's trading limit.
679     content:
680       application/problem+json:
681         schema: { $ref: "#/components/schemas/Problem" }
682   NotFound:
683     description: The resource does not exist, or belongs to another customer.

```

```

684     content:
685       application/problem+json:
686         schema: { $ref: "#/components/schemas/Problem" }
687   Conflict:
688     description: The order's state does not allow this change.
689     content:
690       application/problem+json:
691         schema: { $ref: "#/components/schemas/Problem" }
692   PreconditionFailed:
693     description: The ETag in If-Match is stale.
694     content:
695       application/problem+json:
696         schema: { $ref: "#/components/schemas/Problem" }
697   PreconditionRequired:
698     description: If-Match is missing.
699     content:
700       application/problem+json:
701         schema: { $ref: "#/components/schemas/Problem" }
702   UnprocessableContent:
703     description: The request is well formed but breaks a business rule.
704     content:
705       application/problem+json:
706         schema: { $ref: "#/components/schemas/Problem" }
707         examples:
708           buyingPower:
709             dataValue:
710               type: https://docs.brokerage.example/problems/insufficient-buying-power
711               title: The account does not have enough buying power
712               status: 422
713               detail: This order needs 250000.00 but only 52913.10 is available
714               instance: /accounts/ACC-1001/orders
715               requestId: b1b72c27-9103-4236-9cc0-c2b21d4d86bf
716               required: "250000.00"
717               available: "52913.10"
718   TooManyRequests:
719     description: The client's quota is used up.
720     headers:
721       Retry-After: { $ref: "#/components/headers/Retry-After" }
722     content:
723       application/problem+json:
724         schema: { $ref: "#/components/schemas/Problem" }
725
726   schemas:
727     Decimal:
728       description: An exact decimal number, as a string.
729       type: string
730       pattern: '^~?[0-9]+(\\. [0-9]+)?$'
731       examples: ["187.50"]
732     Link:
733       type: object
734       required: [href]
735       properties:
736         href: { type: string }
737         templated: { type: boolean }
738         type: { type: string }
739         name: { type: string }
740         title: { type: string }
741     Links:
742       description: HAL links, keyed by relation. A relation may hold one link or several.
743       type: object
744       properties:
745         self: { $ref: "#/components/schemas/Link" }
746         curies:
747           type: array
748           items: { $ref: "#/components/schemas/Link" }
749       additionalProperties:
750         anyOf:
751           - $ref: "#/components/schemas/Link"
752           - type: array

```

```

753         items: { $ref: "#/components/schemas/Link" }
754 Templates:
755     description: HAL-FORMS templates, present only for application/prs.hal-forms+json.
756     type: object
757     additionalProperties:
758         type: object
759         required: [method]
760         properties:
761             method: { type: string }
762             contentType: { type: string }
763             target: { type: string }
764             properties: { type: array, items: { type: object } }
765 Problem:
766     description: An RFC 9457 problem document. Extension members depend on the type.
767     type: object
768     required: [type, title, status]
769     properties:
770         type: { type: string, format: uri }
771         title: { type: string }
772         status: { type: integer }
773         detail: { type: string }
774         instance: { type: string, format: uri-reference }
775         requestId: { type: string }
776         errors:
777             type: array
778             items:
779                 type: object
780                 required: [field, message]
781                 properties:
782                     field: { type: string }
783                     message: { type: string }
784 ApiRoot:
785     type: object
786     required: [name, _links]
787     properties:
788         name: { type: string }
789         description: { type: string }
790         _links: { $ref: "#/components/schemas/Links" }
791 ProtectedResourceMetadata:
792     type: object
793     required: [resource, authorization_servers]
794     properties:
795         resource: { type: string, format: uri }
796         resource_name: { type: string }
797         authorization_servers: { type: array, items: { type: string, format: uri } }
798         scopes_supported: { type: array, items: { type: string } }
799         bearer_methods_supported: { type: array, items: { type: string } }
800         tls_client_certificate_bound_access_tokens: { type: boolean }
801
802 # ----- accounts
803 Account:
804     type: object
805     required: [id, number, type, registration, currency, _links]
806     properties:
807         id: { type: string }
808         number:
809             type: string
810             description: Masked; only the last four digits are sent
811             examples: ["****4821"]
812         nickname: { type: string }
813         type: { type: string, enum: [CASH, MARGIN] }
814         registration: { type: string, enum: [INDIVIDUAL, JOINT, ROTH_IRA, TRADITIONAL_IRA] }
815         currency: { type: string, pattern: "^[A-Z]{3}$" }
816         openedOn: { type: string, format: date }
817         _links: { $ref: "#/components/schemas/Links" }
818         _templates: { $ref: "#/components/schemas/Templates" }
819 AccountCollection:
820     type: object
821     required: [_links]

```

```

822     properties:
823       _embedded:
824         type: object
825         properties:
826           bk:accounts: { type: array, items: { $ref: "#/components/schemas/Account" } }
827           _links: { $ref: "#/components/schemas/Links" }
828   Balances:
829     type: object
830     required: [accountId, currency, cash, reservedForOrders, buyingPower, positionsMarketValue,
831               totalEquity, unrealizedPnl, asOf, _links]
832     properties:
833       accountId: { type: string }
834       currency: { type: string }
835       cash: { $ref: "#/components/schemas/Decimal" }
836       reservedForOrders: { $ref: "#/components/schemas/Decimal" }
837       buyingPower: { $ref: "#/components/schemas/Decimal" }
838       positionsMarketValue: { $ref: "#/components/schemas/Decimal" }
839       totalEquity: { $ref: "#/components/schemas/Decimal" }
840       unrealizedPnl: { $ref: "#/components/schemas/Decimal" }
841       asOf: { type: string, format: date-time }
842       _links: { $ref: "#/components/schemas/Links" }
843   Position:
844     type: object
845     required: [instrumentId, symbol, instrumentType, quantity, averageCost, costBasis, lastPrice,
846               marketValue, unrealizedPnl, unrealizedPnlPercent, _links]
847     properties:
848       instrumentId: { type: string }
849       symbol: { type: string }
850       instrumentType: { $ref: "#/components/schemas/InstrumentType" }
851       quantity: { $ref: "#/components/schemas/Decimal" }
852       averageCost: { $ref: "#/components/schemas/Decimal" }
853       costBasis: { $ref: "#/components/schemas/Decimal" }
854       lastPrice: { $ref: "#/components/schemas/Decimal" }
855       marketValue: { $ref: "#/components/schemas/Decimal" }
856       unrealizedPnl: { $ref: "#/components/schemas/Decimal" }
857       unrealizedPnlPercent: { $ref: "#/components/schemas/Decimal" }
858       _links: { $ref: "#/components/schemas/Links" }
859   PositionCollection:
860     type: object
861     required: [_links]
862     properties:
863       _embedded:
864         type: object
865         properties:
866           bk:positions: { type: array, items: { $ref: "#/components/schemas/Position" } }
867       _links: { $ref: "#/components/schemas/Links" }
868   TransactionType:
869     type: string
870     enum: [DEPOSIT, WITHDRAWAL, TRADE, DIVIDEND, FEE]
871   Transaction:
872     type: object
873     required: [id, type, description, amount, occurredAt, settlesOn]
874     properties:
875       id: { type: string }
876       type: { $ref: "#/components/schemas/TransactionType" }
877       description: { type: string }
878       amount: { $ref: "#/components/schemas/Decimal", description: Positive into the account, negative out }
879       instrumentId: { type: string }
880       symbol: { type: string }
881       quantity: { $ref: "#/components/schemas/Decimal" }
882       price: { $ref: "#/components/schemas/Decimal" }
883       orderId: { type: string }
884       occurredAt: { type: string, format: date-time }
885       settlesOn: { type: string, format: date }
886   TransactionPage:
887     type: object
888     required: [_links]
889     properties:
890       _embedded:

```

```

891     type: object
892     properties:
893       bk:transactions: { type: array, items: { $ref: "#/components/schemas/Transaction" } }
894       _links: { $ref: "#/components/schemas/Links" }
895
896 # ----- market data
897 InstrumentType:
898   type: string
899   enum: [EQUITY, ETF, OPTION]
900 Instrument:
901   type: object
902   required: [id, symbol, name, type, currency, tradable, fractionable, multiplier, _links]
903   properties:
904     id: { type: string }
905     symbol: { type: string }
906     name: { type: string }
907     type: { $ref: "#/components/schemas/InstrumentType" }
908     exchange: { type: string, description: ISO 10383 market identifier code }
909     currency: { type: string }
910     tradable: { type: boolean }
911     fractionable: { type: boolean }
912     multiplier: { $ref: "#/components/schemas/Decimal" }
913     option:
914       type: object
915       required: [underlyingId, right, strike, expiration]
916       properties:
917         underlyingId: { type: string }
918         underlyingSymbol: { type: string }
919         right: { type: string, enum: [CALL, PUT] }
920         strike: { $ref: "#/components/schemas/Decimal" }
921         expiration: { type: string, format: date }
922         style: { type: string, enum: [AMERICAN, EUROPEAN] }
923       _links: { $ref: "#/components/schemas/Links" }
924 InstrumentCollection:
925   type: object
926   required: [_links]
927   properties:
928     _embedded:
929       type: object
930       properties:
931         bk:instruments: { type: array, items: { $ref: "#/components/schemas/Instrument" } }
932       _links: { $ref: "#/components/schemas/Links" }
933 QuoteV1:
934   description: Version 1 (deprecated). Flat fields.
935   type: object
936   required: [instrumentId, symbol, last, bid, ask, bidSize, askSize, volume, asOf]
937   properties:
938     instrumentId: { type: string }
939     symbol: { type: string }
940     last: { $ref: "#/components/schemas/Decimal" }
941     bid: { $ref: "#/components/schemas/Decimal" }
942     ask: { $ref: "#/components/schemas/Decimal" }
943     bidSize: { $ref: "#/components/schemas/Decimal" }
944     askSize: { $ref: "#/components/schemas/Decimal" }
945     volume: { type: integer }
946     asOf: { type: string, format: date-time }
947     _links: { $ref: "#/components/schemas/Links" }
948 Quote:
949   description: Version 2.
950   type: object
951   required: [instrumentId, symbol, last, bid, ask, change, changePercent, volume, asOf]
952   properties:
953     instrumentId: { type: string }
954     symbol: { type: string }
955     last: { $ref: "#/components/schemas/Decimal" }
956     bid: { $ref: "#/components/schemas/PriceLevel" }
957     ask: { $ref: "#/components/schemas/PriceLevel" }
958     change: { $ref: "#/components/schemas/Decimal" }
959     changePercent: { $ref: "#/components/schemas/Decimal" }

```

```

960     volume: { type: integer }
961     asOf: { type: string, format: date-time }
962     _links: { $ref: "#/components/schemas/Links" }
963 PriceLevel:
964     type: object
965     required: [price, size]
966     properties:
967         price: { $ref: "#/components/schemas/Decimal" }
968         size: { $ref: "#/components/schemas/Decimal" }
969
970 # ----- orders
971 OrderStatus:
972     type: string
973     enum: [OPEN, PARTIALLY_FILLED, FILLED, PENDING_CANCEL, CANCELLED, REJECTED]
974 OrderSide:
975     type: string
976     enum: [BUY, SELL]
977 OrderType:
978     type: string
979     enum: [MARKET, LIMIT, STOP, STOP_LIMIT]
980 TimeInForce:
981     type: string
982     enum: [DAY, GTC]
983     default: DAY
984 OrderRequest:
985     type: object
986     required: [instrumentId, side, type, quantity]
987     properties:
988         instrumentId: { type: string, minLength: 1 }
989         side: { $ref: "#/components/schemas/OrderSide" }
990         type: { $ref: "#/components/schemas/OrderType" }
991         timeInForce: { $ref: "#/components/schemas/TimeInForce" }
992         quantity: { $ref: "#/components/schemas/Decimal", description: Shares, or contracts for options }
993         limitPrice: { $ref: "#/components/schemas/Decimal", description: Required for LIMIT and STOP_LIMIT }
994         stopPrice: { $ref: "#/components/schemas/Decimal", description: Required for STOP and STOP_LIMIT }
995         clientOrderId: { type: string, maxLength: 64, pattern: "^[A-Za-z0-9._-]*$" }
996 OrderAmendment:
997     type: object
998     minProperties: 1
999     properties:
1000         quantity: { $ref: "#/components/schemas/Decimal" }
1001         limitPrice: { $ref: "#/components/schemas/Decimal" }
1002         stopPrice: { $ref: "#/components/schemas/Decimal" }
1003         timeInForce: { $ref: "#/components/schemas/TimeInForce" }
1004 Fill:
1005     type: object
1006     required: [id, quantity, price, executedAt]
1007     properties:
1008         id: { type: string }
1009         quantity: { $ref: "#/components/schemas/Decimal" }
1010         price: { $ref: "#/components/schemas/Decimal" }
1011         executedAt: { type: string, format: date-time }
1012 Order:
1013     type: object
1014     required: [id, accountId, instrumentId, symbol, instrumentType, side, type, timeInForce, quantity,
1015         status, filledQuantity, remainingQuantity, fills, createdAt, updatedAt, _links]
1016     properties:
1017         id: { type: string }
1018         accountId: { type: string }
1019         clientOrderId: { type: string }
1020         instrumentId: { type: string }
1021         symbol: { type: string }
1022         instrumentType: { $ref: "#/components/schemas/InstrumentType" }
1023         side: { $ref: "#/components/schemas/OrderSide" }
1024         type: { $ref: "#/components/schemas/OrderType" }
1025         timeInForce: { $ref: "#/components/schemas/TimeInForce" }
1026         quantity: { $ref: "#/components/schemas/Decimal" }
1027         limitPrice: { $ref: "#/components/schemas/Decimal" }
1028         stopPrice: { $ref: "#/components/schemas/Decimal" }

```

```

1029     status: { $ref: "#/components/schemas/OrderStatus" }
1030     filledQuantity: { $ref: "#/components/schemas/Decimal" }
1031     remainingQuantity: { $ref: "#/components/schemas/Decimal" }
1032     averageFillPrice: { $ref: "#/components/schemas/Decimal" }
1033     fills: { type: array, items: { $ref: "#/components/schemas/Fill" } }
1034     createdAt: { type: string, format: date-time }
1035     updatedAt: { type: string, format: date-time }
1036     _links: { $ref: "#/components/schemas/Links" }
1037     _templates: { $ref: "#/components/schemas/Templates" }
1038 OrderPage:
1039     type: object
1040     required: [_links]
1041     properties:
1042       _embedded:
1043         type: object
1044         properties:
1045           bk:orders: { type: array, items: { $ref: "#/components/schemas/Order" } }
1046       _links: { $ref: "#/components/schemas/Links" }
1047 OrderPreview:
1048     type: object
1049     required: [accountId, instrumentId, side, type, quantity, acceptable, issues]
1050     properties:
1051       accountId: { type: string }
1052       instrumentId: { type: string }
1053       symbol: { type: string }
1054       side: { $ref: "#/components/schemas/OrderSide" }
1055       type: { $ref: "#/components/schemas/OrderType" }
1056       quantity: { $ref: "#/components/schemas/Decimal" }
1057       estimatedPrice: { $ref: "#/components/schemas/Decimal" }
1058       estimatedNotional: { $ref: "#/components/schemas/Decimal" }
1059       estimatedCommission: { $ref: "#/components/schemas/Decimal" }
1060       estimatedTotal: { $ref: "#/components/schemas/Decimal" }
1061       buyingPower: { $ref: "#/components/schemas/Decimal" }
1062       acceptable: { type: boolean }
1063       issues:
1064         type: array
1065         items:
1066           type: object
1067           required: [type, detail]
1068           properties:
1069             type: { type: string, format: uri }
1070             detail: { type: string }
1071       _links: { $ref: "#/components/schemas/Links" }
1072 OrderEventData:
1073     type: object
1074     required: [orderId, status, filledQuantity, remainingQuantity, occurredAt, _links]
1075     properties:
1076       orderId: { type: string }
1077       status: { $ref: "#/components/schemas/OrderStatus" }
1078       filledQuantity: { $ref: "#/components/schemas/Decimal" }
1079       remainingQuantity: { $ref: "#/components/schemas/Decimal" }
1080       averageFillPrice: { $ref: "#/components/schemas/Decimal" }
1081       occurredAt: { type: string, format: date-time }
1082       _links:
1083         type: object
1084         required: [order]
1085         properties:
1086           order: { $ref: "#/components/schemas/Link" }

```

From the publisher

About the author

Smiroh Dev writes practical, standards-first guides for software engineers who build and secure web systems. Every Smiroh book is built around working code that runs on your own machine, because the best way to understand a standard is to watch it do its job.

Also from Smiroh Publishers

The Ultimate Developer's Guide to OAuth 2.1 and PKCE

A complete OAuth 2.1 deployment on your laptop, with an identity provider, an authorization server, an API and a client, taken apart in hands-on labs. The natural companion to Chapter 20.

The Ultimate Developer's Guide to CORS: Border Control for the Web

Cross-Origin Resource Sharing from first principles: the same-origin policy, preflights, credentials, real incidents and secure configuration, with eight labs.

Design the API you would want to be a client of. Then write the standard down, so everyone else can build it too.

Thank you for reading. Corrections and suggestions are welcome through the publisher's store page.